The Complete System Design Knowledge Base — Preview

From first principles to expert practice

Built from the manuscript sources · 6 chapter(s) in this edition

Part I — Foundations of System Design Thinking

Chapter 1 — What Is System Design?

1.1 Learning objectives
1.2 Prerequisites
1.3 Definition of system design
1.4 Software architecture
1.5 High-level vs low-level design
1.6 Functional requirements
1.7 Non-functional requirements (the “-ilities” catalog)
1.8 Architectural decision-making and ADRs
1.9 Why system design matters (cost of wrong decisions)
1.10 How system design is practiced in industry
1.11 How system design interviews work
1.12 Common beginner mistakes
1.13 Summary and revision notes
1.14 Knowledge checkpoint
1.15 Interview questions
1.16 Practical exercises
1.17 Design challenges
1.18 References
1.19 Open verification items

Chapter 2 — How to Think Like a System Designer

2.1 Learning objectives
2.2 Prerequisites
2.3 Decomposition into components
2.4 System boundaries and context diagrams
2.5 Understanding and negotiating requirements
2.6 Identifying bottlenecks
2.7 Making and stating assumptions
2.8 Evaluating trade-offs
2.9 Designing for failure
2.10 Reasoning about scale
2.11 Back-of-the-envelope capacity estimation
2.12 Architecture documentation
2.13 Summary and revision notes
2.14 Knowledge checkpoint
2.15 Interview questions
2.16 Practical exercises
2.17 Design challenges
2.18 References
2.19 Open verification items

Chapter 3 — The System Design Learning Method

3.1 Learning objectives
3.2 Prerequisites
3.3 The ten-step concept-mastery loop

3.4 Active recall, spaced repetition, the Feynman technique, interleaving and deliberate practice
3.5 Reading papers and engineering blogs
3.6 Drawing architecture from memory
3.7 Building simplified implementations
3.8 Beginner methodology: a 12-month path
3.9 Advanced methodology for experienced engineers
3.10 Tracking progress
3.11 Summary and revision notes
3.12 Knowledge checkpoint
3.13 Interview questions
3.14 Practical exercises
3.15 Design challenges
3.16 References
3.17 Open verification items

Part II — Computer Science Foundations

Chapter 5 — Computer Architecture
5.1 Learning objectives
5.2 Prerequisites
5.3 The CPU: cores, registers, clock and the instruction pipeline
5.4 CPU caches
5.5 RAM and the memory hierarchy
5.6 Storage devices
5.7 I/O operations
5.8 CPU-bound versus I/O-bound workloads
5.9 Memory latency and bandwidth
5.10 NUMA
5.11 SIMD and accelerators
5.12 Throughput versus latency at the hardware level
5.13 Summary and revision notes
5.14 Knowledge checkpoint
5.15 Interview questions
5.16 Practical exercises
5.17 Design challenges
5.18 References
5.19 Open verification items

Chapter 7 — Networking Fundamentals
7.1 Learning objectives
7.2 Prerequisites
7.3 OSI and TCP/IP models
7.4 IP addressing, IPv4/IPv6, subnets and CIDR
7.5 Routing basics, BGP and anycast
7.6 DNS: the Internet’s directory
7.7 TCP: reliable byte streams over an unreliable network
7.8 UDP and when to use it
7.9 HTTP/1.1, HTTP/2, HTTP/3 and QUIC
7.10 TLS 1.3 and HTTPS

7.11 WebSockets (introduction)
7.12 Forward proxies and reverse proxies
7.13 NAT
7.14 Firewalls and security groups
7.15 Latency, bandwidth, the bandwidth-delay product and packet loss
7.16 Connection pooling and keep-alive
7.17 Datacenter networking
7.18 What happens when you type a URL
7.19 Summary and revision notes
7.20 Knowledge checkpoint
7.21 Interview questions
7.22 Practical exercises
7.23 Design challenges
7.24 References
7.25 Open verification items

Chapter 10 — Security Fundamentals for System Designers

10.1 Learning objectives
10.2 Prerequisites
10.3 Threat modeling
10.4 Cryptography essentials
10.5 PKI and certificates
10.6 Password storage
10.7 The OWASP Top 10 in architectural terms
10.8 Secrets management
10.9 Encryption at rest and in transit, KMS and envelope encryption
10.10 Network security, zero trust and mTLS
10.11 DDoS basics
10.12 Audit logging and tamper evidence
10.13 A security checklist for every design
10.14 Summary and revision notes
10.15 Knowledge checkpoint
10.16 Interview questions
10.17 Practical exercises
10.18 Design challenges
10.19 References
10.20 Open verification items

Part I — Foundations of System Design Thinking

Chapter 1 — What Is System Design?

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3.5 h

Every application you have ever used — a photo app, a bank, a search engine — is two things at once. It is *code*, which tells a computer what to do, and it is a *system*, which decides where that code runs, where its data lives, how many copies of each exist, what happens when a machine dies at 3 a.m., and how the whole thing keeps working when ten times more people show up next month. This chapter is about the second thing.

We will follow one fictional product through the whole chapter: **PhotoShare**, a simple photo-sharing app in which people upload photos, follow friends and scroll a feed. PhotoShare starts life on a single rented server with a thousand users. By the end of the chapter it serves ten million daily users across multiple machines and data centers. At every stage you will see *why* the design had to change, *which requirement* forced the change, and *what it cost*. That progression — requirements forcing decisions, decisions carrying costs — is system design.

1.1 Learning objectives

After this chapter you should be able to:

- Define *system design*, *software architecture*, *high-level design* and *low-level design*, and explain how they differ and overlap.
- Distinguish functional requirements from non-functional requirements, and write each in a testable form.
- Name at least fifteen quality attributes (“-ilities”), state a measurable metric for each, and identify common tensions between them.
- Convert an availability target (for example 99.9%) into an allowed-downtime budget, and compute the availability of components in series.
- Estimate average and peak request rates, storage growth and network egress from a user count, using explicit assumptions.
- Write an Architecture Decision Record (ADR) in Nygard’s format and use a weighted decision matrix to compare options.
- Explain why early architectural decisions are expensive to reverse, using a concrete migration-cost calculation.
- Describe how design documents, design reviews and RFC processes work in industry.

- Describe the structure of a 45–60-minute system design interview and what interviewers evaluate.
- Recognize and avoid the twelve most common beginner mistakes in system design.

1.2 Prerequisites

This chapter assumes no prior system design knowledge. Helpful but not required:

- Basic programming in any language — so that “a function”, “a class” and “a web request” are familiar words.
- A rough idea of what happens when a browser loads a web page — treated fully in §8.5 (The request–response lifecycle end to end); here we only need “a client sends a request, a server sends a response”.
- A rough idea of what a database is — treated fully in Ch 11 (Relational Databases); here we only need “a program that stores data durably and answers queries about it”.

Every other term is defined when it first appears.

1.3 Definition of system design

1.3.1 What a “system” is

DEFINITION

Definition — System. A set of interacting **components** (programs, machines, data stores, networks and people) that together deliver a capability no single component delivers alone. Components interact only through **interfaces**: agreed formats and protocols for exchanging requests, responses and data.

DEFINITION

Definition — Component. A unit of a system with a clear responsibility and an interface — for example a web service, a database, a cache, a message queue, a load balancer or a third-party payment provider.

First-principles explanation

Think of a busy restaurant. A single recipe is like a single function in code: it turns inputs (ingredients) into an output (a dish). But a restaurant is not a recipe. It is a *system*: a front-of-house that takes orders, a ticket rail that queues them, stations for grill and salad, a walk-in fridge (storage), a dishwasher (cleanup), a manager who notices when

the grill cook is overwhelmed, and a supplier who delivers flour every Tuesday. The food can be excellent and the restaurant can still fail — because orders get lost, the fridge breaks, or Saturday night brings three times more guests than the kitchen can handle.

Software is the same. The code that resizes a photo can be perfect, and PhotoShare can still fail because the disk filled up, the only server rebooted during a deploy, or a celebrity's post caused a thousand times the normal traffic in ten seconds.

1.3.2 System design

> DEFINITION
>
> **Definition — System design.** The process of deciding the components of a software system, their responsibilities, their interfaces, how data flows and is stored between them, and how they are deployed — so that the system meets its functional requirements *and* its non-functional requirements (performance, scalability, availability, durability, security, cost and others) under stated assumptions about load and failure. The output of the process is also called the *system design*: a set of decisions, diagrams and documents.

Two parts of the definition deserve emphasis.

1. **"So that it meets its requirements."** A design is never good or bad in the abstract — only good or bad *for a stated set of requirements*. A single server is an excellent design for 1,000 users and a terrible one for 10 million.
2. **"Under stated assumptions about load and failure."** Designers must say out loud how much traffic they expect and what kinds of failure they expect to survive. A design that is silent about these is not finished.

Why does this exist?

For most of computing history, a program ran on one machine. Today almost any useful service must handle more data, more users and more failure than one machine can. A modern commodity server fails on the order of once every few years; a fleet of thousands therefore sees failures every day Barroso et al. 2018. Users are spread across continents, and light in optical fiber needs roughly 5 ms per 1,000 km one way, so a single location cannot be fast for everyone (see §7.15 (Latency, bandwidth, BDP, packet loss)). Regulations require some data to stay in certain countries. Each of these forces pushes the system apart into multiple components, and the moment you have multiple components you must decide how they divide the work, how they talk, and what happens when one of them stops. System design exists because those decisions are hard to get right, expensive to get wrong and impossible to avoid.

How it works internally

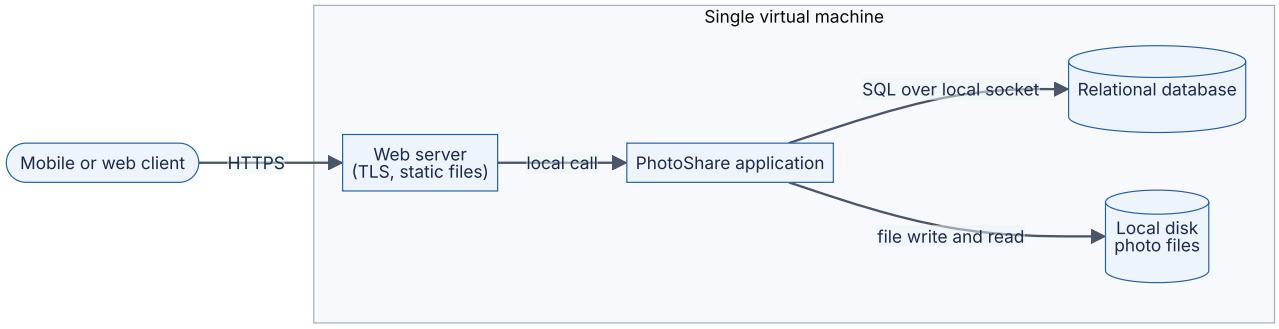
System design is iterative, but in practice most designers follow a loop like this:

1. **Understand the problem.** Who are the users? What must the system do (functional requirements, §1.6)? How well must it do it (non-functional requirements, §1.7)?
2. **Quantify.** Estimate users, request rates, data sizes and growth (back-of-the-envelope estimation; see §2.11 and Ch 89).
3. **Propose a simple design.** Start with the smallest design that could work: a client, a service, a database.
4. **Stress the design against the requirements.** Where does it break first — CPU, memory, disk, network, a single point of failure, a consistency rule? (See §2.6 (Identifying bottlenecks).)
5. **Evolve the design.** Add or change components to fix the weakest point, noting what each change costs.
6. **Record decisions** and their rationale (ADRs, §1.8), and communicate the design (design docs, §1.10).
7. **Validate in production** with monitoring, load tests and incidents, and feed what you learn back into step 1.

Architecture diagram

Here is PhotoShare on launch day. Everything runs on one rented virtual machine.

Figure 1.1 — PhotoShare v0: a single-server design.



Reading the diagram.

- **Components.** A *client* (the mobile app or browser), a *web server* that terminates HTTPS and serves static files, the *PhotoShare application* (business logic: sign-up, upload, feed), a *relational database* holding users, follows and photo metadata, and the *local disk* holding the photo files themselves.
- **Protocols.** The client speaks HTTPS to the web server. Inside the machine, the application talks to the database over a local socket using SQL and reads and writes photo files through the operating system's file system.

- **Direction of data flow.** Uploads flow left to right: client → web server → application → database row plus file on disk. Feed requests flow the same way and responses return along the same path.
- **Synchronous vs asynchronous.** Every arrow is solid: every step is synchronous. The client waits while the application stores the photo and generates any thumbnails.
- **Storage boundaries.** All state — rows and files — lives on one machine's disk. The *subgraph* is the deployment boundary: everything inside shares one fate.
- **Failure points.** The virtual machine is a **single point of failure (SPOF)**: if it reboots, crashes, or its disk fails, PhotoShare is completely down, and photos may be lost if there is no backup.
- **Scaling options.** The only option without redesign is **vertical scaling** — renting a bigger machine (see §18.3). That buys time but has a ceiling and does not remove the SPOF.

This design is not a mistake. For a launch with 1,000 users it is the *right* design: cheap, simple, and easy to change. The rest of the chapter shows which requirements eventually break it.

Mathematical foundations

The first quantitative question in any design is "how much load?" The simplest useful formula converts daily activity into a request rate.

FORMULA

Average and peak request rate

$$\mathrm{RPS}_{\mathrm{avg}} = \frac{\mathrm{DAU} \times r}{86,400} \qquad \mathrm{RPS}_{\mathrm{peak}} = k \times \mathrm{RPS}_{\mathrm{avg}}$$

Symbol	Meaning	Units
$\mathrm{RPS}_{\mathrm{avg}}$	Average request rate over a day	requests/second
DAU	Daily active users	users
r	Requests (of the type being estimated) per user per day	requests/user/day
$86,400$	Seconds in a day	seconds/day
k	Peak factor: ratio of the busiest period's rate to the daily average (dimensionless; 2–5 is a common assumption for consumer apps)	—
$\mathrm{RPS}_{\mathrm{peak}}$	Request rate during the busiest period	requests/second

RPS (requests per second) is the canonical unit in this book; you will also see **QPS** (queries per second) used as a synonym.

WORKED EXAMPLE

Worked example — PhotoShare on launch day.

Assumptions (stated, as always): 1,000 DAU; each user views 50 photos per day; peak factor $k = 3$.

1. Daily photo views: $1,000 \times 50 = 50,000$ views/day.
2. Average rate: $50,000/86,400 \approx 0.58$ RPS.
3. Peak rate: $3 \times 0.58 \approx 1.7$ RPS.

Interpretation. Under two requests per second at peak. A single small virtual machine running an ordinary web framework typically handles hundreds of such requests per second (an order-of-magnitude assumption; measure your own stack, see §23.10).

Engineering implication. Any design more complex than Figure 1.1 would be *over-engineering* at this scale: more cost, more moving parts, more ways to fail, and no benefit users can feel.

Code example

Estimation is arithmetic, but writing it as a tiny program makes your assumptions explicit and easy to change. The same script is reused in §1.9 when PhotoShare grows.

```
        """Back-of-the-envelope estimator for the PhotoShare running example.

        Uses SI units: 1 MB = 10**6 bytes, 1 TB = 10**12 bytes.
        """
        from dataclasses import dataclass

        SECONDS_PER_DAY = 86_400


        @dataclass
        class Workload:
            dau: int                  # daily active users
            views_per_user: float     # photo views per user per day
            uploads_per_user: float   # photo uploads per user per day
            stored_bytes_per_upload: int  # original + thumbnails, bytes
            served_bytes_per_view: int    # bytes sent to the client per view
            peak_factor: float = 3.0      # peak-hour rate / daily average rate


        def estimate(w: Workload) -> dict:
            views_per_day = w.dau * w.views_per_user
            uploads_per_day = w.dau * w.uploads_per_user
            read_rps = views_per_day / SECONDS_PER_DAY
            write_rps = uploads_per_day / SECONDS_PER_DAY
            storage_per_day = uploads_per_day * w.stored_bytes_per_upload
            egress_bits_per_s = views_per_day * w.served_bytes_per_view * 8 /
SECONDS_PER_DAY
            return {
                "read_rps_avg": read_rps,
                "read_rps_peak": read_rps * w.peak_factor,
                "write_rps_avg": write_rps,
                "write_rps_peak": write_rps * w.peak_factor,
                "storage_TB_per_day": storage_per_day / 1e12,
                "storage_PB_per_year": storage_per_day * 365 / 1e15,
                "egress_Gbps_avg": egress_bits_per_s / 1e9,
                "egress_Gbps_peak": egress_bits_per_s * w.peak_factor / 1e9,
            }


        if __name__ == "__main__":
            for label, dau in [("v0 launch", 1_000), ("v3 scale", 10_000_000)]:
                w = Workload(dau=dau, views_per_user=50, uploads_per_user=0.2,
                             stored_bytes_per_upload=2_500_000,
                             served_bytes_per_view=200_000)
                print(label)
                for key, value in estimate(w).items():
                    print(f"  {key:>20}: {value:,.3f}")
```

What to notice:

- Every assumption is a named field with a unit in the comment. When a reviewer disagrees with “50 views per user”, they change one number, not your reasoning.
- The peak factor is explicit. Systems are sized for the peak, not the average.

- Egress is converted from bytes to **bits** (`* 8`) because network links are rated in bits per second.
- Running it prints about 0.58 average read RPS for v0 and about 5,787 average (17,361 peak) read RPS, 5 TB/day of new storage and about 9.3 Gbit/s average egress for v3. Those v3 numbers drive most of §1.9.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Instagram (2011).** About a year after launch, Instagram's engineering team published a description of its stack: a load balancer in front of Django application servers, PostgreSQL for data (partitioned across multiple database instances, with replicas in a different availability zone), Redis and Memcached for feeds and caching, photos stored in Amazon S3 and served through a CDN, and a task queue for asynchronous work. Their stated principles were to keep things simple, avoid reinventing the wheel, and use proven technologies. The problem (rapid growth with a tiny team) drove an architecture built mostly from well-understood, managed components rather than custom infrastructure. The trade-off: less control and some vendor dependence, in exchange for a very small operations burden. *Evidence: Primary* — Instagram Engineering 2011

Advantages and disadvantages

Treating design as an explicit activity — rather than letting structure emerge from whatever code is written first — has a clear profile. **Advantages:** problems are found on a whiteboard instead of in production; trade-offs are chosen deliberately; teams share a mental model; costs are predictable. **Disadvantages:** design takes time; designs made with poor information can be confidently wrong; and heavy upfront design can delay learning from real users. The practical answer is *proportionate* design: spend design effort in proportion to how expensive a decision would be to reverse (§1.8, §2.8).

Failure scenarios

Table 1.1 — Failure scenarios for the design process itself.

What goes wrong	Why	Detection	Recovery	Prevention
System collapses under launch traffic	Load never estimated; peak factor ignored	Latency and error rate spike; saturated CPU or connections	Emergency vertical scaling; shed load	Estimate peak load before launch (§1.3.2); load-test
Total outage when one machine fails	Single point of failure nobody named	Health checks fail; users report outage	Restore from backup onto a new machine	List SPOFs in every design review (§1.10)
Months spent building infrastructure for users who never arrive	Design sized for imagined scale	Low utilization; slipped launch	Simplify; delete unused components	Design for 10× current load, not 1,000×
Nobody remembers why a component exists	Decisions not recorded	Repeated debates; fear of changing anything	Reconstruct history from commits and people	ADRs (§1.8)

Common misconceptions

COMMON MISTAKE

"System design means drawing boxes and arrows." The diagram is the *output*. The substance is the reasoning: which requirements forced each box, what each arrow carries, what happens when each box fails, and what the design costs. A beautiful diagram with no requirements and no numbers is decoration.

COMMON MISTAKE

"System design only matters at Google scale." A three-person startup still chooses where its data lives, how it backs up, and what happens during a deploy. The decisions are smaller, but the cost of getting the *irreversible* ones wrong (data model, identifiers, storage location of user files) is felt at every scale.

1.4 Software architecture

1.4.1 What architecture is

DEFINITION

Definition — Software architecture. "The set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both" [Bass et al. 2021]. The international standard for architecture description defines it as the fundamental concepts or properties of a system in its environment, embodied in its elements, relationships, and in the principles of its design and evolution [ISO/IEC/IEEE 42010].

First-principles explanation

Return to the restaurant. Its *architecture* is not the color of the napkins; it is the set of decisions that would be painful to change once the doors open: where the kitchen sits relative to the dining room, how many burners there are, whether orders go through a single ticket rail or directly to stations, whether there is a walk-in fridge or daily deliveries. You can change the menu weekly; moving the kitchen means closing for months.

Martin Fowler summarized the same idea for software: architecture is about "the important stuff — whatever that is", meaning the decisions that are hard to change and that the team must share an understanding of [Fowler 2003]. The parenthetical "whatever that is" is honest: what counts as architectural depends on the system. For PhotoShare, *where photo files live* is architectural (changing it means migrating terabytes); *which image-resizing library we call* is usually not.

Why does this exist?

Large systems are built by many people over many years. Without a shared structure, each person optimizes locally and the whole becomes inconsistent: three ways to authenticate, four copies of the user table, no one able to say what happens if the cache is down. Architecture exists to (1) make the system understandable enough to reason about, (2) make the most important qualities — performance, availability, security, modifiability — achievable by design rather than by luck, and (3) let teams work in parallel along stable boundaries.

How it works internally

An architecture is described by several **views**, because no single picture answers every question. Kruchten's influential "4+1" model proposes four views plus scenarios that tie them together [Kruchten 1995]:

1. **Logical view** — the main abstractions and their responsibilities (users, photos, feeds, follows).
2. **Process view** — what runs concurrently, and how runtime components communicate (requests, queues, threads).
3. **Development view** — how the code is organized into modules, packages and repositories, and which teams own them.
4. **Physical view** — which machines, data centers and networks the software is deployed on.
5. **+1: Scenarios** — a few important use cases ("a user uploads a photo") traced through all four views to validate them.

The C4 model (context, containers, components, code) is a more recent, lightweight alternative used widely in industry; it is covered in §2.12 (Architecture documentation).

Architectures also follow recognizable **styles** or **patterns** — layered, monolithic, modular monolith, microservices, event-driven, serverless — each a reusable answer to a recurring set of forces. They are the subject of Ch 33 (Architecture Patterns) and Ch 34 (Microservices).

> KEY CONCEPT
>
> **Conway's law.** Melvin Conway observed in 1968 that organizations which design systems are constrained to produce designs that copy the organization's communication structure [Conway 1968]. If three teams build a compiler, you tend to get a three-pass compiler. Architecture and team structure therefore co-evolve: choosing service boundaries is partly choosing team boundaries, and vice versa (see §34.3 (Service boundaries)).

1.4.2 Architecture, system design and design: how the words relate

The terms overlap heavily, and different companies use them differently. The working distinction used in this book is in Table 1.2.

Table 1.2 — Overlapping disciplines: architecture, system design and low-level design.

Aspect	Software architecture	System design	Low-level design
What it is	The significant, hard-to-change structures of a system	Choosing components, data flow and deployment to meet requirements at scale	Internal structure of one component
Typical questions	Monolith or services? Which quality attributes dominate?	How many servers? Which database? Where to cache? What if a node fails?	Which classes, interfaces, algorithms, schemas?
Unit of reasoning	Elements, relations, properties	Services, stores, networks, traffic, failures	Classes, functions, tables, state machines
Time horizon	Years	Months to years	Weeks to months
Typical artifacts	Architecture views, principles, ADRs	Design docs, capacity estimates, deployment diagrams	Class/sequence diagrams, API specs, schema DDL
Where in this book	Ch 33–34	Most of Book I; case studies Ch 52–56	Ch 35–38

In everyday industry usage, "system design" usually means the distributed-systems end of architecture: servers, databases, caches, queues, networks and their failure modes. "Software architecture" is the broader discipline that also includes code-level structure

and organizational concerns. This book treats system design as the large-scale, runtime-focused part of architecture.

Real-world industry applications

INDUSTRY EXAMPLE

Architecture as a role. Many companies do not have a job titled "architect"; architectural decisions are made by senior and staff engineers through design documents and reviews (§1.10). Others have dedicated architects who own cross-team standards. In both models the activity is the same — identifying the hard-to-change decisions and making them deliberately — but the accountability differs. *Evidence: Illustrative* — a general description of common industry practice, not a claim about any specific company.

Advantages and disadvantages

An explicit architecture makes systems understandable, supports parallel work, and lets you predict quality attributes before building. Its costs are upfront effort, the risk of an "ivory-tower" architecture that ignores implementation realities, and rigidity if the architecture is enforced long after the requirements that justified it have changed.

Failure scenarios

Table 1.3 — Architecture-level failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
"Big ball of mud" — every module depends on every other	No agreed structure; boundaries eroded under deadline pressure	Small changes break distant features; slow builds	Incremental refactoring toward modules (§33.5)	Explicit module boundaries; dependency checks in CI
Architecture on paper differs from reality	Diagrams never updated	New engineers' mental model is wrong; incidents surprise everyone	Re-derive diagrams from deployment config and traces	Keep diagrams next to code; review in design reviews
Distributed monolith	Split into services without splitting data or ownership	Every release requires coordinating many services	Merge services or re-cut boundaries	Draw boundaries around data ownership (§34.11, §34.14)

Common misconceptions

COMMON MISTAKE

"Architecture is decided once, at the start." Requirements change, traffic grows, teams reorganize. Good architecture is *evolved* deliberately, with decisions revisited when their context changes — which is exactly why ADRs record the context (§1.8).

1.5 High-level vs low-level design

1.5.1 Definitions

DEFINITION

Definition — High-level design (HLD). A description of a system as a set of major components (services, data stores, queues, caches, external dependencies), their responsibilities, the interfaces and protocols between them, data flows, and deployment topology — at a level of detail where each component is a box whose internals are not yet specified.

DEFINITION

Definition — Low-level design (LLD). A description of the internals of one component: its modules, classes or functions, data structures, algorithms, database schemas, API request and response formats, error handling and state transitions — at a level of detail from which an engineer can write code.

First-principles explanation

A city planner decides where the roads, the water mains and the power substations go, and how much traffic and water each must carry. That is high-level design. An architect then draws the plumbing for one particular building: pipe diameters, which tap connects to which riser, where the shut-off valves are. That is low-level design. Both are design; they answer different questions at different zoom levels, and a good HLD makes the LLD of each part *possible* without dictating it.

Why does this exist?

Splitting design by zoom level lets the most consequential decisions (where data lives, how components communicate, how the system survives failures) be made and reviewed first, by people looking at the whole. It then lets each team design its component in detail independently, as long as it respects the interfaces the HLD fixed.

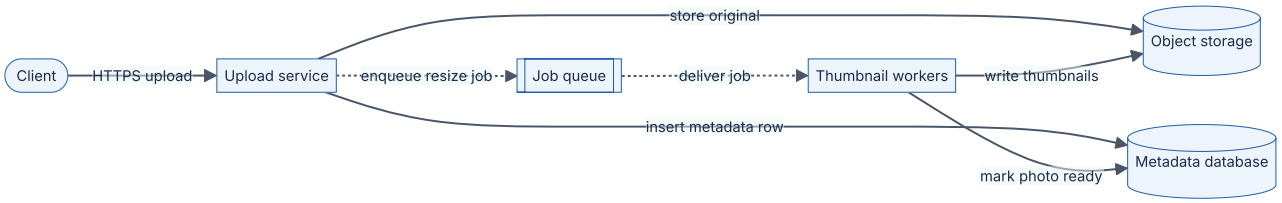
How it works internally

1. The HLD fixes **boundaries and contracts**: which component owns which data and exposes which API.
2. Each component's LLD fills in **internals**: classes, schemas, algorithms, concurrency.
3. LLD work frequently discovers problems the HLD missed (an operation that needs data from two owners; an API that cannot be made idempotent). These flow back to revise the HLD — design is a conversation between zoom levels, not a waterfall.

Architecture diagram

The HLD of PhotoShare's upload path at a later stage (detailed in §1.9) looks like this.

Figure 1.2 — High-level design of the PhotoShare upload path.

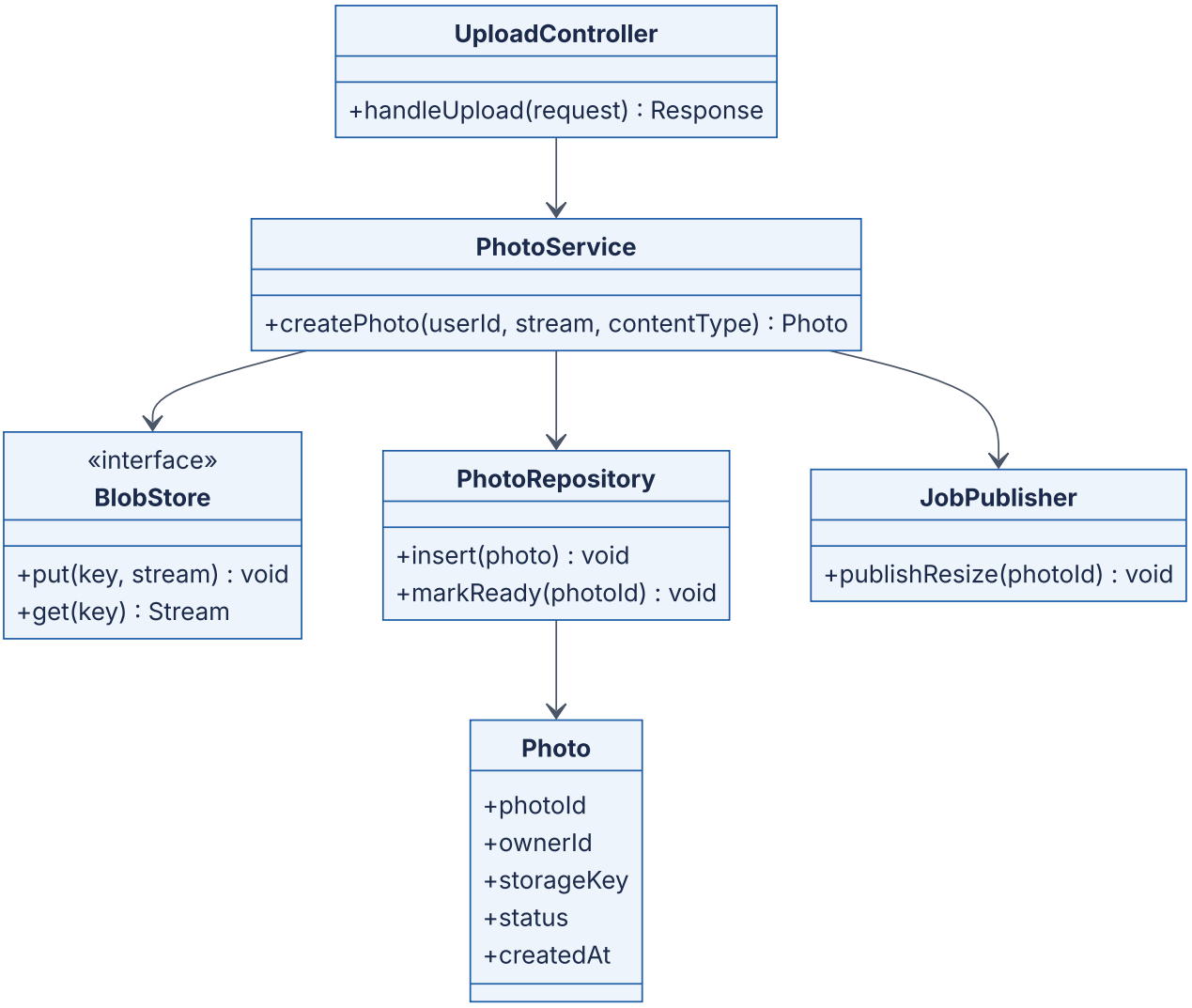


Reading the diagram.

- **Components.** The *upload service* accepts photos; *object storage* holds the image bytes; the *metadata database* holds rows describing each photo (owner, time, status, storage key); the *job queue* buffers work; *thumbnail workers* produce smaller versions of each photo.
- **Protocols.** HTTPS from the client; the object store is typically accessed over an HTTP API; the database over its wire protocol (SQL); the queue over the broker's protocol.
- **Direction and sync/async.** Solid arrows are synchronous: the client waits while the original is stored and the metadata row inserted. Dotted arrows are asynchronous: the resize job is enqueued and processed later, so the client gets a response without waiting for thumbnails.
- **Storage boundaries.** Two stores with different jobs: large immutable blobs in object storage, small mutable rows in the database.
- **Failure points.** If the queue is unavailable, uploads can still succeed but thumbnails are delayed; if workers crash, jobs remain in the queue and are retried. A crash *between* storing the original and inserting the row leaves an orphaned object — a consistency gap the LLD must handle (for example, by a periodic cleanup job).
- **Scaling options.** Upload service and workers scale horizontally by adding instances; the queue absorbs bursts so workers can be sized for average rather than peak load.

Notice what the HLD does *not* say: how the upload service validates file types, which class generates storage keys, or what the table columns are. That is LLD.

Figure 1.3 — Low-level design of the upload component (class view).



Reading the diagram.

- **Components.** `UploadController` handles HTTP concerns (parsing, authentication result, response codes). `PhotoService` holds the business rule "store bytes, record metadata, request thumbnails". `BlobStore` is an *interface*, so the same code can target local disk in tests and object storage in production. `PhotoRepository` wraps database access. `JobPublisher` wraps the queue. `Photo` is the data record.
- **Direction.** Arrows show dependency: the controller depends on the service, which depends on three abstractions. Nothing depends on the controller.
- **Sync/async and storage boundaries.** All calls in this diagram are synchronous in-process calls; the asynchrony from Figure 1.2 is hidden behind `JobPublisher`.
- **Failure points.** `createPhoto` performs three external operations that can each fail; the LLD must define ordering (store bytes first, then the row, then the job) and com-

pensation (a cleanup for orphaned objects). A `status` field (`PENDING` → `READY`) makes partial progress visible.

- **Scaling options.** Scaling is not decided here; this component is stateless, which is what allows the HLD to run many copies of it. Object-oriented principles behind this structure are covered in Ch 35 (Object-Oriented Design) and Ch 36 (SOLID Principles).

Alternatives and trade-offs

Table 1.4 — High-level design vs low-level design.

Aspect	High-level design	Low-level design
What it is	Components, data flows, deployment	Internals of one component
How it works	Boxes, arrows, contracts, capacity numbers	Classes, schemas, algorithms, state machines
Main concern	Quality attributes: scale, availability, latency, cost	Correctness, clarity, testability, extensibility
Typical diagrams	Flowchart/deployment, sequence across services	Class, sequence within a service, state, ER
Cost of a wrong decision	High — often needs data migration	Lower — usually a refactor within one team
Who reviews	Senior engineers across teams	The owning team
Interview format	“Design Instagram” (Ch 87–88, 91)	“Design a parking lot” (Ch 38)
Choose this zoom when	Deciding what to build and where it runs	Deciding how one part is written

Common misconceptions

COMMON MISTAKE

“HLD is for architects, LLD is for junior engineers.” Both require skill. A clumsy LLD (no idempotency, no error handling, a schema that cannot be indexed) can sink a sound HLD. Senior engineers move fluidly between levels — and recognizing *when* an LLD detail is actually an HLD concern (for example, a schema choice that blocks partitioning, see §14.7) is a hallmark of seniority.

COMMON MISTAKE

“Once the HLD is approved, it is fixed.” LLD routinely uncovers HLD flaws. Treat the HLD as a hypothesis that implementation tests.

1.6 Functional requirements

1.6.1 What functional requirements are and how to write them

> DEFINITION
>
> **Definition — Functional requirement (FR).** A statement of a behavior the system must provide: an input, action or event, and the observable result. Functional requirements describe *what* the system does — for example, "a signed-in user can upload a JPEG or PNG photo up to 20 MB, and it appears in their followers' feeds".

First-principles explanation

A restaurant's functional requirements are its menu and services: it serves pasta, takes reservations, offers takeaway. They say nothing about how fast the food arrives or how many guests fit — those are non-functional (§1.7). If you forget a functional requirement ("we also do delivery"), you build the wrong kitchen.

Why does this exist?

Every design decision is justified by some requirement. If you do not write the functional requirements down, two things happen: the team builds features nobody needs (and designs infrastructure for them), and it misses features that change the design fundamentally. "Users can delete their account and all their photos" sounds minor, but it forces you to be able to *find and erase every copy* of a user's data — in replicas, caches, backups, CDN edges and analytics pipelines — which is an architectural property, not a button.

How it works internally

1. **Identify actors.** Who interacts with the system? For PhotoShare: viewers, posters, moderators, advertisers, internal services.
2. **List use cases** per actor, as verb phrases: *upload photo*, *follow user*, *view feed*, *like photo*, *report photo*.
3. **Make each one testable**: specify inputs, limits and the observable outcome. "Upload photos" becomes "upload a JPEG/PNG/HEIC up to 20 MB; on success, return a photo ID within the latency target".
4. **Prioritize** — for example with **MoSCoW**: *Must*, *Should*, *Could*, *Won't (this time)*.
5. **Write the non-goals.** An explicit "out of scope" list prevents the design from silently growing.
6. **Sketch the API.** A rough interface for each must-have use case (detailed API design is §8.6) catches ambiguities early.

Table 1.5 — PhotoShare functional requirements (v1 scope).

ID	Requirement (testable form)	Priority
FR-1	A user can sign up and sign in with email and password	Must
FR-2	A signed-in user can upload a JPEG, PNG or HEIC photo up to 20 MB with an optional caption up to 2,200 characters	Must
FR-3	A user can follow and unfollow another user	Must
FR-4	A user can view a home feed of photos from followed users, newest first, 20 per page	Must
FR-5	A user can like a photo and see the like count	Should
FR-6	A user can delete a photo; it disappears from all feeds	Must
FR-7	A user can delete their account and all associated data	Must (legal)
FR-8	A user can comment on a photo	Could
—	Video, direct messages, stories, search, ads	Won't (v1 non-goals)

An API sketch for the must-haves might read:

```
POST   /v1/photos                (multipart: image, caption)  -> 201 {photoId}
DELETE /v1/photos/{photoId}                                   -> 204
PUT    /v1/users/{userId}/followers/me                        -> 204
DELETE /v1/users/{userId}/followers/me                        -> 204
GET    /v1/feed?cursor={cursor}&limit=20                      -> 200 {items,
nextCursor}
```

What to notice:

- The feed uses a **cursor** rather than a page number, because new photos arrive constantly and page numbers would skip or repeat items (pagination is detailed in §8.6).
- Follow is modeled as an idempotent `PUT`: repeating it has no further effect, which makes client retries safe (see §22.15 (Idempotency)).

Failure scenarios

Table 1.6 — Functional-requirement failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
A late "small" feature forces redesign	Requirement with architectural impact discovered late (e.g., account deletion, data export)	Feature estimate is months, not days	Retrofit data inventory and deletion pipeline	Ask about lifecycle, legal and admin use cases up front
Scope creep	No non-goals list	Design keeps growing; launch slips	Re-prioritize with MoSCoW	Write non-goals explicitly
Ambiguous requirement built two ways	"Feed" never defined (chronological? ranked?)	Teams' implementations disagree	Agree a definition; rework one side	Testable phrasing with examples

Common misconceptions

COMMON MISTAKE

"Functional requirements are the product manager's job, not the designer's." Product managers own *which* features matter; designers must interrogate each feature for architectural consequences. "Newest first" and "ranked by relevance" are one word apart in a requirements document and an entire ML system apart in architecture (see Ch 68 (Recommendation and Ranking Systems Architecture)).

1.7 Non-functional requirements (the "-ilities" catalog)

1.7.1 What non-functional requirements are

DEFINITION

Definition — Non-functional requirement (NFR). A requirement on *how well* the system performs its functions, or on constraints under which it operates, expressed as a measurable target — for example "p99 feed latency ≤ 300 ms at 2,000 RPS" or "monthly availability ≥ 99.9%". NFRs are also called **quality attributes** or, informally, the **"-ilities"** (scalability, availability, durability, …).

First-principles explanation

Two restaurants can serve the identical menu. One seats 20 and serves each dish in 40 minutes; the other seats 400 and serves each dish in 12 minutes, never runs out of bread, and passes every health inspection. Same functional requirements, completely different buildings, staff and suppliers. The difference is non-functional — and it is the non-functional requirements that determine the *architecture*.

Why does this exist?

Functional requirements can usually be met by a single program on a single machine. It is the non-functional requirements — serve 17,000 requests per second, never lose a photo, survive a data-center outage, cost less than $0.01 per user per month — that force systems to be distributed, replicated, cached and partitioned. Bass, Clements and Kazman make this the central thesis of software architecture: architecture is driven primarily by quality attributes, not by functionality [Bass et al. 2021]. Kleppmann similarly frames reliability, scalability and maintainability as the three central concerns when building data-intensive applications [Kleppmann 2017].

How it works internally

An NFR is only useful if it is **measurable**. Vague words — “fast”, “scalable”, “highly available” — cannot be tested and therefore cannot guide decisions. A reliable technique is the **quality attribute scenario** [Bass et al. 2021], which has six parts:

1. **Source** — who or what generates the stimulus (users, an attacker, a failing disk).
2. **Stimulus** — the event (a feed request, a node crash, a login attempt).
3. **Environment** — the conditions (normal load, peak load, during a deploy).
4. **Artifact** — the part of the system affected (feed service, metadata database).
5. **Response** — what the system does.
6. **Response measure** — the number that decides pass or fail.

Example: “**When** an availability zone fails (source/stimulus) **during** peak traffic (environment), the PhotoShare feed service (artifact) continues serving requests from the remaining zones (response) with **no more than 60 seconds** of elevated errors and **p99 latency ≤ 500 ms** after failover (response measure).”

In production, measurable NFRs become **service level indicators (SLIs)** — the measured quantity, such as the fraction of successful requests — and **service level objectives (SLOs)** — the target value for an SLI over a window [Beyer et al. 2016]. A **service level agreement (SLA)** is a contract with consequences (often financial) if an SLO-like target is missed. These are developed fully in §39.6 (SLI, SLO, SLA) and §39.7 (Error budgets).

> DEFINITION
>
> **Definition — Percentile latency (p50, p95, p99, p99.9).** The latency value below which the given percentage of requests fall. If p99 latency is 300 ms, 99% of requests completed in 300 ms or less and 1% took longer. p50 is the median.

Percentiles matter more than averages because users experience the slow requests, and a single page that fans out to many backend calls is slowed by the *slowest* of them [Dean

and Barroso 2013] (fan-out amplification is quantified in §4.8).

1.7.2 The catalog

The catalog is split into qualities observable while the system runs (Table 1.7) and qualities that matter as the system is built, changed and paid for (Table 1.8). Each row gives a measurable metric and a PhotoShare target.

Table 1.7 — Runtime quality attributes.

Attribute	What it means	Typical metric	PhotoShare example target	Deep dive
Latency (performance)	Time to complete one operation	p50/p99 in ms	Feed API p99 ≤ 300 ms	§4.8, Ch 23
Throughput	Work completed per unit time	RPS; MB/s	2,000 feed RPS at peak	Ch 23
Scalability	Ability to handle more load by adding resources	Cost or nodes per 1,000 RPS as load grows	10× users with ≤ 10× cost	Ch 18
Availability	Fraction of time the system serves correctly	% successful requests per 30 days	≥ 99.9% monthly	§4.14, Ch 39
Reliability	Probability of correct operation over time	Error rate; MTBF	< 0.1% failed uploads	Ch 39
Durability	Probability stored data is not lost	Annual object-loss probability	No acknowledged photo lost	§4.16, Ch 15
Consistency	Guarantees about what reads see after writes	Model name; staleness bound in s	Own uploads visible to self immediately; to followers within 5 s	Ch 25
Fault tolerance / resilience	Continued service despite component failure	Failures tolerated; recovery time	Survive loss of one zone	Ch 27, Ch 39
Recoverability (disaster recovery)	Ability to restore after disaster	RTO (time to restore); RPO (data loss window)	RTO 4 h; RPO 15 min	§39.8
Security	Protection of confidentiality, integrity, availability	Vulnerability count; time to patch; auth coverage	All traffic TLS; private photos never served to non-followers	Ch 10, Ch 50
Privacy	Appropriate handling of personal data	Deletion completed within N days; data minimized	Account deletion propagates in ≤ 30 days	Ch 50
Usability / accessibility	Ease and inclusiveness of use	Task success rate; accessibility conformance level	Screen-reader alt text on all photos	—
Elasticity	Speed of adapting capacity to load	Time to add capacity	Scale out in ≤ 5 min	§42.12

Table 1.8 — Development-time and business quality attributes.

Attribute	What it means	Typical metric	PhotoShare example target	Deep dive
Maintainability	Ease of understanding and fixing	Time to fix a bug; onboarding time	New engineer ships in week 1	Ch 35–36
Modifiability / extensibility	Ease of adding features	Number of modules touched per feature	Adding video touches ≤ 3 services	Ch 33
Testability	Ease of verifying behavior	Test coverage of critical paths; test run time	Full test suite ≤ 10 min	Ch 36
Deployability	Ease and safety of releasing	Deploy frequency; change-failure rate	Daily deploys; rollback ≤ 5 min	Ch 43
Observability	Ability to understand internal state from outputs	% of requests traced; time to diagnose	Every request has a trace ID	Ch 40
Operability	Ease of running in production	Pages per week; manual steps per release	≤ 2 pages per on-call week	Ch 39
Interoperability	Ability to work with other systems	Supported standards and API versions	Public API with versioning	§8.15
Portability	Ability to run in other environments	Effort to move cloud or region	Containerized services	Ch 41–42
Compliance	Conformance to laws and standards	Audit findings	EU users' data stored in the EU	Ch 50
Cost efficiency	Resources consumed per unit of value	$ per 1,000 requests; $ per user per month	≤ $0.01 per MAU per month	Ch 51
Sustainability	Energy and carbon per unit of work	kWh or gCO_2e per request	Track and report	Ch 51

The international quality-model standard ISO/IEC 25010 organizes many of these attributes into a formal hierarchy of characteristics and sub-characteristics; it is a useful checklist when you suspect you have forgotten a category [ISO/IEC 25010].

Mathematical foundations

Availability is the NFR most often stated as a number, and the number hides a surprisingly strict budget.

FORMULA

Availability and downtime budget

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \qquad D = (1 - A) \times T$$

Symbol	Meaning	Units
A	Availability: fraction of time the service is up (dimensionless, 0–1)	—
MTBF	Mean time between failures	hours
MTTR	Mean time to repair (restore service)	hours
D	Allowed downtime in the period	minutes (or hours)
T	Length of the period	minutes (or hours)

Table 1.9 — Downtime budgets for common availability targets.

Availability	Downtime per 30-day month	Downtime per 365-day year
99% (“two nines”)	7.2 h	3.65 days
99.9% (“three nines”)	43.2 min	8.76 h
99.95%	21.6 min	4.38 h
99.99% (“four nines”)	4.32 min	52.6 min
99.999% (“five nines”)	25.9 s	5.26 min

WORKED EXAMPLE

Worked example — What does 99.9% allow, and can PhotoShare v0 meet it?

1. Minutes in a 30-day month: $30 \times 24 \times 60 = 43,200$ min.
2. Allowed downtime at 99.9%: $(1 - 0.999) \times 43,200 = 0.001 \times 43,200 = 43.2$ min.
3. PhotoShare v0 (Figure 1.1) is deployed by stopping the application, copying new code and restarting: assume 3 minutes per deploy, and 5 deploys per week ≈ 21.7 deploys per month, giving $21.7 \times 3 \approx 65$ min of planned downtime.
4. Add one unplanned VM failure per quarter with an MTTR of 2 hours, averaging 40 min per month.
5. Total expected downtime $\approx 65 + 40 = 105$ min per month, so $A \approx 1 - 105/43,200 \approx 0.9976$, i.e. 99.76%.

Interpretation. v0 misses 99.9% by more than a factor of two, *before any traffic growth*.

Engineering implication. Meeting 99.9% requires removing deploy downtime (multiple instances behind a load balancer, rolling deploys — §43.4) and shortening MTTR (automatic failover). An availability target, not request volume, is what first forces PhotoShare off a single server.

When a request must pass through several components in sequence, *all* must be up, so availabilities multiply.

FORMULA

Availability of components in series (independent failures)

$$A_{\text{series}} = \prod_{i=1}^{n} A_i$$

Symbol	Meaning	Units
A_{series}	Availability of the whole request path	—
A_i	Availability of component i	—
n	Number of components the request depends on	count

WORKED EXAMPLE

Worked example — A three-tier request path.

A feed request passes through a load balancer ($A = 0.9999$), the feed service ($A = 0.9995$) and the database ($A = 0.9995$).

1. $A_{\text{series}} = 0.9999 \times 0.9995 \times 0.9995$.
2. $0.9995 \times 0.9995 = 0.99900025$.
3. $0.99900025 \times 0.9999 \approx 0.99890$.

Interpretation. About 99.89% — *below* 99.9%, even though every component individually exceeds it.

Engineering implication. Every synchronous dependency you add lowers availability. Designers counter this with redundancy (parallel components; $A_{\text{parallel}} = 1 - \prod(1 - A_i)$ under independence), caching, and making non-critical dependencies asynchronous or optional. The full arithmetic, including the danger of assuming independence, is in §4.14.

Code example

SLOs are increasingly written as configuration that monitoring systems evaluate automatically. The following is illustrative, not the syntax of any specific tool.

```
# illustrative SLO specification for the PhotoShare feed service
service: feed-api
slos:
  - name: feed-availability
    sli: "fraction of GET /v1/feed requests returning non-5xx"
    objective: 0.999          # 99.9 %
    window: 30d               # rolling 30-day window
  - name: feed-latency
    sli: "fraction of GET /v1/feed requests completing in <= 300 ms"
    objective: 0.99           # i.e. p99 <= 300 ms
    window: 30d
```

What to notice:

- A latency SLO is expressed as “the fraction of requests faster than a threshold”, which is equivalent to a percentile target and easier to aggregate across servers.
- The window matters: 99.9% over 30 days allows a single 43-minute outage; 99.9% over every 5-minute window allows almost none.

Real-world industry applications

INDUSTRY EXAMPLE

Google SRE and error budgets. Google's Site Reliability Engineering book argues that 100% is the wrong reliability target for almost everything: users cannot tell the difference beyond a point (their own networks and devices are less reliable), and each additional nine costs disproportionately more. Instead, teams agree an SLO, and the gap between the SLO and 100% becomes an **error budget** that can be "spent" on risky launches; when the budget is exhausted, releases slow down in favor of reliability work. *Evidence: Primary* — [Beyer et al. 2016]

1.7.3 NFRs pull against each other

Quality attributes conflict. Improving one usually costs another, and *naming* the conflict is the heart of design reasoning.

Table 1.10 — Common tensions between quality attributes.

Tension	Why they conflict	Typical resolution	Deep dive
Consistency vs availability (under network partition)	Replicas that cannot communicate must either refuse requests or risk divergent answers	Choose per operation: strong for payments, eventual for like counts	Ch 25
Latency vs consistency (even without partitions)	Coordinating replicas adds round trips	Read from nearby replica; accept bounded staleness	§25.9 (PACELC)
Durability vs write latency	Waiting for multiple disks or zones to confirm takes time	Synchronous replication to a quorum; async to remote region	Ch 27
Availability vs cost	Redundancy means paying for idle capacity	Match nines to business value	Ch 39, Ch 51
Security vs usability	Extra checks add friction	Risk-based authentication	Ch 10
Simplicity (operability) vs scalability	Each scaling technique adds components	Scale only the measured bottleneck	Ch 18
Performance vs modifiability	Optimized code is specialized and harder to change	Optimize hot paths only, after profiling	Ch 23

Failure scenarios

Table 1.11 — Non-functional-requirement failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
System is “fast” in tests, slow for users	Averages measured instead of tail percentiles	p99 dashboards; user complaints	Find tail causes (GC, slow dependencies)	Specify and test p99, not mean
Gold-plated availability	99.999% demanded without business case	Cost far above budget	Renegotiate SLO	Tie each nine to business impact
Data loss despite “replication”	Durability never specified; replicas asynchronous; backups untested	Restore fails during incident	Partial recovery from logs	State RPO; test restores regularly
Privacy breach via forgotten copy	Deletion requirement ignored caches, logs, backups	Audit or regulator finding	Purge pipelines	Data inventory; deletion as an NFR

Common misconceptions

COMMON MISTAKE

“The system must be highly available, scalable and fast.” This is not a requirement; it is a wish. Every system wants these. A requirement has a number, a percentile, a window and a load level: “99.9% of feed requests per 30 days return successfully; p99 ≤ 300 ms at 2,000 RPS”.

COMMON MISTAKE

“Availability and reliability are the same thing.” Availability is the fraction of time the system is *up*; reliability is the probability it operates *correctly* over a period. A system that is up but returns wrong feeds is available and unreliable. See §39.3.

COMMON MISTAKE

“Aim for 100%.” A 100% target is unachievable, makes every change frighteningly risky, and leaves no error budget for shipping features [Beyer et al. 2016].

1.8 Architectural decision-making and ADRs

1.8.1 Architectural decisions

DEFINITION

Definition — Architectural decision. A design choice that addresses an architecturally significant requirement — one that affects structure, quality attributes, interfaces or technology in a way that is costly to change later. Examples: choosing a relational database over a key-value store; storing files in object storage; making photo processing asynchronous.

DEFINITION

Definition — Architecture Decision Record (ADR). A short document that captures one architectural decision: its context, the decision itself, its status, and its consequences. ADRs are numbered, kept in sequence (often in the code repository), and never edited to reverse a decision — a new ADR *supersedes* the old one [Nygard 2011].

First-principles explanation

A court does not just announce “guilty” or “not guilty”; it publishes the reasoning, so later judges can tell whether a new case is similar enough for the ruling to apply. An ADR is a judgment with its reasoning attached. Two years from now, an engineer wondering “why on earth are thumbnails generated asynchronously?” can read the ADR, see that the context was “synchronous resizing pushed upload p99 to 4 seconds”, and decide whether that context still holds.

Why does this exist?

Without records, architectural knowledge lives in people’s heads and leaves when they do. Nygard described the resulting dilemma for a newcomer facing an unexplained decision: either *blindly accept* it (and keep something that may no longer make sense) or *blindly change* it (and possibly break a quality attribute the decision was protecting) [Nygard 2011]. ADRs make a third option possible: read the context, then decide.

How it works internally

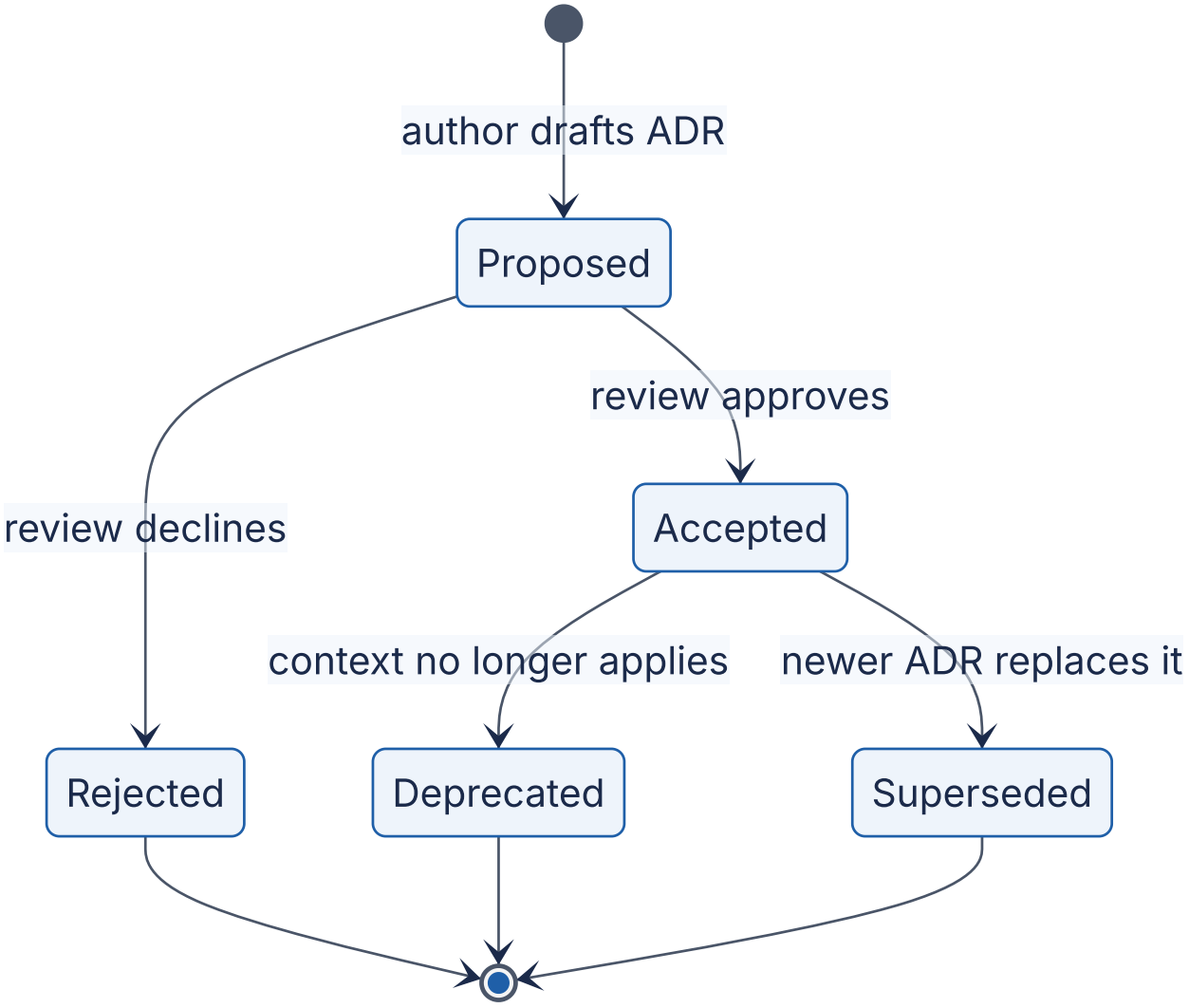
A disciplined decision process looks like this:

1. **Identify the driver.** Which requirement or problem forces a decision now? (For PhotoShare: “the local disk is 80% full and v1 must reach 99.9% availability.”)

2. **Assess reversibility.** Is this a cheap-to-reverse ("two-way door") or expensive-to-reverse ("one-way door") decision? Spend effort proportionally (developed in §2.8).
3. **Enumerate options** — at least two plus "do nothing".
4. **Choose criteria** from the NFRs (§1.7) and weight them.
5. **Evaluate** options against criteria — qualitatively, with a decision matrix, or with a prototype or benchmark when the numbers are uncertain.
6. **Decide and record** the decision as an ADR with status *Proposed*, circulate it, and move it to *Accepted* after review.
7. **Revisit** when the context changes, writing a new ADR that *supersedes* the old one.

Architecture diagram

Figure 1.4 — ADR lifecycle.



Reading the diagram.

- **Components (states).** Each box is the *status* field of an ADR. Nygard's original format lists proposed, accepted, deprecated and superseded; *Rejected* is a common ex-

tension that preserves the record of options the team declined.

- **Transitions.** An ADR starts as *Proposed* and becomes *Accepted* or *Rejected* through review. An accepted ADR is never edited to say the opposite; it becomes *Deprecated* (no longer relevant) or *Superseded* (with a link to the replacing ADR).
- **Direction and sync/async.** This is a state machine, not a data flow; all transitions are human decisions, made asynchronously over days.
- **Storage boundaries.** ADRs typically live as Markdown files in the same repository as the code they govern, so they are versioned and reviewed like code.
- **Failure points.** The common failure is an ADR stuck in *Proposed* forever, or an accepted ADR silently contradicted by code. Design reviews (§1.10) check for both.
- **Scaling options.** At organization scale, teams keep per-repository ADR logs plus a shared log for cross-cutting decisions.

Mathematical foundations

When several criteria matter, a **weighted decision matrix** makes the trade-off explicit and arguable.

FORMULA

Weighted decision score

$$S_j = \sum_{i=1}^{m} w_i \, s_{ij}, \qquad \sum_{i=1}^{m} w_i = 1$$

Symbol	Meaning	Units
S_j	Total score of option j	points (same scale as s_{ij})
w_i	Weight of criterion i (importance; sums to 1)	—
s_{ij}	Score of option j on criterion i (here 1 = poor, 5 = excellent)	points
m	Number of criteria	count

WORKED EXAMPLE

Worked example — Where should PhotoShare store photo files?

Options: (A) keep files on each application server's local disk; (B) a shared network file server (NFS); (C) managed object storage.

Table 1.12 — Decision matrix for photo storage.

Criterion	Weight	A: Local disk	B: NFS server	C: Object storage
Durability	0.30	1	3	5
Scalability	0.25	1	2	5
Operability	0.20	3	2	5
Cost	0.15	5	3	3
Latency	0.10	5	4	3

1. $S_A = 0.30(1) + 0.25(1) + 0.20(3) + 0.15(5) + 0.10(5) = 0.30 + 0.25 + 0.60 + 0.75 + 0.50 = 2.40$.
2. $S_B = 0.30(3) + 0.25(2) + 0.20(2) + 0.15(3) + 0.10(4) = 0.90 + 0.50 + 0.40 + 0.45 + 0.40 = 2.65$.
3. $S_C = 0.30(5) + 0.25(5) + 0.20(5) + 0.15(3) + 0.10(3) = 1.50 + 1.25 + 1.00 + 0.45 + 0.30 = 4.50$.

Interpretation. Object storage wins clearly. A *sensitivity check* matters more than the totals. Suppose a cost-conscious reviewer doubles cost's weight to 0.30 and halves durability's to 0.15:

4. $S_A = 0.15(1) + 0.25(1) + 0.20(3) + 0.30(5) + 0.10(5) = 0.15 + 0.25 + 0.60 + 1.50 + 0.50 = 3.00$.
5. $S_B = 0.15(3) + 0.25(2) + 0.20(2) + 0.30(3) + 0.10(4) = 0.45 + 0.50 + 0.40 + 0.90 + 0.40 = 2.65$.
6. $S_C = 0.15(5) + 0.25(5) + 0.20(5) + 0.30(3) + 0.10(3) = 0.75 + 1.25 + 1.00 + 0.90 + 0.30 = 4.20$.

C still wins by more than a point, so the ranking is robust to this disagreement.

Engineering implication. The matrix does not make the decision; it exposes *which weights* would have to change for the decision to flip, so that reviewers argue about priorities rather than conclusions. Scores of 1–5 are judgments; where a number can be measured (latency, cost per TB), measure it.

Code example

Here is the ADR that records the decision. ADRs are plain Markdown; the block below uses the `text` tag so its content is shown verbatim.

```
# ADR-0003: Store photo binaries in managed object storage

Status: Accepted (2026-03-14). Supersedes ADR-0001 "Store photos on local disk".
Deciders: PhotoShare backend team. Reviewed in design review DR-12.

## Context
- v0 stores photo files on the single VM's local disk (ADR-0001).
- The disk is 80% full; uploads add ~50 GB/day at 100,000 DAU (§1.9 estimate).
- v1 must meet 99.9% monthly availability, which requires at least two
  stateless application servers behind a load balancer. Files on one
  server's local disk are invisible to the other servers.
- There is no off-machine copy of photos; a disk failure loses all photos.

## Decision
Store every original and thumbnail in managed object storage under the key
photos/{photoId}/{variant}. Application servers keep no photo files locally.
Clients download images through the CDN, never directly from app servers.

## Options considered
A. Local disk per server - rejected: blocks horizontal scaling; no durability.
B. Shared NFS server - rejected: new single point of failure; we would
   operate replication and backups ourselves.
C. Managed object storage - chosen (decision matrix score 4.50 vs 2.65, 2.40).

## Consequences
+ Application servers become stateless and can be added or replaced freely.
+ Durability and capacity are the provider's responsibility.
- Upload path gains a network hop (~tens of ms to the object store).
- Storage cost becomes a per-GB monthly bill that grows with users; add a
  lifecycle policy to move old originals to a colder tier (see Ch 15).
- One-time migration of existing files (~2 TB); plan a dual-read period.
- Vendor-specific API; wrap it behind the BlobStore interface (Figure 1.3).
```

What to notice:

- **Context is factual and dated**; it includes the numbers that made the decision necessary. When those numbers change, the ADR tells you the decision may need revisiting.
- **Consequences list negatives**, not only benefits. An ADR with no downsides is advocacy, not a record.
- **Supersession** is explicit in both directions: ADR-0001 should be updated only to change its status to “Superseded by ADR-0003”.
- Nygard’s original format is Title, Status, Context, Decision, Consequences [Nygard 2011]; “Options considered” and “Deciders” are widely used extensions.

Real-world industry applications

INDUSTRY EXAMPLE

Lightweight ADRs in open-source and public-sector projects. After Nygard's 2011 post, the format spread widely; many public repositories contain an ADR numbered 0001 titled "Record architecture decisions", whose decision is simply to adopt ADRs for the project. The appeal is low cost: a page of Markdown reviewed in the same pull-request workflow as code. *Evidence: Secondary* — [Nygard 2011] and the many public repositories that adopt its format; the specific spread is an observation, not a measured statistic.

Advantages and disadvantages

Advantages: cheap to write; versioned with the code; preserves the *why*; speeds onboarding; prevents re-litigating settled questions; forces explicit consequences.
Disadvantages: only as good as team discipline; can become bureaucratic if required for trivial choices; a large ADR log needs an index to remain navigable.

Alternatives and trade-offs

ADRs complement rather than replace design documents (§1.10): a design doc explores a whole problem and may produce several decisions; each significant decision is then distilled into an ADR. Wikis are easier to browse but drift from the code. Commit messages capture *what* changed but rarely the alternatives rejected.

Failure scenarios

Table 1.13 — Decision-making failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Decision re-litigated every quarter	Rationale not recorded	Same debate in multiple meetings	Write the ADR retroactively	ADR for every one-way-door decision
Analysis paralysis	Treating a reversible choice as irreversible	Weeks of debate on a library choice	Time-box; pick and move on	Classify reversibility first (§2.8)
Decision by loudest voice	Criteria never made explicit	Dissenters disengage; later regret	Re-open with an explicit matrix	Agree criteria and weights before scoring
Stale ADR misleads	Context changed, ADR not superseded	Code contradicts accepted ADR	Write a superseding ADR	Review ADR log in design reviews

Common misconceptions

COMMON MISTAKE

“The decision matrix decides.” It structures judgment; it does not replace it. Weights encode values, and a team can make any option win by adjusting them. Its value is making those values visible.

1.9 Why system design matters (cost of wrong decisions)

1.9.1 Decisions differ in their cost of reversal

KEY CONCEPT

The asymmetry of design decisions. Most code-level decisions are cheap to reverse: rename a function, swap a library, redeploy. A few decisions are very expensive to reverse because *data* has been written in their shape: where files live, the primary key format, the partitioning key, the consistency model clients rely on, the public API contract. The job of system design is to spot the expensive few early and decide them deliberately, while keeping everything else cheap to change.

First-principles explanation

Moving furniture is easy; moving a load-bearing wall after the house is built is not. Data is software’s load-bearing wall: once billions of rows or petabytes of files exist in a particular shape, changing that shape means a migration that runs for days or months while the system stays live.

Why does this exist?

Bad design shows up as cost in four currencies: **money** (over-provisioned or emergency hardware), **time** (migrations that consume a team for a quarter), **user trust** (outages, data loss) and **opportunity** (features not built because engineers are fighting fires). Boehm and Basili summarized decades of industry data with the widely quoted observation that fixing a problem after delivery is often far more expensive than fixing it during requirements and design [Boehm and Basili 2001]; the exact multiplier varies greatly between studies and projects, but the direction is consistent.

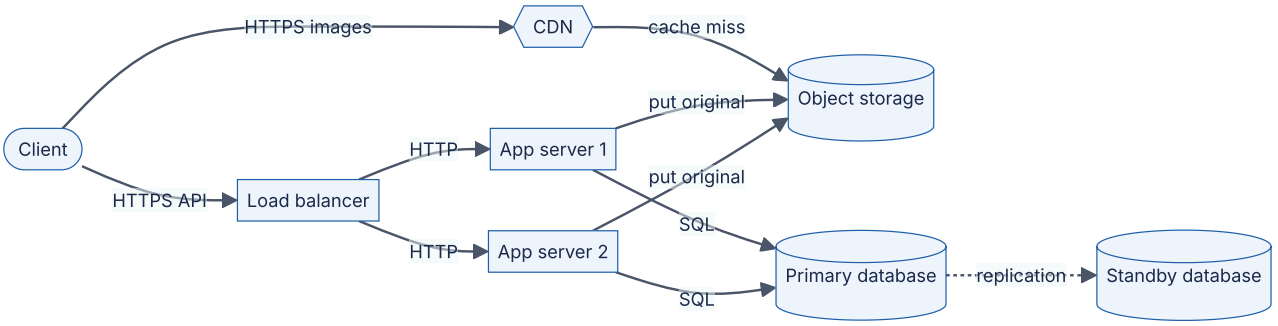
The opposite error — **over-engineering** — has the same currencies. Building a 50-service architecture for 1,000 users spends money and time on problems that do not yet exist, and makes the system harder to change when you learn what users actually want.

How it works internally — PhotoShare grows up

We now evolve PhotoShare from Figure 1.1 through three more stages. Each stage is triggered by a specific requirement that the previous design cannot meet. All estimates come from the script in §1.3.2 with the same assumptions (50 photo views and 0.2 uploads per user per day; 2.5 MB stored per upload; 200 KB served per view; peak factor 3), plus one more: each user loads the feed about 5 times per day, so feed API requests are one-tenth of photo views.

Stage 1 — 100,000 DAU: separate state from compute. Two requirements break v0. The 99.9% availability target cannot be met with deploy downtime (§1.7.2 worked example), and uploads now add $100,000 \times 0.2 \times 2.5$ MB = 50 GB per day to a single local disk. ADR-0003 (§1.8) moves files to object storage; a managed database with a standby replica replaces the local one; two stateless application servers sit behind a load balancer; images are served through a CDN.

Figure 1.5 — PhotoShare v1: stateless application tier and managed storage.



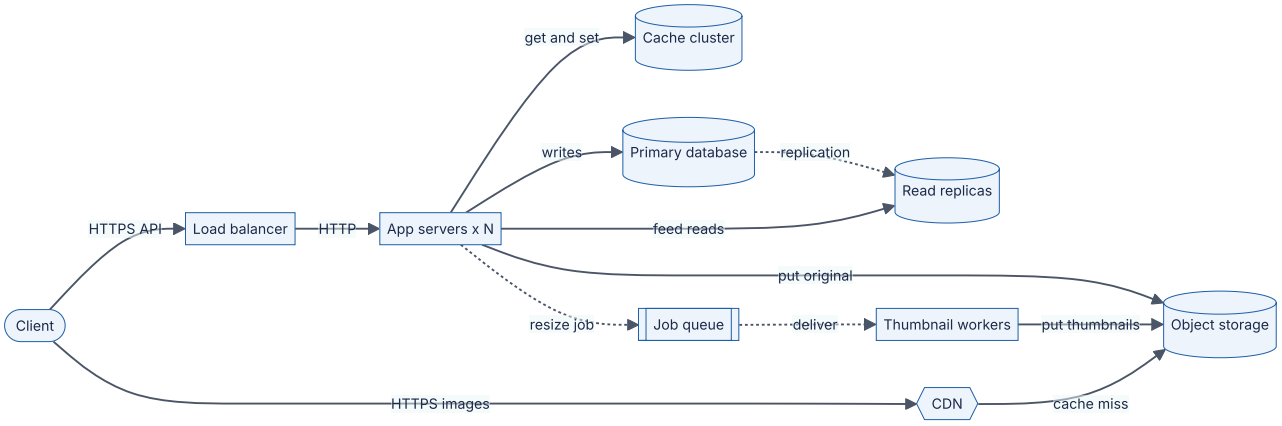
Reading the diagram.

- **Components.** A *load balancer* spreads API requests across two identical *app servers* (see Ch 19 (Load Balancing)). A *primary database* holds metadata, with a *standby* ready to take over. *Object storage* holds image bytes; a *CDN* (an external network of edge caches, drawn as a hexagon) serves images close to users (see Ch 21).
- **Protocols.** HTTPS from clients; plain HTTP inside the private network (or HTTPS, depending on the security model, §10.10); SQL to the database; the object store's HTTP API.
- **Direction and sync/async.** API calls and image fetches are synchronous. Replication from primary to standby is drawn dotted: it is commonly asynchronous, so the standby may lag slightly (replication modes are covered in §14.4).
- **Storage boundaries.** App servers are now **stateless**: they hold no data that cannot be lost. All state lives in the database and object storage. This is what allows any app server to serve any request.

- **Failure points.** Losing one app server halves capacity but keeps the service up. The primary database is still critical: failover to the standby takes time and, with asynchronous replication, may lose the last few seconds of writes.
- **Scaling options.** Add app servers horizontally; scale the database vertically for now.

Stage 2 — 1,000,000 DAU: protect the database. Feed API traffic is now $1,000,000 \times 5 / 86,400 \approx 58$ RPS average and about 174 RPS at peak — modest — but each feed request runs a join across follows and photos, and the database CPU is the bottleneck. Synchronous thumbnail generation pushes upload p99 above 3 seconds. Two additions fix this: a **cache** for feeds, sessions and hot metadata (Ch 20), plus **read replicas** for queries that tolerate slight staleness (§14.3); and an **asynchronous job queue** so uploads return as soon as the original is stored (Figure 1.2; Ch 30).

Figure 1.6 — PhotoShare v2: cache, read replicas and asynchronous processing.



Reading the diagram.

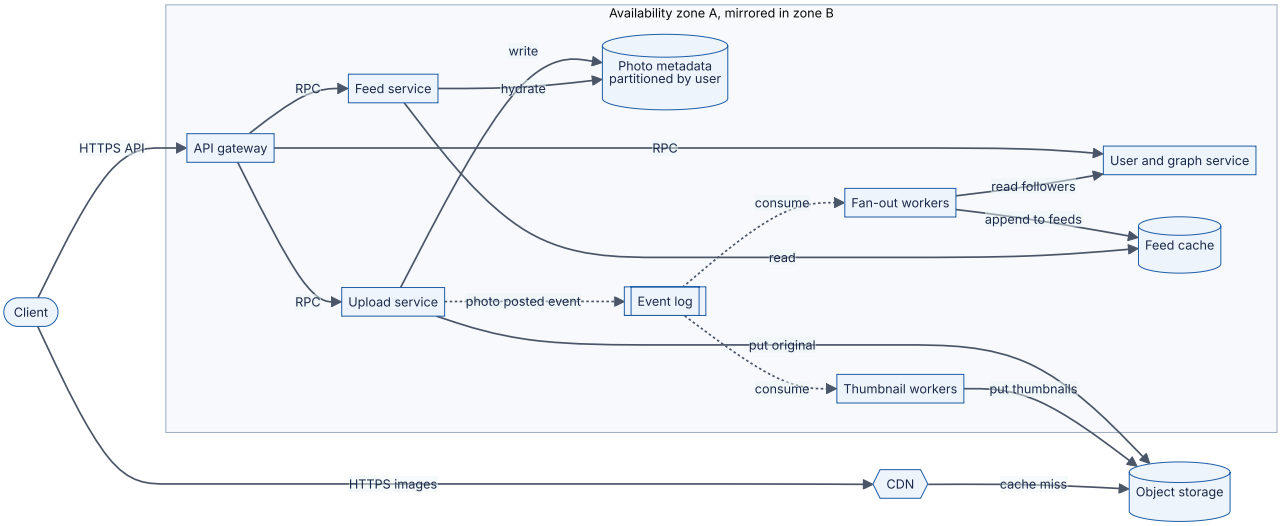
- **Components.** New: a *cache cluster* (in-memory key-value store), *read replicas* of the database, a *job queue* and *thumbnail workers*.
- **Protocols.** The cache uses its own lightweight protocol over TCP; the queue uses the broker's protocol.
- **Direction and sync/async.** Feed reads check the cache first (cache-aside, §20.7) and fall back to a replica on a miss. Uploads are synchronous only until the original is stored and the row written; resizing happens asynchronously (dotted arrows).
- **Storage boundaries.** Data now exists in up to four places — primary, replicas, cache, object storage — so **consistency** becomes a design question: a user who just uploaded may not see the photo if their next read hits a lagging replica. The consistency target in Table 1.7 ("own uploads visible to self immediately") is met by routing a user's reads of their own profile to the primary for a short time after a write (read-your-writes, §25.6).
- **Failure points.** A cache outage sends all reads to the replicas at once and can overload them (a *cache avalanche*, §20.13). A poisoned job that always crashes a worker

must be moved aside to a dead-letter queue (§30.11).

- **Scaling options.** App servers and workers scale horizontally; replicas scale reads; the primary still takes every write.

Stage 3 — 10,000,000 DAU: scale every tier and survive a zone failure. The estimator now gives about 5,800 average and 17,400 peak photo views per second, about 1,700 peak feed API RPS, 5 TB of new photo data per day (1.8 PB per year before replication) and about 9.3 Gbit/s of average image egress (28 Gbit/s at peak). The availability target rises to "survive loss of one availability zone". The design splits into a few services along clear data-ownership lines, partitions the metadata database by user ID (Ch 14), precomputes feeds asynchronously when a photo is posted (*fan-out on write*, examined in §53.3), and runs in at least two zones.

Figure 1.7 — PhotoShare v3: services, partitioned data and multi-zone deployment.



Reading the diagram.

- **Components.** An *API gateway* authenticates and routes requests (§34.6). The *upload*, *feed* and *user and graph* services each own their data. An *event log* (Ch 31) records "photo posted" events; *fan-out workers* insert the new photo ID into each follower's precomputed feed in the *feed cache*; *thumbnail workers* consume the same events. Photo metadata is **partitioned** by user ID across many database nodes.
- **Protocols.** HTTPS at the edge; internal RPC (for example gRPC, §8.8) between gateway and services; the event log's protocol for producers and consumers.
- **Direction and sync/async.** The read path (gateway → feed service → feed cache → metadata) is synchronous and short. The write path returns once the original and metadata are stored; everything else — thumbnails, fan-out — happens asynchronously from the event log, so a user with 100,000 followers does not make the upload slow.

- **Storage boundaries.** Object storage and the CDN are regional or global managed services outside the zone boundary; metadata, the event log and the feed cache are replicated across zones.
- **Failure points.** Loss of a zone: traffic shifts to the mirror zone, which must have spare capacity (≥ 50% headroom with two zones). Fan-out lag: followers see new photos seconds late — acceptable under the "within 5 s" consistency NFR, and visible as event-log consumer lag in monitoring (Ch 40). Celebrity accounts with tens of millions of followers make fan-out on write too expensive; a hybrid with *fan-out on read* for those accounts is the standard fix (§53.3).
- **Scaling options.** Every service is stateless and scales horizontally; metadata scales by adding partitions; the event log by adding partitions and consumers; the CDN absorbs image egress.

Table 1.14 — PhotoShare stages, triggers and costs.

Stage	DAU	Trigger (broken requirement)	Key changes	New cost or complexity
v0 (Fig. 1.1)	1,000	—	One VM	SPOF; no backups
v1 (Fig. 1.5)	100,000	99.9% availability; disk filling	LB, stateless app servers, managed DB plus standby, object storage, CDN	Network hops; managed-service bills; failover semantics
v2 (Fig. 1.6)	1,000,000	DB CPU saturated; upload p99 > 3 s	Cache, read replicas, job queue, workers	Stale reads; cache invalidation; queue operations
v3 (Fig. 1.7)	10,000,000	Zone-failure tolerance; write volume; 28 Gbit/s egress	Services, partitioned metadata, event log, precomputed feeds, multi-zone	Distributed-systems failure modes; more teams and on-call

Mathematical foundations

WORKED EXAMPLE

Worked example — The cost of one wrong early decision.

Suppose PhotoShare had *not* written ADR-0003 and kept files on application-server disks until 1,000,000 DAU, by which time roughly 100 days at 0.5 TB/day had accumulated about 50 TB.

1. Data to move: $50\ \text{TB} = 50 \times 10^{12}$ bytes $= 4 \times 10^{14}$ bits.
2. Available copy bandwidth, without starving production traffic: assume 1 Gbit/s $= 10^{9}$ bit/s.
3. Copy time: $4 \times 10^{14} / 10^{9} = 4 \times 10^{5}$ s ≈ 111 h ≈ 4.6 days of continuous copying.
4. Meanwhile 0.5 TB/day of new uploads keeps arriving, so the migration also needs dual-writes, verification checksums, a cut-over plan and a rollback plan — realistically several engineer-weeks, plus risk of data loss.

Interpretation. Making the decision at v1 with 2 TB took a few hours of copying; deferring it made it 25× larger and turned it into a project.

Engineering implication. Decisions about *where data lives* should be made early and on purpose, even when the system is small. Decisions about *how much compute you run* can safely wait.

Real-world industry applications

INDUSTRY EXAMPLE

Knight Capital (2012). On 1 August 2012, Knight Capital deployed new trading code to its order-routing servers using a manual process; one of eight servers did not receive the new code, and a repurposed configuration flag activated long-unused legacy logic on that server. In about 45 minutes the firm executed millions of unintended trades and lost more than $460 million. The U.S. Securities and Exchange Commission's subsequent order cites, among other failures, the lack of adequate technical controls and written procedures for deployment. The lesson for designers: deployability and operability (Table 1.8) are architectural qualities, not afterthoughts — dead code, reused flags and manual deploys are design decisions with consequences. *Evidence: Primary* — [SEC 2013]

INDUSTRY EXAMPLE

A counter that outgrew its integer. In December 2014 YouTube reported that the view count of the music video "Gangnam Style" had exceeded the maximum value of a signed 32-bit integer (2,147,483,647), and that the counter had been upgraded to a 64-bit integer. Choosing the width of an identifier or counter is a tiny decision on day one and a data migration years later. *Evidence: Secondary* — widely reported at the time; primary source to be confirmed (see §1.19).

Failure scenarios

Table 1.15 — Costly design failure patterns.

What goes wrong	Why	Detection	Recovery	Prevention
Stateful app servers block scaling	Files or sessions stored locally	New servers serve 404s or log users out	Move state to shared stores; migration	Stateless services by default (§18.5)
Identifier overflow	32-bit IDs or counters	Inserts fail at about 2.1 billion	Emergency schema migration	64-bit or globally unique IDs (§17.10)
Premature microservices	Splitting before boundaries are understood	Every feature touches many services	Merge into a modular monolith	Start with a modular monolith (§33.5)
Unplanned celebrity load	Design assumed uniform users	Hot partitions; fan-out lag	Special-case heavy users	Ask about skew in requirements (§14.10)

Common misconceptions

COMMON MISTAKE

"We can always fix the architecture later." You can always fix *code* later. Fixing data shape, storage location and public contracts later costs migrations that grow with the data. Decide those deliberately; defer the rest.

1.10 How system design is practiced in industry

1.10.1 Design documents

DEFINITION

Definition — Design document (design doc). A written proposal, produced before significant implementation, that states a problem, its goals and non-goals, a proposed design, the alternatives considered, and cross-cutting concerns (security, privacy, operations, cost), so that others can review it while changes are still cheap.

First-principles explanation

Before a building is constructed, its drawings are circulated to the structural engineer, the fire inspector and the client. Each reads for different risks, and each objection costs an eraser rather than a demolition crew. A design doc is the software equivalent: the cheapest place to find a design flaw is in a document.

How it works internally

Practice varies, but a widely cited public description of Google's design-doc culture describes informal documents, written in a collaborative editor, typically containing [Ubl 2020]:

1. **Context and scope** — the landscape the system lives in and what is being built.
2. **Goals and non-goals** — including explicit things the system will *not* do.
3. **The actual design** — often with a system-context diagram, APIs and data storage, emphasizing trade-offs rather than implementation detail.
4. **Alternatives considered** — and why they were rejected.
5. **Cross-cutting concerns** — security, privacy, observability and similar.

The same source describes larger design docs of roughly 10–20 pages and shorter "mini design docs" for smaller changes. *Software Engineering at Google* likewise treats design docs as a standard documentation type [Winters et al. 2020]. Amazon is publicly known for narrative documents instead of slide decks, including the "PR/FAQ" (a mock press release and FAQ written before building a product) described by two former Amazon executives [Bryar and Carr 2021].

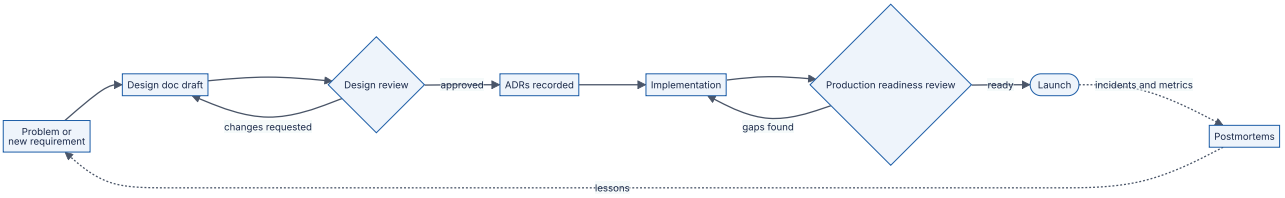
1.10.2 Design reviews and production readiness

A **design review** is a meeting or asynchronous comment thread in which experienced engineers outside the authoring team challenge a design doc. Reviewers typically probe: Are the requirements and numbers right? What are the single points of failure? What hap-

pens under 10× load? How is it deployed and rolled back? How is it monitored? What does it cost? Is there a simpler design?

Close to launch, many organizations run a **production readiness review (PRR)**: a checklist-based assessment that a service is operable — monitoring, alerting, capacity, runbooks, on-call — before it receives production traffic. Google's SRE book describes PRRs as part of how SRE teams engage with services [Beyer et al. 2016]. After incidents, **blameless postmortems** (§39.12) feed lessons back into future designs.

Figure 1.8 — The life of a design in a typical engineering organization.



Reading the diagram.

- **Components.** Activities (rectangles), gates where a decision is made (diamonds) and the launch milestone (stadium).
- **Direction.** Work flows left to right from problem to launch, with two loops back: review feedback to the doc, and readiness gaps to implementation.
- **Sync/async.** Solid arrows are the planned path. Dotted arrows are the slower, asynchronous feedback from production — incidents and metrics — that eventually create new problems to design for.
- **Storage boundaries.** The durable artifacts are the design doc, the ADRs and the postmortems; together they form the organization's architectural memory.
- **Failure points.** Skipping the review gate (designs built unreviewed), a review that becomes a rubber stamp, or postmortem lessons that never feed back into designs.
- **Scaling options.** Large organizations scale review capacity with tiered reviews: lightweight team review for reversible changes, cross-org architecture review for one-way-door decisions.

1.10.3 RFC processes

The term **RFC** ("Request for Comments") comes from the Internet's standards process; RFC 1, "Host Software", was written by Steve Crocker in 1969 [Crocker 1969], and the IETF's current standards process is itself defined in an RFC [Bradner 1996]. Many open-source projects run public proposal processes in the same spirit: Python Enhancement Proposals (PEPs) [Python PEP 1], Rust RFCs [Rust RFCs] and Kubernetes Enhancement Proposals (KEPs) [Kubernetes KEPs]. Inside companies, "RFC" usually means a design doc circulated widely for comment before a decision, often across teams.

Table 1.16 — Design artifacts compared.

Aspect	Design doc	ADR	RFC (company or open source)	PRR
What it is	Proposal for a whole design	Record of one decision	Proposal circulated for broad comment	Operability checklist
When	Before implementation	At decision time	Before cross-team or public change	Before launch
Length	1–20 pages	~1 page	2–20 pages	Checklist
Audience	Team plus reviewers	Future maintainers	Whole organization or community	Operators and SRE
Output	Approved design	Durable rationale	Consensus decision	Launch go/no-go
Choose when	Non-trivial new system or change	Any one-way-door decision	Change affects many teams or users	New service or major change

Real-world industry applications

INDUSTRY EXAMPLE

Open-source RFCs as public design reviews. In the Rust and Python projects, any significant language change is proposed as a document in a public repository, discussed in the open, and accepted or rejected by a designated team, leaving a permanent record of the rationale and alternatives. They are among the best freely available corpora of real design reasoning — and good reading practice (see §3.5). *Evidence: Primary* — [Python PEP 1]; [Rust RFCs]

Failure scenarios

Table 1.17 — Design-process failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Review as rubber stamp	Reviewers lack time or context	Incidents in "reviewed" systems	Postmortem identifies missed risk	Named reviewers with explicit checklists
Design by committee	Every reviewer's preference absorbed	Bloated design; slipped dates	Author re-asserts goals and non-goals	Clear decision owner
Doc written after the fact	Process seen as paperwork	Doc describes code, lists no alternatives	Treat as documentation, not review	Review before implementation starts

1.11 How system design interviews work

1.11.1 Format

DEFINITION

Definition — System design interview. A 45–60-minute interview in which a candidate designs a system from an open-ended prompt (for example, “design a photo-sharing service”) in conversation with an interviewer, who evaluates the candidate’s process, technical judgment and communication rather than a single correct answer.

First-principles explanation

The interview is a compressed design review in which you are the author and the interviewer is a friendly but probing reviewer. There is no single correct answer because real design has none; what is being assessed is whether you would be a good person to have in a real design review.

How it works internally

Formats differ between companies and levels (Ch 90 separates verified public information from general advice), but most follow a shape like Figure 1.9. Popular preparation guides describe a similar sequence — understand the problem and scope, propose a high-level design, dive deep, wrap up [Xu 2020].

Figure 1.9 — A typical 45-minute system design interview.



Reading the diagram.

- **Components.** Six phases, each with a suggested time budget for a 45-minute slot.
- **Direction.** Left to right. The dotted arrow shows that deep dives often send you back to revise the high-level design — that is expected, not a failure.
- **What each phase produces.** A short list of FRs and numeric NFRs; a few key numbers (peak RPS, storage per year); a handful of API calls and core tables; a diagram like Figure 1.6 or 1.7; a deep analysis of one or two components (feed generation, partitioning, consistency); a summary of bottlenecks, failure modes and what you would do next.
- **Failure points.** Spending 20 minutes on requirements, or jumping to the diagram with no requirements at all; going silent; designing for scale nobody asked for.
- **Scaling to longer formats.** In a 60-minute slot, the extra time usually goes to deep dives.

What interviewers evaluate

Table 1.18 — Interview signals: weak vs strong.

Signal	Weak	Strong
Requirements	Starts drawing immediately	Asks targeted questions; states FRs, numeric NFRs and non-goals
Estimation	Skips, or computes to six digits	Order-of-magnitude numbers that drive decisions
Design	Lists technologies by name	Explains what each component is for and why it is needed
Trade-offs	"X is the best"	"X gives A at the cost of B; given our NFRs, that's acceptable"
Depth	Shallow everywhere	Deep on the hardest part of *this* problem
Failure thinking	Assumes nothing breaks	Names SPOFs, failure modes and recovery
Communication	Monologue or silence	Thinks aloud, checks in, adapts to hints

Expectations rise with level: a junior candidate is expected to produce a working design with guidance; a senior candidate to drive the conversation and identify the hard parts unprompted; a staff-level candidate to reason about evolution, organizational and cost concerns, and multiple viable designs (rubrics by level are in Ch 88).

> INTERVIEW TIP
>
> **Interview tip — the first two minutes.** For "design PhotoShare", a strong opening sounds like: *"Let me confirm scope: upload, follow, and a home feed of followed users' photos — chronological, not ranked? I'll leave out video, search and messaging. For scale, shall I assume around 10 million daily users, read-heavy — maybe 50 views per upload? And for quality targets: feed p99 under a few hundred milliseconds, uploads must never be lost, and a few seconds of delay before followers see a new photo is fine?"* In thirty seconds you have stated FRs, non-goals, scale, read/write ratio, latency, durability and consistency — and given the interviewer a chance to redirect.

Variants include **low-level/object-oriented design** interviews (Ch 38), **ML system design** interviews (Ch 92), and product-flavored versus infrastructure-flavored prompts. Full preparation is in Book III (Ch 87–92).

1.12 Common beginner mistakes

Most beginner mistakes are versions of one error: *designing without stating requirements and numbers*. Table 1.19 lists the twelve most common.

Table 1.19 — Common beginner mistakes and their fixes.

#	Mistake	Symptom	Fix	See
1	Jumping to a solution	Diagram before requirements	Spend the first minutes on FRs and NFRs	§1.6–1.7
2	Vague NFRs	“Fast and scalable”	Numbers, percentiles, windows	§1.7
3	No estimation	Arbitrary component counts	Back-of-the-envelope RPS, storage, bandwidth	§1.3.2, Ch 89
4	Over-engineering	Microservices, Kafka and sharding for 1,000 users	Design for ~10× current load; evolve	§1.9
5	Buzzword architecture	Technologies named, purpose unexplained	Justify every box by a requirement	§1.3
6	Ignoring failure	No answer to “what if this node dies?”	Name SPOFs; add redundancy where the NFR demands it	Ch 39
7	Averages instead of tails	p99 never mentioned	Specify and reason in percentiles	§1.7.1, §4.8
8	Stateful app servers	Sessions or files on local disk	Push state into shared stores	§18.5
9	One database for everything, forever	DB is every bottleneck	Cache, replicas, partitioning — when measured	Ch 14, Ch 20
10	Silent trade-offs	Only advantages stated	Always name the cost of each choice	§1.7.3
11	Forgetting the data lifecycle	No deletion, retention or backup plan	Treat deletion, backup and retention as requirements	§1.6, §39.9
12	Forgetting operations	No monitoring, deploy or rollback story	Include observability and deployability	Ch 40, Ch 43

COMMON MISTAKE

The two-sided trap. Mistakes 4 and 9 are opposites, and beginners often swing from one to the other. The cure for both is the same: let *measured or estimated* load and *stated* requirements decide when each component is added. Figure 1.1 was right for v0; Figure 1.7 was right for v3; neither is right for the other.

RECOMMENDATION

A five-question self-check before any design discussion. (1) What must it do? (2) How well — in numbers? (3) How big — in RPS, bytes and growth? (4) What fails, and what happens then? (5) What did I give up, and why is that acceptable?

1.13 Summary and revision notes

SUMMARY

- **System design** is deciding components, responsibilities, interfaces, data flow, storage and deployment so a system meets functional *and* non-functional requirements under stated load and failure assumptions. A design is only good relative to its requirements.
- **Software architecture** is the set of significant, hard-to-change structures of a system; system design is its large-scale, runtime-focused part. Describe architecture with multiple views (4+1, C4).
- **HLD** fixes components, boundaries and contracts; **LLD** fixes the internals of one component. Design moves between the two levels.
- **Functional requirements** say what the system does; write them testably, prioritize them, and list non-goals. Some “small” FRs (account deletion) have large architectural consequences.
- **Non-functional requirements** say how well; they drive the architecture. Make them measurable (number, percentile, window, load) using quality attribute scenarios, SLIs and SLOs.
- 99.9% availability allows 43.2 minutes of downtime per 30 days; components in series multiply availabilities, so each synchronous dependency lowers the total.
- Quality attributes conflict (consistency vs availability, durability vs latency, availability vs cost). Name the tension and choose per operation.
- Record one-way-door decisions as **ADRs** (title, status, context, decision, consequences); supersede, never rewrite. Use weighted matrices to make priorities explicit.
- Decisions about **where data lives and in what shape** are expensive to reverse; decisions about compute quantity are cheap. Avoid both under- and over-engineering.
- Industry practice: **design docs** → **design review** → ADRs → implementation → **production readiness review** → launch → **postmortems**; RFC processes broaden review.
- Interviews compress a design review into 45–60 minutes: requirements, estimation, API and data, high-level design, deep dives, trade-offs.

1.14 Knowledge checkpoint

1. In one sentence each, define *system design* and *software architecture*.
2. Why is “a single server” a good design for PhotoShare v0 but a bad one for v3?
3. Give two things that belong in an HLD and two that belong in an LLD.
4. Rewrite “uploads should be fast” as a measurable NFR.

5. What are the six parts of a quality attribute scenario?
6. How much downtime does 99.95% availability allow in a 30-day month?
7. Three components with availabilities 0.999, 0.999 and 0.9999 are in series. What is the path’s availability?
8. Distinguish availability from reliability with an example.
9. Distinguish an SLI, an SLO and an SLA.
10. What are the five sections of Nygard’s ADR format?
11. Why is an accepted ADR superseded rather than edited?
12. A product has 2 million DAU, each making 30 API requests per day, with peak factor 3. Estimate average and peak RPS.
13. Which PhotoShare requirement first forced the move off a single server — load or availability? Why?
14. Name the five typical sections of a design doc as described in public accounts of Google’s practice.
15. Name four of the twelve beginner mistakes in Table 1.19 and the fix for each.

Answers

1. System design: choosing components, interfaces, data flow, storage and deployment so a system meets its functional and non-functional requirements under stated load and failure. Software architecture: the set of structures needed to reason about a system — its elements, their relations and the properties of both [Bass et al. 2021].
2. At 1,000 DAU the peak load is about 1.7 RPS and simplicity dominates. At 10 million DAU the load (about 17,400 photo views per second at peak, 5 TB/day of new data), the zone-failure requirement and the 99.9% availability target all exceed what one machine — itself a single point of failure — can deliver.
3. HLD: the list of services and data stores; the protocol and sync/async nature of each interaction (also deployment topology, data ownership). LLD: class structure and interfaces; table schemas and indexes (also algorithms, error handling, state machines).
4. For example: “For photos up to 20 MB under normal load at up to 70 uploads per second, 99% of uploads return a photo ID within 2 s, measured server-side over a 30-day window.”
5. Source, stimulus, environment, artifact, response, response measure.
6. $(1 - 0.9995) \times 43,200 = 21.6$ minutes.
7. $0.999 \times 0.999 \times 0.9999 = 0.998001 \times 0.9999 \approx 0.99790$, about 99.79%.
8. Availability is the fraction of time the system is up and serving; reliability is the probability it operates correctly over a period. A feed service that responds quickly but sometimes shows other users’ private photos is available but unreliable (and insecure).
9. SLI: the measured quantity (fraction of successful requests). SLO: the target for that SLI over a window (≥ 99.9% per 30 days). SLA: a contract with consequences if a stated target is missed.

10. Title, Status, Context, Decision, Consequences.
11. So that the historical reasoning remains intact: future readers can see what was decided, under what context, and what replaced it. Editing would erase the record of why the earlier choice was once right.
12. Requests/day $= 2 \times 10^6 \times 30 = 6 \times 10^7$; average $= 6 \times 10^7 / 86,400 \approx 694$ RPS; peak $\approx 3 \times 694 \approx 2,083$ RPS.
13. Availability. At 100,000 DAU the load was still small, but deploy downtime plus occasional VM failures gave roughly 99.76%, below the 99.9% target; only multiple stateless servers behind a load balancer remove deploy downtime and the single point of failure.
14. Context and scope; goals and non-goals; the actual design; alternatives considered; cross-cutting concerns [Ubl 2020].
15. Any four, e.g. jumping to a solution → clarify FRs and NFRs first; vague NFRs → add numbers, percentiles and windows; over-engineering → design for about 10× current load and evolve; ignoring failure → name SPOFs and the recovery path.

1.15 Interview questions

Basic

Q1. What is the difference between functional and non-functional requirements? Give an example of each for a URL shortener.

Answer. Functional requirements describe behavior — what the system does: “given a long URL, return a short code; visiting the short URL redirects to the long one”. Non-functional requirements describe quality — how well, under what constraints, with numbers: “redirects complete with p99 ≤ 50 ms server-side at 10,000 RPS; the service is available 99.95% per month; a created mapping is never lost”. The NFRs drive the design: the latency and read volume suggest caching (Ch 20), and the durability requirement suggests a replicated store (Ch 27). The full design is in §52.1.

Q2. What is a single point of failure, and how would you find them in a design?

Answer. A component whose failure alone makes the whole system (or a critical function) unavailable. Trace each critical request path end to end and, for every component on it, ask “if this one instance disappears now, what happens?”. Typical SPOFs are a lone database primary, a single load balancer, one availability zone, a shared configuration service, and a person who is the only one able to deploy. Removing a SPOF usually means redundancy plus automatic failover, but each removal costs money and complexity, so remove the ones the availability NFR requires.

Intermediate

Q3. Your product manager asks for “five nines”. How do you respond?

Answer. First translate: 99.999% allows about 26 seconds of downtime per 30-day month — less than a typical database failover and less than most deploy rollbacks. Then ask what the business impact of downtime actually is and which user journeys need the target (checkout, perhaps; profile editing, probably not). Explain the cost: multi-zone or multi-region active–active deployment, automated failover, rigorous change management, and every synchronous dependency must itself exceed five nines because availabilities multiply in series. Propose per-journey SLOs (for example 99.95% for checkout, 99.9% elsewhere) with an error budget, and revisit with data [Beyer et al. 2016].

Q4. When would you introduce a cache, and what new problems does it bring?

Answer. When measurements or estimates show a read-heavy workload where a backing store is the bottleneck or too slow for the latency NFR, and where data can tolerate some staleness or be invalidated reliably — as at PhotoShare v2 (Figure 1.6). New problems: stale reads and invalidation (§20.10); cold-start and stampedes when popular keys expire together (§20.11); a cache outage overwhelming the database (§20.13); memory cost; and a new component to operate. State the expected hit ratio and what happens on a miss and on cache failure.

Advanced

Q5. Walk through how you would evolve a single-server photo-sharing app to 10 million DAU. What triggers each change?

Answer. Estimate first: at 10 million DAU with 50 views and 0.2 uploads per user per day, about 5,800 average and 17,400 peak photo views per second, about 1,700 peak feed API RPS, 5 TB/day of new data and about 28 Gbit/s peak egress. Then evolve by trigger, not by fashion. (1) Availability target and disk growth → stateless app servers behind a load balancer, managed database with standby, object storage and CDN; data location is decided early because it is expensive to change later. (2) Database CPU and upload latency → cache with cache-aside, read replicas with read-your-writes routing, async thumbnail jobs. (3) Zone-failure tolerance and write volume → services split by data ownership, metadata partitioned by user ID, an event log driving fan-out-on-write feed precomputation with a fan-out-on-read path for celebrity accounts, and multi-zone deployment with capacity headroom. At each stage, name the new failure modes (stale reads, cache avalanche, consumer lag, hot partitions) and how you would observe them.

Q6. Two senior engineers disagree about whether to adopt a new distributed database. How would you drive the decision?

Answer. Classify reversibility first: a primary datastore holding production data is a one-way door, so it deserves a design doc and review. Agree the decision drivers (specific NFRs with numbers), criteria and weights *before* scoring options, including "do nothing" and an incremental option (for example partitioning the existing database). Replace opinion with evidence where possible: a time-boxed prototype under representative load, failure-injection tests, and an estimate of migration and operational cost (team skills, on-call burden). Score with a weighted matrix, run a sensitivity check to see which weight changes would flip the result, and record the outcome as an ADR with explicit negative consequences and the conditions under which it should be revisited. The goal is a decision both engineers can commit to even if one disagrees.

1.16 Practical exercises

1. **Run and extend the estimator.** Run the §1.3.2 script. Add fields for feed API requests (5 per user per day) and replication factor (3), and print API RPS and replicated storage per year for 1 million and 10 million DAU.
2. **Write NFRs.** Pick an app you use daily. Write five measurable NFRs, one of them as a full six-part quality attribute scenario.
3. **Availability arithmetic.** A request path has a CDN (99.99%), API gateway (99.95%), service (99.9%) and database (99.95%). Compute the path availability and the monthly downtime it implies. Which component would you make redundant first, and why?
4. **Write an ADR.** Write ADR-0004 for PhotoShare: "Generate thumbnails asynchronously". Include context with numbers, at least two options considered, and at least two negative consequences.
5. **Read a real design record.** Choose one accepted Python PEP or Rust RFC. Identify its context, decision, alternatives and consequences, and compare its structure with Nygard's ADR format.

1.17 Design challenges

1. **PhotoShare goes private.** Add the requirement "accounts can be private; only approved followers can see their photos". Which components in Figure 1.7 change? How do you stop the CDN from serving private photos to anyone with the URL? (Hint: signed URLs with short TTLs; see Ch 21.)
2. **PhotoShare in two regions.** EU regulators require EU users' photos and metadata to stay in the EU. Sketch the changes to Figure 1.7, list the new failure modes, and state which NFR tensions from Table 1.10 become sharper. (See Ch 44.)
3. **Cost ceiling.** The business caps infrastructure cost at $0.01 per monthly active user per month. Using the v3 numbers, identify the two largest cost drivers (con-

sider storage growth and egress) and propose design changes — storage tiers, image formats, CDN cache hit ratio — with estimated savings. (See Ch 51.)

1.18 References

1. Barroso, L. A., Hölzle, U., Ranganathan, P. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd ed. Morgan & Claypool, 2018.
2. Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
3. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. https://sre.google/sre-book/table-of-contents/
4. Boehm, B., Basili, V. R. "Software Defect Reduction Top 10 List." *IEEE Computer*, 34(1), 2001.
5. Bradner, S. *The Internet Standards Process — Revision 3*. RFC 2026, IETF, 1996. https://www.rfc-editor.org/rfc/rfc2026
6. Bryar, C., Carr, B. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021.
7. Conway, M. E. "How Do Committees Invent?" *Datamation*, 14(4), 1968.
8. Crocker, S. *Host Software*. RFC 1, 1969. https://www.rfc-editor.org/rfc/rfc1
9. Dean, J., Barroso, L. A. "The Tail at Scale." *Communications of the ACM*, 56(2), 2013.
10. Fowler, M. "Who Needs an Architect?" *IEEE Software*, 20(5), 2003.
11. Instagram Engineering. "What Powers Instagram: Hundreds of Instances, Dozens of Technologies." Instagram Engineering blog, 2011. https://instagram-engineering.com/what-powers-instagram-hundreds-of-instances-dozens-of-technologies-adf2e22da2ad
12. ISO/IEC 25010. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO/IEC, 2011; revised 2023.
13. ISO/IEC/IEEE 42010. *Software, systems and enterprise — Architecture description*. ISO/IEC/IEEE, 2011; revised 2022.
14. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
15. Kruchten, P. "Architectural Blueprints — The '4+1' View Model of Software Architecture." *IEEE Software*, 12(6), 1995.
16. Kubernetes Authors. *Kubernetes Enhancement Proposals (KEPs)*. GitHub. https://github.com/kubernetes/enhancements
17. Nygard, M. "Documenting Architecture Decisions." Cognitect blog, 15 November 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
18. Warsaw, B., Hylton, J., Goodger, D., Coghlan, A. *PEP 1 — PEP Purpose and Guidelines*. Python Software Foundation. https://peps.python.org/pep-0001/

19. The Rust Project. *Rust RFCs*. GitHub. https://github.com/rust-lang/rfcs
20. U.S. Securities and Exchange Commission. *In the Matter of Knight Capital Americas LLC* (administrative proceeding order). SEC, 2013.
21. Ubl, M. “Design Docs at Google.” *Industrial Empathy*, 2020. https://www.industrialempathy.com/posts/design-docs-at-google/
22. Winters, T., Manshreck, T., Wright, H. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O’Reilly Media, 2020.
23. Xu, A. *System Design Interview – An Insider’s Guide*. Independently published, 2020.

1.19 Open verification items

- [Nygard 2011] and [Ubl 2020]: URLs could not be fetched from the authoring environment (network-restricted); existence and dates confirmed only via search results. Re-check URLs and the Ubl page-length claim (“roughly 10–20 pages”; “mini design docs”).
- [SEC 2013]: confirm release number and URL of the Knight Capital order, and the figures “one of eight servers”, “about 45 minutes” and “more than $460 million”.
- YouTube “Gangnam Style” 32-bit counter (December 2014): locate a primary source (YouTube/Google post) or keep as Secondary.
-
- [Boehm and Basili 2001]: confirm wording of the after-delivery cost-of-fix item; the commonly quoted “100×” multiplier is deliberately not stated.
- [ISO/IEC 25010] and [ISO/IEC/IEEE 42010]: confirm exact current titles of the 2023 and 2022 revisions.
-
-

Chapter 2 — How to Think Like a System Designer

> **Level:** 1–2 · **Track:** Shared · **Estimated reading time:** 3 h

Chapter 1 told you *what* system design is. This chapter is about *how* experienced designers think while they do it. It does not cover specific technologies (databases, caches, message brokers). Those come in Parts II–VI. What you get here is a set of thinking tools that apply to every design problem in this book, whether it is a URL shortener, a payment system or a large-language-model serving platform.

To keep the tools concrete, the whole chapter follows one running problem from a vague request to a documented architecture:

> WORKED EXAMPLE
>
> **Running problem — QuickBite.** A start-up asks you to “build the backend for our food-delivery app.” Customers browse nearby restaurants, put dishes in a cart, pay, and watch a courier bring the food. Restaurants receive orders on a tablet and mark them ready. Couriers receive delivery jobs on a phone and share their location while driving. QuickBite runs in one country today and wants to grow.
>
> QuickBite is a hypothetical company invented for teaching. Every number attached to it in this chapter is an assumption we make and state openly. None of them is a measurement of any real food-delivery company.

2.1 Learning objectives

After this chapter you should be able to:

- **Decompose** a product description into components with clear responsibilities, using cohesion, coupling and data ownership as criteria.
- **Draw** a system-context diagram that separates what you are building from the people and external systems it depends on.
- **Elicit, quantify and negotiate** functional and non-functional requirements, turning vague wishes (“fast”, “real-time”, “never down”) into testable numbers.
- **Identify the bottleneck** of a request path using the resource model (CPU, memory, disk, network, locks and other software resources), the utilization law and Little’s

Law.

- **State assumptions** explicitly, rank them by how much the design depends on them, and plan how to check them.
- **Evaluate trade-offs** with a weighted decision matrix, and judge how much process a decision needs by asking whether it is reversible (a “two-way door”) or not (a “one-way door”).
- **Design for failure** by listing failure modes, limiting blast radius and computing the availability of a dependency chain.
- **Reason about scale** in orders of magnitude, and predict which parts of a design break at 10×, 100× and 1000× the load.
- **Perform** a complete back-of-the-envelope capacity estimate with explicit arithmetic.
- **Document** an architecture with the C4 model, a design document and Architecture Decision Records (ADRs).

2.2 Prerequisites

- Functional vs non-functional requirements — §1.6 and §1.7. This chapter turns them into numbers and negotiates them.
- High-level vs low-level design — §1.5. Decomposition here stays at the high level.
- Powers of ten and units (KB vs KiB) — §4.3 and §4.5. Every estimate in §2.10 and §2.11 depends on them. This chapter uses SI prefixes throughout (1 KB = 10^3 bytes, 1 GB = 10^9 bytes).
- A rough idea of what a server, a database and a network request are. No deeper knowledge is assumed. Every other term is defined when first used.

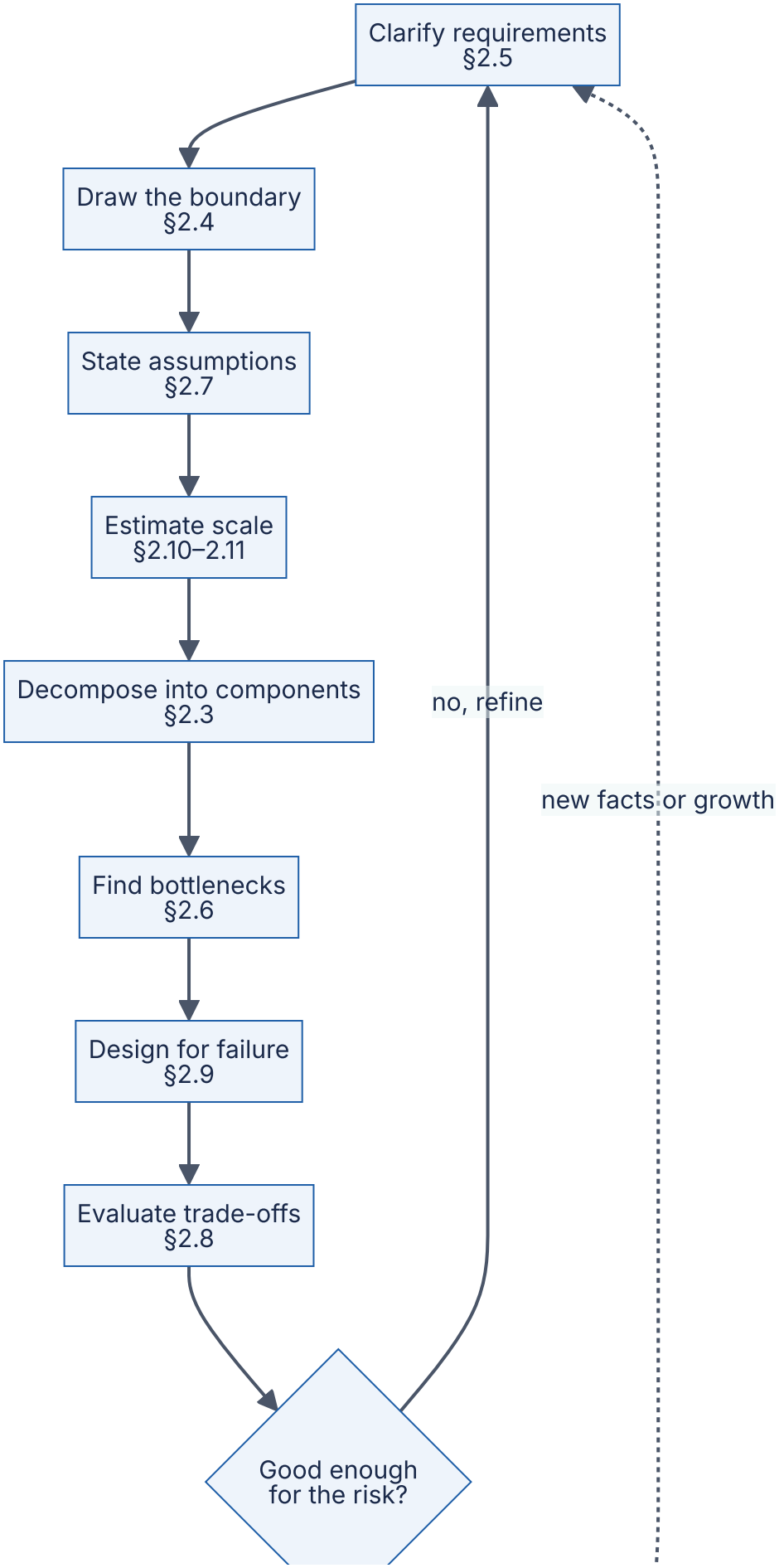
Helpful but optional: Little’s Law (§4.11) and availability arithmetic (§4.14). This chapter introduces both informally and points forward to the full treatment.

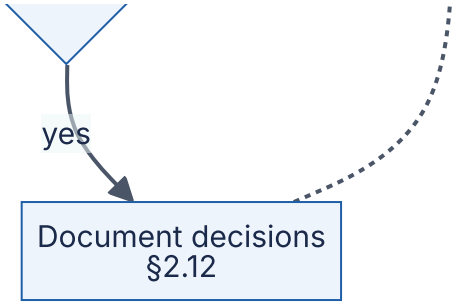
2.3 Decomposition into components

2.3.1 The thinking loop and where decomposition fits

Before the first tool, look at the whole process. Experienced designers do not move in a straight line from requirements to boxes and arrows. They work in a loop. They make a rough pass through every concern, find the riskiest part, and go round again at higher resolution. The sections of this chapter are the tools used at each step of that loop. They are presented one at a time for teaching, but in practice you switch between them constantly.

Figure 2.1 — The system designer’s thinking loop.


Clarify requirements §2.5
Draw the boundary §2.4
State assumptions §2.7
Estimate scale §2.10–2.11
Decompose into components §2.3
Find bottlenecks §2.6
Design for failure §2.9
Evaluate trade-offs §2.8
Good enough for the risk?
no, refine
new facts or growth



Reading the diagram. Each rectangle is a thinking activity, labeled with the section that teaches it. It is not a software component. Solid arrows show the usual order within one pass. The diamond is the decision every designer makes at the end of a pass: is the design good enough *for the risk involved*? A prototype for ten users clears that bar after one rough pass. A payment system may need many passes. The "no, refine" edge returns to requirements because a weak design is very often caused by a misunderstood requirement. The dotted arrow from documentation back to requirements is asynchronous: weeks or months later, production data or business growth disproves an assumption and starts the loop again. Nothing in this loop "fails" in the software sense. The failure to watch for is getting stuck, for example by polishing component diagrams (§2.3) before estimating scale (§2.11). The way to avoid that is to time-box each pass.

2.3.2 What decomposition is

DEFINITION

Definition — Decomposition. Dividing a system into smaller parts (**components**) so that each part has one clear responsibility, a well-defined interface, and as few dependencies on other parts as possible. Each part can then be understood, built, scaled and changed mostly on its own.

First-principles explanation

Think of a restaurant kitchen. Nobody "does the cooking." There is a grill station, a fry station, a pastry station and a pass where plates are checked before they leave. Each station has its own tools and ingredients and gets work through a narrow interface: the ticket. When the fryer breaks, the grill keeps working. When dessert orders double, you add a pastry cook without touching the grill.

Software decomposition works the same way. You split a big, fuzzy problem ("the food-delivery backend") into parts. Each part answers one question well ("what is on the menu?", "where is the courier?"). The parts interact through narrow, explicit interfaces, usually an API or a message.

Why does this exist?

A system nobody can hold in their head cannot be reasoned about. Decomposition gives you four concrete benefits:

1. **Comprehension.** One component fits in one engineer's head.
2. **Independent scaling.** In QuickBite, courier location updates arrive thousands of times per second, but restaurants edit menus a few times per day. Separating them lets each be scaled on its own terms.
3. **Failure isolation.** A bug in the recommendation feature should not stop customers from paying.
4. **Team autonomy.** Different teams can own different components. Conway's observation that organizations produce designs that copy their own communication structures [Conway 1968] means team boundaries and component boundaries end up shaping each other whether you plan it or not.

David Parnas made the classic argument for *how* to choose the parts. Divide a system so that each module hides a design decision that is likely to change, rather than by the steps of a processing flowchart [Parnas 1972]. That principle, **information hiding**, is still the best single test of a good boundary.

How it works internally

A practical decomposition procedure:

1. **List the nouns and verbs.** From the product description: *customer, restaurant, menu, dish, cart, order, payment, courier, delivery, location, notification*. Verbs: *browse, search, add to cart, place order, pay, accept, prepare, assign, pick up, track, deliver, rate*.
2. **Group by data ownership.** Ask "who is the single source of truth for this data?" Menus belong with restaurants. Orders and their states belong together. Courier locations are a separate, high-volume stream.
3. **Group by rate of change and load profile.** Things that change at very different speeds, or see very different traffic, usually belong apart. Menus are read-heavy and rarely written. Location is write-heavy and short-lived.
4. **Group by consistency need.** Money movements need strong guarantees: no double charges. The display of a courier's position can tolerate being a few seconds stale.
5. **Name each component with one sentence of responsibility.** If the sentence needs "and", consider splitting it.
6. **Draw the interactions.** Mark each one as synchronous (the caller waits) or asynchronous (the caller hands off a message and moves on).
7. **Check against scenarios.** Walk the top three user journeys through the diagram. Every step must have exactly one owner.

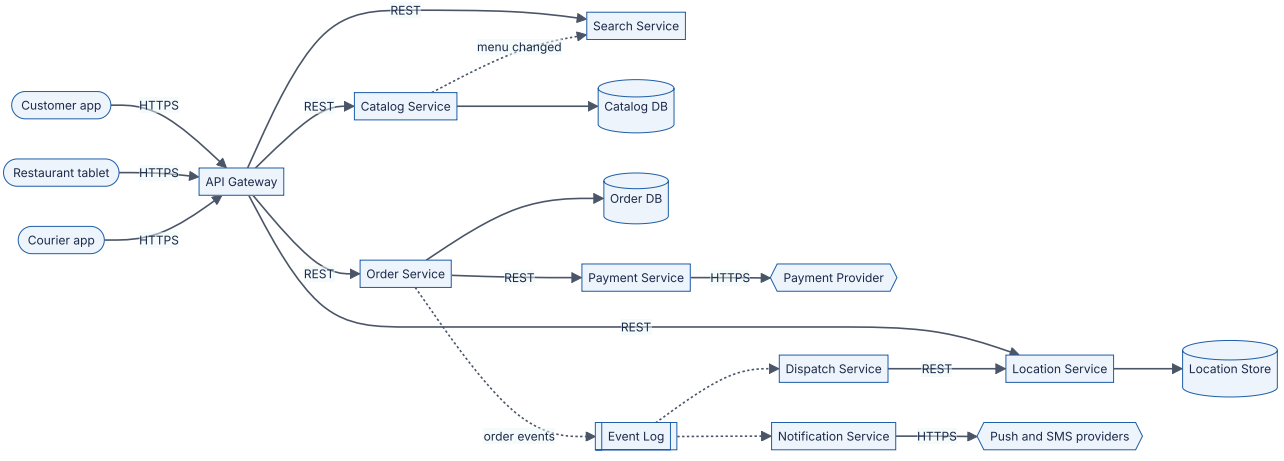
Applying this to QuickBite gives the components in Table 2.1.

Table 2.1 — QuickBite components and their single responsibilities.

Component	Responsibility (one sentence)	Owns data	Load profile	Consistency need
API gateway	Authenticates clients and routes requests	None (stateless)	All traffic	n/a
Catalog service	Serves restaurants, menus and prices	Restaurants, menus	Read-heavy	Minutes of staleness OK
Search service	Finds restaurants by location, cuisine, text	Search index (derived)	Read-heavy	Derived, can lag
Order service	Owns the life cycle of an order	Orders, order items	Moderate writes	Strong
Payment service	Charges and re-funds via a payment provider	Payment records	Low volume	Strong, idempotent
Dispatch service	Assigns couriers to ready orders	Assignments	Bursty	Strong per assignment
Location service	Ingests and serves courier positions	Recent positions	Write-heavy	Seconds of stale-ness OK
Notification service	Sends push, SMS and email	Delivery log	Bursty, async	At-least-once

Architecture diagram

Figure 2.2 — QuickBite first-pass decomposition.



Reading the diagram. Three client types (stadiums) reach the system only through the **API gateway** over HTTPS. The gateway is stateless, so it scales horizontally behind a load balancer (Chapter 19). Solid arrows are synchronous request/response calls. Placing an order calls the order service, which calls the payment service, which calls the external

payment provider (hexagon). That chain is the critical path, and every link in it can fail (§2.9). Dotted arrows are asynchronous. Once an order is committed, the order service publishes an event to the **event log** (subroutine shape; see Chapter 30). The dispatch and notification services consume it at their own pace, so a slow SMS provider cannot slow down checkout. Each cylinder is a storage boundary owned by exactly one service. No service reads another service's database, which keeps data ownership clear (§34.11). The catalog service pushes menu changes asynchronously to the search service, whose index is a *derived* copy that may lag by seconds. The location store is the heaviest write path, as §2.11 will show, so it scales independently, for example with an in-memory store partitioned by city. Likely failure points: the payment provider (external, outside your control), the order database (single source of truth) and the event log (if it stops, dispatch stops). Each needs a mitigation in §2.9.

> NOTE
>
> **Components are logical, not deployment units.** Figure 2.2 shows *logical* components. On day one, QuickBite might deploy all of them as one process: a **modular monolith** (§33.5). Decomposition decides where the *boundaries* are. Deployment decides how many *processes* there are. §2.8 makes that second decision with a decision matrix.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **Amazon's move to services.** In a 2006 interview, Amazon's CTO Werner Vogels described how Amazon's early monolithic application was restructured into services. Each service owned its data and could only be reached through a published interface. Direct database access between teams was ruled out. He linked this to team ownership ("you build it, you run it"). The design choice was data ownership behind interfaces, the same rule used in Figure 2.2. The trade-off he acknowledged was added operational and coordination complexity in exchange for team independence and scalability. *Evidence: Primary* — [Gray 2006]

Advantages and disadvantages

Good decomposition makes reasoning, scaling, failure isolation and team ownership easier. It also has costs. Every boundary that crosses a network turns a function call (~nanoseconds) into a remote call (~0.5 ms or more within a datacenter, see §4.17), and that call can fail independently. Data split across components loses cross-component transactions (Chapter 28). Too many components multiply operational load: more deployments, dashboards and on-call rotations.

Alternatives and trade-offs

Table 2.2 — Ways to choose component boundaries.

Approach	Splits by	Works well when	Main risk
By technical layer	UI / logic / data	Small systems, one team	Every feature change touches every layer
By business capability	Orders, catalog, payments	Most product systems	Needs domain understanding up front
By volatility (Parnas)	Decisions likely to change	Long-lived systems	Requires predicting change
By load profile	Read-heavy vs write-heavy	Very uneven traffic	Can fragment one domain concept
By domain-driven bounded context	Consistent language and model	Large organizations	Heavy analysis cost (§34.4–34.5)

In practice you combine them. QuickBite splits by business capability first, then splits *location* out of *dispatch* because of its very different load profile.

Failure scenarios

Table 2.3 — Decomposition failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Distributed monolith	Services share a database or must deploy together	Every release touches many services; cross-service schema locks	Merge tightly coupled services	Enforce data ownership; review coupling
Chatty interfaces	Boundary cuts through a hot loop	One user request causes dozens of internal calls; high p99	Batch calls or move the boundary	Walk scenarios through the diagram before building
Orphan responsibility	No component owns a step, e.g. refunds	Bugs bounced between teams	Assign an owner explicitly	Step 7 of the procedure: one owner per step
Premature split	Boundaries drawn before the domain is understood	Frequent cross-boundary changes	Re-merge into a modular monolith	Start coarse; split when evidence appears

Common misconceptions

COMMON MISTAKE

“More components means a better design.” The number of boxes is not a quality measure. A good decomposition has *few, cohesive* components with *narrow* interfaces. A useful test: if two components change together in most pull requests, they are probably one component.

COMMON MISTAKE

“Decompose by technology.” “The Redis service” or “the Kafka service” are not responsibilities. Components are named after what they do for the business (“location service”). The technology they use is an internal detail that may change.

2.4 System boundaries and context diagrams

2.4.1 Boundaries and the context diagram

DEFINITION

Definition — System boundary. The line that separates what you are designing and are responsible for from everything outside it: users, other teams’ systems and third-party services. You control what is inside. You can only *depend on* what is outside.

DEFINITION

Definition — Context diagram. A diagram that shows the whole system as a single box, surrounded by the people (actors) and external systems that interact with it, with each interaction labeled. Context diagrams come from structured analysis [DeMarco 1979] and are Level 1 of the C4 model [Brown C4].

First-principles explanation

Before renovating a house, a builder walks the property line. Fences, the street, the neighbor’s tree and the water main are not part of the job, but every one of them constrains it. A context diagram is that walk around the property line. It answers three questions before any internal design starts: *who uses this?*, *what does it depend on?* and *what is not our job?*

Why does this exist?

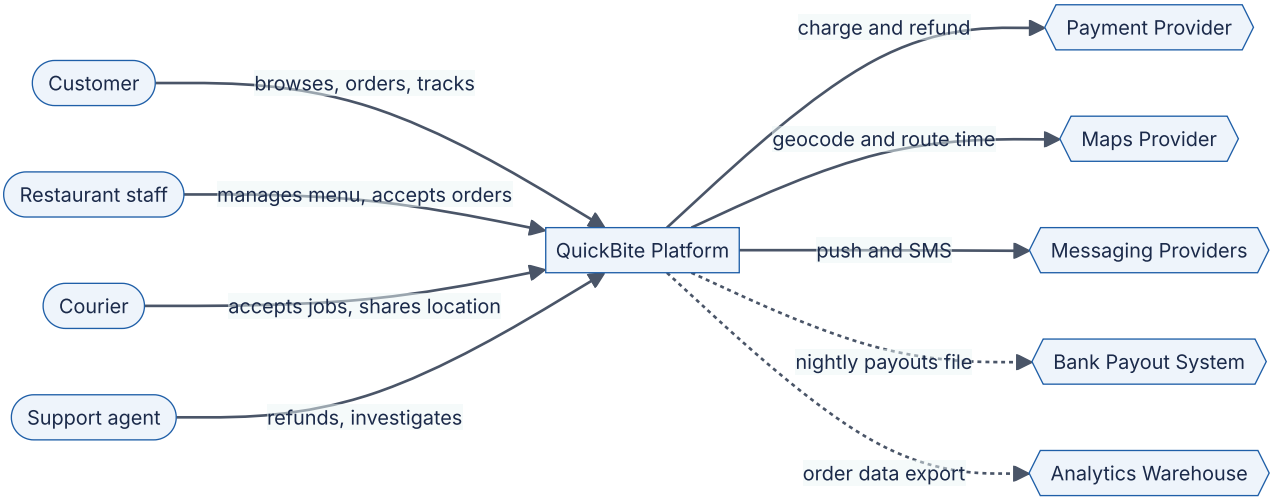
Many expensive design failures are scope failures, not technical ones. A team builds a payment system and finds out late that tax calculation was expected. Or it assumes an external maps API can take 10,000 requests per second when the contract allows 100. Drawing the boundary early brings these surprises to the surface while they are still cheap to fix. It also gives non-engineers, such as product managers, legal and finance, a diagram they can actually check.

How it works internally

1. Draw your system as one box. Resist the urge to draw its insides.
2. Around it, place every **actor** (human role) and every **external system**.
3. Label each interaction with *what* flows (not *how*): “places orders”, “sends card charge”.
4. For every external system, note three facts: its **availability promise** (its SLA, see §39.6), its **rate limits** and its **latency**. These numbers are inherited constraints you do not control.
5. Write down what is explicitly **out of scope**.

Architecture diagram

Figure 2.3 — QuickBite system-context diagram.



Reading the diagram. The single rectangle is the whole QuickBite platform, deliberately shown with no internal detail. Four human actors (stadiums) interact with it. Note the **support agent**, an actor often forgotten in first drafts, who needs refund and investigation tools. Five external systems (hexagons) sit outside the boundary. Solid arrows are synchronous, user-facing interactions. A customer waits while a card charge happens, so the payment provider’s latency and availability become part of QuickBite’s own checkout latency and availability. Dotted arrows are asynchronous batch flows. Payouts to restau-

rants and data exports run nightly and can be retried without users noticing. Each hexagon is a failure point outside your control. The maps provider is especially dangerous because both search (delivery-time estimates) and dispatch depend on it. Hexagons also bring rate limits and per-call costs, so scaling QuickBite may be limited by *contracts* rather than by servers.

WORKED EXAMPLE

Worked example — Inherited constraints from the boundary. Suppose the maps provider's contract allows 500 requests per second and charges $0.005 per route-time request. (These figures are assumptions for QuickBite, not any real provider's price list.)

1. If every restaurant listing in search results needs a live route-time estimate, and search peaks at 2,000 requests per second showing 20 restaurants each, QuickBite would need 2,000 × 20 = 40,000 route requests per second.
2. That is 40,000 / 500 = 80 times the contract limit.
3. Cost at that rate for one peak hour: 40,000 × 3,600 × $0.005 = $720,000.

Interpretation. Calling the maps provider per listing is impossible, regardless of how good the internal architecture is. **Engineering implication:** estimate delivery times internally, for example from precomputed travel-time grids per city zone, and call the provider only once per *placed order*. The boundary diagram exposed this constraint before any code was written.

Real-world industry applications

INDUSTRY EXAMPLE

C4 system-context diagrams. Simon Brown's C4 model makes the system-context diagram its first and most abstract level. It recommends showing the software system as one box with its users and the other software systems it depends on, and states that the diagram should be understandable by non-technical stakeholders. *Evidence: Primary* — [Brown C4]

Failure scenarios and misconceptions

Table 2.4 — Boundary failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Hidden dependency outage	An external system was never listed	Incident traced to an unlisted vendor	Add fallback; add it to the diagram	Inventory every outbound call
Rate-limit wall	Vendor limits not read	HTTP 429 errors at peak	Cache, batch, negotiate a higher tier	Step 4: record limits per external system
Scope creep	Boundary never agreed	Late "but we assumed you'd do X"	Re-plan	Explicit out-of-scope list signed off

COMMON MISTAKE

"External systems are always available." A third-party SaaS API typically promises somewhere between 99.9% and 99.99% availability, as of 2026. At 99.9% that is up to about 43 minutes of downtime per 30-day month. Your design must decide what QuickBite does during those minutes (§2.9).

2.5 Understanding and negotiating requirements

2.5.1 Eliciting and negotiating requirements

DEFINITION

Definition — Requirement. A statement of what the system must do (a **functional requirement**, e.g. "a customer can cancel an order before the restaurant accepts it") or how well it must do it (a **non-functional requirement**, e.g. "99% of order placements complete within 800 ms"). A requirement is **testable** if you could write an automated check or a measurement that proves it is met.

First-principles explanation

A customer asks a tailor for "a nice suit." A bad tailor starts cutting. A good tailor asks: for what occasion? Which climate? What budget? Ready by when? The good tailor is not being difficult. "Nice" can mean a hundred different suits, and only one of them is what the customer actually wants. Requirements gathering is the tailor's questions. **Negotiation** is the conversation that follows: "a hand-stitched suit by Friday for that budget is not possible, but a made-to-measure one is."

Why does this exist?

Vague requirements cause two opposite failures. **Under-engineering** builds a system that falls over on launch day because nobody asked how many users would arrive. **Over-engineering** spends six months building multi-region active–active replication for a prod-

uct with 200 users, because someone said “it must never go down.” Both come from the same root cause: a requirement nobody turned into a number.

Requirements also conflict. “Show the courier’s exact position in real time” pushes against “keep infrastructure costs under budget” and “protect courier privacy.” Someone has to decide how those conflicts are resolved, and the designer’s job is to make that decision *visible and deliberate*.

How it works internally

A reliable elicitation and negotiation procedure:

1. **Identify users and their top journeys.** For QuickBite: *browse and order*, *track delivery*, *restaurant accepts and prepares*, *courier delivers*.
2. **Write functional requirements as user-observable behaviors**, and rank them with **MoSCoW** priorities: Must, Should, Could, Won’t (this time).
3. **For each journey, ask the “-ility” questions** (§1.7): How many users? How fast? How available? How consistent? How durable? How secure? What does it cost?
4. **Quantify every non-functional requirement** as a metric, a threshold and a measurement window. “Fast” becomes “p99 < 800 ms measured over 5-minute windows”. The notation p99 means the 99th percentile: 99% of requests are faster than this value (§4.8).
5. **Differentiate by journey.** Availability and latency targets rarely need to be uniform. Checkout deserves a stricter target than the restaurant’s sales-history report.
6. **Surface conflicts and negotiate.** For each expensive requirement, show its cost and offer a cheaper alternative that still meets the underlying need.
7. **Record what was agreed, and what was explicitly rejected.**

Table 2.5 — QuickBite requirements after negotiation.

ID	Requirement (as first stated)	Negotiated, testable version	Priority
F1	“Customers can order food”	Customer can place an order of 1–30 items from one restaurant and pay by card or wallet	Must
F2	“Real-time tracking”	Courier position shown on a map, refreshed every 5 s; position at most 10 s old at p95	Must
F3	“Restaurants get orders instantly”	New order appears on the restaurant tablet within 5 s at p99	Must
F4	“Smart recommendations”	Deferred to phase 2; phase 1 shows “popular near you”	Won’t (now)
N1	“Never goes down”	Order placement available 99.95% per 30-day month; browsing 99.9%	Must
N2	“Fast”	Place-order p99 < 800 ms excluding payment-provider time; browse p99 < 300 ms	Must
N3	“Never lose an order”	Once the customer sees “order confirmed”, the order survives the loss of any single node or zone (RPO = 0 for orders)	Must
N4	“Never double-charge”	Each order is charged at most once, even if the client retries (idempotency, §22.15)	Must
N5	“Menus always correct”	Menu or price changes visible to customers within 60 s; price is re-checked at order time	Should

Notice what negotiation did to N1. “Never goes down” is impossible: no system reaches 100%. It was also unnecessary for most features. The negotiated version gives each journey its own target and makes the target measurable. RPO in N3 is the *recovery point objective,* the amount of recent data you may lose in a disaster (§39.8).

Mathematical foundations

The most useful number in a negotiation is the downtime a target actually allows.

FORMULA

Allowed downtime for an availability target

$$T_{down} = (1 - A) \times T_{period}$$

Symbol	Meaning	Units
T_{down}	Maximum downtime allowed in the period	minutes
A	Availability target, as a fraction	dimensionless (e.g. 0.9995)
T_{period}	Length of the measurement period	minutes

WORKED EXAMPLE

Worked example — What “three and a half nines” buys QuickBite. A 30-day month has 30 × 24 × 60 = 43,200 minutes.

1. Browsing at 99.9%: T_{down} = (1 − 0.999) × 43,200 = 0.001 × 43,200 = **43.2 minutes per month**.
2. Order placement at 99.95%: (1 − 0.9995) × 43,200 = 0.0005 × 43,200 = **21.6 minutes per month**.
3. A naive “five nines” (99.999%): 0.00001 × 43,200 = **0.43 minutes**, about 26 seconds per month.

Interpretation. Twenty-six seconds a month is less than the time a person takes to notice an alert, open a laptop and log in. Five nines requires fully automated detection and failover across independent zones or regions, which multiplies cost and complexity. **Engineering implication:** for a regional food-delivery start-up, 99.95% on checkout is ambitious but reachable with redundancy inside one region (§39.4). Five nines would be over-engineering, and the payment provider’s own availability probably caps the achievable number anyway (§2.9).

Real-world industry applications

INDUSTRY EXAMPLE

Service level objectives at Google. Google's Site Reliability Engineering book argues that 100% is the wrong reliability target for almost everything. Users cannot tell the difference between very high availability and 100%, because their own devices and networks are less reliable than that. Each extra nine costs a lot. Google's practice is to agree an explicit **service level objective (SLO)** with product owners and treat the remaining unreliability as an **error budget** to be spent on releasing features. This is requirement negotiation turned into an ongoing process (§39.6–39.7). *Evidence: Primary* — [Beyer et al. 2016]

Advantages, alternatives and trade-offs

Up-front negotiation costs meeting time, and stakeholders may not know the answers yet. When they do not, do not block. Make an assumption (§2.7), label it, and design so that it is cheap to change if wrong. In interviews, spend roughly the first five to ten minutes of a 45-minute session on requirements. In industry, requirements live in a design document (§2.12) and get re-negotiated as data arrives.

Failure scenarios

Table 2.6 — Requirements failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Launch-day overload	Peak load never quantified	Errors and latency spike at launch	Emergency scaling, load shedding (§22.9)	Ask "how many, at peak?"
Gold-plating	"Never down" accepted literally	Cost far exceeds the budget; delivery late	Simplify	Translate every adjective into a number
Silent conflict	Two stakeholders want contradictory things	Rework after review	Escalate with options and costs	Step 6: surface conflicts explicitly
Uniform targets	One SLO applied to all features	Expensive redundancy on unimportant paths	Tier the features	Step 5: differentiate by journey

Common misconceptions

COMMON MISTAKE

“Requirements are given to the designer.” In practice they are *discovered and negotiated* by the designer together with stakeholders. A designer who only takes orders will build exactly the wrong thing, very efficiently.

INTERVIEW TIP

Interview tip. Say your negotiated requirements out loud and write them on the board as numbers: “I’ll assume 2 million daily active users, checkout at 99.95%, tracking refreshed every 5 seconds — does that match what you have in mind?” Interviewers reward this because it shows you design *to a target*, not by habit.

2.6 Identifying bottlenecks

2.6.1 The resource model and the bottleneck

DEFINITION

Definition — Bottleneck. The resource whose capacity limits the throughput of the whole system for a given workload. Throughput cannot rise above the bottleneck’s capacity no matter how much the other resources are improved. Formally, it is the resource with the highest **service demand** per request (defined below).

First-principles explanation

Picture a highway that narrows from four lanes to one for a bridge. Widening the highway before or after the bridge to eight lanes changes nothing: cars still cross the bridge one lane at a time. The bridge is the bottleneck. Worse, traffic does not slow down evenly. It piles up in a queue *in front of* the bridge. That is why a system near its bottleneck shows rising latency long before it shows errors.

Two consequences follow, and they explain most performance work:

1. **Improving a non-bottleneck resource gives zero improvement in throughput.** Faster CPUs do not help if the database disk is the limit.
2. **Fixing the bottleneck moves it.** Once the bridge is widened, the next narrowest point becomes the new bottleneck. Performance work is a sequence of “find the bottleneck, relieve it, find the next one.”

Why does this exist?

Systems are made of finite resources. When a QuickBite lunch rush arrives, the first resource to run out decides whether customers see a spinner or an error. If you cannot name that resource, you will add capacity in the wrong place, and pay for servers that make no difference.

How it works internally: the resource model

Every request consumes a small amount of several **resources**. The classical hardware four plus one software category cover almost every case:

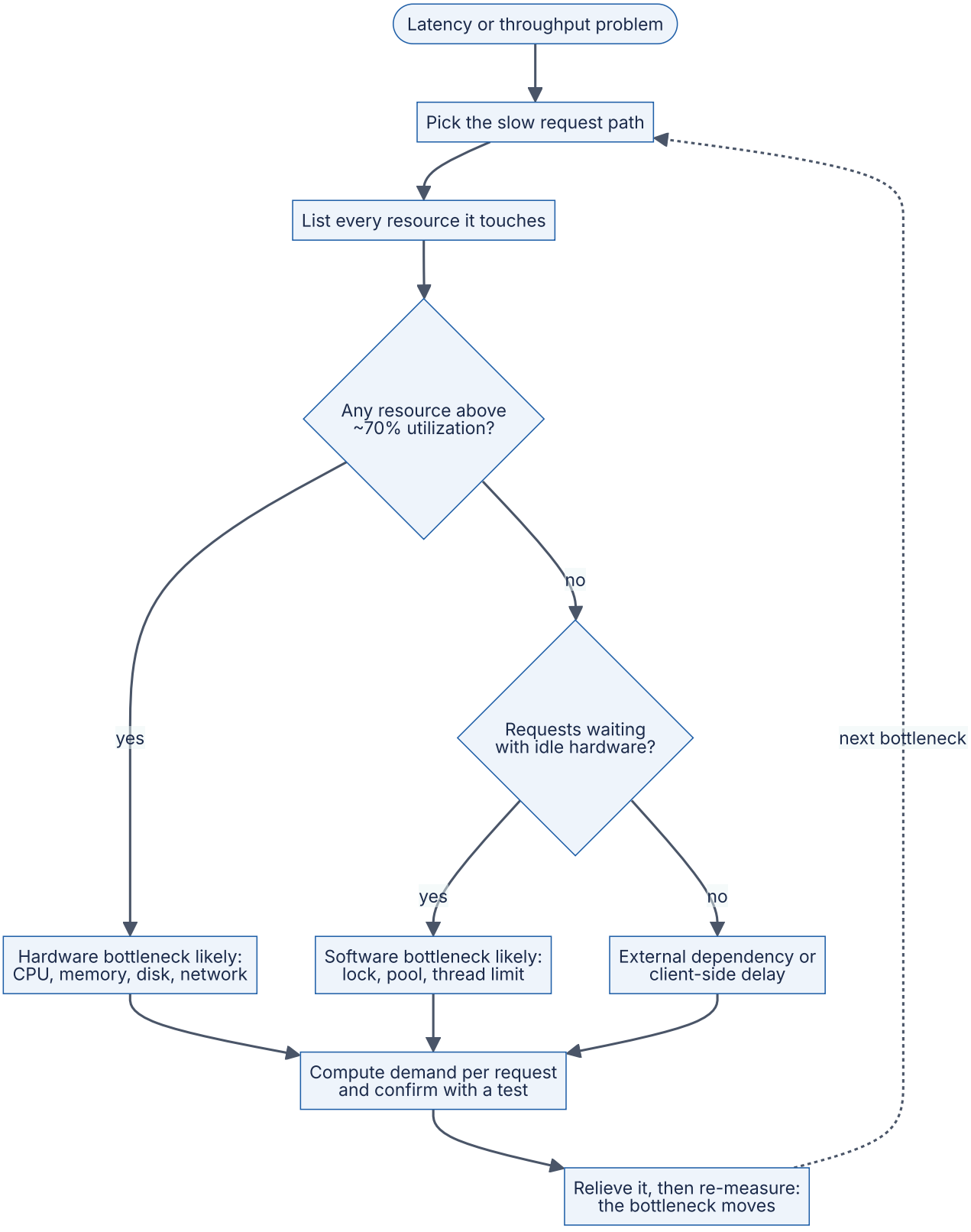
Table 2.7 — The resource model.

Resource	What a request consumes	Utilization signal	Saturation signal	Typical relief
CPU	Cycles for parsing, serialization, business logic, encryption	% busy per core	Run-queue length > core count	Optimize hot code, cache results, add cores or nodes
Memory	Working set, buffers, caches, per-connection state	% used	Swapping, OOM kills, GC pauses	Reduce footprint, add RAM, partition data
Disk	Reads and writes (IOPS), bytes per second, fsync latency	% time busy, IOPS used vs limit	Long I/O queue, rising await time	Cache, batch writes, faster media, partition
Network	Bytes in and out, packets, connections	Bandwidth used vs link capacity	Dropped packets, retransmits	Compress, CDN, bigger links, keep traffic local
Locks and other software resources	Exclusive access to a row, mutex, thread, connection-pool slot, file handle	Fraction of time held	Waiters queued; pool exhausted	Shorten hold time, finer-grained locks, bigger pools, partition the hot item

The last row is the one beginners miss. A **software resource**, such as a database row lock, a thread pool or a connection pool, is just as finite as a CPU. It can be the bottleneck while every hardware dashboard shows idle machines.

The standard diagnostic method is Brendan Gregg's **USE method**: for every resource, check **U**tilization, **S**aturation and **E**rrors [Gregg USE]. Figure 2.4 turns it into a procedure.

Figure 2.4 — Finding the bottleneck with the USE method.



Reading the diagram. This is a reasoning procedure, not a running system. You start from a symptom (stadium) and pick one slow request path, because bottlenecks are workload-specific: browsing and ordering have different ones. The first decision checks hardware utilization. Around 70% is a common warning threshold, because queueing delay grows steeply as utilization approaches 100% (§4.10). The second decision is the key di-

agnostic insight: *requests are waiting while the hardware is idle*. This almost always means a software resource such as a lock or a pool is saturated. If neither applies, the time is being spent outside your servers, for example in a third-party API or on a slow mobile network. Every branch leads to the same discipline: compute the demand per request and confirm the hypothesis with a measurement or load test (Chapter 23). The dotted edge back to the start is the loop that never ends: relieving one bottleneck exposes the next.

Mathematical foundations

Two laws from operational analysis of computer systems [Denning and Buzen 1978] are enough to find most bottlenecks with arithmetic alone.

FORMULA

Service demand, the utilization law and the bottleneck bound

$$D_i = V_i \times S_i \qquad U_i = X \times D_i \qquad X_{max} = \frac{c_i}{D_{max}}$$

Symbol	Meaning	Units
D_i	Service demand: total busy time one request places on resource i	seconds per request
V_i	Visits: how many times one request uses resource i	visits per request
S_i	Service time per visit to resource i	seconds per visit
U_i	Utilization of resource i, per unit of capacity	fraction (0–1)
X	System throughput	requests per second
c_i	Number of parallel units of resource i (e.g. cores)	units
X_{max}	Maximum throughput, set by the resource with the largest demand per unit	requests per second

The utilization law for a resource with c_i parallel units is $U_i = X \times D_i / c_i$. The bottleneck is the resource with the largest D_i / c_i.

FORMULA

Little’s Law (concurrency)

$$L = \lambda \times W$$

Symbol	Meaning	Units
L	Average number of requests in the system (or holding a resource) at once	requests
λ	Average arrival rate	requests per second
W	Average time each request spends in the system (or holds the resource)	seconds

Little’s Law holds for any stable system, regardless of the arrival pattern [Little 1961]. Full treatment in §4.11.

WORKED EXAMPLE

Worked example — Finding QuickBite's checkout bottleneck.

Situation. At dinner peak QuickBite receives 60 order placements per second (this number is derived in §2.11). Customers complain that checkout sometimes spins for 5 seconds and then fails, yet CPU dashboards show the database at only 25% utilization. The order transaction, as first written, does this:

1. Begin a database transaction.
2. Insert the order and its items.
3. Update the restaurant's row in `restaurant_load` to increment `open_orders` (used to stop accepting orders when a kitchen is overwhelmed). This takes a **row lock** until commit.
4. Call the payment service, which calls the payment provider. Average 300 ms.
5. Commit.

Measured per-order demands (assumed values for the exercise):

Table 2.8 — Per-order resource demand in the checkout path.

Resource	Demand per order D_i	Parallel units c_i	D_i/c_i	Max orders/s = $1/(D_i/c_i)$	Utilization at 60/s
API tier CPU	8 ms	32 cores	0.25 ms	4,000	60 × 0.25 ms = 1.5%
Database CPU	5 ms	8 cores	0.625 ms	1,600	3.8%
Database WAL disk (fsync)	2 ms	1 device	2 ms	500	12%
Network (25 KB per order, 10 Gbps link)	0.02 ms	1 link	0.02 ms	50,000	0.1%
DB connection pool (20 connections)	320 ms held	20	16 ms	62.5	96%
Row lock on one popular restaurant	320 ms held	1	320 ms	3.1	see below

Step 1 — Hardware. Every hardware resource is far below 70% utilization. The largest is the WAL disk at 12%, so hardware is not the bottleneck. Following Figure 2.4, "requests waiting with idle hardware" points to a software resource.

Step 2 — Connection pool, by Little's Law. Each order holds a database connection from step 1 to step 5: about 300 ms of payment time plus about 20 ms of database work, so W = 0.32 s. Then $L = \lambda \times W = 60 \times 0.32 =$ **19.2 connections busy on av-**

erage, out of a pool of 20. Utilization is 19.2 / 20 = 96%. Any burst above 62.5 orders/s (20 / 0.32) makes requests queue for a connection, and queueing at 96% utilization is severe (§4.10).

Step 3 — The row lock. A celebrity chef's restaurant runs a promotion and receives 8 orders per second. Each order holds that restaurant's row lock for 0.32 s, so the row can serve at most 1 / 0.32 = **3.1 orders per second**. Arrivals (8/s) exceed capacity (3.1/s), so the queue grows without bound until requests time out at 5 s. That matches the complaint exactly. This is the bottleneck for that workload: a single **hot row**.

Step 4 — Relieve it. Move the slow external call *out of* the database transaction. First authorize the payment, keyed by an idempotency key so retries cannot double-charge (§22.15). Then run a short transaction (insert order, update counter, commit) of about 15 ms. Then capture the payment, or publish an event to do so asynchronously.

Step 5 — Re-compute. Hold time W drops from 0.32 s to 0.015 s.

- Pool: $L = 60 \times 0.015 =$ **0.9 connections busy**, which is 4.5% of the pool.
- Hot row: maximum 1 / 0.015 ≈ **66 orders/s per restaurant**, comfortably above 8.
- The bottleneck has moved. It is now the WAL disk at 500 orders/s, about 8× headroom over today's peak. That is the next thing to watch as QuickBite grows (group commit and partitioning address it, §13.5 and Chapter 14).

Interpretation. Adding database servers or CPUs would have cost money and fixed nothing: the limit was a lock held during a network call. **Engineering implication:** never hold a lock or a pooled connection across a call to a slow or external system. In general, *hold time multiplied by arrival rate* (Little's Law) tells you how many units of any software resource you need.

Code example

A short, runnable script that turns the table above into a bottleneck report. Use it to check your own estimates.

```
        # Bottleneck finder using the utilization law (U = X * D / c).
        resources = {
            # name: (demand per request in seconds, parallel units)
            "api_cpu":   (0.008, 32),
            "db_cpu":    (0.005, 8),
            "wal_disk":  (0.002, 1),
            "network":   (0.00002, 1),
            "db_pool":   (0.320, 20),
            "hot_row":   (0.320, 1),
        }
        arrival_rate = 60.0  # requests per second

        for name, (demand, units) in sorted(
                resources.items(), key=lambda kv: kv[1][0] / kv[1][1], reverse=True):
            max_x = units / demand
            util = arrival_rate * demand / units
            flag = "SATURATED" if util >= 1 else ("HOT" if util >= 0.7 else "ok")
            print(f"{name:9s} max={max_x:9.1f}/s  util@{arrival_rate:.0f}/s={util:7.1%}
{flag}")
```

What to notice:

- Resources are sorted by D_i/c_i, so the first line printed is the bottleneck.
- `hot_row` shows a utilization above 100% (1920%) at 60/s, because the script applies the *whole* arrival rate to one row. That is the worst case where every order goes to a single restaurant. In the worked example only 8/s hit the hot row, which still exceeds its 3.1/s capacity. Edit the script to model per-key arrival rates.
- Changing `0.320` to `0.015` for the pool and the row reproduces Step 5 and shows the bottleneck moving to `wal_disk`.

Real-world industry applications

> INDUSTRY EXAMPLE
>
> **The USE method at Netflix and beyond.** Brendan Gregg developed the USE method while working on performance analysis and has published it, together with per-operating-system checklists, as a way to find resource bottlenecks quickly without guessing. It explicitly includes software resources such as mutex locks, thread pools and file descriptors alongside CPUs, memory, disks and network interfaces. *Evidence: Primary* — [Gregg USE], [Gregg 2020]

Alternatives and trade-offs

The USE method is resource-centric. Its complement is the **RED method** (Rate, Errors, Duration per service), which is request-centric and better for spotting *which* service is slow in a microservice graph. The two work well together (§23.3). Profiling (§23.4) finds

which code consumes a resource. Load testing (§23.5) confirms a bottleneck hypothesis empirically. The arithmetic above tells you where to point those tools.

Failure scenarios

Table 2.9 — Bottleneck-analysis failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Scaling the wrong tier	Added servers to a non-bottleneck	Throughput unchanged after scaling	Re-run USE analysis	Compute demand per resource first
Hot key or hot row	Skewed popularity, e.g. a viral restaurant	One key's latency far above the others	Shorten hold time, shard the counter	Model per-key rates, not just totals
Pool exhaustion	Long hold times, often across remote calls	"Timeout acquiring connection" errors with idle DB	Move remote calls out of transactions	Little's Law sizing
Retry amplification	Clients retry into a saturated resource	Load rises as success rate falls	Retry budgets, backoff with jitter (§22.12)	Cap retries; shed load

Common misconceptions

COMMON MISTAKE

"Low CPU means the system has spare capacity." CPU is one resource of five. QuickBite's checkout failed with every machine mostly idle, because the bottleneck was a lock. Always check software resources when latency is high and utilization is low.

COMMON MISTAKE

"Averages are enough." A system can be fine on average and saturated on one hot key, one shard or one minute of the day. Bottleneck analysis must use *peak* rates and *per-partition* rates (§14.10).

2.7 Making and stating assumptions

2.7.1 The assumption register

DEFINITION

Definition — Design assumption. A belief about the workload, the environment or the business that the design depends on but that has not yet been verified, for example “80% of requests are reads” or “the payment provider responds within 1 s at p99”. An assumption is **stated** when it is written down with its value, its source, your confidence in it and how it will be checked.

First-principles explanation

A weather forecaster does not refuse to forecast because tomorrow is unknown. She says “70% chance of rain,” which is an explicit, checkable statement of uncertainty. System designers face the same situation: you never have all the facts before you must design. The professional response is not to wait. It is to assume *out loud*, so that everyone can see which parts of the design rest on which beliefs.

Why does this exist?

Every design rests on assumptions anyway. The only choice is whether they are visible. Hidden assumptions are dangerous because nobody checks them, and when one turns out wrong, nobody knows which parts of the design must change. Stated assumptions turn surprises into planned decision points.

How it works internally

1. **Write each assumption as a number with a source.** “2 M daily active users (marketing forecast for year 1).”
2. **Rate confidence**: high (measured), medium (comparable data exists), low (guess).
3. **Do a sensitivity check.** Ask: *if this is wrong by 10×, does the design change?* Assumptions where the answer is “yes” are **load-bearing**.
4. **Attach a check and a tripwire** to every load-bearing assumption: how you will verify it, and the observed value that will trigger a redesign.
5. **Prefer designs that are robust to the assumptions you are least sure of.**

Table 2.10 — QuickBite assumption register (excerpt).

ID	Assumption	Value	Source	Confidence	Load-bearing?	Check and tripwire
A1	Daily active users	2 M	Business forecast	Low	Yes: sizes every tier	Weekly DAU dashboard; redesign review at 5 M
A2	Orders per DAU per day	0.25	Industry intuition	Low	Yes: order DB and payments	Measure in beta
A3	Busiest hour share of daily orders	20%	Assumed dinner peak	Medium	Yes: peak sizing	Hourly order histogram
A4	Courier location update interval	4 s	Product decision	High	Yes: location write load	Config value, reviewed with product
A5	Payment provider p99 latency	1 s	Vendor documentation	Medium	Moderate: timeouts, pools	Client-side latency histogram
A6	Average menu size	100 items × 1 KB	Sample of 50 menus	Medium	No: menus fit in memory even at 10×	Catalog size metric

The sensitivity column is the payoff. A6 is not load-bearing: all menus total about 5 GB (§2.11), and 50 GB would still fit in a small cache cluster, so being 10× wrong changes nothing structural. A4 *is* load-bearing: halving the interval to 2 s doubles the heaviest write path in the system.

WORKED EXAMPLE

Worked example — Sensitivity of the location tier to assumption A4. Peak active couriers are assumed to be 20,000.

1. Update interval 4 s: 20,000 / 4 = **5,000 writes/s**.
2. Product asks for 1 s “for smoother maps”: 20,000 / 1 = **20,000 writes/s**, 4× the load.
3. At an assumed 200 bytes per update including protocol overhead, ingress rises from 5,000 × 200 = 1 MB/s to 20,000 × 200 = 4 MB/s. That is modest for the network, but 4× the write operations for the store.

Interpretation. Network bandwidth is not the issue. The *write rate* to the location store is. **Engineering implication:** keep the 4 s interval on the server and have the client app *interpolate* the marker’s movement between updates. Customers see smooth motion at a quarter of the cost. The assumption register made this negotiation possible because the cost of the request was visible immediately.

INDUSTRY EXAMPLE

Assumptions in published design-review practice. Google's internal design-doc culture, as described by a Google engineer, asks authors to state context, goals and *non-goals*, and to discuss alternatives and trade-offs, so that reviewers can challenge the premises behind a design and not only its mechanics. *Evidence: Secondary* — [Ubl 2020]

Failure scenarios and misconceptions

Table 2.11 — Assumption failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Stale assumption	The world changed, e.g. a new market launched	Metrics drift past the tripwire	Run the redesign review	Tripwires on load-bearing assumptions
Assumption mistaken for fact	Repeated until nobody remembers it was a guess	Nobody can name its source	Re-label and re-check	"Source" and "confidence" columns
Average-case assumption	Peak and skew ignored	Outages at peaks or on hot keys	Add peak and skew factors	Always state a peak factor

COMMON MISTAKE

"Stating assumptions shows you don't know the answer." The opposite is true. In interviews and design reviews, explicit assumptions signal seniority. They show you know *which* unknowns matter and have a plan for each one.

2.8 Evaluating trade-offs

2.8.1 Weighted decision matrices

DEFINITION

Definition — Trade-off. A situation where improving one desired property of a system makes another worse. Examples: lower latency at higher cost, stronger consistency at lower availability during network partitions, faster delivery at higher operational risk. Every design decision is a trade-off. If you cannot name what an option makes worse, you have not understood it yet.

First-principles explanation

Buying a car means trading price against comfort, fuel economy against power, cargo space against parking ease. No car wins on everything. A sensible buyer decides *which properties matter most to them*, scores the candidates, and also notices how hard the choice is to undo: leasing for a year is easy to reverse, buying a car and selling it a month later loses money. System design decisions have exactly the same two dimensions, **weighted fit** and **reversibility**.

Why does this exist?

Without a method, design decisions are made by whoever argues loudest or by habit ("we always use X"). A structured method makes the criteria and their importance explicit. It lets people disagree about *weights* (a business question) separately from *scores* (an engineering question), and it leaves a record for the ADR (§2.12).

How it works internally: the weighted decision matrix

1. List the **options**, including "do nothing" or "the simplest thing" where it makes sense.
2. List the **criteria** that come from the requirements (§2.5), not from personal taste.
3. Assign **weights** that sum to 1 (or 100%), agreed with stakeholders.
4. **Score** each option against each criterion on a fixed scale, e.g. 1–5, with a one-line justification per score.
5. Compute weighted totals.
6. Run a **sensitivity check**: does the winner change if a weight moves by ±10 percentage points? If it does, the decision is close and the deciding factor should be discussed explicitly.
7. Check **reversibility** before committing (below).

This is a lightweight form of the concept-selection matrices used in engineering design, such as Pugh's method [Pugh 1991].

FORMULA

Weighted decision score

$$S_j = \sum_{i=1}^{n} w_i \times s_{ij}, \qquad \sum_{i=1}^{n} w_i = 1$$

Symbol	Meaning	Units
S_j	Total weighted score of option j	score points (same scale as s_{ij})
w_i	Weight of criterion i	fraction (0–1)
s_{ij}	Score of option j on criterion i	score points, e.g. 1–5
n	Number of criteria	count

WORKED EXAMPLE

Worked example — Modular monolith or microservices for QuickBite v1? The team is eight engineers and must launch in four months. Criteria and weights come from Table 2.5 and from business constraints.

Table 2.12 — Weighted decision matrix for QuickBite's deployment architecture.

Criterion	Weight	Modular monolith	Microservices (8 services)
Time to launch with 8 engineers	0.30	5	2
Operational simplicity	0.20	5	2
Independent scaling of hot paths	0.20	3	5
Failure isolation	0.15	3	4
Team autonomy as headcount grows	0.15	3	5
Weighted total	**1.00**	**3.90**	**3.35**

Arithmetic for the monolith: 0.30×5 + 0.20×5 + 0.20×3 + 0.15×3 + 0.15×3 = 1.50 + 1.00 + 0.60 + 0.45 + 0.45 = **3.90**. For microservices: 0.30×2 + 0.20×2 + 0.20×5 + 0.15×4 + 0.15×5 = 0.60 + 0.40 + 1.00 + 0.60 + 0.75 = **3.35**.

Sensitivity. Move 10 points of weight from time-to-launch to independent scaling (0.20 and 0.30). Monolith: 0.20×5 + 0.20×5 + 0.30×3 + 0.45 + 0.45 = 1.00 + 1.00 + 0.90 + 0.90 = **3.80**. Microservices: 0.20×2 + 0.20×2 + 0.30×5 + 0.60 + 0.75 = 0.40 + 0.40 + 1.50 + 1.35 = **3.65**. The monolith still wins, so the result is robust to a reasonable disagreement about weights.

A hybrid surfaces. The one criterion where the monolith loses badly is independent scaling, and §2.11 shows that the location path dominates write load. Deploying the **location service separately** and everything else as a modular monolith captures most of the scaling benefit at a fraction of the operational cost.

Interpretation. The matrix did not "decide". It made the reasoning inspectable and revealed a better third option. **Engineering implication:** record the matrix and the sensitivity result in the ADR, and name the trigger for revisiting it, for example "more than 30 engineers or more than 3 teams blocked on shared releases."

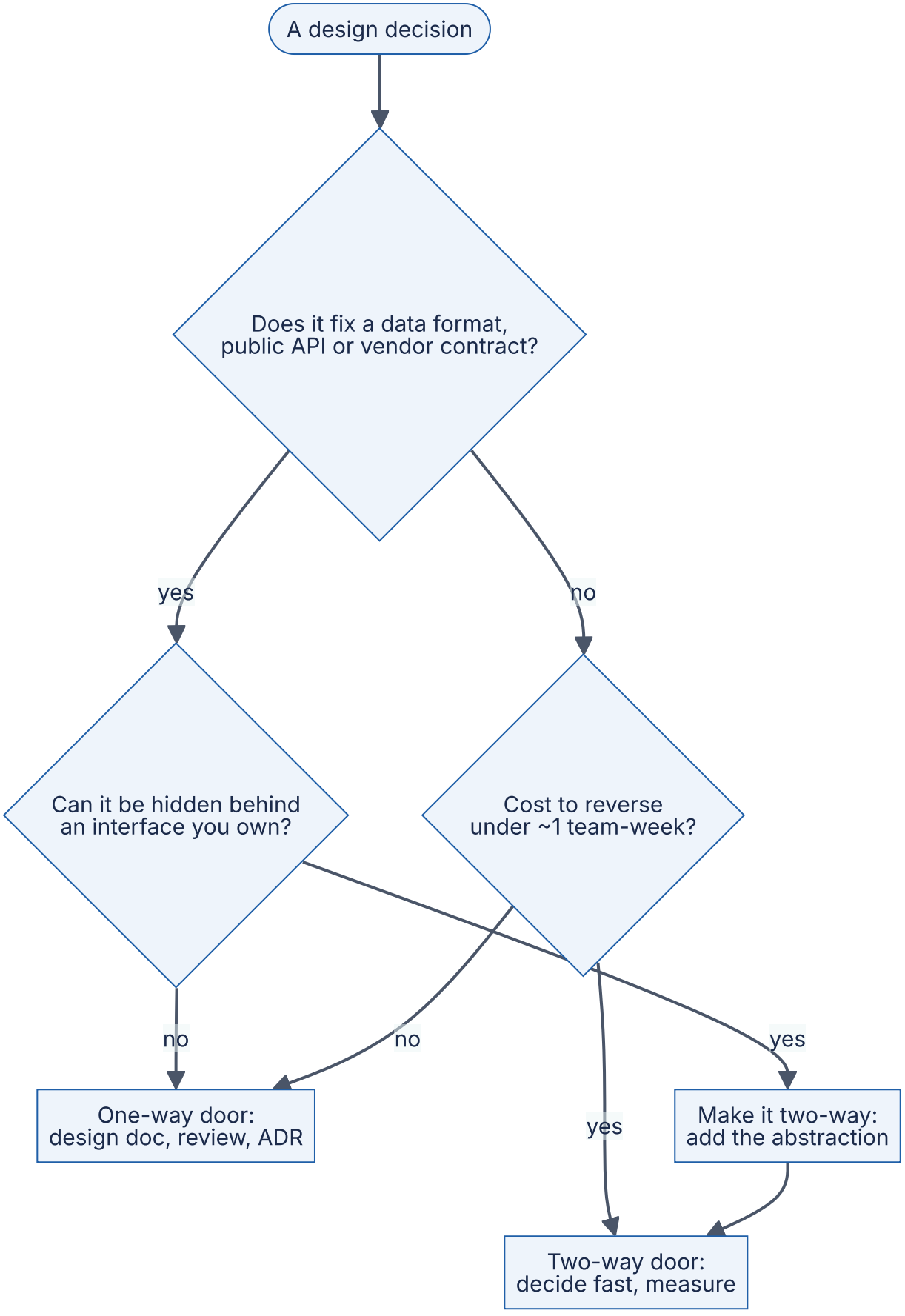
2.8.2 Reversibility: one-way and two-way doors

The matrix measures *fit*. It says nothing about the *cost of being wrong*. In his 2015 letter to shareholders, Jeff Bezos distinguished **Type 1** decisions, which are consequential and irreversible or nearly so ("one-way doors") and deserve slow, careful deliberation, from

Type 2 decisions, which are reversible (“two-way doors”) and should be made quickly by individuals or small groups with good judgment. He warned that large organizations tend to apply the heavyweight Type 1 process to Type 2 decisions, and that this causes slowness and too little experimentation [Bezos 2016].

In architecture, reversibility is mostly about **data and contracts**. Code behind a stable interface is cheap to change. Data formats that have accumulated years of records, and public APIs that thousands of clients depend on, are not.

Figure 2.5 — Choosing the decision process by reversibility.


A design decision
Does it fix a data format, public API or vendor contract?
yes
no
Can it be hidden behind an interface you own?
Cost to reverse under ~1 team-week?
no
no
yes
yes
One-way door: design doc, review, ADR
Make it two-way: add the abstraction
Two-way door: decide fast, measure

Reading the diagram. A decision (stadium) passes through up to three questions (diamonds). The first asks whether the decision fixes something that is costly to change later: stored data formats, public APIs, multi-year vendor contracts. If it does, the second asks whether an interface you own can hide it. A repository layer in front of the database, or an internal payment service in front of the payment provider, turns a one-way door into a two-way one (box T2). That is often the best move available. Decisions that fix nothing durable, and are cheap to reverse, are two-way doors: decide quickly and let measurement correct you. Everything else is a one-way door and earns the full process: a design document, a review and an ADR (§2.12). The "~1 team-week" threshold is a heuristic. Calibrate it to your organization. There is no software data flow here. The failure mode this procedure prevents is organizational: spending weeks debating a two-way door, or rushing a one-way door.

Table 2.13 — QuickBite decisions classified by reversibility.

Decision	Door	Why
Web framework for the catalog service	Two-way	Internal; can be rewritten behind the same API
Cache eviction policy	Two-way	A configuration change
Primary key and ID format for orders	One-way	Embedded in receipts, URLs, partner systems, years of data
Public order-status API shape for restaurant POS partners	One-way	External clients depend on it; versioning needed (§8.15)
Payment provider	One-way unless abstracted	Stored payment tokens are provider-specific
Monolith vs separate location service	Mostly two-way	Modular boundaries make extraction later feasible

INDUSTRY EXAMPLE

Amazon's Type 1 / Type 2 decisions. The two-door model comes from Amazon's 2015 shareholder letter. Bezos presented it as a way for a growing company to keep moving fast: reserve heavyweight deliberation for irreversible choices and push reversible ones down to small groups. *Evidence: Primary* — [Bezos 2016]

Alternatives, failure scenarios and misconceptions

Alternatives to a weighted matrix include simple **pros/cons lists** (fine for two-way doors), **Pugh matrices** (score relative to a baseline option, [Pugh 1991]) and formal methods such as the **Architecture Tradeoff Analysis Method (ATAM)** for large systems, which ties quality-attribute scenarios to architectural decisions [Bass et al. 2021].

Table 2.14 — Trade-off evaluation failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Matrix theatre	Scores reverse-engineered to justify a choice already made	Justifications are vague	Re-score independently	Have two people score blind
Missing option	The best option was never listed	A reviewer proposes it late	Add and re-score	Always include "simplest thing" and a hybrid
Analysis paralysis on a two-way door	One process for every decision	Weeks of debate on reversible items	Time-box; pick and measure	Classify reversibility first
Rushed one-way door	Treated like a quick choice	Expensive migration later	Migration project	Figure 2.5 before deciding

COMMON MISTAKE

"There is a best architecture." There is only a best architecture *for a set of weighted requirements and constraints*. Change the team size, the deadline or the load, and the winner changes. This is why the requirements and assumptions behind a decision must be recorded alongside it.

2.9 Designing for failure

2.9.1 Failure as a normal event

DEFINITION

Definition — Designing for failure. Treating component failure as a *normal, expected event* rather than an exception. For every component and dependency, the design states what happens when it is slow, returns errors or disappears, and makes sure the **blast radius** (the set of users and features affected) is as small as possible.

First-principles explanation

Aircraft are not designed on the assumption that engines never fail. They are designed to fly on one engine. Buildings have fire doors that confine a fire to one section. Designing for failure applies the same idea to software: accept that every part *will* fail at some point, and decide in advance how the rest of the system keeps working.

Why does this exist?

At scale, failure is a matter of arithmetic, not bad luck. With 1,000 servers that each fail on average once every three years, the fleet sees roughly one failure a day ($1{,}000 / (3 \times 365) \approx 0.9$). Networks drop packets, disks wear out, deploys ship bugs and third parties

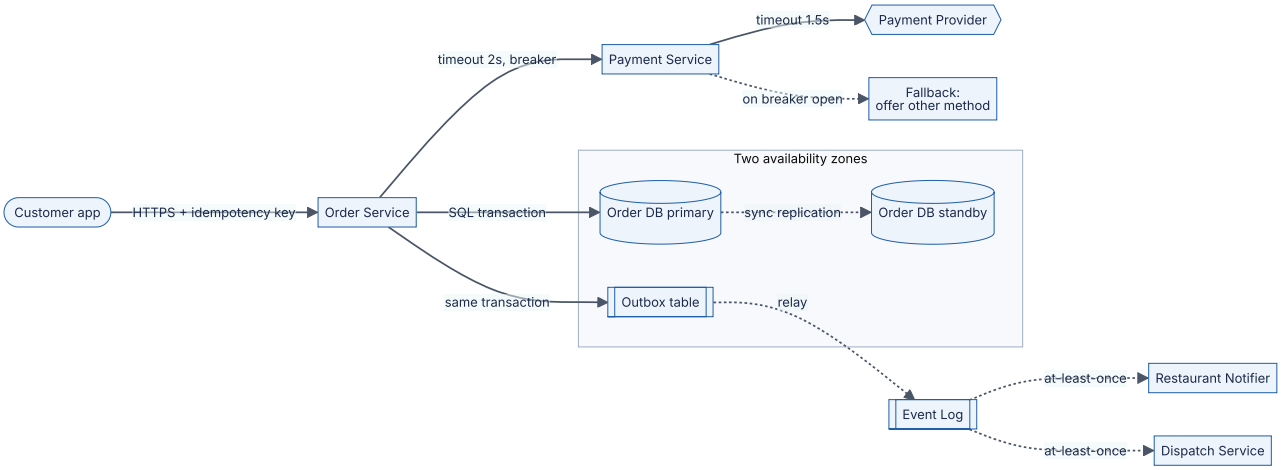
have outages. James Hamilton's widely cited guidance for internet-scale services starts from this premise: expect failures, and design every component so that failures are handled automatically and without drama [Hamilton 2007]. The "fallacies of distributed computing," such as "the network is reliable," catalog the assumptions that make systems brittle (§24.5).

How it works internally

A failure-mode walk-through, sometimes called a lightweight **failure mode and effects analysis (FMEA)**:

1. For each component and each arrow in the decomposition diagram, ask: *what if it is down? Slow? Returning wrong data?*
2. Decide the **desired behavior**: fail fast, retry, degrade gracefully or queue for later.
3. Apply standard mechanisms: **timeouts** on every remote call, **retries with backoff and jitter** only for idempotent operations, **circuit breakers** to stop calling a failing dependency, **bulkheads** to isolate resource pools (all in Chapter 22), **redundancy** across zones (§39.4) and **asynchrony** to take non-essential work off the critical path.
4. Remove things from the **critical path**, the chain of synchronous calls a user must wait for. Each item removed from it raises availability (formula below).

Figure 2.6 — QuickBite checkout designed for failure.



Reading the diagram. The customer's request carries an **idempotency key**, so a retry after a network error cannot create a second order or a second charge (requirement N4). The synchronous critical path is customer → order service → payment service → payment provider. Each hop has an explicit **timeout**, and the payment call sits behind a **circuit breaker**. If the provider is failing, the breaker opens and the customer is offered another payment method within milliseconds, instead of waiting for timeouts (dotted fallback). The order database runs inside a subgraph spanning two **availability zones**: a primary with a synchronously replicated standby, which meets N3 (RPO = 0). Instead of calling

the restaurant notifier and dispatch directly, the order service writes an event to an **outbox table** in the same transaction as the order (§32.5). A relay then publishes it to the event log. Restaurant notification and dispatch are therefore *off the critical path*: if either is down, orders still succeed, and the events are delivered when they recover (at-least-once, so consumers must be idempotent, §32.10). Remaining single points of failure to watch: the payment provider (mitigated by the fallback, or by a second provider later) and the event-log relay (mitigated by running two relays).

FORMULA

Availability of components in series

$$A_{series} = \prod_{i=1}^{n} A_i$$

Symbol	Meaning	Units
A_{series}	Availability of a request path that needs *all* n components	fraction (0–1)
A_i	Availability of component i	fraction (0–1)
n	Number of components on the synchronous path	count

This assumes independent failures. Parallel (redundant) components and correlated failures are treated in §4.14.

WORKED EXAMPLE

Worked example — Taking the restaurant notifier off the critical path. Assumed monthly availabilities: order service 99.95%, order DB 99.95%, payment provider 99.9%, restaurant notifier 99.9%.

1. Notifier called synchronously during checkout: 0.9995 × 0.9995 × 0.999 × 0.999 = 0.9990 × 0.9980 ≈ **0.9970**, i.e. 99.70%. That allows 0.003 × 43,200 ≈ **130 minutes** of checkout failure per month.
2. Notifier asynchronous via the outbox: 0.9995 × 0.9995 × 0.999 ≈ 0.9990 × 0.999 ≈ **0.9980**, i.e. 99.80%, about **86 minutes** per month.
3. Requirement N1 is 99.95%, or 21.6 minutes. Even the improved path misses it, because the payment provider alone allows 43 minutes.

Interpretation. A series chain can never be more available than its weakest link, and every extra synchronous dependency lowers the total. **Engineering implication:** either renegotiate N1 to exclude provider outages (a common SLO definition), or add a second payment provider in parallel. With two independent providers at 99.9% each, the pair is unavailable only when both fail: 1 − (0.001 × 0.001) = 99.9999%. This is a one-way-door-sized decision for a start-up (contracts, token migration), so it goes through Figure 2.5.

INDUSTRY EXAMPLE

Netflix's Chaos Monkey. Netflix publicly described the "Simian Army," starting with Chaos Monkey, a tool that randomly terminates production instances during business hours. Because engineers know instances will disappear, they must build services that tolerate it, and weaknesses are found while engineers are at work rather than at 3 a.m. This is designing for failure enforced by continuous practice (§39.11). *Evidence: Primary* — [Izrailevsky and Tseitlin 2011]

Table 2.15 — Failure scenarios for QuickBite checkout.

What goes wrong	Why	Detection	Recovery	Prevention
Payment provider outage	External dependency	Error rate on provider calls; breaker opens	Fallback method; queue payment capture	Second provider; breaker
Duplicate order on retry	Client retried after a timeout	Two orders with the same cart and customer	Refund the duplicate	Idempotency key (N4)
Order DB primary lost	Hardware or zone failure	Health checks, replication monitor	Automatic failover to standby	Synchronous standby in another zone
Event relay stalls	Bug or crash	Outbox backlog age metric	Restart; events retained in outbox	Two relays; alert on backlog age
Retry storm	Many clients retry at once	Load spikes as success falls	Shed load, open breakers	Exponential backoff with jitter, retry budgets

COMMON MISTAKE

"Redundancy alone makes a system reliable." Two replicas that share a power supply, a configuration push or a software bug fail together. Redundancy only helps against *independent* failures, so look for correlated ones: same zone, same deploy, same dependency.

2.10 Reasoning about scale

2.10.1 Orders of magnitude

DEFINITION

Definition — Order of magnitude. A factor of ten. Two quantities differ by k orders of magnitude when their ratio is about 10^k. Reasoning in orders of magnitude means estimating only the *power of ten* of a quantity (is it 10, 1,000 or 1,000,000 requests per second?) because that, not the exact value, decides the architecture.

First-principles explanation

Cooking dinner for 4 people, for 40 and for 4,000 are not the same task done more times. For 4 you use your home kitchen. For 40 you need a rented hall, borrowed pans and helpers. For 4,000 you need an industrial kitchen, refrigerated trucks, food-safety inspections and shift managers. Each 10× step changes the *kind* of solution, not just its size. Software behaves the same way, and this is why experienced designers first ask "what order of magnitude?"

Why does this exist?

Most design choices are insensitive to a factor of 2 but sensitive to a factor of 10. A single PostgreSQL primary comfortably handles QuickBite's 60 orders per second. At 600 it needs careful tuning and read replicas. At 6,000 it likely needs partitioning (Chapter 14). Knowing the order of magnitude tells you which of these architectures you are in, and how far you are from the next one.

How it works internally

1. Estimate the key rates and sizes to within a factor of about 3 (§2.11).
2. Compare them with reference capacities (Table 2.17 and §4.17) to find which tier is closest to a limit.
3. For each 10× step of growth, ask which components hit a *qualitatively* new problem.

Table 2.16 — What changes for QuickBite at each order of magnitude.

Scale	Orders/day	Peak orders/s	What breaks or changes	Typical response
1× (one city)	5,000	~0.6	Nothing; one server and one DB suffice	Modular monolith, managed DB with standby
10× (region)	50,000	~6	Browse reads dominate; menu queries load the DB	Cache menus, CDN for images (Chapters 20–21)
100× (country)	500,000	~60	Location writes (~5,000/s); hot restaurants; payment pool sizing	Separate location store; short transactions; async events
1000× (multi-country)	5,000,000	~600	Single order-DB write limits; data residency laws; multi-region latency	Partition orders by city or region; regional cells (§33.12, Chapter 44)

WORKED EXAMPLE

Worked example — Where does QuickBite's single order database run out? From §2.6 the WAL disk caps the order DB at about 500 orders/s, assuming 2 ms of fsync demand per order.

1. At 100× (today's design point) peak is 60 orders/s: utilization = 60 / 500 = 12%.
2. At 1000× peak is 600 orders/s: 600 / 500 = 120%, so the database saturates.
3. Group commit, which flushes many transactions in one fsync, might cut per-order disk demand 4× (an assumed factor), giving 2,000 orders/s capacity and 30% utilization at 1000×. That buys roughly one more order of magnitude.

Interpretation. The current design has about 8× headroom on its most constrained resource. **Engineering implication:** no partitioning work is needed now (that would be over-engineering), but partitioning by city should be *designed* before order volume reaches about 5× today's level, giving time to build it.

INDUSTRY EXAMPLE

"Design for 10× growth, plan to rewrite before 100×." Jeff Dean of Google has given talks on building large distributed systems in which he advises designing for about ten times the current scale while expecting a significant redesign before about a hundred times. The point is that one architecture rarely spans more than one or two orders of magnitude gracefully. *Evidence: Secondary* — talk slides; see Open verification items. [Dean 2009]

COMMON MISTAKE

"Design for Google scale from day one." Designing QuickBite v1 for 1000× its launch load would cost months of extra work, and the assumptions it relied on would likely be wrong by the time that scale arrived. Design for the next order of magnitude, keep the one after that *possible*, and use tripwires (§2.7) to know when to start.

2.11 Back-of-the-envelope capacity estimation

2.11.1 The estimation method

DEFINITION

Definition — Back-of-the-envelope estimation. A quick calculation, done with round numbers and stated assumptions, that gives the order of magnitude of a system's load and resource needs (requests per second, storage, bandwidth, memory, number of nodes). The aim is to be within a factor of about 2–3 of the truth with a few minutes of arithmetic. This section introduces the method. Chapter 89 is the full handbook.

First-principles explanation

Enrico Fermi famously estimated quantities such as the number of piano tuners in Chicago by multiplying a chain of reasonable guesses: population, households with pianos, tunings per year, tunings a tuner can do. The individual errors partly cancel out, and the answer lands near the right power of ten. Capacity estimation is the same technique applied to users, requests and bytes.

Why does this exist?

Estimation tells you *which architecture you are in* (§2.10) before you spend money. It also exposes surprises early. In QuickBite's case, the largest write load turns out to be courier GPS updates, not orders, a fact that reshapes the decomposition.

How it works internally

1. Write down the assumptions (they come from the register in §2.7).
2. Convert daily volumes to an **average rate**, then to a **peak rate**.
3. Multiply by **per-request sizes** to get bandwidth, and by **retention** and **copies** to get storage.
4. Divide peak load by **per-node capacity at a target utilization** to get node counts, then add spare capacity for failures.
5. Round sensibly, compare against reference capacities, and name the dominant cost or bottleneck.

FORMULA

Average and peak request rate

$$R_{avg} = \frac{U \times a}{86,400} \qquad R_{peak} = \frac{U \times a \times h}{3,600} \times b$$

Symbol	Meaning	Units
R_{avg}	Average request rate over a day	requests/s
R_{peak}	Design peak request rate	requests/s
U	Daily active users	users
a	Actions of this type per user per day	requests/user/day
h	Share of daily volume in the busiest hour	fraction
b	Burst factor above the busiest-hour average	dimensionless
86,400 / 3,600	Seconds per day / per hour	s

FORMULA

Storage and node count

$$S = n_{day} \times s \times T \times k \qquad N = \lceil \frac{R_{peak}}{c \times u} \rceil + N_{spare}$$

Symbol	Meaning	Units
S	Total storage needed	bytes
n_{day}	Records created per day	records/day
s	Size per record, including indexes and overhead	bytes
T	Retention period	days
k	Number of stored copies (replication)	dimensionless
N	Nodes needed	nodes
c	Throughput one node sustains	requests/s
u	Target utilization, leaving head-room for queueing (§4.10)	fraction
N_{spare}	Extra nodes so a failure or deploy does not overload the rest	nodes

Table 2.17 — Rough reference capacities for estimation (order of magnitude only, as of 2026).

Resource	Rough capacity	Note
Round trip within one datacenter	~0.5 ms	See §4.17 and Appendix F for dated values
Round trip across a continent	~50–100 ms	Bounded by the speed of light in fiber
Stateless API node, simple JSON requests	~10^3 RPS	Varies by 10× with language and work per request
Relational DB primary, simple indexed writes	~10^3–10^4 writes/s	Depends on fsync, schema, hardware
In-memory key-value store node	~10^5 ops/s	Small values, pipelining helps
1 Gbps network link	125 MB/s	1 Gbps ÷ 8 bits per byte

These numbers are deliberately rough. Measure your own system before relying on any of them.

WORKED EXAMPLE

Worked example — Full capacity estimate for QuickBite (the 100× design point). SI units throughout (1 KB = 10^3 bytes).

Assumptions (from Table 2.10, plus new ones marked "new"). U = 2,000,000 DAU. Orders a = 0.25 per DAU per day. Browse reads 30 per DAU per day (new). Busiest-hour share h = 0.20 for both. Burst factor b = 2 (new; covers promotions). 20,000 couriers active at peak, updating every 4 s. Order record 2 KB including items and status history (new). Average browse response 20 KB with images served separately by a CDN (new).

Step 1 — Orders. - Daily: 2,000,000 × 0.25 = **500,000 orders/day**. - Average: 500,000 / 86,400 ≈ **5.8 orders/s**. - Busiest hour: 500,000 × 0.20 = 100,000 orders in one hour; 100,000 / 3,600 ≈ 27.8 orders/s. - Design peak: 27.8 × 2 ≈ 55.6, rounded up to **60 orders/s**. This is the figure used in §2.6.

Step 2 — Browse reads. - Daily: 2,000,000 × 30 = 60,000,000 reads/day. Average: 60,000,000 / 86,400 ≈ **694 RPS**. - Busiest hour: 60,000,000 × 0.20 / 3,600 ≈ 3,333 RPS. Design peak: × 2 ≈ **7,000 RPS** (rounded up). - Read-to-order ratio at peak: 7,000 / 60 ≈ 117:1. Browsing is read-heavy and highly cacheable.

Step 3 — Courier locations and tracking. - Writes at peak: 20,000 / 4 s = **5,000 writes/s**. That is about 83× the order write rate, so this is the dominant write path. - Orders in the delivery leg at once, by Little's Law (§2.6): arrivals 27.8 orders/s × 900 s average delivery leg (15 min, new assumption) ≈ **25,000 orders**. - Tracking reads: if half of those customers have the app open and it refreshes every 5 s, 25,000 × 0.5 / 5 = **2,500 reads/s**.

Step 4 — Storage. - Orders: 500,000 × 2 KB = 1 GB/day. Over 5 years: 1 GB × 365 × 5 ≈ 1.8 TB. Add 50% for indexes: ≈ 2.7 TB. Two copies (primary + standby): **≈ 5.5 TB**. One database server can hold this. - Locations: assume 6,000 couriers active on average across the day, giving 6,000 / 4 = 1,500 writes/s. Per day: 1,500 × 86,400 = 129.6 million points. At 100 bytes each: ≈ 13 GB/day. Keep 30 days hot: 13 × 30 ≈ **390 GB**, then downsample to an archive. - Menus: 50,000 restaurants × 100 items × 1 KB = **5 GB**, small enough to cache entirely in memory (§20.3).

Step 5 — Bandwidth. - Browse egress at peak: 7,000 × 20 KB = 140 MB/s = 140 × 8 = **1.12 Gbps**, which needs more than one 1 Gbps link or a load balancer tier with 10 Gbps links. - Location ingress: 5,000 × 200 bytes = 1 MB/s = **8 Mbps**, negligible.

Step 6 — API nodes for browsing. With c = 1,000 RPS per node, u = 0.6 and N_spare = 2 (one zone or one deploy batch down): 7,000 / (1,000 × 0.6) = 11.7, rounded up to 12, plus 2 = **14 nodes**.

Step 7 — Interpretation.

Table 2.18 — QuickBite estimate summary (100× design point).

Quantity	Value	Dominant consequence
Peak orders	60/s	One relational primary is enough (§2.6: ~500/s limit)
Peak browse reads	7,000 RPS	Cache menus; ~14 API nodes
Peak location writes	5,000/s	Separate, in-memory, partitioned location store
Tracking reads	2,500/s	Serve from the location store, not the order DB
Order storage (5 years, 2 copies)	~5.5 TB	Single DB cluster fine; plan partitioning at ~5×
Location storage (30 days hot)	~390 GB	Time-partitioned; expire old data with a TTL
Peak egress (API, excluding images)	~1.1 Gbps	CDN for images; 10 Gbps load balancer links

Engineering implication. The estimate confirms the decomposition in Figure 2.2. It also justifies the one deviation from the monolith in §2.8: location must scale independently because it carries about 83× the write rate of orders. Nothing here needs partitioning yet, and that is as important to know as what does.

COMMON MISTAKE

"Estimation needs precise numbers." Round aggressively: 86,400 ≈ 10^5 seconds per day is fine for a first pass. What matters is stating assumptions, keeping units consistent and reaching the right power of ten. Spending ten minutes on the third significant digit while never computing peak load is the classic mistake.

2.12 Architecture documentation

2.12.1 Structure and rationale

DEFINITION

Definition — Architecture documentation. The written and diagrammed record of a system's structure and of the decisions that shaped it. It is written so that people who were not in the room (new team members, reviewers, your future self) can understand *what* the system is and *why* it is that way.

First-principles explanation

A map of a city at one scale cannot answer every question. You need a country map to see where the city is, a city map to see districts, and a street map to find a door. Architecture documentation works the same way: diagrams at several zoom levels for *structure*, and a logbook of decisions for *reasons*. Without the logbook, the next team sees a strange design choice and "fixes" it, reintroducing the problem it solved.

Why does this exist?

Knowledge in heads leaves when people do. Design decisions also tend to be revisited endlessly when their reasons are not written down. Three lightweight artifacts cover most needs:

Table 2.19 — Three architecture documentation artifacts.

Artifact	Answers	Written when	Lifetime
C4 diagrams	What are the parts and how do they connect, at each zoom level?	Before building; updated as the system changes	Living
Design document	What problem, what design, what alternatives, what risks?	Before a significant piece of work	Snapshot, then archived
Architecture Decision Record (ADR)	What did we decide, in what context, with what consequences?	At each significant decision	Immutable; superseded, never edited

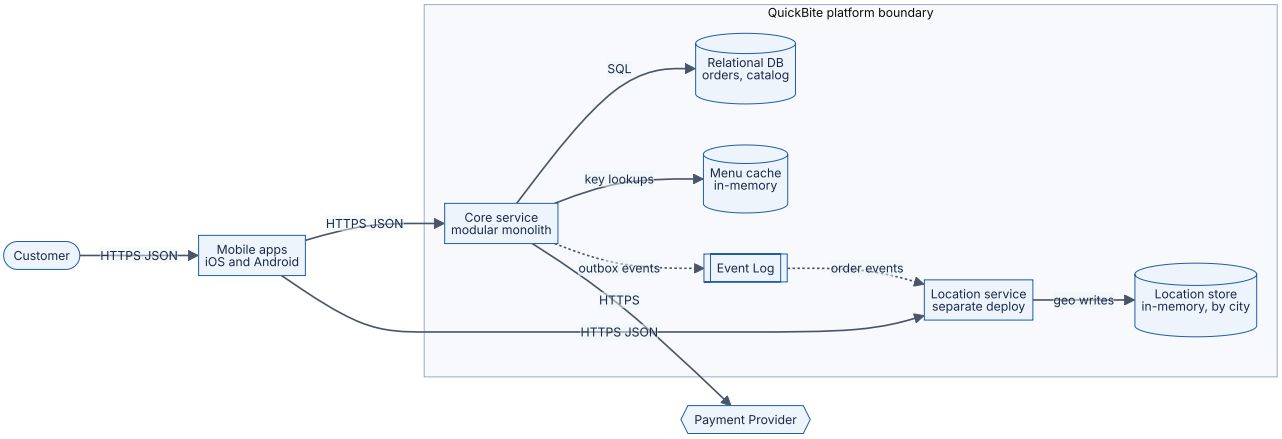
2.12.2 The C4 model

The **C4 model**, created by Simon Brown, describes a software system at four zoom levels [Brown C4]:

1. **Context.** The system as one box, with its users and external systems (Figure 2.3).
2. **Containers.** The separately deployable or runnable units (applications, databases, queues) and the technologies and protocols between them. In C4, "container" means a runtime unit, not specifically a Docker container.
3. **Components.** The major building blocks inside one container.
4. **Code.** Classes and functions. This level is usually generated from code or skipped.

This book draws C4 views with standard flowchart diagrams rather than an experimental C4 diagram syntax (see the style guide).

Figure 2.7 — QuickBite C4 container view, after the §2.8 decision.



Reading the diagram. This is C4 level 2 for QuickBite after the trade-off decision in §2.8. The **mobile apps** are containers too, because they are separately deployed runtime units. The subgraph is the platform's deployment boundary, matching the single box in Figure 2.3. The **core service** is the modular monolith: catalog, orders, payments, dispatch and notifications run in one process with internal module boundaries from Table 2.1. It talks SQL to the relational database (storage boundary for orders and catalog) and does key lookups against an in-memory **menu cache**, which keeps 7,000 browse RPS off the database (§2.11). The **location service** is deployed separately because of its 5,000 writes/s. It stores positions in an in-memory store partitioned by city and learns which courier is on which order from asynchronous order events (dotted). Synchronous paths (solid) are user-facing. Asynchronous paths (dotted) tolerate delay. Failure points match §2.9: the payment provider (external), the relational DB (standby in a second zone) and the event log. Scaling options: add core-service nodes horizontally, add location-store partitions per city, and later extract further modules into services if the tripwires from §2.7 fire.

2.12.3 Design documents

A **design document** (also called an RFC or a tech spec) is written *before* significant work, so that reviewers can find problems while changes are still cheap. Google engineers describe a widely copied structure: context and scope, goals and non-goals, the design itself, alternatives considered, and cross-cutting concerns such as security, privacy and observability [Ubl 2020]. A minimal skeleton:

```
Title, author(s), reviewers, status, date
1. Context and scope        - what exists today, what problem we solve
2. Goals and non-goals      - testable requirements (Table 2.5 style)
3. Assumptions              - register with confidence and tripwires (Table 2.10)
4. Estimates                - capacity numbers (Table 2.18)
5. Design                   - C4 context and container diagrams; APIs; data model
6. Alternatives considered  - decision matrix, reversibility (Table 2.12, Figure 2.5)
7. Failure modes            - FMEA table (Table 2.15)
8. Cross-cutting concerns   - security, privacy, observability, cost
9. Rollout and migration    - phases, flags, rollback plan
10. Open questions
```

What to notice:

- Sections 2–7 map directly onto the thinking tools of this chapter. A design document is the written output of the thinking loop in Figure 2.1.
- “Non-goals” and “Alternatives considered” are the sections reviewers value most, because they show what was rejected and why.

2.12.4 Architecture Decision Records

An **Architecture Decision Record (ADR)** is a short text file recording *one* significant decision. Michael Nygard’s original format has five parts: title, status, context, decision and consequences [Nygard 2011]. ADRs are numbered, stored in the code repository next to the code they affect, and never edited after acceptance. A changed decision gets a *new* ADR that **supersedes** the old one, so the history of reasoning is preserved.

```
ADR-007: Deploy the location service separately from the core monolith

Status: Accepted (2026-10-30). Supersedes: none.

Context: Peak location writes are estimated at 5,000/s, about 83x peak order
writes (design doc section 4). The core service is a modular monolith chosen
by weighted matrix (score 3.90 vs 3.35 for microservices; robust to +/-10 pt
weight changes). Location is the only module whose load profile differs by
more than 10x from the rest.

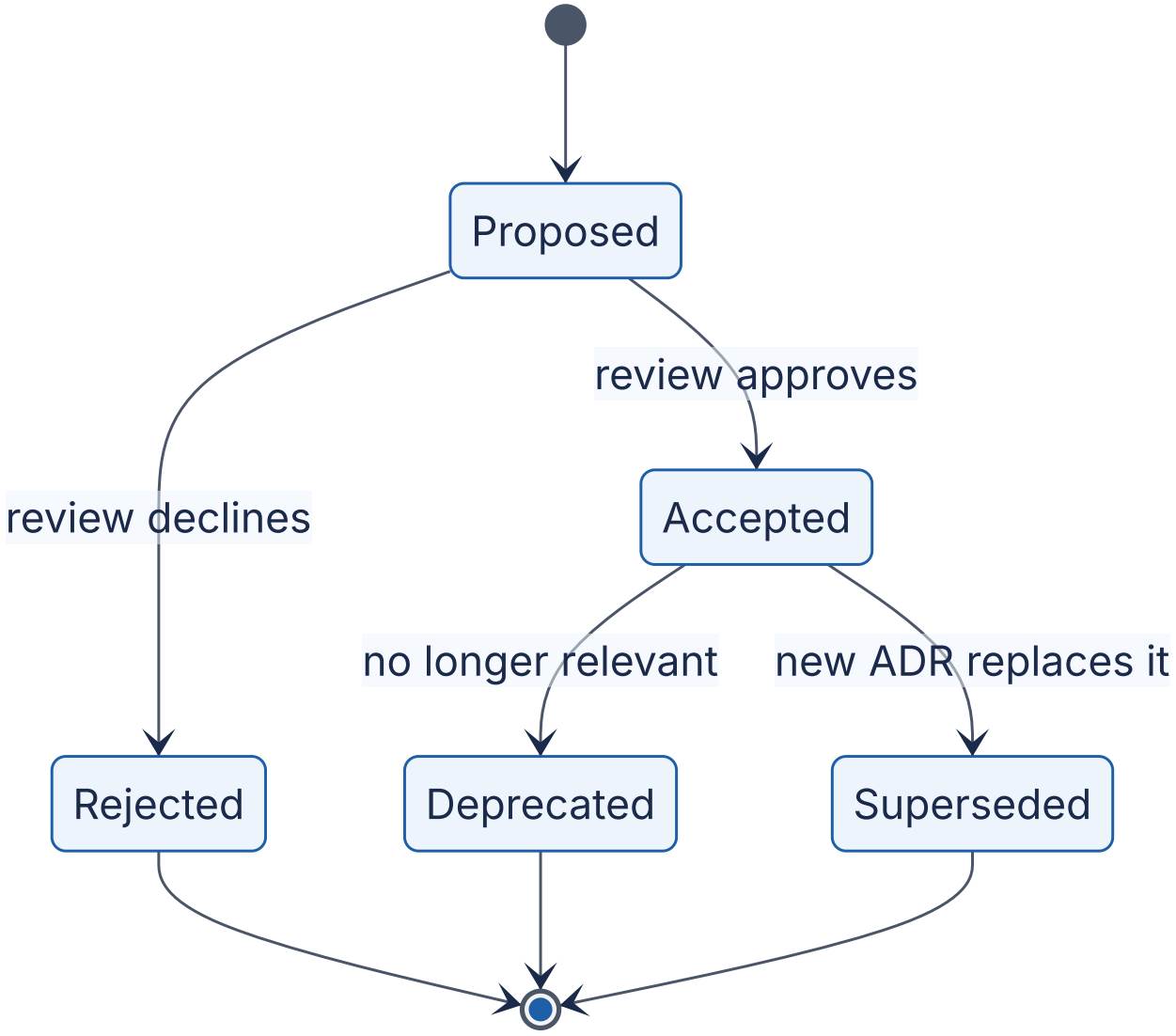
Decision: Run the location module as its own deployable service with its own
in-memory, city-partitioned store. All other modules remain in the monolith.

Consequences: (+) location scales independently; a location outage cannot
block checkout. (-) one more deployment pipeline and on-call surface; courier
assignment data reaches it asynchronously, with seconds of lag.
Revisit if: engineering headcount exceeds 30, or another module's peak load
exceeds 1,000 writes/s.
```

What to notice:

- The **context** carries the numbers and assumptions. Without them, a future reader cannot tell whether the decision is still valid.
- **Consequences** list the downsides honestly. An ADR with only positives is a sales pitch, not a record.
- “Revisit if” turns the assumption tripwires of §2.7 into an explicit trigger.

Figure 2.8 — ADR life cycle.



Reading the diagram. Each state is the *status* field of an ADR, not a running software component. An ADR starts as **Proposed**, often in a pull request, so that review happens with the usual code-review tooling. Review moves it to **Accepted** or **Rejected**. Rejected ADRs are kept, because knowing an option was considered and declined saves future debate. An accepted ADR is never rewritten. When circumstances change it becomes **Superseded**, with a link to the new ADR, or **Deprecated** if the thing it governed no longer exists. All transitions are human, asynchronous actions. The failure mode is decay:

ADRs that are never updated to Superseded make the record misleading, so check ADR status during design reviews.

INDUSTRY EXAMPLE

ADRs and C4 in practice. Nygard's 2011 post proposed ADRs as a lightweight alternative to large architecture documents that nobody reads, and the format has since spread widely, with open-source templates and tooling. Simon Brown's C4 model is likewise published openly with notation guidance and is used to standardize diagrams across teams. *Evidence: Primary* — [Nygard 2011], [Brown C4]

Table 2.20 — Documentation failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Diagram drift	Diagrams not updated with the code	New joiners find components missing	Redraw from the running system	Diagrams in the repo; review with code changes
Lost rationale	Decisions made in chat or meetings	"Why is it like this?" has no answer	Write retroactive ADRs	ADR required for one-way doors
Unreadable tome	80-page document nobody reads	No review comments	Split into C4 views plus ADRs	Keep docs short; link out to details

COMMON MISTAKE

"Documentation slows us down." Writing an ADR takes about half an hour. Re-arguing an undocumented decision, or reversing it by accident, costs days. Document one-way doors always and two-way doors only when they are surprising.

2.13 Summary and revision notes

SUMMARY

- **Design is a loop, not a line** (Figure 2.1): requirements → boundary → assumptions → estimates → decomposition → bottlenecks → failure → trade-offs → documentation, repeated at higher resolution until the design is good enough *for the risk*.
- **Decompose** by business capability, data ownership, load profile and likely change (information hiding). One sentence of responsibility per component. One owner per data store.
- **Draw the boundary first.** External systems bring availability, latency, rate limits and costs you do not control. QuickBite's maps contract ruled out per-listing route calls before any code existed.
- **Turn adjectives into numbers.** "Never down" became 99.95% for checkout (21.6 min/month). "Real-time" became a 5 s refresh with at most 10 s staleness.
- **The bottleneck is the resource with the largest demand per unit of capacity.** Check CPU, memory, disk, network *and* software resources (locks, pools). Utilization law: $U = X \times D/c$. Little's Law: $L = \lambda W$. QuickBite's checkout was limited by a row lock held across a payment call, not by hardware.
- **State assumptions** with source, confidence, sensitivity and a tripwire. A load-bearing assumption is one where a 10× error changes the design.
- **Evaluate trade-offs** with a weighted matrix plus a sensitivity check. Then classify reversibility: one-way doors get deliberation and an ADR, two-way doors get speed.
- **Design for failure.** Timeouts, idempotent retries, circuit breakers, redundancy across zones, and moving work off the critical path. A series path's availability is the product of its parts.
- **Think in orders of magnitude.** Each 10× brings qualitatively new problems. Design for the next 10×, keep the following one possible.
- **Estimate on the back of an envelope.** Average → peak → bandwidth → storage → nodes. QuickBite's heaviest write path is courier location (5,000/s), about 83× its order writes.
- **Document** structure with C4 (context, containers, components, code), plans with design docs, and decisions with immutable, superseding ADRs.

2.14 Knowledge checkpoint

1. Name four criteria for choosing component boundaries.
2. What does a context diagram deliberately leave out, and why?
3. Convert "the site must be fast" into a testable requirement.

4. How many minutes of downtime per 30-day month does 99.95% allow?
5. Which five resource categories make up the resource model?
6. A database is at 20% CPU but requests time out waiting. What class of bottleneck is most likely?
7. Each request holds a pooled connection for 250 ms and 100 requests/s arrive. How many connections are busy on average?
8. What makes an assumption “load-bearing”?
9. Why run a sensitivity check on a weighted decision matrix?
10. Give one example each of a one-way door and a two-way door in system design.
11. Three components in series each have 99.9% availability. What is the path’s availability?
12. 1 million DAU each perform 50 reads per day, 15% of them in the busiest hour. What is the busiest-hour read rate?
13. What are the four levels of the C4 model?
14. Why is an accepted ADR never edited?

Answers

1. Business capability, data ownership, load profile or rate of change, consistency need, and likely future change (information hiding). Any four.
2. The system’s internal structure. It focuses attention on actors, external dependencies and scope, and stays readable for non-technical stakeholders.
3. For example: “p99 latency of the search endpoint below 300 ms, measured over 5-minute windows at up to 7,000 RPS.”
4. $(1 - 0.9995) \times 43{,}200 = 21.6$ minutes.
5. CPU, memory, disk, network, and locks or other software resources (pools, threads, file handles).
6. A software resource: a lock, a connection pool or a thread pool. Requests are waiting while hardware is idle.
7. $L = \lambda W = 100 \times 0.25 = 25$ connections.
8. If it were wrong by about 10×, the design would have to change.
9. To learn whether the winner depends on a contested weight. If small weight changes flip the result, the decision is close and the deciding factor must be discussed explicitly.
10. One-way: the order ID format, a public partner API, the stored data model. Two-way: a cache eviction policy, an internal framework behind a stable API.
11. $0.999^3 \approx 0.997$, i.e. 99.7%.
12. $1{,}000{,}000 \times 50 \times 0.15 / 3{,}600 \approx 2{,}083$ RPS (before any burst factor).
13. Context, Containers, Components, Code.
14. To preserve the history of reasoning. A changed decision gets a new ADR that supersedes the old one, so readers can see what was believed when.

2.15 Interview questions

Basic

Q1. You are asked to “design a food-delivery app.” What do you do in the first five minutes?

Answer. Clarify before designing. Name the users (customers, restaurants, couriers, support) and the top journeys. Agree the functional scope and say explicitly what is out of scope. Then quantify: daily active users, orders per day, peak factor, latency targets per journey, availability for checkout, and consistency needs (“no double charges”). State the assumptions you are making and confirm them with the interviewer. Finish with a quick estimate (orders/s, reads/s, location writes/s) so that the architecture you draw is sized to a target.

Q2. What is a bottleneck, and why does adding servers sometimes not help?

Answer. It is the resource with the largest demand per unit of capacity. It caps throughput for the whole path. Adding capacity anywhere else changes nothing, as with widening a highway before a one-lane bridge. Servers add CPU and memory, but if the limit is a database disk, a hot row lock or a connection pool, throughput stays the same. Find it with the USE method and the utilization law before scaling anything.

Intermediate

Q3. Checkout latency is high at peak but every server’s CPU is below 30%. Walk through your diagnosis.

Answer. Low utilization with high latency points to waiting on a software resource or an external dependency. Look at per-hop latencies in traces. Check pool wait times, lock waits in the database and the latency of external calls. Apply Little’s Law to pools: busy connections = arrival rate × hold time. A common cause is holding a transaction (and therefore a row lock and a connection) across a remote payment call. That gives long hold times, pool exhaustion and hot-row queues. The fix is to shorten the critical section: do the external call outside the transaction, use an idempotency key, and keep transactions to milliseconds. Re-measure afterwards, because the bottleneck moves.

Q4. How do you decide between a modular monolith and microservices for a new product?

Answer. Derive criteria from requirements and constraints: time to market, team size, operational maturity, independent scaling needs, failure isolation. Weight and score them, and run a sensitivity check. For a small team with a deadline, a modular monolith usually wins. Then look for components whose load profile differs by an order of magnitude or

more (QuickBite's location service) and extract only those. Note reversibility: well-kept module boundaries make later extraction a two-way door. Record the decision and its revisit triggers in an ADR.

Advanced

Q5. Your checkout SLO is 99.95%, but the payment provider guarantees only 99.9%. What are your options?

Answer. In series, the path cannot beat its weakest link, so 99.95% is unreachable as stated. Options: (1) redefine the SLO to exclude declared third-party outages, which is common but must be agreed with the business; (2) add a second provider in parallel, giving roughly $1 - 0.001^2$ for independent failures, at the cost of contracts, token handling and reconciliation, a one-way door; (3) accept the order and defer payment capture (authorize later, queue retries) for trusted customers, trading fraud risk for availability; (4) graceful degradation, such as offering alternative payment methods when the breaker opens. Present the costs and reversibility of each and let the business choose.

Q6. QuickBite expands from one country to ten. What changes, and what would you do first?

Answer. That is roughly 10×, so expect qualitative change. Order writes reach ~600/s and saturate the single primary's WAL disk (§2.10). Data residency laws may require in-country storage. Cross-continent round trips of ~100 ms hurt latency. Payment providers differ by country. First, re-run the estimate and assumption register. Then partition by geography into regional cells, each with its own order database and location store, which addresses scale, residency, blast radius and latency together. Keep a thin global layer for identity and routing only. Treat the partition key (region or city) as a one-way door with a full design doc. Migrate incrementally, one country per cell, with rollback plans.

2.16 Practical exercises

1. **Context diagram.** Draw a system-context diagram for a ride-hailing app. List every external system with an assumed SLA, rate limit and latency, and write three out-of-scope statements.
2. **Requirements negotiation.** Rewrite these as testable requirements and propose a cheaper acceptable alternative for each: "instant search", "zero data loss", "global availability".
3. **Bottleneck arithmetic.** Edit the script in §2.6 so that 10% of orders go to the single most popular restaurant. At what total order rate does the hot row saturate with a 320 ms hold time? With 15 ms?
4. **Estimation.** Repeat the §2.11 estimate for 20 M DAU and a 2 s location update interval. Which tier changes order of magnitude first?

5. **ADR.** Write an ADR for QuickBite's choice of an idempotency-key scheme for payments, including consequences and revisit triggers.

2.17 Design challenges

1. **Grocery delivery variant.** QuickBite adds grocery delivery: orders of 40–100 items, with stock levels that change constantly and substitutions. Rework Table 2.1, the assumption register and the bottleneck analysis. Where does inventory consistency create new locks or hot rows, and how do you keep them short?
2. **Holiday surge.** On one public holiday per year, orders spike to 8× the normal dinner peak for three hours. Compare three strategies: over-provision permanently, autoscale, or queue orders with a "your order is being placed" experience. Use a weighted decision matrix and classify each strategy's reversibility. Estimate the cost difference, stating your price assumptions.

2.18 References

1. Bass, L., Clements, P., Kazman, R. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
2. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. https://sre.google/sre-book/table-of-contents/
3. Bezos, J. *2015 Letter to Shareholders*. Amazon.com, Inc., 2016 (published with the 2015 annual report).
4. Brown, S. *The C4 model for visualising software architecture*. c4model.com, accessed 2026. https://c4model.com
5. Conway, M. E. *How Do Committees Invent?* Datamation 14(4), 1968.
6. Dean, J. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, LADIS Workshop, 2009.
7. DeMarco, T. *Structured Analysis and System Specification*. Yourdon Press, 1979.
8. Denning, P. J., Buzen, J. P. *The Operational Analysis of Queueing Network Models*. ACM Computing Surveys 10(3), 1978.
9. Gray, J. *A Conversation with Werner Vogels: Learning from the Amazon Technology Platform*. ACM Queue 4(4), 2006.
10. Gregg, B. *The USE Method*. brendangregg.com. https://www.brendangregg.com/usemethod.html
11. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
12. Hamilton, J. *On Designing and Deploying Internet-Scale Services*. Proceedings of the 21st Large Installation System Administration Conference (LISA '07), USENIX, 2007.

13. Izrailevsky, Y., Tseitlin, A. *The Netflix Simian Army*. Netflix Technology Blog, 2011.
14. Little, J. D. C. *A Proof for the Queuing Formula: $L = \lambda W$*. Operations Research 9(3), 1961.
15. Nygard, M. *Documenting Architecture Decisions*. Cognitect blog, 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
16. Parnas, D. L. *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM 15(12), 1972.
17. Pugh, S. *Total Design: Integrated Methods for Successful Product Engineering*. Addison-Wesley, 1991.
18. Ubl, M. *Design Docs at Google*. Industrial Empathy (blog), 2020. https://www.industrialempathy.com/posts/design-docs-at-google/

2.19 Open verification items

- **[Bezos 2016]** The exact wording and canonical URL (Amazon IR site or SEC exhibit) of the Type 1 / Type 2, “one-way / two-way door” passage. The chapter paraphrases it. sec.gov could not be reached from the authoring session.
- **[Dean 2009]** Confirm that the “design for ~10× growth, plan to rewrite before ~100×” advice appears in the LADIS 2009 keynote slides and not only in other Dean talks (e.g. Stanford, 2007). Hence the label is *Secondary*.
- **[Ubl 2020]** Confirm the publication year and URL. Only secondary summaries were checked.
- **[Gray 2006]** Confirm that “you build it, you run it” and the no-direct-database-access rule appear in this interview.
- **[DeMarco 1979]** The first edition is dated 1978 or 1979 depending on the source.
- **[Izrailevsky and Tseitlin 2011]** Confirm authors and date of the Netflix blog post.
- Table 2.17 capacities and the “99.9–99.99% third-party SLA” range are rough, as of 2026. Reconcile them with Appendix F and §4.17.
- The style guide asks for longer runnable programs in `code/ch02/`. The §2.6 script is short and was not copied there.

Chapter 3 — The System Design Learning Method

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3.5 h

System design is a large field. This book covers it in 93 chapters, and the field keeps changing while you read. No one masters it by reading straight through once. Engineers who become strong designers usually have two things: a deep stock of well-understood building blocks, and a reliable way to add new blocks. This chapter gives you that way. It is a *method*, so it applies to every later chapter, from B-trees (Chapter 11) to LLM inference (Chapter 73).

3.1 Learning objectives

After this chapter you should be able to:

- Apply the ten-step concept-mastery loop to any system-design concept and produce the artifact each step requires.
- Explain, with evidence, why retrieval practice and spaced practice beat rereading and highlighting, and schedule reviews to match.
- Read a systems paper using Keshav's three-pass method and judge an engineering blog post's claims by the strength of its evidence.
- Draw a reference architecture from memory, score it against a checklist, and repair the gaps.
- Scope a simplified implementation that keeps the core mechanism and leaves out production concerns.
- Follow a 12-month beginner path through this book, or build a gap-driven plan if you are already experienced.
- Track your progress with a self-assessment rubric, flashcards and a design journal.

3.2 Prerequisites

- **What system design is** — §1.3–§1.5. You need to know what counts as a "concept" or "building block" before you can learn concepts methodically.
- **Trade-off reasoning** — §2.8. Step 6 of the loop asks you to analyze trade-offs. Chapter 2 gives you the vocabulary.

- **Basic Python** — enough to read a 60-line program with a class, a list and a dictionary. Sections 3.3 and 3.4 contain short runnable programs.
- **Hashing (informal)** — you should know that a hash function maps any input to a fixed-size number. Chapter 17 (§17.3) covers hashing in depth; you do not need it yet.

3.3 The ten-step concept-mastery loop

3.3.1 Why a loop rather than a reading list

> DEFINITION
>
> **Definition — Concept-mastery loop.** A fixed sequence of ten learning activities applied to one system-design concept. Each activity produces a concrete artifact (a definition, a diagram, a program, a design). The concept counts as mastered only when you can rebuild every artifact without notes.

First-principles explanation

Think about learning to cook one dish well. Reading the recipe tells you what the dish is. Knowing why each ingredient is there tells you what goes wrong if you leave one out. Watching a chef shows you technique. But you only really know the dish after you have cooked it yourself, burned it once, fixed it, and then cooked it for someone else without the recipe in front of you. The loop applies the same idea to a concept such as caching or consistent hashing.

Why does this exist?

Most engineers learn system design by *exposure*: they read blog posts, watch videos and skim chapters. Exposure feels productive because the material seems familiar on the second reading. Familiarity is not the same as the ability to *use* a concept. Learning researchers call this mismatch an illusion of competence. Rereading and highlighting are among the least effective study techniques measured [Dunlosky et al. 2013]. A design interview or a design review asks you to *produce*: choose a mechanism, draw it, estimate it and defend it. The loop makes production the main activity.

How it works internally

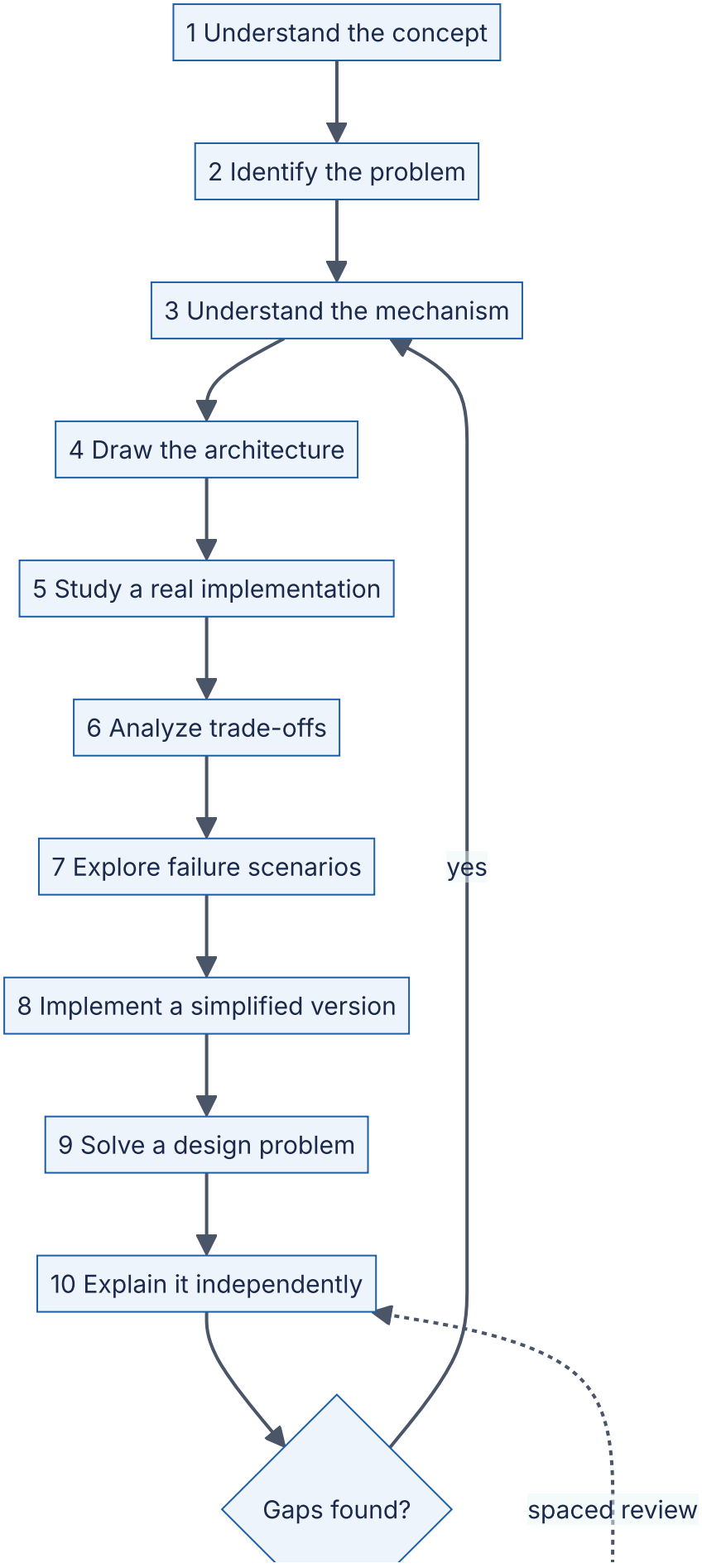
Table 3.1 — The ten steps, the question each answers, and the artifact each produces.

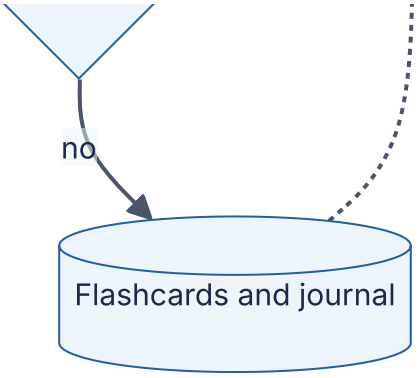
#	Step	Question it answers	Artifact	Typical time (L1–L2)
1	Understand the concept	What is it, in one precise sentence?	Definition in your own words	15 min
2	Identify the problem it solves	What breaks without it?	A “before” scenario with numbers	20 min
3	Understand the mechanism	How does it work, step by step?	Numbered mechanism description	45 min
4	Draw the architecture	Where does it sit and what talks to it?	Diagram drawn from memory	20 min
5	Study an actual implementation	How did a real system build it?	Notes on one paper, docs page or codebase	60–120 min
6	Analyze trade-offs	What does it cost, and versus what?	Comparison table against alternatives	30 min
7	Explore failure scenarios	How does it break?	Failure table (what, why, detect, recover, prevent)	30 min
8	Implement a simplified version	Can I build the core?	Runnable program of 50–300 lines with a measurement	2–4 h
9	Solve a design problem	Can I use it in context?	A small design that uses the concept	45–60 min
10	Explain it independently	Can I teach it without notes?	2–5 minute spoken or written explanation	15 min

A full pass takes about 6–10 hours for one substantial concept. Plan for that. A concept such as “HTTP status codes” does not need all ten steps. A concept such as consistent hashing, Raft or the KV cache does.

The steps are ordered so that each one gives the next one something to work with. You cannot weigh trade-offs (6) until you know the mechanism (3). You should not implement (8) before you know the failure modes (7), because the failures tell you what your simulation must be able to show. Step 10 then checks all the earlier steps. Wherever your explanation falters, you return to that step. That return is why it is a loop and not a line.

Figure 3.1 — The ten-step concept-mastery loop.


1 Understand the concept
2 Identify the problem
3 Understand the mechanism
4 Draw the architecture
5 Study a real implementation
6 Analyze trade-offs
7 Explore failure scenarios
8 Implement a simplified version
9 Solve a design problem
10 Explain it independently
Gaps found?
yes
spaced review



Reading the diagram. The ten rectangles are learning activities, done in order. The solid arrows mean “finish this before starting the next”. The decision node after step 10 is your self-test. If you cannot explain some part without notes, you go back to step 3 (mechanism), because most gaps turn out to be gaps in the mechanism. If you can, the concept goes into long-term storage: the cylinder represents your flashcard deck and design journal (§3.10). The dotted arrow is asynchronous. Days or weeks later, a scheduled review asks you to repeat step 10 from memory. The weak point in this process is skipping steps 7 and 8, which feel optional but are where most understanding is built. To scale the method across a whole book, run several concepts’ loops interleaved, not one after another (§3.4.5).

Common misconceptions

COMMON MISTAKE

Misconception — “I understood it when I read it, so I know it.” Understanding while reading is recognition: the text is in front of you and gives you the cues. Using a concept in a design is recall: you have to bring it up with no cues. Only steps 4, 8, 9 and 10 test recall. If you skip them, you will not know what you are unable to do.

3.3.2 The loop demonstrated: consistent hashing

The rest of §3.3 runs one concept through all ten steps. It shows the artifacts at the depth we expect from you. We use **consistent hashing** because it is compact enough for one section and still has a real mechanism, real failure modes and real implementations. Chapter 14 (§14.8) and Chapter 20 (§20.14) treat it in full. Treat what follows as a sample of how to study, not the canonical treatment.

Step 1 — Understand the concept

DEFINITION

Definition — Consistent hashing. A way of assigning keys to nodes so that, when a node is added or removed, only about K/N of the K keys change owner, where N is the number of nodes. Both keys and nodes are hashed onto the same circular number space (the **ring**). Each key belongs to the first node found by moving clockwise from the key's position [Karger et al. 1997].

Self-check artifact: one sentence in your own words, with no jargon you cannot define. For example: "It spreads keys over servers so that adding a server moves only that server's fair share of keys."

Step 2 — Identify the problem it solves

Suppose you run four cache nodes and route each key with `node = hash(key) mod 4`. Traffic grows, so you add a fifth node and the rule becomes `hash(key) mod 5`. For most keys, `hash mod 4` and `hash mod 5` point to different nodes. Those keys now live on the wrong node, so nearly every request misses the cache and falls through to the database at the very moment you were trying to add capacity.

FORMULA

Fraction of keys remapped when growing from N to $N+1$ nodes

$$f_{\text{mod}} = 1 - \frac{1}{N+1} = \frac{N}{N+1} \qquad f_{\text{ring}} \approx \frac{1}{N+1}$$

Symbol	Meaning	Units
N	Number of nodes before the change	nodes
f_{mod}	Expected fraction of keys whose owner changes under `hash mod N` (uniform hashing)	dimensionless (0–1)
f_{ring}	Expected fraction of keys whose owner changes under consistent hashing	dimensionless (0–1)

The f_{mod} result comes from a simple observation. A key stays on the same node only if `h mod N` equals `h mod (N+1)`. For uniformly distributed hashes, that happens for about $1/(N+1)$ of keys. Under consistent hashing, the new node takes over only the keys that fall in its arcs of the ring. On average that is its fair share, $1/(N+1)$.

WORKED EXAMPLE

Worked example — Adding a fifth cache node.

1. Setup: $N = 4$, $K = 100,000,000$ cached keys, cache hit ratio 95 %, and 50,000 reads per second (RPS) reach the cache tier.
2. Naive scheme: $f_{\text{mod}} = 4/5 = 0.80$. About $0.80 \times 10^8 = 8 \times 10^7$ keys are now on the wrong node.
3. Ring scheme: $f_{\text{ring}} = 1/5 = 0.20$. About 2×10^7 keys move. They move only *to the new node*, which starts out empty.
4. Database impact right after the change, before the cache refills. Under mod-N, about 80 % of reads miss, plus the normal 5 % on the rest. The database sees about $0.80 \times 50,000 + 0.05 \times 0.20 \times 50,000 = 40,000 + 500 = 40,500$ RPS instead of the usual $0.05 \times 50,000 = 2,500$ RPS. That is a 16× surge. Under the ring, about 20 % of reads miss, plus the normal 5 % on the remaining traffic. The database sees about $0.20 \times 50,000 + 0.05 \times 0.80 \times 50,000 = 10,000 + 2,000 = 12,000$ RPS, a 4.8× surge.

Interpretation. Both schemes cause a miss spike. Consistent hashing makes it about 3.4× smaller ($40,500/12,000$) and limits it to one node's share of keys.
Engineering implication. If your database can absorb about 12,000 RPS, consistent hashing lets you add cache nodes without an outage. With mod-N you would need to warm the new layout ahead of time or add nodes only during low traffic. Chapter 20 (§20.11, cache stampede) covers the remaining spike.

Self-check artifact: a "before" scenario with numbers, like the one above.

Step 3 — Understand the mechanism

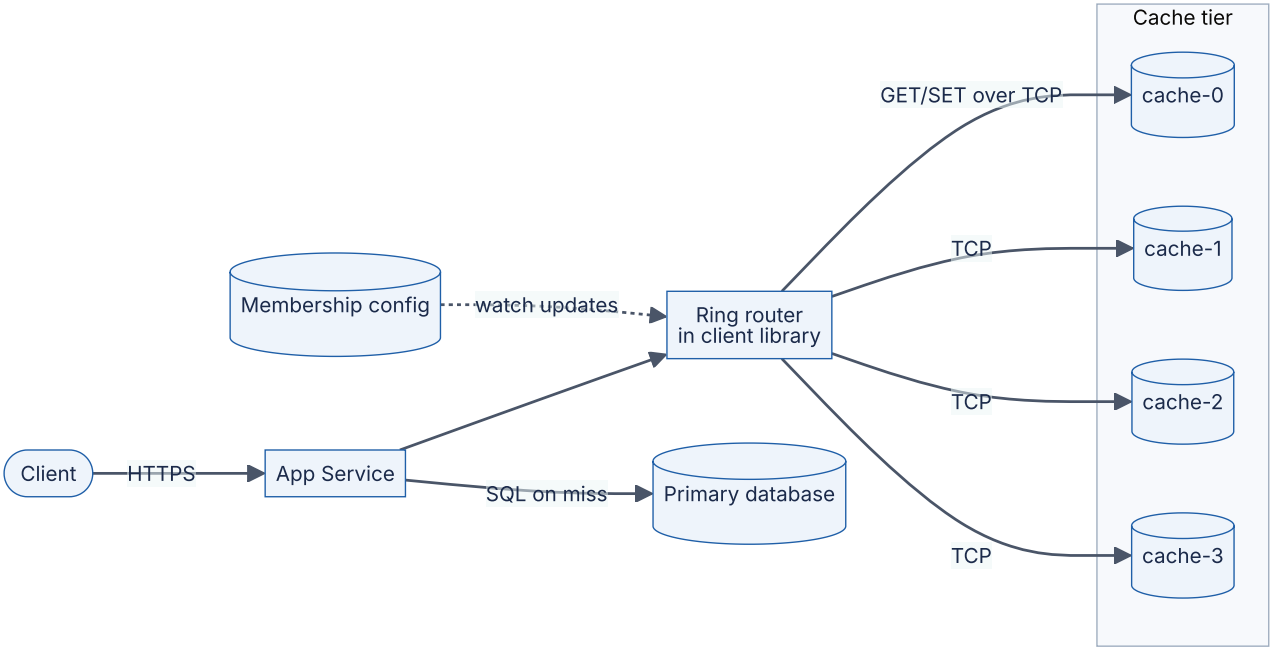
1. Pick a hash function that maps strings to integers in $[0, 2^m)$. Treat that range as a circle in which $2^m - 1$ is next to 0.
2. For each node, compute V positions by hashing distinct labels such as `"cache-2#vn17"`. Each position is a **virtual node** (vnode). Store all $N \times V$ positions in a sorted array, each mapped to its physical node.
3. To look up a key, hash it and binary-search the sorted array for the first position greater than the key's hash. If there is none, wrap around to index 0. The node that owns that position owns the key. A lookup costs $O(\log(NV))$.
4. To add a node, insert its V positions. Each one takes over the arc between its predecessor and itself. Every other key keeps its owner.
5. To remove a node, delete its positions. Each of its arcs goes to the next position clockwise. Because the vnodes are scattered around the ring, the removed node's load is spread over many survivors instead of all landing on one neighbor.
6. For replication, walk clockwise and collect the first R *distinct physical* nodes. Dynamo calls this list the **preference list** [DeCandia et al. 2007].

Why do virtual nodes matter? With $V = 1$, each node owns one arc, and arc lengths from N random points vary widely. One node can easily own two or three times its fair share. With V vnodes, each node's load is the sum of V arcs, and sums of many random lengths stay close to the average. The relative spread shrinks roughly in proportion to $1/\sqrt{V}$. Step 8 measures this.

Step 4 — Draw the architecture

Close the book and draw where consistent hashing sits in a distributed cache. Then compare your drawing with Figure 3.2.

Figure 3.2 — Consistent hashing in a client-routed distributed cache.



Reading the diagram. A client calls the app service over HTTPS. Inside the app service, a client library holds the ring: the sorted vnode positions and their owners. It is in-process, so a lookup is a binary search in memory, typically microseconds. The ring router sends each `GET` / `SET` to exactly one cache node over a TCP connection, such as the Memcached or Redis protocol. All of these are synchronous calls. On a miss, the app service reads the primary database over SQL and fills the cache (cache-aside, §20.7). The dotted arrow from the membership config is asynchronous. A coordination service or config file tells every app instance when nodes join or leave (§26.8–§26.9). There are two storage boundaries: the in-memory cache tier, which is disposable, and the database, which is durable. Failure points: a cache node that dies sends its keys to clockwise successors and causes a miss spike, and app instances that see *different* membership send the same key to different nodes. To scale, add cache nodes (each takes about $1/(N+1)$ of the keys) or raise V to even out the load.

Self-check artifact: your own drawing, annotated with which calls are synchronous and where state lives. If your drawing left out the membership channel, you have found your first gap. Section 3.6 explains how to score drawings like this.

Step 5 — Study an actual implementation

Now read how real systems built it. Pick *one* primary source and read it with the three-pass method (§3.5). For consistent hashing, three sources are worth knowing:

INDUSTRY EXAMPLE

Akamai and the original paper. Karger and colleagues introduced consistent hashing to spread web-cache load without hot spots, in a setting where different clients may see different sets of caches. Their paper defines the properties a good scheme needs. Two of them are *balance*, meaning keys spread evenly, and *monotonicity*, meaning that adding a cache moves keys only onto the new cache, never between old caches. The paper also uses multiple copies of each cache point on the circle to improve balance, which is the idea behind virtual nodes. *Evidence: Primary* — [Karger et al. 1997]

INDUSTRY EXAMPLE

Amazon Dynamo. Dynamo partitions its key space with consistent hashing and assigns each physical node several virtual nodes. This lets a more powerful machine take more vnodes, and it spreads a failed node's load over many survivors. Each key is replicated on the first R distinct physical nodes clockwise, its preference list (the paper calls this count N). The paper compares three strategies for placing tokens. It reports that dividing the ring into equal-sized partitions, decoupled from node placement, gave the best load balance and the smallest membership metadata. *Evidence: Primary* — [DeCandia et al. 2007]

INDUSTRY EXAMPLE

Apache Cassandra. Cassandra's original design hashes keys onto a ring with an order-preserving hash function and assigns each node one position. The paper acknowledges the load imbalance this causes. It handles the imbalance by analyzing load on the ring and moving lightly loaded nodes so they take load from heavily loaded ones. *Evidence: Primary* — [Lakshman and Malik 2010]

When you read an implementation, write down where it *departs* from the textbook mechanism and why. Departures are usually the most instructive part. Dynamo moved from random tokens to fixed equal-sized partitions because random tokens made bootstrapping

and archiving slow. That is an operational reason, not an algorithmic one, and only a real implementation would show it to you.

Step 6 — Analyze trade-offs

Table 3.2 — Key-to-node assignment schemes compared.

Dimension	`hash mod N`	Ring + virtual nodes	Rendezvous (HRW) hashing	Jump consistent hash
What it is	Modulo of key hash	Keys and vnodes on a circle	Each key picks the node with the highest score `hash(key, node)`	Arithmetic function from key to bucket number
Keys moved on add	~$N/(N+1)$	~$1/(N+1)$	~$1/(N+1)$	~$1/(N+1)$
Lookup cost	$O(1)$	$O(\log NV)$	$O(N)$ (scores every node)	$O(\log N)$, no table
Memory	None	$O(NV)$ positions	$O(N)$ node list	None
Load balance	Excellent	Good; improves with V	Excellent	Excellent
Arbitrary node removal	Remaps almost everything	Yes	Yes	No; buckets must be numbered $0..N-1$ and removed from the end
Weighted nodes	Awkward	Natural (more vnodes)	Possible with weighted scores	No
Choose when	N never changes	General-purpose caches and stores	Small N, simple code	Storage shards that only grow

Sources: [Karger et al. 1997], [Thaler and Ravishankar 1998], [Lamping and Veach 2014]. Two further variants are worth knowing by name. **Maglev hashing** builds a fixed-size lookup table for $O(1)$ lookups in load balancers and accepts a little extra disruption in exchange [Eisenbud et al. 2016]. **Consistent hashing with bounded loads** caps every node at $(1+\varepsilon)$ times the average load and sends overflow keys clockwise [Mirrokni et al. 2018].

Self-check artifact: a table like Table 3.2 written from memory, plus one sentence saying which scheme you would pick for a 6-node Redis cache and why.

Step 7 — Explore failure scenarios

Table 3.3 — Failure scenarios for consistent hashing.

What goes wrong	Why	Detection	Recovery	Prevention
One node overloaded	Too few vnodes, so arcs are uneven	Per-node RPS and memory skew	Add vnodes to the others; rebalance	$V \geq 100$; bounded-load variant
Hot key	One key gets a large share of traffic; hashing cannot split one key	Top-K key metrics (§17.6)	Replicate the hot key; add a local cache tier	Key splitting, request coalescing (§20.15)
Successor overload after node death	With $V = 1$, the clockwise neighbor inherits *all* of the dead node's keys	Neighbor CPU and miss-ratio spike	Bring up a replacement node	Virtual nodes spread the load over many successors
Split views of membership	Clients update membership at different times	Same key read from two nodes; inconsistent reads	Converge config; flush the affected range	Single source of truth with watches; versioned ring
Miss storm on resize	New node starts cold	Database RPS spike	Rate-limit fills; serve stale data	Pre-warm; add nodes off-peak; stampede protection (§20.11)
Poor hash function	Clustering of positions	Skewed arc lengths at build time	Change the hash, which remaps everything	Use a well-mixed hash (e.g., MurmurHash, xxHash, MD5 truncated)

The split-view row is the one beginners miss. The algorithm assumes every client has the same ring. Keeping that true is a distributed-systems problem in its own right, and you will meet it again in Chapter 26.

Step 8 — Implement a simplified version

Keep the mechanism and leave out everything else: no networking, no replication, no persistence. The goal is a program that lets you *see* the two properties you studied, minimal movement and load balance.

```
    """Simplified consistent-hashing ring with virtual nodes (teaching version)."""
    import bisect
    import hashlib
    from collections import Counter


    def h(key: str) -> int:
        """Map a string to a 64-bit integer position on the ring."""
        digest = hashlib.md5(key.encode("utf-8")).digest()
        return int.from_bytes(digest[:8], "big")


    class HashRing:
        def __init__(self, nodes=(), vnodes: int = 100):
            self.vnodes = vnodes
            self._positions = []   # sorted ring positions
            self._owner = {}       # position -> physical node name
            for node in nodes:
                self.add_node(node)

        def add_node(self, node: str) -> None:
            for i in range(self.vnodes):
                pos = h(f"{node}#vn{i}")
                bisect.insort(self._positions, pos)
                self._owner[pos] = node

        def remove_node(self, node: str) -> None:
            for i in range(self.vnodes):
                pos = h(f"{node}#vn{i}")
                self._positions.remove(pos)
                del self._owner[pos]

        def get_node(self, key: str) -> str:
            """Return the first node clockwise from the key's position."""
            if not self._positions:
                raise LookupError("ring is empty")
            idx = bisect.bisect_right(self._positions, h(key))
            if idx == len(self._positions):  # wrap around past the top
                idx = 0
            return self._owner[self._positions[idx]]


    def mod_n_node(key: str, nodes: list) -> str:
        """The naive scheme: hash(key) mod N."""
        return nodes[h(key) % len(nodes)]


    if __name__ == "__main__":
        keys = [f"user:{i}" for i in range(100_000)]
        before = [f"cache-{i}" for i in range(4)]
        after = before + ["cache-4"]

        moved = sum(mod_n_node(k, before) != mod_n_node(k, after) for k in keys)
        print(f"mod-N : {moved / len(keys):.1%} of keys moved (4 -> 5 nodes)")

        for v in (1, 100):
```

```
    ring = HashRing(before, vnodes=v)
    old = {k: ring.get_node(k) for k in keys}
    ring.add_node("cache-4")
    moved = sum(old[k] != ring.get_node(k) for k in keys)
    load = Counter(ring.get_node(k) for k in keys)
    spread = max(load.values()) / (len(keys) / len(after))
    print(f"ring vnodes={v:<3}: {moved / len(keys):.1%} moved, "
          f"busiest node at {spread:.2f}x the mean load")
```

Running it with Python 3.8 or later prints:

```
mod-N : 79.9% of keys moved (4 -> 5 nodes)
ring vnodes=1  : 24.4% moved, busiest node at 2.60x the mean load
ring vnodes=100: 21.4% moved, busiest node at 1.08x the mean load
```

What to notice:

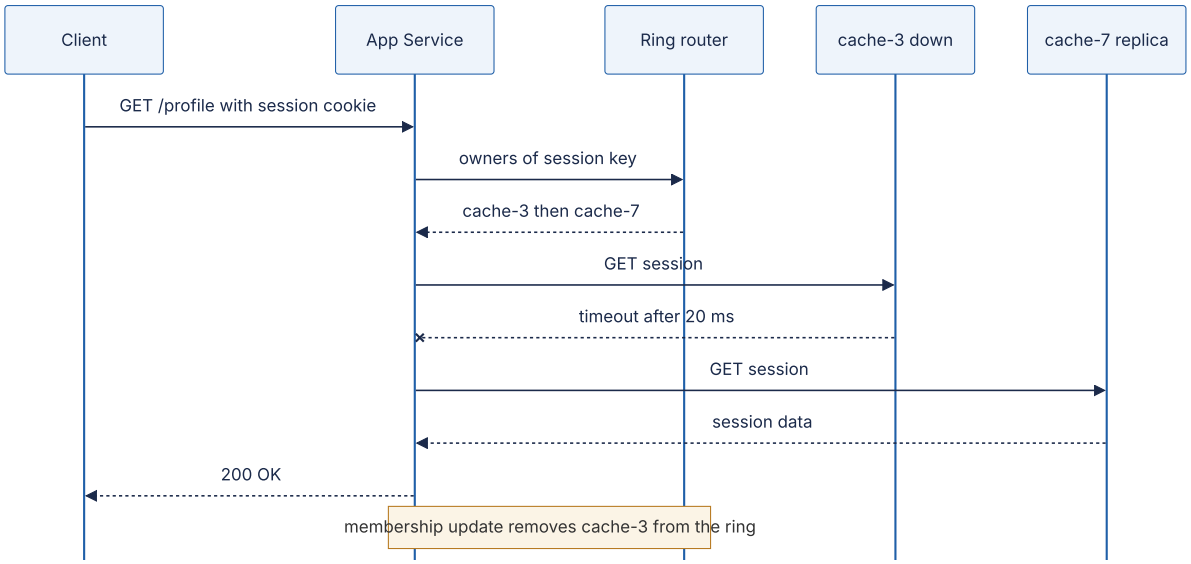
- `h()` uses MD5 only as a *mixing* function, which is fine here because nothing depends on it being cryptographically secure. Python's built-in `hash()` is randomized per process for strings, so two app instances would build different rings. That is exactly the split-view failure from Table 3.3, reproduced by accident.
- `bisect_right` plus the wrap-around check *is* the clockwise walk. Those two lines are the whole algorithm.
- The measurement agrees with the formula. Mod-N moved 79.9 % against a predicted 80 %. The ring moved about 20 % (21.4 % with 100 vnodes).
- With one vnode per node, the busiest node carries 2.6× its fair share. With 100 vnodes it carries 1.08×. You have now *seen* why virtual nodes exist, which is worth more than reading it.
- `bisect.insort` costs $O(NV)$ per insert. That is fine at this scale. A production ring would rebuild a sorted array once per membership change.

Step 9 — Solve a design problem

Use the concept inside a small design. **Problem:** a session store for 20 million concurrently active users. Each session is about 2 KB. Reads run at 200,000 RPS and writes at 20,000 RPS. A cache node may fail without logging out more than about 10 % of users.

A sketch of the solution. Data volume is $2 \times 10^7 \times 2\,\text{KB} = 40\,\text{GB}$ (SI units). With replication factor 2 that is 80 GB. Ten nodes with 16 GB of usable memory each hold it with headroom. Assign keys with a ring of 10 nodes × 150 vnodes. Write each session to the first two distinct physical nodes clockwise. If one node dies, its share (~10 %) has a replica on many different successors thanks to the vnodes, so no user is logged out and no single successor doubles its load. Figure 3.3 shows the read path during that failure.

Figure 3.3 — Session read when the primary owner has failed.



Reading the diagram. The participants are the client, the app service, the in-process ring router, the failed owner (cache-3) and the second replica on the preference list (cache-7). All arrows are synchronous request/response calls. The client uses HTTPS, and the app talks to the cache nodes over TCP. The crossed arrow is the failure: the call to cache-3 times out after a short, explicit deadline (§22.11). The app then tries the next node on the preference list. The cost to the user is one timeout of added latency, here 20 ms. The note shows the asynchronous repair: once failure detection (§27.10) removes cache-3, the router stops trying it and the extra latency goes away. The storage boundary is the cache tier, and each session lives on two nodes. To scale, add nodes. Each one takes about $1/(N+1)$ of the sessions.

Step 10 — Explain it independently

Close everything. Set a timer for three minutes and explain consistent hashing aloud, or in writing, to an imagined colleague who knows what a hash function is but nothing else. A good explanation covers five beats:

1. **The problem**: mod-N remaps about $N/(N+1)$ of keys when N changes.
2. **The trick**: put nodes and keys on the same ring and walk clockwise.
3. **The consequence**: only about $1/(N+1)$ of keys move, and only to or from the changed node.
4. **The refinement**: virtual nodes fix uneven arcs and spread failure load.
5. **The limits**: it does nothing for hot keys, and every client must share the same membership view.

Record yourself if you can. Then mark each beat as “fluent”, “hesitant” or “missing”. Each beat you marked “hesitant” or “missing” sends you back to the matching step: beats 1–2 to steps 2–3, beat 4 to step 8, beat 5 to step 7. Then turn the shaky beats into flash-cards (§3.10.2).

RECOMMENDATION

Recommendation — Keep the artifacts. Store all ten artifacts for each concept in one folder or journal entry (§3.10.3). Months later, rebuilding a concept from your own artifacts is far quicker than the first loop was. That folder becomes your personal edition of this book.

3.4 Active recall, spaced repetition, the Feynman technique, interleaving and deliberate practice

The loop tells you *what* to do with a concept. This section explains *how to practice* so that what you learn lasts. The techniques come from cognitive psychology. They were tested mostly on facts, vocabulary, science texts and mathematics, not on architecture, so applying them to system design is a reasonable extrapolation rather than a measured result.

3.4.1 What the evidence says, in one table

Dunlosky and colleagues reviewed ten widely used study techniques. They rated each one's *utility* by how well it generalized across learners, materials and test conditions [Dunlosky et al. 2013].

Table 3.4 — Study techniques by rated utility, and their system-design form.

Technique	Rated utility	System-design form
Practice testing (retrieval practice)	High	Flashcards; drawing from memory; mock design sessions
Distributed practice (spacing)	High	Scheduled reviews days and weeks apart (§3.4.3)
Elaborative interrogation	Moderate	Asking "why does Dynamo use vnodes rather than one token?"
Self-explanation	Moderate	Step 10 of the loop; the Feynman technique (§3.4.4)
Interleaved practice	Moderate	Mixing caching, queuing and replication problems in one session
Summarization	Low	Writing chapter summaries (useful only with good training)
Highlighting / underlining	Low	Marking up PDFs
Keyword mnemonic	Low	Acronyms for lists of terms
Imagery for text	Low	Picturing a text's content
Rereading	Low	Reading a chapter a second time

The ratings of the top five are the ones that matter. The two techniques that feel most like studying, highlighting and rereading, rated low. The two that feel most like being tested, practice testing and spacing, rated high.

KEY CONCEPT

Key insight — Learning versus performance. How well you do *during* practice is a poor guide to how much you are learning. Conditions that make practice feel harder, such as spacing, interleaving and testing yourself before you feel ready, often produce better long-term retention. Bjork called these **desirable difficulties** [Bjork 1994]; see [Soderstrom and Bjork 2015] for a review. If a study session felt smooth and easy, be suspicious of it.

3.4.2 Active recall (retrieval practice)

DEFINITION

Definition — Active recall. Deliberately retrieving information from memory without looking at the source, for example by answering a question, drawing a diagram or explaining aloud. Its measured benefit for later retention is called the **testing effect**.

In a well-known experiment, students read short prose passages under one of three schedules. They either studied four times (SSSS), studied three times and then took one free-recall test (SSST), or studied once and took three tests (STTT). On a test five minutes later, more studying gave better recall. On a test one week later, the order reversed. The repeated-testing group recalled about 61 % of the idea units, against about 40 % for the repeated-study group, even though the testing group had seen the text only once [Roediger and Karpicke 2006]. A later study found that retrieval practice also beat building concept maps, an “active” elaborative technique, on a test of meaningful learning [Karpicke and Blunt 2011].

Applied to system design. Every reading session should end with retrieval: close the book, and write or draw what you just learned. The checkpoints at the end of each chapter in this book exist for this reason. Answer them before you open the `<details>` block.

3.4.3 Spaced repetition

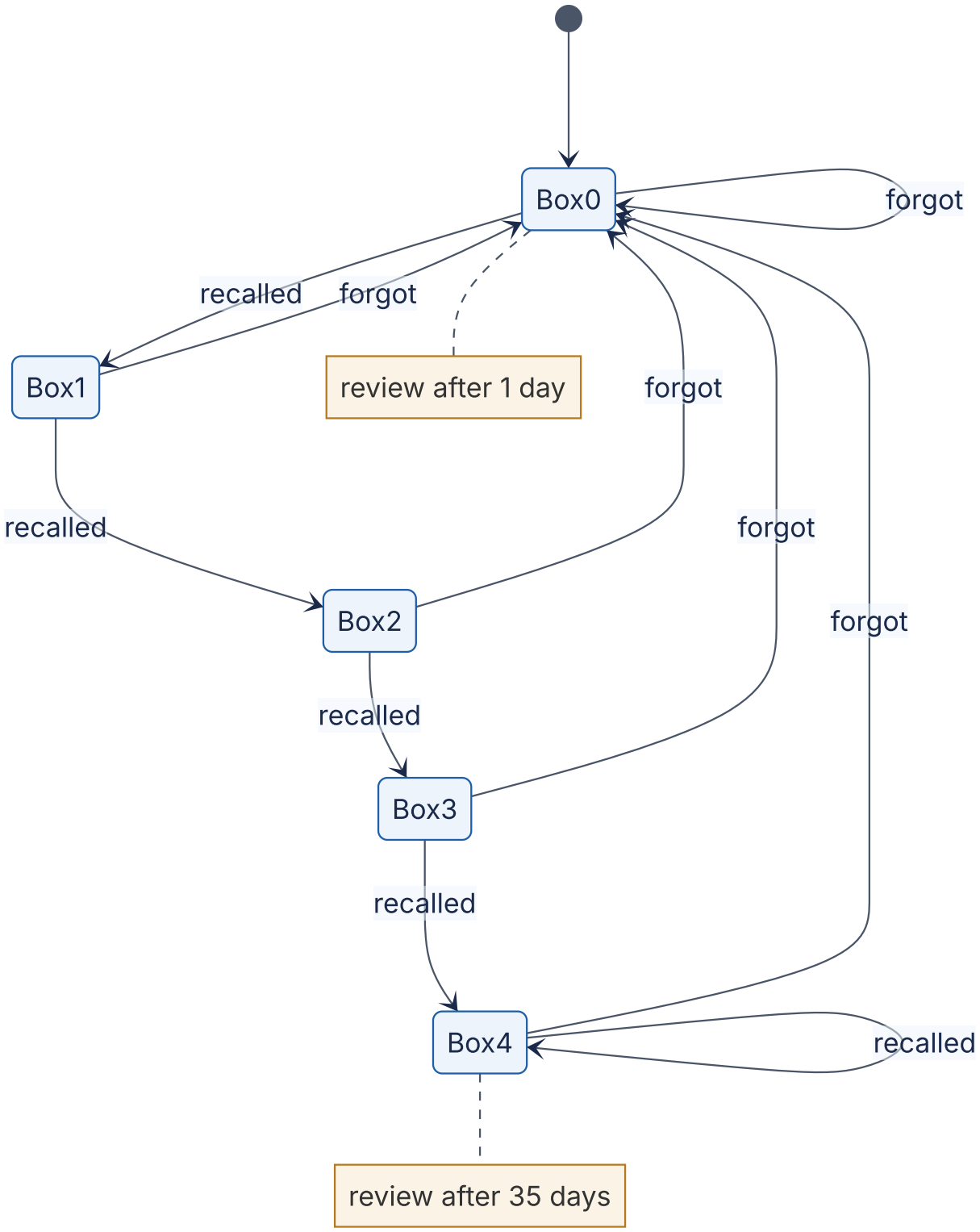
DEFINITION

Definition — Spaced repetition. Reviewing material at increasing intervals, so that each review happens around the time you would otherwise begin to forget it. The underlying finding, that spacing study sessions apart beats massing them together, is the **spacing effect**.

The spacing effect is one of the oldest results in experimental psychology. It goes back to Ebbinghaus's experiments on himself in the 1880s [Ebbinghaus 1885/1913]. A meta-analysis of hundreds of comparisons confirmed that spaced practice beats massed practice for verbal recall [Cepeda et al. 2006]. A follow-up study asked *how far apart* reviews should be. The best gap between study sessions depended on how long you need to remember the material. Expressed as a fraction of that retention interval, the best gap was about 20–40 % for a one-week retention interval and fell to about 5–10 % for a one-year interval [Cepeda et al. 2008].

The practical form of spaced repetition is the **Leitner system**: flashcards move through numbered boxes. A correct answer promotes a card to the next box, which is reviewed less often. A wrong answer sends it back to box 0. Software such as Anki automates the same idea with adaptive intervals.

Figure 3.4 — A five-box Leitner schedule for flashcards.



Reading the diagram. Each state is a box with its own review interval: 1, 3, 7, 16 and 35 days for boxes 0 to 4. The intervals are a reasonable starting point, not an empirically optimal set. Each transition is one review. “Recalled” moves a card one box up, and the top box keeps it there. “Forgot” sends it back to box 0. Data flows one card at a time. All of the state is the box number and due date per card, so a spreadsheet is enough storage.

The failure point is a backlog: if you skip a week, the reviews that were due pile up. The fix is to cap new cards per day, not to skip reviews. The schedule scales because mastered cards cost one review every five weeks or so.

The scheduler itself is small enough to build in a few minutes, which makes it a good first "simplified implementation" (§3.7):

```
        """Minimal Leitner-box scheduler for system-design flashcards."""
        from dataclasses import dataclass, field
        from datetime import date, timedelta

        INTERVALS = [1, 3, 7, 16, 35]  # days until next review, per box (box 0..4)


        @dataclass
        class Card:
            prompt: str
            box: int = 0
            due: date = field(default_factory=date.today)

            def review(self, recalled: bool, today: date) -> None:
                # Success promotes one box; failure sends the card back to box 0.
                self.box = min(self.box + 1, len(INTERVALS) - 1) if recalled else 0
                self.due = today + timedelta(days=INTERVALS[self.box])


        def due_cards(cards, today):
            return [c for c in cards if c.due <= today]


        if __name__ == "__main__":
            start = date(2026, 1, 5)
            card = Card("Why do virtual nodes reduce load imbalance?", due=start)
            today = start
            for ok in [True, True, False, True, True, True]:
                card.review(ok, today)
                print(f"{today}  recalled={ok!s:<5} -> box {card.box}, next {card.due}")
                today = card.due
```

Output:

```
2026-01-05  recalled=True  -> box 1, next 2026-01-08
2026-01-08  recalled=True  -> box 2, next 2026-01-15
2026-01-15  recalled=False -> box 0, next 2026-01-16
2026-01-16  recalled=True  -> box 1, next 2026-01-19
2026-01-19  recalled=True  -> box 2, next 2026-01-26
2026-01-26  recalled=True  -> box 3, next 2026-02-11
```

What to notice:

- `field(default_factory=date.today)` matters. A plain `date.today()` default would be evaluated once, when the class is defined, and every card would share that date. This is the kind of bug you only find by building the thing.
- One failure on 15 January wipes out two promotions. That is harsh by design: a card you forgot needs to be relearned, not just reviewed.
- After six reviews spread over five weeks, the card is due in 16 days. Spaced repetition costs little per item, and the cost keeps falling.

> WORKED EXAMPLE
>
> **Worked example — Daily review load.** You add 5 new cards per study day, 5 days a week, for a year: about $5 \times 5 \times 52 = 1,300$ cards. In the steady state, assume 70 % of cards sit in boxes 3–4 (reviewed about every 16–35 days, call it every 25 days), 20 % in box 2 (every 7 days) and 10 % in boxes 0–1 (every 2 days on average). Daily reviews $\approx 1,300 \times (0.70/25 + 0.20/7 + 0.10/2) = 1,300 \times (0.028 + 0.029 + 0.050) \approx 139$ cards. At about 8 seconds per card that is roughly 19 minutes a day. **Interpretation.** A year of system-design vocabulary stays fresh for under 20 minutes a day, *if* most cards are mastered. **Implication.** Keep cards atomic so they move up the boxes quickly. Cards that bounce between boxes 0 and 1 dominate the load, so rewrite or split them.

3.4.4 The Feynman technique (self-explanation)

The “Feynman technique” is a popular name for a four-step routine. You pick a concept, explain it in plain language as if to a newcomer, find the places where your explanation breaks down or falls back on jargon, and go back to the source to fix those places. Richard Feynman did not publish this method under that name. The label was attached later. The research behind it is the work on **self-explanation**: learners who explain material to themselves while studying understand it better than those who only read it [Chi et al. 1994]. Dunlosky and colleagues rated self-explanation and elaborative interrogation as having moderate utility [Dunlosky et al. 2013].

In system design, the routine is step 10 of the loop, plus one rule: **every “it just handles that” is a gap.** If your explanation of Kafka says “the broker handles durability”, you have not explained durability. You have named it. Push until you reach a mechanism, such as “the leader waits for in-sync replicas to append the record before acknowledging when `acks=all`” (§31.6, §31.9).

3.4.5 Interleaving

DEFINITION

Definition — Interleaving. Mixing different kinds of problems or concepts within one practice session (A B C A C B), instead of finishing all practice on one kind before moving to the next (AAA BBB CCC, called **blocking**).

In one study, students who practiced mixed (shuffled) mathematics problems did worse during practice but much better on a delayed test than students who practiced in blocks [Rohrer and Taylor 2007]. Interleaving also helped people learn to tell categories apart, such as painters' styles, even though most participants *believed* blocking had worked better for them [Kornell and Bjork 2008].

A design problem never tells you which chapter it comes from. Interleaving trains the skill that matters most in a real design: noticing *which* tool a situation calls for. In practice, once you have studied caching, queues and replication, practice with mixed questions such as "this read path is slow; is it a cache problem, a fan-out problem, or a replication-lag problem?" instead of ten caching questions in a row.

3.4.6 Deliberate practice

DEFINITION

Definition — Deliberate practice. Effortful practice designed to improve performance. It targets specific weaknesses, has clear goals, gives immediate feedback and is usually guided by a teacher or coach [Ericsson et al. 1993].

Ericsson and colleagues argued that differences in expert performance largely reflect accumulated deliberate practice. Later work found the picture more mixed. A meta-analysis found that deliberate practice explained about 12 % of the variance in performance across domains, and less in education and professions than in games, music and sports [Macnamara et al. 2014]. The honest conclusion is this: practice that targets weaknesses and comes with feedback helps, but hours alone do not guarantee expertise.

For a system designer, the scarce ingredient is **feedback**. You can get it from mock interviews with a scoring rubric (Chapter 88), from design reviews by senior engineers (§3.9.2), from comparing your design with published ones (Chapters 52–56), and from running your own simplified implementation to see whether it behaves as you predicted.

COMMON MISTAKE

Misconception — “I’m a visual learner, so I should only watch videos.” Matching instruction to self-reported “learning styles” has not been shown to improve outcomes in properly designed experiments [Pashler et al. 2008]. Diagrams do help with system design, but because the *subject* is spatial and structural, not because of your learning style. Everyone should draw.

3.5 Reading papers and engineering blogs

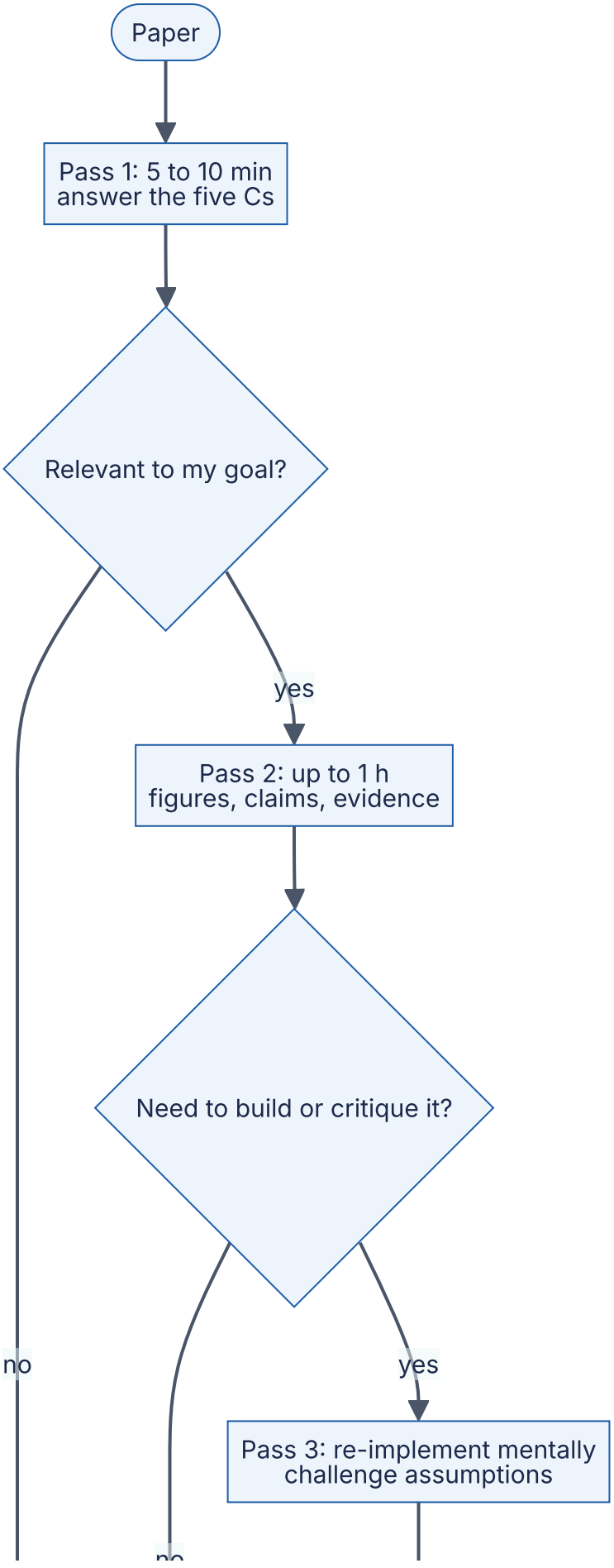
Step 5 of the loop sends you to primary sources. In system design, those are mostly conference papers (SOSP, OSDI, NSDI, USENIX ATC, VLDB, SIGMOD), official documentation and company engineering blogs. Each needs a different way of reading.

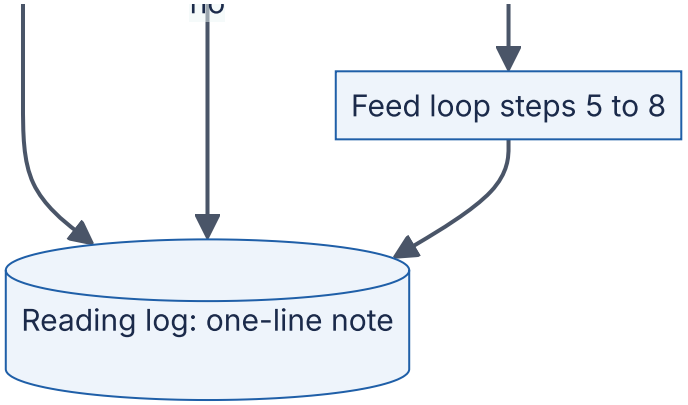
3.5.1 Keshav’s three-pass method

Keshav’s two-page note is the standard advice for reading systems papers. Read a paper in up to three passes, and decide after each pass whether the next one is worth the time [Keshav 2007].

1. **First pass (about 5–10 minutes).** Read the title, abstract and introduction. Read the section headings and nothing under them. Glance at any mathematics, then read the conclusions and scan the references. Afterwards, you should be able to answer the **five Cs**: *Category* (what type of paper is it?), *Context* (which other work does it build on?), *Correctness* (do the assumptions look valid?), *Contributions* (what is new?) and *Clarity* (is it well written?).
2. **Second pass (up to about an hour).** Read carefully but skip proofs. Study the figures, graphs and axes. Mark the references you have not read. Afterwards, you should be able to summarize the paper’s main thrust, with evidence, to someone else.
3. **Third pass (about 4–5 hours for a beginner, about an hour for an experienced reader).** Virtually re-implement the paper. Make the same assumptions, recreate the work, and compare your version with the authors’. This pass exposes hidden assumptions and missing citations.

Figure 3.5 — The three-pass reading decision flow.


Paper
Pass 1: 5 to 10 min
answer the five Cs
Relevant to my goal?
yes
Pass 2: up to 1 h
figures, claims, evidence
Need to build or critique it?
no
yes
Pass 3: re-implement mentally
challenge assumptions



Reading the diagram. A paper enters at the top. Each rectangle is a pass with a time budget, and each diamond is a decision to stop or go deeper. Every path ends at the reading log, the cylinder. Even a paper you rejected after five minutes gets a one-line note, so you will not reread it by accident. Only the third pass feeds steps 5–8 of the loop. The usual failure is starting at the third pass: reading page one in detail and giving up on page four. To scale this across many papers, do many first passes and only a few third passes. For most engineers, about one paper in five deserves the third.

When you apply the method to a *systems* paper, add two questions to the five Cs. **What was the workload?** Dynamo’s design follows from Amazon’s shopping-cart writes, which had to be “always writeable”. **What did they choose not to provide?** GFS gave up a POSIX-compatible interface and offered relaxed consistency [Ghemawat et al. 2003]. A paper’s non-goals tell you as much as its goals.

Table 3.5 — A starter reading list mapped to this book.

Paper	Read with	Why this one first
Lamport, *Time, Clocks, and the Ordering of Events* [Lamport 1978]	§24.10	Short; defines "happened-before"
The Google File System [Ghemawat et al. 2003]	§15.5	Clear workload-driven trade-offs
MapReduce [Dean and Ghemawat 2004]	§29.4	Simple model; good first third-pass paper
Dynamo [DeCandia et al. 2007]	§14.8, §27.7	Assembles many chapter concepts in one design
The Chubby Lock Service [Burrows 2006]	§26.14	Engineering lessons, not just algorithms
ZooKeeper [Hunt et al. 2010]	§26.8	Coordination as a service
Kafka [Kreps et al. 2011]	§31.3	The log as a central abstraction
Raft [Ongaro and Ousterhout 2014]	§26.7	Written explicitly to be understandable
The Tail at Scale [Dean and Barroso 2013]	§23.6–§23.7	Magazine-length; shapes how you think about latency
Spanner [Corbett et al. 2012]	§28.8	Advanced; read after Chapters 25–26

3.5.2 Reading engineering blogs critically

Company engineering blogs are where the field's current practice gets written down. They are also written to recruit engineers and to promote the company, and they rarely go through peer review. Read them as **evidence of what one team did under its own constraints**, not as recommendations for you.

Table 3.6 — Critical-reading checklist for engineering posts.

Ask	Why	Red flag
What was the workload and scale?	Solutions depend on the workload	No numbers at all, or only "massive scale"
What was the baseline?	A speed-up needs something to compare against	"10× faster" with no stated "than what"
What did they try that failed?	Shows what the real constraint was	Only a success story
What did it cost?	Every choice costs something	No downsides mentioned
When was it written?	Systems and versions change	No date; describes a deprecated version
Can it be checked?	Separates evidence from narrative	No benchmark method; no code; no follow-up
Who benefits from you believing it?	Vendor posts sell products	"Our product solved it" with no alternatives considered

This book uses the same discipline in its **evidence labels** (§9 of the style guide): Primary, Secondary or Illustrative. Label your own notes the same way. A claim you read in a conference talk summary is Secondary, even if it sounds authoritative.

3.6 Drawing architecture from memory

3.6.1 Why drawing from memory works

Drawing a system from memory is retrieval practice (§3.4.2) on structured material. It also builds what Chase and Simon called **chunks**. In their studies, chess masters reconstructed real game positions from a brief glance far better than novices. On random piece placements, the masters' advantage largely disappeared [Chase and Simon 1973]. Their expertise was stored as meaningful patterns, not as better raw memory. Reference architectures are the chunks of system design: "CDN → load balancer → stateless service → cache → primary/replica database", or "producer → partitioned log → consumer group". Once each of these is a single chunk, your working memory is free to reason about what is *different* in the problem in front of you.

3.6.2 The protocol

1. **Choose a target**: a figure from a chapter you finished at least two days ago.
2. **Draw blind for 10 minutes** on paper or a whiteboard. Use the book's shapes (§6 of the style guide): stadiums for clients, rectangles for services, cylinders for stores, subroutine shapes for queues.
3. **Annotate edges**: protocol, synchronous or asynchronous, direction.
4. **Compare** with the source and score your drawing with Table 3.7.
5. **Write the diff**: every missing component and every wrong edge becomes a flashcard.
6. **Redraw** after the next spacing interval (3 days, then 7, then 16).

Table 3.7 — Scoring rubric for a drawing from memory (0–2 points each, 12 maximum).

Criterion	0	1	2
Components	Major pieces missing	All major pieces present	Also includes supporting pieces (config, discovery, monitoring)
Data flow	Arrows missing or wrong	Correct direction	Read and write paths distinguished
Sync vs async	Not marked	Partly marked	Every edge correct
Protocols	None	Some	Every edge labeled
Storage boundaries	Unclear where state lives	Stores shown	Durability of each store stated
Failure points	None marked	One marked	Each single point of failure plus its mitigation

A score of 10–12 on a figure means you can use that architecture fluently. Below 7 means you should go back to loop step 3 for the components you missed.

INTERVIEW TIP

Interview tip. Interviewers do not score how pretty your diagram is. They notice whether you mark the read and write paths separately and whether you can point at the single points of failure without being asked. Those are the two highest-value rows of Table 3.7, and both can be trained.

3.7 Building simplified implementations

3.7.1 Why build at all

Feynman's blackboard at the time of his death carried the line "What I cannot create, I do not understand." Building a small version of a mechanism forces you to make every decision the prose glossed over: what happens on wrap-around, what a timeout returns, which component owns a piece of state. The consistent-hashing ring in §3.3 and the Leitner scheduler in §3.4.3 each surfaced a bug that reading would not have, a process-randomized `hash()` and a shared default date.

3.7.2 Scoping rules

1. **Keep the mechanism; fake the environment.** Simulate the network with function calls and the clock with a counter. Simulate disks with dictionaries.
2. **One file, under 300 lines.** If it grows past that, you are building a product, not a learning model.

3. **Build in a measurement.** Every simplified implementation should print a number that tests a claim from the chapter: keys moved, hit ratio, p99 latency, messages lost.
4. **Inject one failure.** Kill a node, drop a message or skew a clock, then check whether the behavior matches Step 7's failure table.
5. **Compare with a real system afterwards.** Read the corresponding part of a real codebase or its documentation and list three things it does that yours does not. That list is your map to production concerns.

Table 3.8 — Simplified implementations that pay off, by chapter.

Build	Chapter	The measurement that proves it works
LRU cache with TTL	§20.8–§20.9, §38.10	Hit ratio against cache size on a Zipf-like key stream
Token-bucket rate limiter	§22.4	Accepted rate converges to the refill rate
Consistent-hash ring	§14.8	Keys moved ≈ $1/(N+1)$; max/mean load against vnode count
Bloom filter	§17.4	Measured false-positive rate matches the formula
Append-only log with offsets	§31.3, §31.8	Consumer restarts from its committed offset
Leader election with leases	§26.5, §26.11	At most one leader per lease epoch under clock skew
Raft log replication (simulated network)	§26.7	No committed entry lost when a minority of nodes crash
Retry with exponential backoff and jitter	§22.12	Load on a recovering server, with and without jitter

Chapter 93 turns several of these into full projects with deployment, testing and monitoring. The simplified versions here are the step before that.

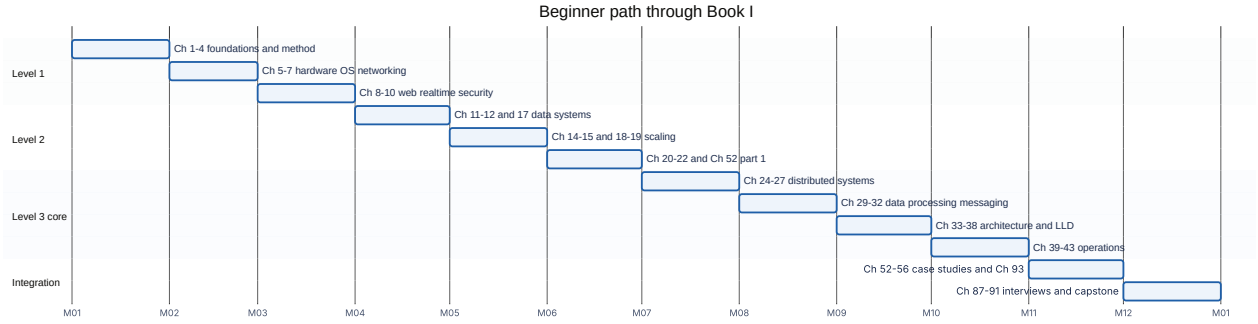
WARNING / TRADE-OFF

Warning — The toy-to-production gap. A 100-line Raft simulation that passes your tests is not a correct Raft. Consensus implementations fail in rare interleavings of events that small tests seldom reach, which is why Jepsen-style testing exists (§25.11). Treat your build as a model of the mechanism, never as evidence that the real problem is easy.

3.8 Beginner methodology: a 12-month path

This path is for readers starting at Level 1: you can program, but you have not designed distributed systems. It assumes 8–10 hours a week. That is enough for about four full concept loops a month plus lighter reading. The target is **Level 2 by month 6 and Level 3 on core Book I material by month 12**. Levels 4 and 5 (senior/staff, principal) depend on production experience that no reading plan can replace. Part IX and §3.9 are where you go next.

Figure 3.6 — The 12-month beginner path (months shown as M01–M12).



Reading the diagram. Each bar is one month of study, and bars run one after another. The `after` dependencies mean that each block assumes you have finished the one before it. The calendar dates are placeholders, so read the axis as month numbers. The three sections match the proficiency levels you should reach by the end of each phase. The fourth section, Integration, combines everything rather than adding new material. The common failure is slipping in months 7–8. Distributed systems is the steepest part of the path, so plan for those two months to take three. There are two ways to scale the plan. At 15+ hours a week, the whole path fits in about eight months. An ML-track reader can swap month 9 for Chapters 57–60 and month 10 for Chapters 66 and 72–75.

Table 3.9 — Month-by-month plan with loop targets and milestones.

Month	Chapters	Full-loop concepts (examples)	Milestone artifact
1	1–4	Latency vs throughput; Little's Law; availability arithmetic	Flashcard deck started; first design journal entry
2	5–7	Memory hierarchy; event loops; TCP handshake; DNS	Request lifecycle drawn from memory (§8.5 preview)
3	8–10	REST design; OAuth 2.0 flow; TLS; threat model	API spec for a URL shortener (§52.1)
4	11–12, 17	B+ tree index; isolation levels; Bloom filter	Bloom filter build (Table 3.8)
5	14–15, 18–19	Replication lag; consistent hashing; L4 vs L7 balancing	Hash ring build (§3.3)
6	20–22; 52.1–52.3	Cache-aside; cache stampede; token bucket	**L2 checkpoint**: rate limiter design (§52.3), timed at 45 min
7	24–27	Lamport clocks; linearizability; Raft; quorums	Raft election simulation
8	29–32	Partitioned log; consumer groups; outbox; saga	Log with offsets build
9	33–38	Modular monolith; bounded contexts; LLD of an LRU cache	LLD exercise from §38.10
10	39–43	SLOs and error budgets; tracing; canary releases	One postmortem study (§3.9.3)
11	52–56; 93.1–93.2	Five case studies through all 16 template sections	One beginner project from Ch 93 deployed
12	87–89, 91	Estimation drills; interview pacing	**L3 checkpoint**: three mock interviews scored with the Ch 88 rubric

Chapters 13, 16, 23 and 28 and all of Part IX are **deferred to year two**. They are L3–L5 material that builds on everything above. Read their introductions when a case study points to them, but do not run full loops on them yet.

RECOMMENDATION

Recommendation — The weekly rhythm. Use two weekday sessions of 2 hours for loop steps 1–7 on one concept, and one weekend session of 3 hours for steps 8–9. Do 15–20 minutes of flashcards on five days. Close the week with a 30-minute review: one drawing from memory of a concept from last month and one journal entry. This rhythm builds in spacing, interleaving and retrieval without any extra planning.

3.9 Advanced methodology for experienced engineers

If you already build production systems, you do not need a path from Chapter 1. You need to know **where your gaps are**. Experience tends to be deep in the systems you have run and thin everywhere else. A payments engineer may know idempotency keys (§22.15) and sagas very well and know little about CDNs or GPU serving.

3.9.1 Gap analysis

Rate yourself from 0 to 4 on each of the book's 19 parts, where 0 means "cannot define the main terms" and 4 means "could teach it and have operated it in production". Set a target for each part from your role or the job you want. Then rank the parts by weighted gap.

FORMULA

Gap priority

$$P_i = (T_i - C_i) \times W_i$$

Symbol	Meaning	Units
P_i	Priority score for part i	points (0–12)
T_i	Target proficiency for part i	0–4 scale
C_i	Current self-rated proficiency (use 0 if $C_i > T_i$ would make P_i negative)	0–4 scale
W_i	Relevance weight: 1 = nice to have, 2 = useful, 3 = essential for role	dimensionless (1–3)

WORKED EXAMPLE

Worked example — A backend engineer moving to an infrastructure role.

Table 3.10 — Gap-analysis scores for one engineer.

Part	T	C	W	$P = (T - C) \times W$
V Distributed systems	4	2	3	$2 \times 3 = 6$
VIII Operations	4	3	3	$1 \times 3 = 3$
IV Scalability and performance	3	2	2	$1 \times 2 = 2$
XIII Training infrastructure	2	0	1	$2 \times 1 = 2$

Interpretation. Part V comes first with a score of 6, three times any other part. Parts IV and XIII tie at 2. Their gaps differ, but weighting by relevance brings them level. **Implication.** Spend the next quarter on Chapters 24–28 with full loops, and keep the others to first-pass reading. Re-rate yourself each quarter, because a self-rating is only a guess until a design review or a mock interview tests it.

3.9.2 Design reviews as a learning instrument

Design reviews (§1.10) are deliberate practice with expert feedback already built in. Use them on purpose. **As an author**, write a design document for a real change and ask reviewers to challenge specific sections, such as failure modes or capacity numbers. **As a reviewer**, read others' designs with Table 3.7 and Table 3.3 in mind, and write at least one question about failure handling and one about cost. **As an observer**, sit in on reviews outside your team. Hearing how senior engineers question a design teaches you what to ask before you can answer it yourself. After each review, write down in your journal one thing you would have missed.

3.9.3 Postmortem study

Production incidents are the most honest record of how systems fail. Many organizations publish blameless postmortems. The SRE book describes postmortem culture as a way to learn from failure without assigning blame [Beyer et al. 2016]. Curated collections, such as Dan Luu's public list of postmortems, make it easy to find many of them [Luu, postmortems]. Research that analyzes failures across many systems shows patterns that single reports cannot. One study of failures in distributed data-intensive systems found that most catastrophic failures came from incorrect handling of non-fatal errors that the software had already signaled [Yuan et al. 2014]. A study of hundreds of public cloud outage reports catalogued their root causes and how far their impact spread [Gunawi et al. 2016].

Figure 3.7 — The postmortem study loop.



Reading the diagram. The hexagon is an external source, a published report. Each rectangle is an analysis step, done in order. The two most valuable steps are drawing the architecture yourself, since reports rarely include a complete diagram, and separating the *trigger* (what started it) from the *contributing factors* (why it spread and why recovery was slow). Mapping each factor to a chapter turns an anecdote into a lesson you can review. The rule you generalize goes into durable storage, your journal and flashcards. The dotted arrow is a spaced review in which you redraw the architecture from memory. The failure point in this process is stopping at the trigger. “Someone typed the wrong command” is never the whole story.

INDUSTRY EXAMPLE

Amazon S3, US-EAST-1, February 2017. While debugging a billing-system issue, an engineer ran an established playbook command meant to remove a small number of servers. An input was entered incorrectly, and a much larger set of servers was removed, including servers supporting two S3 subsystems: the index subsystem and the placement subsystem. Both needed a full restart, which took longer than expected because those subsystems had not been fully restarted in years while S3 had grown substantially. In response, AWS changed the tool to remove capacity more slowly and to refuse to take any subsystem below its minimum required capacity. It also prioritized splitting the index subsystem into smaller cells. **Lessons mapped to chapters:** guard rails on operational tools (§43.3), blast-radius reduction with cells (§33.12, Chapter 44), and recovery time as a design property (§39.8). *Evidence: Primary* — [AWS 2017]

3.9.4 Reading production incident reports critically

Published reports are edited for a public audience and tend to leave out organizational causes and near misses. Read them with Table 3.6 in mind. Separate **what the report states** from **what you infer**. Look for the **detection gap**, the time from trigger to first alert, which is often more instructive than the bug. Ask what the incident shows about the **design**, not the operator: why could one command remove so much capacity? One report a week gives you about 50 failure case studies a year, and that breadth is a large part of what distinguishes a Level 4 engineer.

3.10 Tracking progress

3.10.1 Self-assessment rubric

Rate yourself each quarter on the seven dimensions below, and ask a peer or mentor to rate you as well. A large gap between their rating and yours is itself useful information.

Table 3.11 — Self-assessment rubric mapped to the five proficiency levels.

Dimension	L1 Beginner	L2 Intermediate	L3 Advanced	L4 Senior/Staff	L5 Principal
Concept knowledge	Defines core terms	Explains building blocks	Explains internals	Knows edge cases and variants	Shapes how the field uses the concepts
Mechanism depth	Describes what it does	Describes how, step by step	Implements a simplified version	Debugs production behavior	Designs new mechanisms
Trade-off reasoning	Names one pro and one con	Compares two options	Compares options with numbers	Weighs cost, organization and time	Sets direction for many teams
Estimation	Needs a template	Estimates RPS and storage	Estimates the bottleneck resource	Builds a capacity model	Owns fleet-level planning
Failure reasoning	Rarely considers failure	Lists failure modes	Designs detection and recovery	Limits the blast radius	Sets reliability strategy
Communication	Describes components	Draws a clear diagram	Leads a 45-minute design discussion	Runs design reviews	Writes standards others adopt
Implementation	Follows tutorials	Builds small services	Builds simplified distributed mechanisms	Ships and operates such systems	Builds platforms others build on

Appendix E gives the full definitions of the levels. Your overall level is roughly the level reached by your *weakest* three dimensions, not your strongest, because a design is only as strong as its weakest reasoning.

3.10.2 Flashcard templates

Make each card **atomic** (one fact or one relationship) and prefer **why** and **when** cards over **what** cards. Six card types cover most of system design:

```
[DEFINITION]  Q: Define linearizability in one sentence.
[MECHANISM]   Q: In a hash ring, which node owns key k?  (answer in two steps)
[NUMBER]      Q: Rough round-trip time within one datacenter? (cite Appendix F, with
year)
[TRADE-OFF]   Q: When would you choose jump hash over a ring with vnodes?
[FAILURE]     Q: A cache node dies under mod-N hashing. What happens to the database,
and why?
[DIAGRAM]     Q: Draw cache-aside read and write paths. (check against Fig. 20.x)
```

Each card should also record its source section (for example `src: §3.3 step 6`), so a failed card leads you straight to the material it came from.

3.10.3 Design journal template

```
Date:              Concept / problem:              Loop steps completed: 1-10
1. One-sentence definition (from memory):
2. Problem it solves, with a number:
3. Mechanism in 5 steps or fewer:
4. Drawing score (Table 3.7): __ / 12   Missing:
5. Source studied + evidence label:
6. Best alternative and when I would pick it:
7. Scariest failure mode and its mitigation:
8. Build: repo/file + the measurement it printed:
9. Design problem used in + what I would change next time:
10. Explanation beats fluent / hesitant / missing:
Cards created:                    Next review date:
```

3.10.4 The concept tracker

Table 3.12 — Concept tracker (example rows).

Concept	Section	Steps done	Drawing score	Last review	Next review	Confidence (1–5)
Consistent hashing	§14.8	1–10	11/12	2027-05-03	2027-05-19	4
Raft leader election	§26.7	1–7	8/12	2027-07-10	2027-07-13	2
Cache stampede	§20.11	1–4, 6–7	9/12	2027-06-02	2027-06-09	3

Look at the tracker every week. Three numbers tell you whether the method is working: **flashcard retention**, meaning the fraction recalled on review, where around 85–90 % is a healthy target; **average drawing score**, which should rise over a quarter; and **loops completed per month**, about four on the beginner path. If retention falls below about 80 %, you are adding cards faster than you are learning them. Slow down. If drawing scores stall, put more time into loop step 3.

3.11 Summary and revision notes

SUMMARY

- Mastery means you can **produce** the concept without notes: define it, draw it, build it, use it and explain it. Recognizing it when you read it does not count.
- The **ten-step loop** turns each concept into ten artifacts. Step 10 (explain independently) checks the other nine, and any gap sends you back to step 3.
- Consistent hashing shows the loop's value. The formula predicts that ~80 % of keys move under mod-N and ~20 % under a ring when going from 4 to 5 nodes. A 60-line build confirmed it and showed that virtual nodes cut the busiest node's load from 2.6× to 1.08× the mean.
- **Retrieval practice** and **spacing** have the strongest evidence. Rereading and highlighting have the weakest [Dunlosky et al. 2013]. Self-explanation and interleaving have moderate support. Deliberate practice helps, but it explains only part of the differences in expertise.
- Read papers in **three passes** [Keshav 2007]. Read blogs as evidence of one team's choices under their own constraints, and label every claim Primary, Secondary or Illustrative.
- **Drawing from memory** builds chunks. Score your drawings out of 12 on components, flows, sync/async, protocols, storage and failure points.
- **Simplified implementations** keep the mechanism, fake the environment, print a measurement and inject a failure.
- Beginners: follow the 12-month path to reach L2 by month 6 and L3 on core Book I material by month 12. Experienced engineers: rank gaps with $P = (T - C) \times W$, use design reviews as deliberate practice and study one incident report a week.
- Track flashcard retention (~85–90 %), drawing scores and loops per month. Judge your level by your weakest dimensions.

3.12 Knowledge checkpoint

1. What artifact does each of loop steps 2, 4, 8 and 10 produce?
2. Why does the loop return to step 3, not step 1, when step 10 reveals a gap?
3. Using the formula, what fraction of keys move when a mod-N cluster grows from 9 to 10 nodes? And under consistent hashing?
4. Why does Python's built-in `hash()` make a poor ring hash for a multi-process service?
5. Name two techniques Dunlosky et al. rated high utility and two they rated low.
6. In Roediger and Karpicke's experiment, which condition did best at five minutes and which at one week?

7. According to Cepeda et al. (2008), does the best review gap, as a fraction of the retention interval, grow or shrink as the retention interval gets longer?
8. What is the difference between interleaved and blocked practice, and why does interleaving suit system design?
9. List Keshav's five Cs.
10. Which pass of the three-pass method feeds loop steps 5–8, and roughly how long does it take a beginner?
11. Give three questions from the blog-reading checklist.
12. What did Chase and Simon's random-position condition show about expertise?
13. State four scoping rules for a simplified implementation.
14. Compute the gap priority for a part with target 3, current 1 and weight 2.
15. In a postmortem study, why separate the trigger from the contributing factors?
16. Your flashcard retention has fallen to 72 %. What should you change?

Answers

1. Step 2: a "before" scenario with numbers. Step 4: a diagram drawn from memory. Step 8: a runnable program that prints a measurement. Step 10: a 2–5 minute explanation given without notes.
2. Most gaps in an explanation are gaps in the mechanism. The definition (step 1) is rarely what is missing.
3. Mod-N: $N/(N+1) = 9/10 = 90\%$. Consistent hashing: about $1/(N+1) = 10\%$.
4. Python randomizes `hash()` for strings in each process, so different processes would build different rings. That is a split-view failure.
5. High: practice testing and distributed practice. Low: any two of rereading, highlighting, summarization, keyword mnemonic and imagery for text.
6. At five minutes, repeated study (SSSS) did best. At one week, repeated testing (STTT) did best, about 61 % against 40 %.
7. It shrinks: from about 20–40 % for a one-week interval to about 5–10 % for a one-year interval.
8. Interleaving mixes problem types within a session, while blocking finishes one type before starting the next. Real design problems do not say which tool they need, and interleaving trains you to choose.
9. Category, Context, Correctness, Contributions, Clarity.
10. The third pass, which takes about 4–5 hours for a beginner.
11. Any three of: What was the workload and scale? What was the baseline? What failed? What did it cost? When was it written? Can it be checked? Who benefits?
12. On random positions, the masters' advantage largely disappeared. Their expertise was stored as meaningful patterns (chunks), not as better general memory.
13. Keep the mechanism and fake the environment; one file under 300 lines; build in a measurement; inject one failure; compare with a real system afterwards.
14. $P = (3 - 1) \times 2 = 4$.

15. The trigger explains how the incident started. The contributing factors explain why it spread and why recovery was slow, and those are what design can change.
16. Add fewer new cards, and rewrite or split the cards that keep falling back to box 0. Cards that are too big or vague cause most of the failures.

3.13 Interview questions

Interviewers sometimes ask how you learn, especially for senior roles. The concept used to demonstrate the loop, consistent hashing, also comes up often in interviews.

Basic

Q1. How do you get up to speed on a technology you have never used? Answer. Describe a method, not a list of resources: find the problem it solves and what came before it; first-pass the official architecture docs or paper; draw the architecture from memory and check it; build a toy that measures something you care about; read its known failure modes or incident reports; and write a one-page summary for your team, which exposes what you do not yet understand.

Q2. Why does `hash(key) mod N` cause problems when you add a cache node? Answer. With uniform hashing, about $N/(N+1)$ of keys change owner, so most of the cache misses at once and the load falls on the database. Consistent hashing moves only about $1/(N+1)$ of keys, all of them to the new node.

Intermediate

Q3. What do virtual nodes fix in consistent hashing, and what do they cost? Answer. With one position per node, arc lengths vary widely, so one node can carry two or three times its share. A dead node's whole load also lands on a single successor. With V vnodes per node, load evens out, the spread shrinking roughly as $1/\sqrt{V}$, and failure load is spread over many nodes. The costs are $O(NV)$ memory for the ring, larger membership metadata and more key ranges to manage during rebalancing.

Q4. Tell me about a postmortem you learned from. Answer. Choose a public report, such as the 2017 S3 disruption. Describe the trigger (a mistyped input to a capacity-removal command), the contributing factors (no minimum-capacity safeguard, and subsystems that had not been fully restarted in years at their current scale), and the design changes (slower removal, capacity floors, cell partitioning). Then say what you changed in your own work because of it.

Advanced

Q5. Your organization's design reviews keep missing failure modes that later cause incidents. How would you change the process? Answer. Require a failure table in the design template (what goes wrong, why, detection, recovery, prevention); give one reviewer the explicit failure-hunting role; link each design to postmortems of similar architectures; and track whether later incidents were or were not anticipated in the design. That is deliberate practice at team scale, with feedback from real outcomes.

Q6. A ring-based cache shows one node at 3× average CPU even with 200 vnodes per node. Diagnose it. Answer. With 200 vnodes, the ring itself should keep the busiest node within roughly 10 % of the mean, so look elsewhere. The likely causes are a hot key, which hashing cannot split; clients with inconsistent membership sending one range to the wrong node; or a heterogeneous node, such as a smaller instance or a noisy neighbor. Check top-K key metrics and compare ring versions across clients. Fixes include replicating hot keys, adding a local cache tier, bounded-load hashing and versioned membership.

3.14 Practical exercises

1. **Run the loop.** Take the cache-aside pattern (§20.7) through all ten steps. Turn in the ten artifacts, and use the journal template (§3.10.3) for at least one of them.
2. **Extend the ring.** Add `get_nodes(key, r)` to the §3.3 `HashRing` so that it returns r distinct physical nodes. Measure the fraction of keys whose replica set changes when you remove a node.
3. **Measure the vnode curve.** For $V \in \{1, 10, 50, 100, 500\}$, plot max/mean load across 10 nodes. Check whether the excess over 1.0 shrinks roughly as $1/\sqrt{V}$.
4. **Three passes.** Read the Dynamo paper with Keshav's method. Turn in the five Cs after pass 1, a one-paragraph summary after pass 2, and, after pass 3, a list of three assumptions that would not hold for a bank ledger.
5. **Critique a post.** Choose a recent engineering blog post about a migration. Score it against Table 3.6 and rewrite its main claim with an evidence label.

3.15 Design challenges

1. **Design your own learning system.** Treat your next six months of study as a system. State requirements (hours per week, target level), components (deck, journal, tracker, build repository), data flows (reading → artifacts → cards → reviews) and failure modes (backlog, burnout, illusion of competence). Add detection and recovery for each failure mode.
2. **A team incident library.** Design a lightweight internal system in which engineers log public and internal postmortems, tag them with this book's chapters and receive

spaced “redraw this architecture” prompts. Estimate storage and request rates for an organization of 2,000 engineers.

3. **Gap-driven onboarding.** Design a 90-day onboarding plan for an experienced engineer joining an ML-infrastructure team from a web-backend background. Use the gap formula, Table 3.11 and specific chapters from Book II.

3.16 References

1. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems* (Ch. 15, “Postmortem Culture: Learning from Failure”). O’Reilly, 2016. https://sre.google/sre-book/postmortem-culture/
2. Bjork, R. A. *Memory and metamemory considerations in the training of human beings*. In J. Metcalfe and A. P. Shimamura (eds.), *Metacognition: Knowing about Knowing*, MIT Press, 1994.
3. Burrows, M. *The Chubby Lock Service for Loosely-Coupled Distributed Systems*. OSDI, 2006.
4. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., Rohrer, D. *Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis*. Psychological Bulletin 132(3), 2006.
5. Cepeda, N. J., Vul, E., Rohrer, D., Wixted, J. T., Pashler, H. *Spacing Effects in Learning: A Temporal Ridgeline of Optimal Retention*. Psychological Science 19(11), 2008. https://pubmed.ncbi.nlm.nih.gov/19076480/
6. Chase, W. G., Simon, H. A. *Perception in Chess*. Cognitive Psychology 4(1), 1973.
7. Chi, M. T. H., de Leeuw, N., Chiu, M.-H., LaVancher, C. *Eliciting Self-Explanations Improves Understanding*. Cognitive Science 18(3), 1994.
8. Corbett, J. C., et al. *Spanner: Google’s Globally-Distributed Database*. OSDI, 2012.
9. Dean, J., Barroso, L. A. *The Tail at Scale*. Communications of the ACM 56(2), 2013.
10. Dean, J., Ghemawat, S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI, 2004.
11. DeCandia, G., et al. *Dynamo: Amazon’s Highly Available Key-value Store*. SOSP, 2007.
12. Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., Willingham, D. T. *Improving Students’ Learning With Effective Learning Techniques: Promising Directions From Cognitive and Educational Psychology*. Psychological Science in the Public Interest 14(1), 2013.
13. Ebbinghaus, H. *Memory: A Contribution to Experimental Psychology* (trans. H. A. Ruger and C. E. Bussenius). Teachers College, Columbia University, 1913 (German original 1885).
14. Eisenbud, D. E., et al. *Maglev: A Fast and Reliable Software Network Load Balancer*. NSDI, 2016.

15. Ericsson, K. A., Krampe, R. T., Tesch-Römer, C. *The Role of Deliberate Practice in the Acquisition of Expert Performance*. Psychological Review 100(3), 1993.
16. Ghemawat, S., Gobioff, H., Leung, S.-T. *The Google File System*. SOSP, 2003.
17. Gunawi, H. S., et al. *Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages*. ACM SoCC, 2016.
18. Hunt, P., Konar, M., Junqueira, F. P., Reed, B. *ZooKeeper: Wait-free Coordination for Internet-scale Systems*. USENIX ATC, 2010.
19. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., Lewin, D. *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*. ACM STOC, 1997.
20. Karpicke, J. D., Blunt, J. R. *Retrieval Practice Produces More Learning than Elaborative Studying with Concept Mapping*. Science 331(6018), 2011.
21. Keshav, S. *How to Read a Paper*. ACM SIGCOMM Computer Communication Review 37(3), 2007.
22. Kornell, N., Bjork, R. A. *Learning Concepts and Categories: Is Spacing the "Enemy of Induction"?* Psychological Science 19(6), 2008.
23. Kreps, J., Narkhede, N., Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB Workshop, 2011.
24. Lakshman, A., Malik, P. *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review 44(2), 2010.
25. Lamping, J., Veach, E. *A Fast, Minimal Memory, Consistent Hash Algorithm*. arXiv:1406.2294, 2014. https://arxiv.org/abs/1406.2294
26. Lamport, L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM 21(7), 1978.
27. Luu, D. *A collection of postmortems* (GitHub repository). https://github.com/danluu/post-mortems
28. Macnamara, B. N., Hambrick, D. Z., Oswald, F. L. *Deliberate Practice and Performance in Music, Games, Sports, Education, and Professions: A Meta-Analysis*. Psychological Science 25(8), 2014.
29. Mirrokni, V., Thorup, M., Zadimoghaddam, M. *Consistent Hashing with Bounded Loads*. ACM-SIAM SODA, 2018. https://arxiv.org/abs/1608.01350
30. Ongaro, D., Ousterhout, J. *In Search of an Understandable Consensus Algorithm*. USENIX ATC, 2014.
31. Pashler, H., McDaniel, M., Rohrer, D., Bjork, R. *Learning Styles: Concepts and Evidence*. Psychological Science in the Public Interest 9(3), 2008.
32. Roediger, H. L., Karpicke, J. D. *Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention*. Psychological Science 17(3), 2006.
33. Rohrer, D., Taylor, K. *The Shuffling of Mathematics Problems Improves Learning*. Instructional Science 35, 2007.
34. Soderstrom, N. C., Bjork, R. A. *Learning Versus Performance: An Integrative Review*. Perspectives on Psychological Science 10(2), 2015.

35. Thaler, D. G., Ravishankar, C. V. *Using Name-Based Mappings to Increase Hit Rates*. IEEE/ACM Transactions on Networking 6(1), 1998.
36. Yuan, D., et al. *Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems*. OSDI, 2014.
37. Amazon Web Services [AWS 2017]. *Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region*. 2017. https://aws.amazon.com/message/41926/

3.17 Open verification items

- [AWS 2017] URL and wording of the S3 post-event summary could not be fetched from this environment (network restriction); the summary in §3.9.3 is from the author's knowledge of the report and should be checked against the primary text.
- Roediger and Karpicke (2006) Experiment 2 one-week figures (~61 % STTT vs ~40 % SSSS) were confirmed only through secondary summaries; verify against the paper's Figure/Results.
- Macnamara et al. (2014): headline 12 % of variance confirmed; the domain ordering (lower in education and professions than in games, music, sports) is from memory and should be checked.
- Attribution of "What I cannot create, I do not understand" to Feynman's blackboard (Caltech Archives, 1988): widely documented; add an archival citation if one is required.
- Leitner intervals (1, 3, 7, 16, 35 days) are a teaching choice, not an empirically derived schedule; the text says so.
- The "~1/√V" load-spread scaling is stated as an approximation; a formal bound should be cited or derived in §14.8.

Part II — Computer Science Foundations

Chapter 5 — Computer Architecture

> **Level:** 1 · **Track:** Shared · **Estimated reading time:** 3 h

Every distributed system, however large, is built out of single machines. When a database is “slow”, a queue “backs up” or a service “needs more threads”, the real cause almost always comes down to one of a small number of physical facts: it takes about 100 ns to fetch a value from main memory, a spinning disk can perform only about 100 random reads per second, a network card can only move so many bytes per second, and a CPU core can do useful work only when its data is close at hand. This chapter teaches those facts from the ground up. You need no prior knowledge of hardware. By the end you will be able to explain *why* system designers favor sequential I/O, batching, caching and data locality, and you will be able to back those preferences with numbers.

Throughout the chapter we use decimal units for estimation (KB, MB, GB = 10^3, 10^6, 10^9 bytes) and binary units where hardware is built in powers of two (KiB, MiB, GiB = 2^{10}, 2^{20}, 2^{30} bytes), for example cache sizes and page sizes. Hardware numbers are labeled “as of , approximate”. They change every product generation, so learn the **orders of magnitude and the ratios**, which are far more stable than any single figure.

5.1 Learning objectives

After this chapter you should be able to:

- Describe what a CPU core does on every clock cycle, and explain pipelining, branch prediction, superscalar and out-of-order execution, and simultaneous multithreading (hyperthreading) in plain language.
- Calculate CPU execution time from instruction count, cycles per instruction and clock rate, and estimate the cost of branch mispredictions.
- Explain how L1, L2 and L3 caches, cache lines and associativity work, and calculate average memory access time for a multi-level cache.
- Explain cache coherence using the MESI protocol, recognize false sharing in code, and fix it.
- Draw the memory hierarchy from registers to archival storage with dated latency, capacity and cost figures.
- Calculate hard disk access time from seek time, rotational latency and transfer time, and explain why random I/O on a disk is hundreds of times slower than sequential I/O.

- Explain how flash SSDs store data (pages, blocks, erase-before-write), what write amplification is, and why NVMe outperforms SATA.
- Compare programmed I/O, interrupts, DMA, polling and kernel bypass.
- Diagnose whether a workload is CPU-bound or I/O-bound and size a thread pool accordingly.
- Estimate how long it takes to scan 1 GB from RAM, SSD, HDD or the network, and explain the difference between latency and bandwidth.
- Explain NUMA and its effect on multi-socket servers.
- Explain what SIMD and hardware accelerators do and when they matter.
- Map hardware facts to system design decisions: sequential I/O, batching, caching, data locality and queue depth.

5.2 Prerequisites

- Powers of two and powers of ten — §4.3 (cache sizes, addresses and capacities are powers of two).
- Units: bits, bytes, rates and prefixes — §4.5 (we convert between ns, µs, ms and between MB/s and Gbit/s constantly).
- Little's Law — §4.11 (we use it to relate queue depth, latency and throughput for memory and SSDs).
- Latency numbers every engineer should know — §4.17 (this chapter explains the hardware *behind* those numbers).
- Basic programming in any language; the code examples use C and Python, and every line that matters is explained.

5.3 The CPU: cores, registers, clock and the instruction pipeline

5.3.1 What a CPU does

> DEFINITION
>
> **Definition — Central processing unit (CPU).** The hardware that executes a program's **instructions**: simple operations such as "add these two numbers", "load a value from memory address X", or "if this value is zero, jump to instruction Y". A modern CPU package contains several independent **cores**, each of which can execute its own stream of instructions.

First-principles explanation

Imagine a restaurant kitchen. The **chef** is a CPU core. The **recipe** is the program: a numbered list of small steps. The chef's **two hands and the cutting board** are the **regis-**

ters, a tiny number of slots holding the values being worked on right now. The **counter beside the stove** is the cache, the **walk-in fridge** at the back of the kitchen is main memory (RAM), and the **supplier's warehouse across town** is the disk. The chef works at astonishing speed with whatever is in hand, but any trip to the fridge or the warehouse stops the cooking. Most of the techniques in this chapter are tricks for keeping the chef's hands full.

The analogy also explains cores: a kitchen with eight chefs, each following their own recipe, can prepare eight dishes at once — provided they do not keep reaching for the same pan.

Why does this exist?

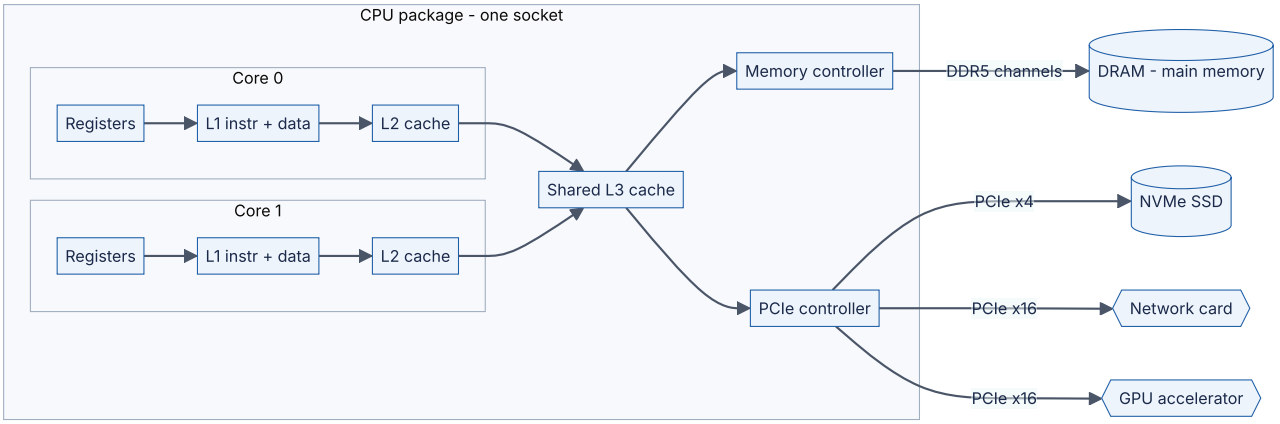
Early computers ran one instruction at a time and memory was about as fast as the processor. Since the 1980s processor speed has grown much faster than memory speed — the so-called **memory wall** [Hennessy & Patterson 2017, ch. 2]. Every feature in a modern core (caches, pipelines, prediction, out-of-order execution) exists to keep a very fast execution engine busy despite slow memory and unpredictable control flow.

How it works internally

At its simplest a core repeats the **fetch–decode–execute cycle**:

1. **Fetch.** Read the next instruction from memory at the address held in the **program counter** (PC) register.
2. **Decode.** Work out what the instruction means: which operation, which registers, which memory address.
3. **Execute.** Perform the operation in an execution unit — the **arithmetic logic unit** (ALU) for integer maths, the floating-point unit for decimals, the load/store unit for memory.
4. **Memory access.** If the instruction reads or writes memory, do so (via the caches, §5.4).
5. **Write back.** Store the result in the destination register and advance the program counter (or jump, if the instruction was a branch).

Figure 5.1 — Inside a single-socket server.



Reading the diagram. The package (the chip you hold in your hand) contains several **cores**; we show two, but server parts in 2025 carry anywhere from 8 to well over 100. Each core has private **registers**, a private **L1 cache** split into an instruction half and a data half, and usually a private **L2 cache**. All cores share an **L3 cache**, which connects to the **memory controller** and the **PCIe controller**. The memory controller drives several **DDR5 channels** to the DRAM modules (main memory). The PCIe controller connects storage (NVMe SSDs), the network card and accelerators such as GPUs over **PCIe lanes** (x4 = four lanes, x16 = sixteen). Arrows point outward from the core because the core initiates requests; data flows back along the same path. All of these paths are synchronous from the core's point of view (an instruction waits for its load), except device I/O, which is asynchronous via DMA and interrupts (§5.7). Every arrow is a potential bottleneck: the further right you go, the higher the latency and the lower the bandwidth per byte of capacity. The storage boundary that matters most is the one between DRAM (volatile, lost on power failure) and the SSD (persistent). The machine scales up by adding cores, memory channels, sockets (§5.10) and PCIe devices; beyond that you scale out across machines, which is the subject of most of this book.

5.3.2 Registers, the clock and the "iron law" of performance

DEFINITION

Definition — Register. A tiny, extremely fast storage slot inside a core, typically 64 bits wide for general-purpose registers. x86-64 exposes 16 general-purpose integer registers to programs; ARMv8-A (AArch64) exposes 31. Reading a register costs essentially nothing beyond the instruction that uses it.

DEFINITION

Definition — Clock rate. The frequency at which a core's internal circuitry steps from one state to the next, measured in hertz (cycles per second). A 4 GHz core has a **cycle time** of $1 / (4 \times 10^9)$ s = 0.25 ns.

The clock is like a metronome in the kitchen: on every tick each part of the core can advance its work by one step. For decades chip makers raised clock rates every year. Around 2005 this stopped: higher frequencies needed more voltage, and power (and therefore heat) grows faster than linearly with voltage and frequency. Clock rates have hovered around 2–6 GHz since (as of 2025, approximate), so the industry turned to doing **more work per cycle** and putting **more cores** on each chip [Hennessy & Patterson 2017, ch. 1]. This “end of free speed-ups” is why concurrency and horizontal scaling dominate modern system design.

How long a program takes is governed by three numbers multiplied together.

FORMULA

CPU execution time (the “iron law”)

$$T_{CPU} = N_{instr} \times CPI \times t_{cycle} = \frac{N_{instr} \times CPI}{f}$$

Symbol	Meaning	Units
T_{CPU}	CPU time to run the program	seconds
N_{instr}	Number of instructions executed	instructions
CPI	Average clock cycles per instruction (its inverse is IPC, instructions per cycle)	cycles/instruction
t_{cycle}	Duration of one clock cycle	seconds/cycle
f	Clock frequency, $1/t_{cycle}$	cycles/second (Hz)

WORKED EXAMPLE

Worked example — Running a program and paying for mispredicted branches.

A program executes $N_{instr} = 4 \times 10^9$ instructions on a 4 GHz core ($t_{cycle} = 0.25$ ns) at an average CPI of 0.5 (the core completes two instructions per cycle on average).

1. $T_{CPU} = 4 \times 10^9 \times 0.5 \times 0.25 \times 10^{-9}$ s $= 0.5$ s.

Now suppose 20% of instructions are branches, 5% of those branches are mispredicted (§5.3.4), and each misprediction costs 15 cycles.

2. Extra CPI from mispredictions $= 0.20 \times 0.05 \times 15 = 0.15$ cycles per instruction.
3. New CPI $= 0.5 + 0.15 = 0.65$.
4. New $T_{CPU} = 4 \times 10^9 \times 0.65 \times 0.25 \times 10^{-9} = 0.65$ s.

Cross-check: $4 \times 10^9 \times 0.2 \times 0.05 = 4 \times 10^7$ mispredictions $\times$ 15 cycles $= 6 \times 10^8$ cycles $= 6 \times 10^8 \times 0.25$ ns $= 0.15$ s extra. ✓

Interpretation. One branch in a hundred instructions going the “wrong way” costs 30% of the run time. **Engineering implication.** Raw clock speed is only one of three factors; CPI — which depends on caches, branches and dependencies — usually varies far more between two versions of the same code than clock rate varies between two CPUs.

5.3.3 The instruction pipeline

DEFINITION

Definition — Instruction pipeline. A technique in which the core splits instruction execution into stages and works on different stages of several consecutive instructions at the same time, like an assembly line.

First-principles explanation

A laundry has a washer, a dryer and a folding table. Doing one load start-to-finish before starting the next wastes the washer while the dryer runs. Instead you start load 2 in the washer the moment load 1 moves to the dryer. Each load still takes as long as before (its **latency** is unchanged), but a load *finishes* far more often (its **throughput** rises). A CPU pipeline is the same idea applied to instructions. This distinction between latency and throughput recurs at every level of system design (§5.12).

How it works internally

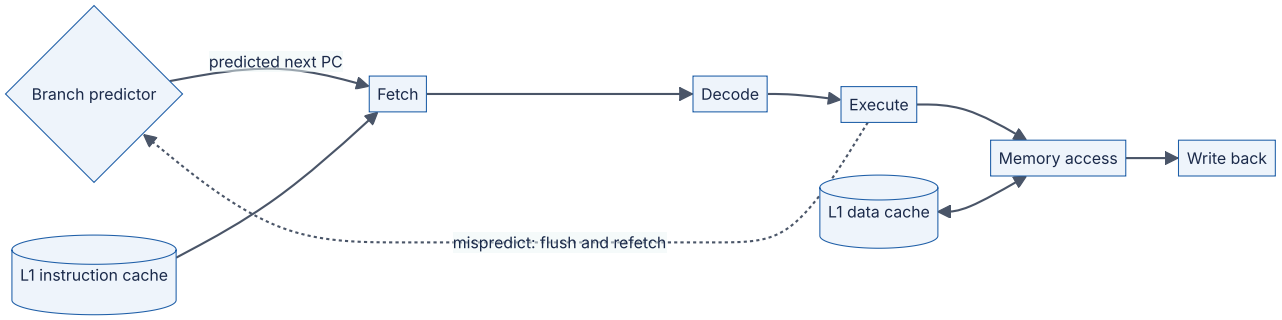
The classic teaching pipeline has five stages: instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and write back (WB) [Hennessy & Patterson 2017, appendix C]. Real cores have between roughly 10 and 20 stages (as of 2025, approximate), but the principle is identical.

Table 5.1 — Five instructions flowing through a five-stage pipeline.

Cycle →	1	2	3	4	5	6	7	8	9
Instruction 1	IF	ID	EX	MEM	WB				
Instruction 2		IF	ID	EX	MEM	WB			
Instruction 3			IF	ID	EX	MEM	WB		
Instruction 4				IF	ID	EX	MEM	WB	
Instruction 5					IF	ID	EX	MEM	WB

Without pipelining, five instructions would take 5 × 5 = 25 cycles. With it they take 9 cycles, and once the pipeline is full one instruction completes **every** cycle. Each individual instruction still takes 5 cycles.

Figure 5.2 — A pipelined core with a branch predictor.



Reading the diagram. Instructions move left to right through five stages; each arrow is a hand-off on a clock edge, so the whole chain is synchronous. The **fetch** stage reads instructions from the **L1 instruction cache**; the **memory** stage reads and writes the **L1 data cache** (bidirectional arrow). The **branch predictor** tells fetch which instruction comes next before the core actually knows. The dotted arrow is the failure path: when the execute stage discovers that a branch went the other way, the work in all earlier stages is wrong and is thrown away (flushed), and fetching restarts from the correct address. Pipelines fail to deliver one instruction per cycle when an instruction needs a result not yet produced (a **data hazard**), when two instructions need the same unit (a **structural hazard**), or when the next instruction is unknown (a **control hazard**). Designers scale this structure by making it wider (§5.3.5) rather than much deeper, because deeper pipelines make each misprediction more expensive.

5.3.4 Branch prediction

DEFINITION

Definition — Branch prediction. Hardware that guesses the outcome of a conditional jump (a **branch**, e.g. an `if` or a loop condition) before it is computed, so that the pipeline can keep fetching instructions. A wrong guess is a **misprediction** and forces a pipeline flush.

A long pipeline needs to know which instruction to fetch next on every cycle, but a branch's outcome is only known several stages later. Waiting would leave the pipeline empty for 10–20 cycles at every `if`, and typical code has a branch every five to ten instructions. So the core *guesses*. Modern predictors record the recent history of each branch and of the branches before it, and are right well over 95% of the time on typical code (approximate; it depends heavily on the program) [Hennessy & Patterson 2017, ch. 3]. A misprediction costs roughly 15–20 cycles on recent high-performance x86 cores (as of 2025, approximate).

The practical lesson: **predictable code runs faster**. Loops over sorted data, or code whose `if` almost always goes the same way, mispredict rarely. A filter over random data where each element passes with 50% probability mispredicts about half the time. Compilers can sometimes replace such branches with branch-free "conditional move" instructions, and data-processing engines often sort or partition data first precisely to make branches predictable.

5.3.5 Superscalar and out-of-order execution (intuition)

DEFINITION

Definition — Superscalar execution. A core that can issue and complete more than one instruction per cycle by having several execution units working in parallel.

DEFINITION

Definition — Out-of-order (OoO) execution. A technique in which the core executes instructions as soon as their inputs are ready rather than strictly in program order, while still making the results *appear* in program order (it **retires** them in order).

Back to the kitchen. A good chef reading a recipe does not stand idle while the water boils; they skip ahead to chopping onions, which does not depend on the water, and come back when the pot is ready. That is out-of-order execution. And a chef with two hands and

several burners can sear, stir and plate in the same moment; that is superscalar execution.

Internally, the core decodes a window of upcoming instructions — several hundred on recent high-end cores (as of 2025, approximate) — into a buffer called the **reorder buffer**. Each instruction waits in a **scheduler** until its inputs are ready, then dispatches to any free execution unit. Results are held until every earlier instruction has completed and are then retired in order, which preserves the illusion of sequential execution for the programmer. The key algorithmic idea dates to Tomasulo's work at IBM in 1967 [Tomasulo 1967].

Why does this matter to a system designer? Because the main thing OoO hides is **memory latency**. If a loop issues ten independent loads, an OoO core can have all ten outstanding at once and pay roughly one memory latency instead of ten. If each load depends on the previous one — walking a linked list, chasing pointers through a tree — nothing can overlap and the core pays the full latency every time. This is why arrays usually beat linked lists by a wide margin in practice, even when asymptotic complexity is identical, and why B-trees with wide nodes (§11.8) beat binary trees on real hardware.

5.3.6 Multiple cores and simultaneous multithreading (hyperthreading)

DEFINITION

Definition — Multi-core processor. A single chip containing several complete cores, each with its own pipeline and private caches, sharing the last-level cache and memory.

DEFINITION

Definition — Simultaneous multithreading (SMT). A technique in which one physical core runs two (occasionally more) instruction streams at once, interleaving them through the same execution units. Intel markets its implementation as **Hyper-Threading Technology**. Each stream appears to the operating system as a separate **logical CPU** (or **hardware thread**).

Even with out-of-order execution, a single thread often leaves execution units idle — for example while waiting for a cache miss. SMT gives the core a second recipe to work from: when thread A stalls, the core issues instructions from thread B. The core duplicates only the per-thread state (registers, program counter) and shares nearly everything else, so the extra silicon is small. The benefit depends heavily on the workload. Intel's original description reported gains of up to about 30% on some server benchmarks [Marr et al. 2002]; workloads that already saturate the execution units or the caches may gain nothing or even slow down, because two threads now compete for the same L1 and L2.

Three consequences for system design:

- **A "vCPU" in the cloud is usually a hardware thread, not a core.** On most x86 instance types at the major cloud providers, two vCPUs share one physical core (as of 2025; check your provider's documentation per instance family). Do not assume 16 vCPUs give 16 cores' worth of throughput.
- **Benchmarks must state whether SMT is on.** Doubling thread count from 8 to 16 on an 8-core, 16-thread machine will not double CPU-bound throughput.
- **Security.** Because SMT siblings share caches and execution units, they can leak information to each other through timing side channels. Some operators disable SMT for multi-tenant isolation, and some recent client CPUs ship without it (as of 2024).

Failure scenarios

Table 5.2 — CPU-level failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Service throughput flat after doubling vCPUs	vCPUs were SMT siblings; the workload saturates execution units	Per-core utilization near 100%; IPC drops as threads are added	Size by physical cores; scale out	Benchmark with physical-core counts in mind
Hot loop far slower on production data than in tests	Test data was sorted or uniform; production data makes branches unpredictable	`perf stat` shows high branch-miss rate	Sort/partition input; use branch-free code	Benchmark on realistic data distributions
Throttled CPU under sustained load	Thermal or power limits lower the clock rate	Clock frequency drops in telemetry	Improve cooling; spread load	Capacity-plan at sustained, not turbo, frequency

Common misconceptions

COMMON MISTAKE

"A higher GHz number means a faster CPU." Clock rate is only one factor in the iron law. A 3 GHz core with an IPC of 4 completes more work than a 4 GHz core with an IPC of 2. Compare CPUs on benchmarks of your own workload.

COMMON MISTAKE

"The CPU runs my code one line at a time, in order." It overlaps dozens to hundreds of instructions and executes them out of order; only the *visible* results are in order. That is why independent operations can be nearly free while dependent chains are slow.

5.4 CPU caches

5.4.1 Why caches exist

DEFINITION

Definition — CPU cache. A small, fast memory built from SRAM (static RAM) on the processor chip that holds copies of recently used main-memory data, so that most loads and stores do not have to wait for DRAM.

First-principles explanation

Picture a researcher in a library. The book on the **desk** is instantly available. The **shelf behind the chair** holds a few dozen books and takes a few seconds to reach. The **library stacks** on the same floor take a minute. The **off-site depository** takes a day. The researcher keeps the books they are using right now on the desk and the ones they will probably need soon on the shelf. When they fetch a book from the stacks, they bring back its neighbors too, because research tends to move along the shelf.

A CPU cache is that desk and shelf. It works because programs exhibit **locality of reference**:

- **Temporal locality** — data used recently is likely to be used again soon (a loop counter, a hot configuration object).
- **Spatial locality** — data near recently used data is likely to be used soon (the next element of an array, the next field of a struct).

Why does this exist?

A DRAM access costs about 80–120 ns (as of 2025, approximate), which at 4 GHz is 320–480 cycles. A core that could issue several instructions per cycle would spend almost all of its time waiting. Making all of main memory as fast as a register is physically impossible: fast memory (SRAM) needs about six transistors per bit, sits on the expensive processor die, and the longer the wires to it, the slower it gets. So designers build a **hierarchy**: tiny and very fast near the core, larger and slower further away.

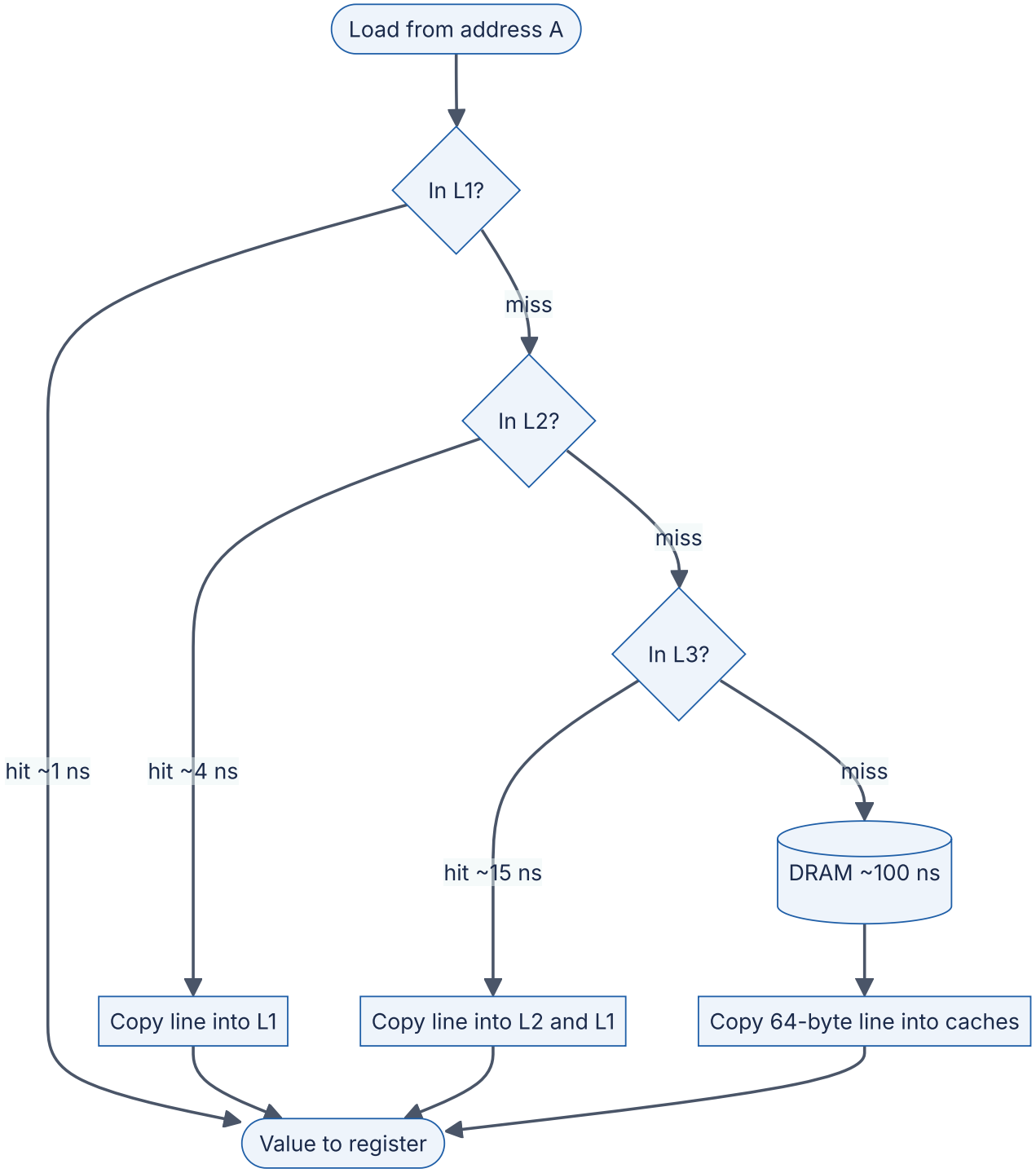
5.4.2 Cache levels

Table 5.3 — Typical CPU cache levels (as of 2024–2025, approximate; sizes and latencies vary by product).

Level	Typical size	Shared by	Latency (cycles)	Latency at ~4 GHz	Analogy
Registers	Tens of 64-bit values visible per thread	One hardware thread	0–1	< 0.3 ns	Hands
L1 data	32–64 KiB	One core	~4–6	~1–1.5 ns	Desk
L2	0.5–3 MiB	One core (usually)	~12–18	~3–5 ns	Shelf behind you
L3 (last-level cache)	8 MiB to several hundred MiB	All cores on the chip or chiplet	~40–80	~10–20 ns	Stacks on your floor
DRAM	GBs to TBs	Whole socket (or machine)	~300–500	~80–120 ns	Off-site depository

Sources for the ranges: vendor optimization manuals and independent measurements summarized in [Hennessy & Patterson 2017, ch. 2] and [Drepper 2007]; check `lscpu` on your own machine. The machine on which this chapter's examples were tested reports 48 KiB L1d and 2 MiB L2 per core.

Figure 5.3 — What happens on a load instruction.



Reading the diagram. A load instruction checks each level in turn. A **hit** returns the data at that level's latency; a **miss** passes the request to the next level. When the data finally arrives it is copied into the faster levels so the next access hits (this is temporal locality at work), possibly **evicting** some other line to make room. Each step is synchronous for the instruction that needs the value, but an out-of-order core keeps running independent instructions while it waits, and several misses can be outstanding at once (§5.9.2). The storage boundary is between L3 (on chip, SRAM) and DRAM (off chip). The main failure mode is a **miss storm**: a working set larger than the caches turns most loads into

100 ns trips. The way to scale is not a bigger cache — you cannot buy one — but a smaller, denser working set and access patterns the hardware can predict.

5.4.3 Cache lines and locality

DEFINITION

Definition — Cache line. The fixed-size unit in which data moves between memory and caches. On x86-64 and most ARM server cores a line is 64 bytes; some designs (for example Apple's M-series cores) use 128 bytes (as of 2024). A load of a single byte brings its whole line into the cache.

Because a whole line arrives at once, reading the first element of an array also brings in the next seven 8-byte elements "for free". Hardware **prefetchers** go further: when they detect that you are walking through memory in a regular pattern, they start fetching the next lines before you ask. Sequential access is therefore the best case in the entire memory system, and random access the worst.

Code example — row-major vs column-major traversal

```c
/* traverse.c — build: gcc -O2 -std=c11 traverse.c -o traverse */
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 8192                /* 8192 x 8192 ints = 256 MiB */

static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void) {
    int *m = malloc((size_t)N * N * sizeof *m);
    if (!m) return 1;
    for (size_t i = 0; i < (size_t)N * N; i++) m[i] = 1;

    long long sum = 0;
    double t0 = now();
    for (size_t r = 0; r < N; r++)          /* row-major: consecutive addresses
*/
        for (size_t c = 0; c < N; c++)
            sum += m[r * N + c];
    double t1 = now();
    for (size_t c = 0; c < N; c++)          /* column-major: jumps of N*4 bytes
*/
        for (size_t r = 0; r < N; r++)
            sum += m[r * N + c];
    double t2 = now();

    printf("row-major:    %.3f s\n", t1 - t0);
    printf("column-major: %.3f s\n", t2 - t1);
    printf("checksum: %lld\n", sum);         /* stops the compiler deleting
loops */
    free(m);
    return 0;
}
```

What to notice:

- Both loops add exactly the same 67,108,864 integers; only the **order** differs.
- The row-major loop touches consecutive 4-byte integers, so each 64-byte line serves 16 additions and the prefetcher streams lines ahead of time.
- The column-major loop jumps 8192 × 4 = 32,768 bytes between consecutive accesses, so every access lands on a different line and, because the matrix is 256 MiB, most of those lines have been evicted before the loop comes back for their neighbors.

- On the test machine (a cloud x86-64 VM, 2026) the row-major loop took 0.038 s and the column-major loop 0.667 s — about **17× slower** for identical arithmetic. Your ratio will differ, but the direction will not.
- `sum` is printed so that the optimizing compiler cannot prove the loops are useless and delete them — a classic benchmarking pitfall (§23.5).

5.4.4 Associativity: where a line may live

A cache must find a line quickly, so it cannot search every slot. Instead the memory address decides which **set** (group of slots) a line may occupy.

DEFINITION

Definition — Set-associative cache. A cache divided into S sets of W slots (**ways**). A line at a given address may be stored in any of the W ways of exactly one set. $W = 1$ is a **direct-mapped** cache; a cache with a single set holding every line is **fully associative**.

The analogy is a coat check with numbered racks. Your ticket number decides the rack (the set); within the rack your coat can go on any free hook (any way). If many guests' tickets map to the same rack, that rack overflows even though others are empty — a **conflict miss**.

FORMULA

Address decomposition in a set-associative cache

$$S = \frac{C}{B \times W}, \qquad \text{offset bits} = \log_2 B, \qquad \text{index bits} = \log_2 S$$

$$\text{set index} = \left\lfloor \frac{A}{B} \right\rfloor \bmod S, \qquad \text{tag} = \left\lfloor \frac{A}{B \times S} \right\rfloor$$

Symbol	Meaning	Units
C	Cache capacity	bytes
B	Line (block) size	bytes
W	Associativity (ways per set)	lines
S	Number of sets	sets
A	Memory address	byte address
tag	High-order address bits stored with the line to identify it	—

WORKED EXAMPLE

Worked example — Where does address 0x12345 go in a 32 KiB, 8-way cache with 64-byte lines?

1. Sets: $S = 32\,768/(64 \times 8) = 32\,768/512 = 64$ sets.
2. Offset bits $= \log_2 64 = 6$; index bits $= \log_2 64 = 6$.
3. Address $A = 0x12345 = 74\,565$.
4. Offset within the line $= 74\,565 \bmod 64 = 74\,565 - 1165 \times 64 = 74\,565 - 74\,560 = 5$.
5. Line number $= \lfloor 74\,565/64 \rfloor = 1165$; set index $= 1165 \bmod 64 = 1165 - 18 \times 64 = 13$.
6. Tag $= \lfloor 1165/64 \rfloor = 18$ (0x12).

Check in binary: 0x12345 = `0001 0010 0011 0100 0101`; the low 6 bits `000101` $= 5$ ✓, the next 6 bits `001101` $= 13$ ✓, the remaining bits `10010` $= 18$ ✓.

Interpretation. Addresses that differ by a multiple of $B \times S = 64 \times 64 = 4096$ bytes land in the **same set**. **Engineering implication.** A loop that strides through memory in steps of exactly 4 KiB (for example, reading one field from each of many page-aligned 4 KiB records) can use only one set — 8 lines, 512 bytes — of a 32 KiB cache. Power-of-two strides are a known performance trap; padding a row or record by one cache line often removes it [Drepper 2007, §6.2].

5.4.5 Average memory access time

FORMULA

Average memory access time (AMAT), applied recursively to each level

$$AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times (t_{L3} + m_{L3} \times t_{mem}))$$

Symbol	Meaning	Units
t_{L1}, t_{L2}, t_{L3}	Hit time at each cache level	ns
t_{mem}	DRAM access time	ns
m_{L1}	Fraction of all accesses that miss L1	dimensionless (0–1)
m_{L2}, m_{L3}	**Local** miss rate: fraction of accesses *reaching* that level that miss it	dimensionless (0–1)

The same formula, with two levels, is the **effective access time** of any cache in front of a slower store: $EAT = h \times t_{hit} + (1 - h) \times t_{miss}$, where h is the hit ratio. You will use it again for application caches (§20.3).

WORKED EXAMPLE

Worked example — Good locality versus poor locality.

Assume t_{L1} = 1 ns, t_{L2} = 4 ns, t_{L3} = 15 ns, t_{mem} = 90 ns.

Good locality (array scans, small hot data): m_{L1} = 0.05, m_{L2} = 0.20, m_{L3} = 0.30.

1. L3 term: $15 + 0.30 \times 90 = 15 + 27 = 42$ ns.
2. L2 term: $4 + 0.20 \times 42 = 4 + 8.4 = 12.4$ ns.
3. AMAT $= 1 + 0.05 \times 12.4 = 1 + 0.62 = 1.62$ ns.

Poor locality (random lookups in a large hash table): m_{L1} = 0.30, m_{L2} = 0.60, m_{L3} = 0.70.

1. L3 term: $15 + 0.70 \times 90 = 15 + 63 = 78$ ns.
2. L2 term: $4 + 0.60 \times 78 = 4 + 46.8 = 50.8$ ns.
3. AMAT $= 1 + 0.30 \times 50.8 = 1 + 15.24 = 16.24$ ns.

Interpretation. The same hardware delivers memory ten times slower on average for the second workload. **Engineering implication.** Data layout (compact structs, arrays instead of pointer-linked nodes, hot fields grouped together) can change performance by an order of magnitude without changing the algorithm. The application-level twin of this example: raising a cache hit ratio from 90% to 99% in front of a 100 µs SSD read, with a 1 µs hit, cuts effective access time from $0.9 \times 1 + 0.1 \times 100 = 10.9$ µs to $0.99 \times 1 + 0.01 \times 100 = 1.99$ µs, about 5.5× faster.

5.4.6 Cache coherence and the MESI protocol

DEFINITION

Definition — Cache coherence. The guarantee that all cores observe a single, consistent sequence of values for each memory location, even though each core keeps private copies in its own caches.

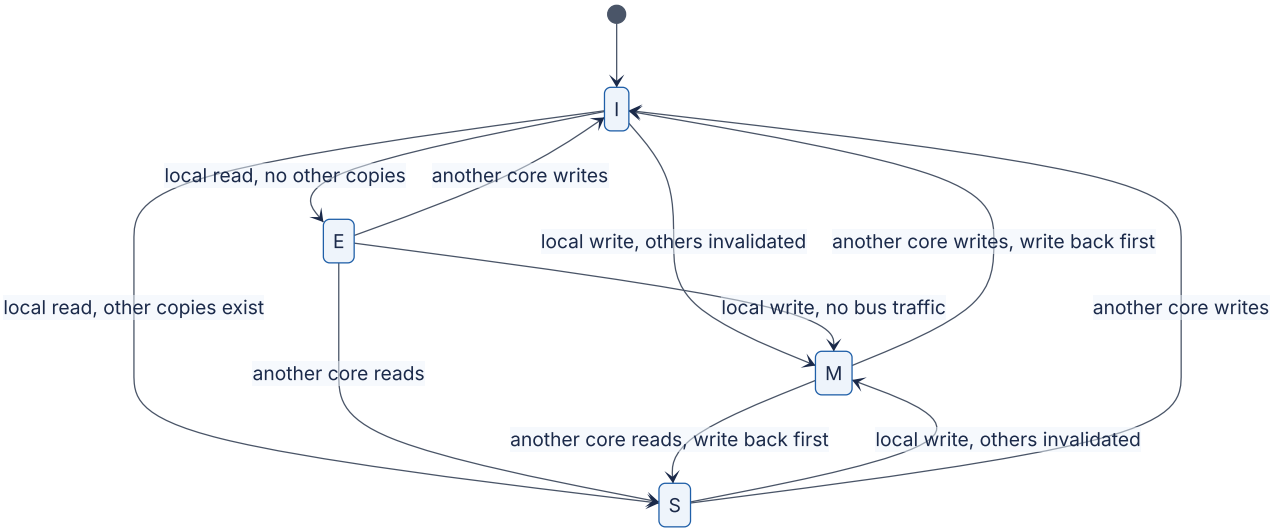
Suppose cores 0 and 1 both cache the variable `x`. Core 0 writes `x = 5`. If core 1 keeps reading its stale copy, the program breaks. Coherence protocols prevent this by tracking, for every cached line, which cores hold it and whether it has been changed. The textbook protocol is **MESI**, named after the four states a line can be in within one core's cache [Papamarcos & Patel 1984; Sorin, Hill & Wood 2011]:

- **Modified (M)** — this core has the only copy and has changed it; memory is stale. The core must write the line back before anyone else may use it.
- **Exclusive (E)** — this core has the only copy, and it matches memory. The core may write to it without telling anyone.

- **Shared (S)** — several cores may hold read-only copies that match memory.
- **Invalid (I)** — this cache does not hold a valid copy.

The analogy is a shared document with a single “edit pen”. Anyone may hold a read-only photocopy (Shared). To write, you must collect every photocopy and shred it (invalidate them), then you hold the pen (Modified). Real processors use extensions — Intel describes MESIF, AMD MOESI — that add states to reduce memory traffic, but the core idea is the same.

Figure 5.4 — MESI state transitions for one cache line in one core’s cache.



Reading the diagram. Each box is the state of *one* 64-byte line in *one* core’s cache; every core runs this machine independently for every line. “Local” events come from this core’s own instructions; “another core” events arrive as coherence messages over the on-chip interconnect (a ring or mesh linking cores, caches and memory controllers) or, between sockets, over the socket-to-socket link. Transitions caused by our own reads and writes are synchronous with the instruction; invalidations and write-backs triggered by other cores arrive asynchronously. Note the two expensive arcs: moving from Shared to Modified requires invalidating every other copy and waiting for acknowledgements, and a remote read of a Modified line forces a write-back. When two cores repeatedly write the same line, it ping-pongs between their caches, each transfer costing tens of nanoseconds on one chip and more across sockets. Coherence traffic is the reason “just add cores” stops scaling for shared, frequently written data.

5.4.7 False sharing

DEFINITION

Definition — False sharing. A performance problem in which threads on different cores write to *different* variables that happen to sit in the *same* cache line, so the coherence protocol bounces the line between cores as if they were sharing data.

Coherence works at the granularity of a whole line, not a variable. Two counters 8 bytes apart, each updated by its own thread, look to the hardware exactly like one heavily contended variable. The program is logically correct and has no locks, yet it scales *negatively*.

Code example — demonstrating and fixing false sharing in C

```c
/* false_sharing.c — build: gcc -O2 -std=c11 -pthread false_sharing.c -o fs */
#define _POSIX_C_SOURCE 199309L
#include <pthread.h>
#include <stdalign.h>
#include <stdatomic.h>
#include <stdint.h>
#include <stdio.h>
#include <time.h>

#define ITERATIONS 50000000ULL   /* 5 x 10^7 increments per thread */
#define CACHE_LINE 64            /* x86-64 line size; check yours */

/* Variant A: two counters 8 bytes apart -> same cache line. */
struct packed {
    _Atomic uint64_t a;
    _Atomic uint64_t b;
};

/* Variant B: each counter starts its own 64-byte cache line. */
struct padded {
    alignas(CACHE_LINE) _Atomic uint64_t a;
    alignas(CACHE_LINE) _Atomic uint64_t b;
};

static struct packed P;
static struct padded Q;

static void *worker(void *arg) {
    _Atomic uint64_t *counter = arg;
    for (uint64_t i = 0; i < ITERATIONS; i++)
        atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
    return NULL;
}

static double run(_Atomic uint64_t *x, _Atomic uint64_t *y) {
    struct timespec t0, t1;
    pthread_t ta, tb;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    pthread_create(&ta, NULL, worker, (void *)x);
    pthread_create(&tb, NULL, worker, (void *)y);
    pthread_join(ta, NULL);
    pthread_join(tb, NULL);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    return (double)(t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
}

int main(void) {
    printf("gap between counters: packed=%td B, padded=%td B\n",
           (char *)&P.b - (char *)&P.a, (char *)&Q.b - (char *)&Q.a);
    printf("packed (false sharing): %.2f s\n", run(&P.a, &P.b));
    printf("padded (separate lines): %.2f s\n", run(&Q.a, &Q.b));
    return 0;
}
```

What to notice:

- Each thread touches **only its own counter**; there is no logical sharing and no lock.
- In `struct packed` the counters are 8 bytes apart, so they share a 64-byte line. In `struct padded`, `alignas(64)` forces each counter onto its own line (the struct grows from 16 to 128 bytes).
- `atomic_fetch_add_explicit` compiles to a locked read-modify-write instruction, which requires the core to hold the line in the Modified state — every increment by one thread therefore steals the line from the other.
- On the 4-vCPU test VM (2026), three runs gave 1.57–1.69 s for the packed layout versus 0.31–0.34 s for the padded layout: about **5× slower purely because of memory layout**. With plain (non-atomic) increments, the effect is smaller and noisier because store buffers and compiler optimizations hide part of it — another reason to measure rather than assume.
- The fix costs memory (112 wasted bytes here). Pad only data that is genuinely written by different threads at high frequency: per-thread counters, queue head and tail indices, per-core statistics.

Real-world industry applications

INDUSTRY EXAMPLE

LMAX Disruptor (low-latency financial exchange). LMAX built an inter-thread messaging library for its trading platform and found that the head and tail sequence counters of a queue, written by different threads, suffered false sharing. The Disruptor pads its sequence counters to occupy their own cache lines and avoids locks on the hot path; the authors attribute a large part of its throughput advantage over conventional bounded queues to this “mechanical sympathy” with the cache hierarchy. Trade-off: padded structures waste memory and the design assumes a known line size. *Evidence: Primary* — [Thompson et al. 2011]

INDUSTRY EXAMPLE

Netflix (JVM service on larger cloud instances). When Netflix moved a stateless Java service to an instance type with three times the vCPUs, throughput rose only about 25% instead of the expected ~3×. Using hardware performance counters, engineers traced the problem to cache-line contention inside the JVM’s type-checking code — first false sharing, then, after a workaround, true sharing of a frequently written cache field. Fixing it brought throughput close to the expected scaling. Trade-off: diagnosing it required low-level counters that are not exposed on every cloud instance type. *Evidence: Primary* — [Filanovsky & Sane 2022]

Runtimes offer help: Java's internal `@Contended` annotation, C++17's `std::hardware_destructive_interference_size`, and explicit padding fields in Go and Rust.

Advantages and disadvantages of caches

Caches make the common case (good locality) almost as fast as registers, with no effort from the programmer. Their disadvantages are that performance becomes **data-dependent and hard to predict**, that coherence traffic limits how well shared, written data scales across cores, and that shared caches create interference between unrelated workloads — one tenant's large scan can evict another tenant's hot data (the **noisy neighbor** problem, Ch 47).

Failure scenarios

Table 5.4 — Cache-related failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Throughput falls as threads are added	False sharing or a hot shared counter	`perf c2c` on Linux; high cross-core cache-to-cache transfers	Pad or shard per-thread state	Per-thread or per-core data; aggregate occasionally
Latency spikes when a batch job runs on the same host	Batch job's scan evicts the service's working set from shared L3	LLC miss rate rises in sync with the batch job	Move the job; pin to other cores	Cache partitioning (e.g., Intel RDT) or separate hosts
Mysterious slowdown at certain array sizes	Power-of-two stride causes conflict misses in one set	Performance cliff at sizes like 4096 or 8192	Pad rows/records	Avoid power-of-two strides in hot loops
Pointer-heavy structure slow despite good big-O	Each hop is a dependent cache miss	Low IPC, high cache-miss counts	Switch to arrays / B-tree-like wide nodes	Design data structures for locality

Common misconceptions

> COMMON MISTAKE
>
> **"Lock-free code always scales."** Lock-free code still writes to shared cache lines. If many cores update the same atomic counter, the line ping-pongs between them and throughput can be *lower* than with one thread. Scalable designs avoid shared writes altogether (per-core counters, sharded state).

COMMON MISTAKE

“Caches only matter to C programmers.” Every language runs on the same hardware. Java object headers, Python object pointers and Go slices all have cache footprints; the gap between “array of values” and “array of pointers to objects” is often several-fold in any language.

5.5 RAM and the memory hierarchy

5.5.1 Main memory (DRAM)

DEFINITION

Definition — DRAM (dynamic random-access memory). The technology used for main memory. Each bit is stored as charge in a tiny capacitor that leaks and must be **refreshed** every few tens of milliseconds. DRAM is **volatile**: its contents are lost when power is removed.

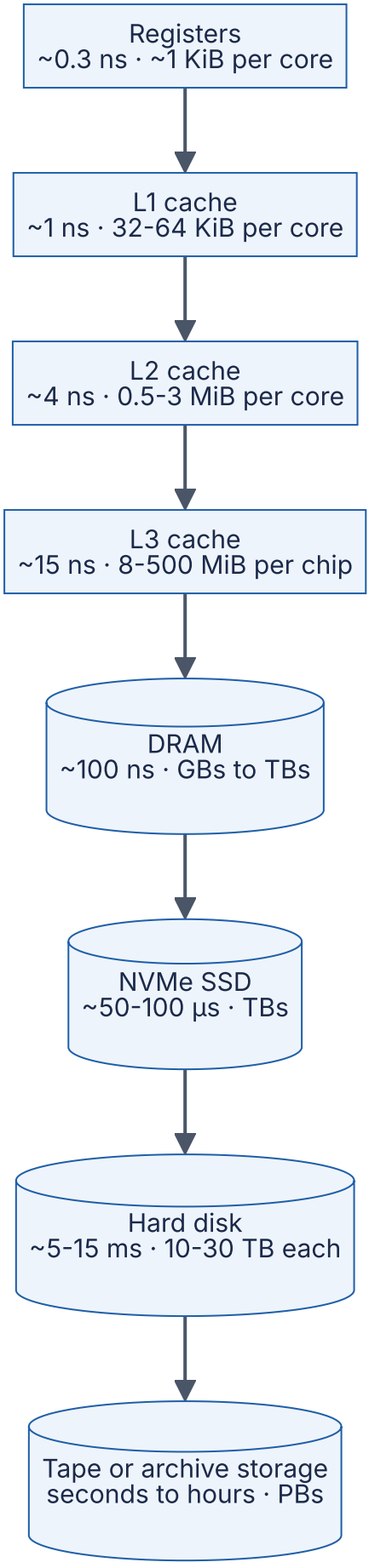
DRAM uses one transistor and one capacitor per bit, versus about six transistors for the SRAM in caches, so it is far denser and cheaper — but slower. Memory chips sit on modules (DIMMs) that plug into **channels**, each an independent 64-bit data path to the CPU’s memory controller (DDR5 splits each module’s path into two 32-bit subchannels, still 64 bits in total). Inside a chip, bits are arranged in banks of rows and columns; reading a row that is already “open” is faster than opening a new one, which is one more reason sequential access is cheaper than random access [Drepper 2007, §2]. Servers use **ECC** (error-correcting code) memory, which stores extra check bits to correct single-bit errors.

5.5.2 The memory hierarchy

DEFINITION

Definition — Memory hierarchy. The layered arrangement of storage technologies in a computer, from small, fast and expensive per byte (registers, caches) to large, slow and cheap per byte (disks, tape), in which each level acts as a cache for the level below it.

Figure 5.5 — The memory and storage hierarchy (as of 2024–2025, approximate).


Registers
~0.3 ns · ~1 KiB per core
L1 cache
~1 ns · 32-64 KiB per core
L2 cache
~4 ns · 0.5-3 MiB per core
L3 cache
~15 ns · 8-500 MiB per chip
DRAM
~100 ns · GBs to TBs
NVMe SSD
~50-100 µs · TBs
Hard disk
~5-15 ms · 10-30 TB each
Tape or archive storage
seconds to hours · PBs

Reading the diagram. Read it top to bottom as “faster, smaller, costlier per byte” turning into “slower, larger, cheaper per byte”. Each arrow means “misses at this level are served by the next level”. The levels above DRAM are managed by hardware automatically; the boundary between DRAM and SSD is managed by software — the operating system’s page cache (§6.6) or your database’s buffer pool (§13.4). The most important storage boundary is **volatility**: everything from DRAM upward vanishes on power loss, so durability requires reaching the SSD, disk or a remote replica (§6.14). Each step down costs roughly 3–1000× in latency. Systems “scale” this pyramid by keeping the working set as high up as possible: more RAM to cache hot data, SSDs instead of disks for random access, and disks or object storage only for cold, sequentially read data.

Table 5.5 — Memory hierarchy numbers (as of 2024–2025, approximate; prices are volatile).

Level	Typical capacity	Random-access latency	Sequential bandwidth	Approx. cost per GB	Persistent?
L1 / L2 / L3 cache	KiB to hundreds of MiB	~1 / ~4 / ~15 ns	Hundreds of GB/s per core (L1)	Not sold separately	No
DRAM (DDR5)	16 GB – several TB per server	~80–120 ns	~38 GB/s per DDR5-4800 channel; ~300–600 GB/s per socket	~$2–5	No
NVMe SSD (TLC flash)	1–30+ TB	~50–100 µs read	~3–7 GB/s (PCIe 4.0 x4); ~10–14 GB/s (PCIe 5.0 x4)	~$0.05–0.15	Yes
SATA SSD	0.25–8 TB	~80–150 µs read	~0.55 GB/s	~$0.05–0.10	Yes
HDD (7200 RPM)	4–30+ TB	~5–15 ms	~0.15–0.28 GB/s	~$0.015–0.03	Yes
LTO-9 tape	18 TB native per cartridge	Seconds to minutes (mount, wind)	~0.4 GB/s native	< $0.01	Yes

Sources: interface limits from the DDR5, PCIe and SATA specifications (bandwidth arithmetic in §5.9); device ranges from vendor datasheets of the period; LTO-9 figures from the LTO Program; general structure from [Hennessy & Patterson 2017, ch. 2] and [Bryant & O’Hallaron 2015, ch. 6]. Prices are retail/street approximations for 2024 and changed sharply in 2025–2026 for DRAM and flash; treat them as ratios, not quotes.

The ratios are the lesson: DRAM costs roughly 30–100× more per GB than flash, and flash roughly 3–5× more than disk, while latency falls by about 1,000× from DRAM to flash and 100× again from flash to disk. Choosing where data lives is an economic decision as much as a technical one.

5.6 Storage devices

5.6.1 Hard disk drives (HDDs)

DEFINITION

Definition — Hard disk drive (HDD). A storage device that records data magnetically on spinning platters, read and written by heads mounted on a moving arm (the **actuator**). Data is organized in concentric **tracks** divided into **sectors** of 512 or 4096 bytes.

First-principles explanation

Think of a record player. To play a particular song, you lift the arm and move it to the right groove (the **seek**), then wait for the start of the song to come round under the needle (**rotational latency**), then listen to the song (**transfer**). Moving the arm and waiting for the platter take milliseconds — millions of CPU cycles — whereas reading the data once the head is in place is fast. An HDD rewards you for reading long runs of adjacent data and punishes you for jumping around.

FORMULA

HDD access time

$$T_{access} = T_{seek} + T_{rot} + T_{transfer}, \qquad T_{rot} = \frac{1}{2} \times \frac{60}{RPM}, \qquad T_{transfer} = \frac{S}{R}$$

$$IOPS_{random} \approx \frac{1}{T_{access}}$$

Symbol	Meaning	Units
T_{seek}	Average time to move the head to the target track	ms
T_{rot}	Average rotational latency: half a revolution	ms
RPM	Spindle speed	revolutions per minute
S	Size of the request	bytes
R	Sustained media transfer rate	bytes/second
$IOPS_{random}$	Random I/O operations per second for one outstanding request	operations/second

WORKED EXAMPLE

Worked example — Random versus sequential reads on a 7200 RPM disk.

Assume T_{seek} = 8.5 ms (a typical desktop-class average), R = 200 MB/s (as of 2024, approximate; outer tracks are faster than inner tracks).

1. One revolution: $60/7200 = 0.008\,33$ s $= 8.33$ ms, so $T_{rot} = 8.33/2 = 4.17$ ms.
2. Transfer of a 4 KiB block: $4096/(200 \times 10^6) = 2.05 \times 10^{-5}$ s $= 0.02$ ms.
3. Random 4 KiB read: $T_{access} = 8.5 + 4.17 + 0.02 = 12.69$ ms.
4. Random IOPS $\approx 1/0.012\,69 \approx 79$ operations/s, so random throughput $= 79 \times 4096 \approx 0.32$ MB/s.
5. Sequential throughput ≈ 200 MB/s, which is $200/0.32 \approx 620\times$ the random throughput.
6. Reading 100 MB contiguously after one positioning: $12.67 + (10^8/2 \times 10^8) \times 1000 = 12.67 + 500 = 512.67$ ms, an effective $10^8/0.512\,67 \approx 195$ MB/s — 97.5% of the maximum.

Interpretation. For small random reads, 99.8% of the time is mechanical positioning (12.67/12.69). **Engineering implication.** On disks, design for large sequential transfers: append-only logs, LSM trees (§13.8), large blocks in distributed file systems (§15.5) and batch scans. Even a 15,000 RPM enterprise disk (T_{rot} = 2 ms, seek ≈ 3.5 ms) reaches only about $1/0.0055 \approx 180$ random IOPS.

HDDs remain the cheapest online storage per byte (Table 5.5), so they still hold most of the world's bulk data in object stores and backup systems — where access is overwhelmingly sequential [Ruemmler & Wilkes 1994].

5.6.2 Solid-state drives (SSDs) and flash memory

DEFINITION

Definition — Solid-state drive (SSD). A storage device with no moving parts that stores data in **NAND flash** memory cells, managed by an on-board controller running a **flash translation layer** (FTL).

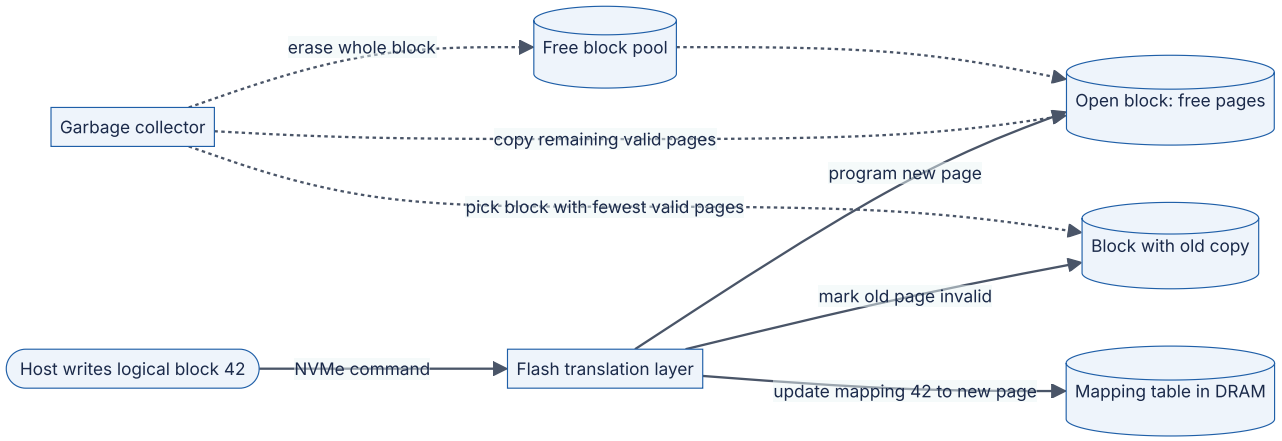
Flash is fast to read and has no seek time, but it has three peculiar rules that shape everything about SSD behavior [Agrawal et al. 2008]:

1. **Read and write in pages.** The smallest unit you can read or program (write) is a **page**, typically 4–16 KiB (as of 2024, approximate).
2. **Erase in blocks.** A page cannot be overwritten in place. It must first be erased, and erasure works only on a whole **erase block** of hundreds of pages (several MiB in modern chips).

3. **Cells wear out.** Each block survives a limited number of program/erase (P/E) cycles — on the order of 1,000–3,000 for TLC (three bits per cell) flash and fewer for QLC (four bits per cell) (as of 2024, approximate; vendors rarely publish exact figures).

The analogy is a notebook written in pen. You cannot rub out one line; to reuse a page you must tear out a whole chapter, and the binding can only take so many tear-outs. So when you "update" a line, you write the new version on a fresh page and cross out the old one. Periodically you copy the still-valid lines from a messy chapter to a fresh one and tear the old chapter out. That is exactly what an SSD controller does.

Figure 5.6 — How an SSD handles an overwrite and garbage collection.



Reading the diagram. The host addresses data by **logical block address** (LBA); the FTL translates LBAs to physical flash pages using a mapping table held in the SSD's own DRAM (backed up to flash). An overwrite from the host (solid arrows, synchronous with the write command) becomes "program a fresh page, then repoint the mapping", and the old page is merely marked invalid. In the background (dotted arrows, asynchronous), the **garbage collector** chooses a block with few valid pages, copies the survivors elsewhere and erases the block, returning it to the free pool. The failure point is garbage collection under sustained random writes: when free blocks run out, the host's writes wait behind GC, producing latency spikes. Drives mitigate this with **over-provisioning** (spare capacity invisible to the host) and the host helps with **TRIM**/deallocate commands that tell the drive which pages no longer hold useful data. The mapping table is also why SSDs need power-loss protection capacitors in enterprise models.

5.6.3 Write amplification and endurance

FORMULA

Write amplification factor and drive lifetime

$$WAF = \frac{\text{bytes written to flash}}{\text{bytes written by the host}}, \qquad \text{Lifetime (days)} \approx \frac{C \times PE}{W_{host} \times WAF}$$

Symbol	Meaning	Units
WAF	Write amplification factor (≥ 1 for a drive without compression)	dimensionless
C	Raw flash capacity	bytes
PE	Rated program/erase cycles per block	cycles
W_{host}	Host write volume per day	bytes/day

WORKED EXAMPLE

Worked example — How long will this SSD last?

A 1 TB TLC SSD, assumed rating $PE = 3000$ cycles, so total flash write budget $= 10^{12} \times 3000 = 3 \times 10^{15}$ bytes $= 3$ PB. A database writes $W_{host} = 500$ GB/day (0.5 drive writes per day).

1. With sequential, log-structured writes, $WAF \approx 2.5$: flash writes $= 500 \times 2.5 = 1250$ GB/day $= 1.25$ TB/day. Lifetime $= 3000$ TB/1.25 TB/day $= 2400$ days ≈ 6.6 years.
2. With small random in-place updates on a nearly full drive, $WAF \approx 5$: flash writes $= 2.5$ TB/day. Lifetime $= 3000/2.5 = 1200$ days ≈ 3.3 years.

Interpretation. The same host workload wears the drive out twice as fast purely because of the write *pattern*. **Engineering implication.** Write amplification compounds: an LSM tree's own compaction (§13.11) multiplies with the SSD's internal WAF. Prefer large, sequential, aligned writes; leave free space (or over-provisioning) on write-heavy drives; and monitor the drive's wear indicators (SMART / NVMe health log).

5.6.4 SATA versus NVMe

SSDs first plugged into the same **SATA** interface designed for hard disks. SATA III signals at 6 Gbit/s, and after its 8b/10b line encoding (10 bits on the wire per data byte) delivers at most 600 MB/s; real drives reach about 550 MB/s. Its command protocol, AHCI, offers

one command queue of 32 entries — ample for a disk that can do 100 operations per second, but a straitjacket for flash that can do hundreds of thousands.

NVMe (Non-Volatile Memory Express) is a protocol designed for flash that attaches drives directly to **PCIe** lanes. It supports up to about 65,535 I/O queues, each up to 65,536 commands deep [NVM Express Base Specification], so every CPU core can have its own submission and completion queue pair with no locking between cores. A PCIe 4.0 x4 link carries roughly 7.9 GB/s of payload after encoding (16 GT/s × 4 lanes × 128/130 ÷ 8 ≈ 7.9 GB/s), and fast drives deliver about 7 GB/s sequential and around a million random 4 KiB reads per second at high queue depth (as of 2024, approximate).

5.6.5 IOPS, throughput and queue depth

DEFINITION

Definition — IOPS. Input/output operations per second: the number of independent read or write requests a device completes per second, quoted for a stated request size (usually 4 KiB) and **queue depth** (number of requests outstanding at once).

FORMULA

Little's Law applied to a storage device

$$IOPS = \frac{QD}{L}, \qquad \text{Throughput} = IOPS \times S$$

Symbol	Meaning	Units
QD	Average number of requests outstanding (queue depth)	requests
L	Average latency per request	seconds
S	Request size	bytes

WORKED EXAMPLE

Worked example — Why a single-threaded reader cannot exploit an NVMe SSD.

An NVMe SSD has a random-read latency of $L = 80$ µs.

1. One request at a time ($QD = 1$): $IOPS = 1/(80 \times 10^{-6}) = 12\,500$; throughput $= 12\,500 \times 4096 = 51.2$ MB/s.
2. To reach the drive's rated 10^6 IOPS: $QD = IOPS \times L = 10^6 \times 80 \times 10^{-6} = 80$ requests outstanding.

Interpretation. A program that issues one blocking read, waits, then issues the next will use about 1% of a fast SSD. **Engineering implication.** To use modern storage you need **concurrency in the I/O path**: many threads, asynchronous I/O (io_uring, §6.13), or batched requests. The same law governs memory (§5.9.2) and network connections (§7.15).

5.6.6 Persistent memory and newer tiers

Persistent memory (PMem) is byte-addressable like DRAM — the CPU reads it with ordinary load instructions — but keeps its contents without power. The best-known product, Intel Optane persistent memory, offered larger capacities than DRAM at read latencies of a few hundred nanoseconds (approximate); Intel announced in 2022 that it was winding the business down. The ideas survive in capacitor-backed DRAM inside storage controllers and in **CXL** (Compute Express Link), which attaches extra memory over the PCIe physical layer at somewhat higher latency than local DRAM. Programming persistent memory requires explicit cache-line flushes and fences, because data still in a volatile CPU cache is not durable.

Table 5.6 — Storage device comparison (as of 2024–2025, approximate).

	HDD	SATA SSD	NVMe SSD	Persistent memory / CXL memory
What it is	Spinning magnetic platters	Flash behind a disk-era interface	Flash on PCIe with a flash-native protocol	Byte-addressable memory on the memory bus or CXL
How it works	Mechanical seek + rotation	FTL, 1 queue × 32 commands	FTL, up to ~64K queues	CPU loads/stores
Advantages	Cheapest per TB; good sequential speed	Drop-in replacement for disks	Very high IOPS and bandwidth	Nanosecond-scale access, large capacity
Disadvantages	~100 random IOPS; mechanical failure	Interface caps at ~550 MB/s	Wear-out; GC latency spikes	Niche, costly; Optane discontinued
Performance	~80–200 IOPS; ~0.2 GB/s	~50–100K IOPS; 0.55 GB/s	~0.5–1.5M IOPS; 3–14 GB/s	~100s ns latency
Cost per GB	~$0.02	~$0.05–0.10	~$0.05–0.15	Above DRAM-adjacent tiers
Choose when	Cold, bulk, sequential data	Legacy servers, budget	Databases, logs, anything random-access	Very large in-memory datasets; specialized logging

Real-world industry applications

INDUSTRY EXAMPLE

Apache Kafka (append-only log on ordinary disks). Kafka's designers observed that sequential disk throughput is hundreds of times higher than random-access throughput and that the operating system's page cache already keeps recently written data in RAM. Kafka therefore stores each partition as an append-only file, reads and writes it sequentially, and relies on the page cache rather than an in-process cache, which lets brokers sustain high throughput even on hard disks. Trade-off: the design favors sequential consumers; random lookups by key are not what it is for. *Evidence: Primary* — [Apache Kafka Documentation, Design]

Failure scenarios

Table 5.7 — Storage failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
p99 write latency spikes on a busy SSD	Garbage collection when free blocks are exhausted	Latency spikes correlate with sustained writes	Throttle writes; free space	Over-provision; TRIM; sequential writes
SSD wears out early	High write amplification from small random writes	SMART "percentage used" climbs fast	Replace drive; rebuild replica	Log-structured writes; endurance-rated drives
Database slow after moving from SSD to cheaper HDD tier	Random-access workload on ~100 IOPS devices	Queue depth high, low throughput, high await	Move back; add cache	Match the device to the access pattern
Data lost after power failure despite successful write	Data was still in volatile cache (OS, drive)	Corruption on restart	Restore from replica/backup	`fsync` and drives with power-loss protection (§6.14)

COMMON MISTAKE

"SSDs make access patterns irrelevant." Sequential I/O on an SSD is still faster than random I/O, writes are amplified by small random updates, and a single blocking thread cannot exploit the device's parallelism. SSDs shrink the penalty for randomness from ~600× to a few times; they do not remove it.

5.7 I/O operations

DEFINITION

Definition — I/O (input/output). Any transfer of data between the CPU/memory and a device outside it: storage, network cards, GPUs, keyboards. A **device controller** on each device does the physical work; the CPU communicates with it through **device registers** and shared memory.

The central question of I/O design is: *what does the CPU do while a slow device works?* Back in the kitchen, ordering from the supplier can be handled four ways. You can stand at the phone repeating "is it ready yet?" (**polling**). You can do something else and let the supplier ring a bell when the delivery arrives (**interrupts**). You can have the supplier's driver carry the goods straight into the fridge without you lifting a box (**DMA**). Or, for the busiest restaurants, you can bypass the front office entirely and deal with the driver directly (**kernel bypass**).

5.7.1 Programmed I/O

In **programmed I/O** (PIO) the CPU itself moves every word between the device and memory, reading and writing device registers in a loop. It is simple and still used for tiny,

infrequent transfers (configuring a device), but for bulk data it wastes the most expensive resource in the machine — CPU cycles — on copying.

5.7.2 Interrupts

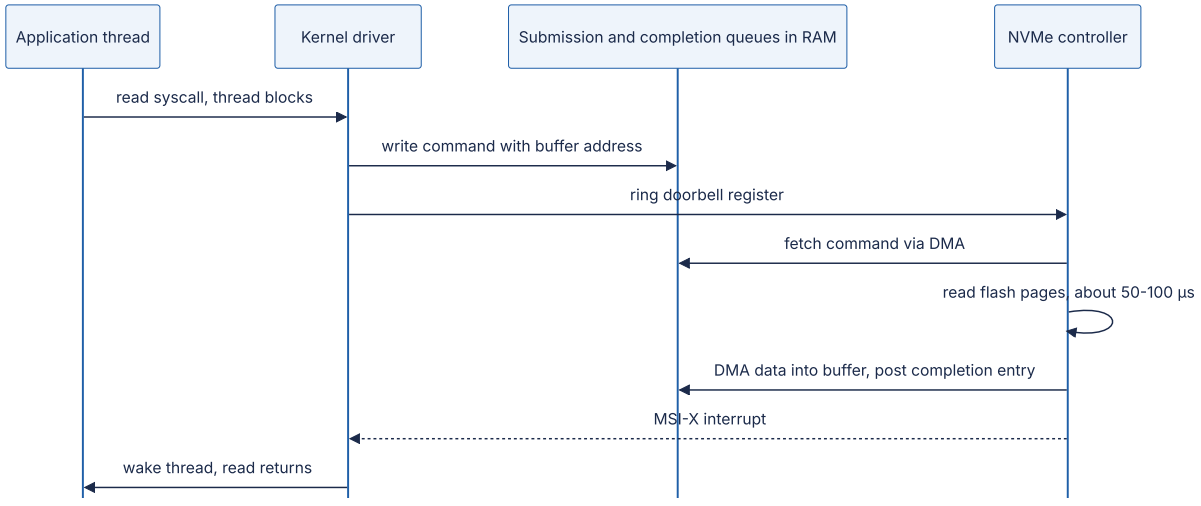
An **interrupt** is a hardware signal from a device that makes the CPU suspend its current work, run a short **interrupt handler** in the kernel, and then resume. The CPU starts an operation, goes off to run other threads, and is notified on completion. Modern PCIe devices raise **message-signaled interrupts** (MSI-X), which can be steered to a specific core. Interrupts are efficient when events are rare, but each one costs on the order of a microsecond or more of CPU time (handler entry, cache pollution, possible context switch; approximate). At millions of events per second — a single 10 Gbit/s link can deliver up to about 14.9 million minimum-size Ethernet frames per second (10^{10} bit/s ÷ (84 bytes × 8) on the wire, counting preamble and inter-frame gap) — one interrupt per packet would overwhelm the CPU.

5.7.3 Direct memory access (DMA)

DEFINITION

Definition — Direct memory access (DMA). A mechanism that lets a device controller read from or write to main memory directly, without the CPU copying each word; the CPU only sets up the transfer and is told when it finishes.

Figure 5.7 — Reading a block from an NVMe SSD with DMA and an interrupt.



Reading the diagram. Four participants: the application thread, the kernel's NVMe driver, the command queues living in ordinary RAM, and the SSD's controller. The application's `read` system call is synchronous *for the thread* — it blocks — but the CPU core is free to run other threads in the meantime. The kernel writes a command describing the request and the destination buffer into a submission queue in memory, then writes

to a **doorbell** register on the device over PCIe. From then on the device does the work: it fetches the command, reads flash, and **DMAs** the data straight into the buffer — no CPU copying. Finally it posts a completion entry and raises an interrupt (dashed arrow: asynchronous notification). The storage boundary is crossed only by the device. Failure points: the device may time out (the driver resets it and fails the I/O), and a buggy device with unrestricted DMA could overwrite arbitrary memory, which is why servers use an **IOMMU** to restrict which addresses each device may access. Scaling comes from many queue pairs (one per core) and many outstanding commands (§5.6.5).

5.7.4 Polling and interrupt coalescing

When events are very frequent, **polling** — a core repeatedly checking a completion queue — is cheaper than taking an interrupt per event, at the cost of burning a core even when idle. Practical systems mix the two. Network cards **coalesce** interrupts (one interrupt per batch of packets or per few microseconds), and Linux's NAPI network subsystem switches from interrupts to polling under load, then back again when traffic subsides. This is the batching principle of §5.12 appearing inside the kernel. How programs see all this — blocking calls versus non-blocking interfaces such as `epoll` and `io_uring` — is covered in §6.13; underneath, the device always works in parallel with the CPU, and software's job is to keep enough requests in flight.

5.7.5 Kernel bypass (introduction)

For each I/O, the conventional path crosses the user/kernel boundary, runs generic driver code, and often copies data between kernel and user buffers. At 10–100 Gbit/s or a million IOPS, those per-operation costs (each a few hundred nanoseconds to microseconds) become the bottleneck. **Kernel bypass** lets an application drive the device directly from user space:

- **DPDK** (Data Plane Development Kit) — user-space network drivers that poll NIC queues; used in software routers, load balancers and packet processors [DPDK Documentation].
- **SPDK** (Storage Performance Development Kit) — the same approach for NVMe storage [SPDK Documentation].
- **RDMA** (remote direct memory access) — network cards that read and write a *remote* machine's memory without involving the remote CPU, used in HPC and some databases and ML clusters (Ch 49, Ch 65).
- **io_uring** (Linux 5.1, 2019) — not a full bypass, but shared submission/completion rings between application and kernel that remove most per-operation system calls and can poll [Axboe 2019].

Bypass trades generality, security isolation and operational simplicity for raw speed: you dedicate cores to polling, give up the kernel's TCP stack or file system, and take on more

of the driver's responsibilities. Most services never need it; network appliances, exchanges and storage engines sometimes do.

Table 5.8 — I/O mechanisms compared.

Mechanism	CPU cost per transfer	Latency	Best for	Drawback
Programmed I/O	Very high (CPU copies every word)	Low for tiny transfers	Device configuration	Wastes CPU on bulk data
Interrupt-driven + DMA	Low per byte; ~µs per interrupt	Adds interrupt + wake-up time	General-purpose I/O	Interrupt storms at high event rates
Polling	A dedicated core, even when idle	Lowest	Very high event rates	Burns CPU and power
Coalescing / hybrid (NAPI)	Low	Slightly higher for single events	Network stacks under variable load	Tuning trade-off between latency and CPU
Kernel bypass (DPDK/SPDK/RDMA)	Lowest per operation	Lowest	Packet processing, storage engines, HPC	Complexity; loses kernel features

5.8 CPU-bound versus I/O-bound workloads

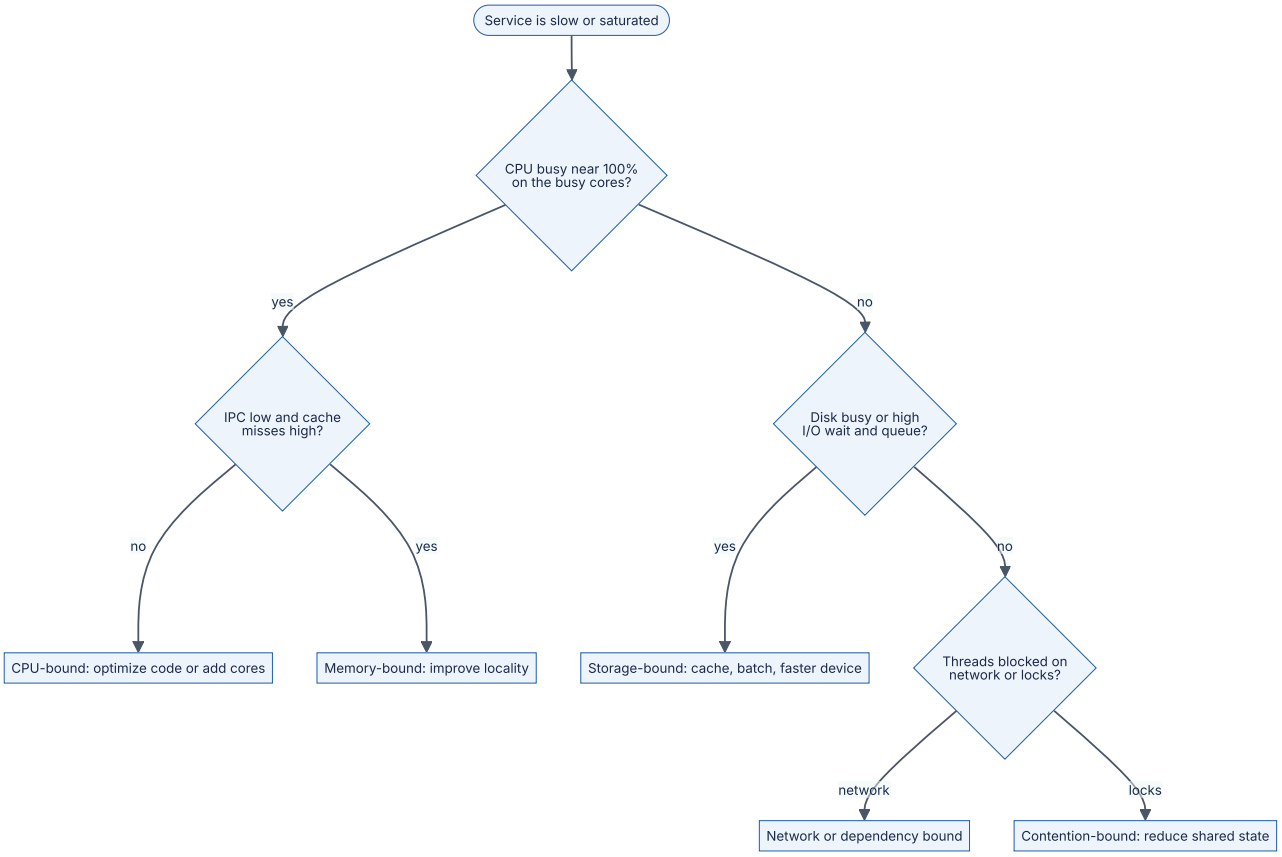
DEFINITION

Definition — CPU-bound and I/O-bound. A workload is **CPU-bound** when its speed is limited by how fast the processor can execute instructions; giving it a faster CPU or more cores speeds it up. It is **I/O-bound** when it spends most of its time waiting for storage or the network; a faster CPU barely helps, but more concurrency or faster devices do. A related category, **memory-bound**, is limited by memory latency or bandwidth (§5.9) — the CPU looks "busy" but is mostly stalled.

A cook who spends all day chopping is CPU-bound: hire more cooks. A cook who spends all day waiting for deliveries is I/O-bound: hiring more cooks only adds more people waiting — what helps is ordering many deliveries at once, or keeping stock closer to hand (caching).

5.8.1 How to tell

Figure 5.8 — Diagnosing what limits a workload.



Reading the diagram. Start at the top with a symptom and walk the decisions using measurements, not guesses. CPU utilization per core (not the machine average, which hides one saturated core) separates "computing" from "waiting". Among busy CPUs, instructions per cycle and cache-miss counters (from `perf stat` on Linux) separate genuine computation from stalls on memory. Among idle CPUs, disk utilization and queue length (`iostat -x`) reveal storage limits, and thread dumps or off-CPU profiles reveal whether threads wait on network calls to dependencies or on locks. Each leaf names a different remedy; applying the wrong one (more threads for a CPU-bound service, a faster CPU for an I/O-bound one) wastes money. This is the resource model of §2.6 applied to one machine; the systematic version is Brendan Gregg's USE method (§23.3) [Gregg 2020].

Table 5.9 — Signals that distinguish CPU-bound from I/O-bound workloads.

Signal	CPU-bound	I/O-bound
Per-core CPU utilization	Near 100% in user time	Low or moderate
`time` output for a command	user + sys ≈ real (× cores used)	real ≫ user + sys
Effect of a faster CPU	Proportional speed-up	Little effect
Effect of more threads	Helps up to the core count, then hurts	Helps until the device or dependency saturates
Typical examples	Compression, encryption, image resizing, JSON parsing at scale, model inference on CPU	Database queries over the network, file servers, API gateways, web crawlers

A caution about the Linux `%iowait` metric: it reports idle time during which some I/O was outstanding. It is a hint, not a measure of I/O capacity; a fully busy CPU can hide I/O waits, and on multi-core machines the number is easy to misread [Gregg 2020].

5.8.2 Why it matters for thread pools

FORMULA

Thread pool sizing heuristic [Goetz et al. 2006, §8.2]

$$N_{threads} = N_{cores} \times U_{target} \times (1 + \frac{W}{C})$$

Symbol	Meaning	Units
$N_{threads}$	Threads in the pool	threads
N_{cores}	Cores available to the pool	cores
U_{target}	Target CPU utilization (0–1)	dimensionless
W	Average time a task spends waiting (I/O, remote calls)	seconds
C	Average CPU time per task	seconds

WORKED EXAMPLE

Worked example — Sizing pools for two services on an 8-core machine.

1. *Image-thumbnail service* (CPU-bound): $W \approx 0$, so $N = 8 \times 1 \times (1 + 0) = 8$ threads. Goetz et al. suggest $N_{cores} + 1$ to cover the occasional page fault. Using 64 threads would add context switches and cache thrashing, not throughput.
2. *Order API* (I/O-bound): each request uses $C = 5$ ms of CPU and waits $W = 50$ ms on the database. At $U_{target} = 1$: $N = 8 \times (1 + 50/5) = 8 \times 11 = 88$ threads. At $U_{target} = 0.8$, leaving headroom: $8 \times 0.8 \times 11 = 70.4 \approx 70$ threads.
3. Maximum throughput of the order API is set by CPU: 8 cores$/5$ ms $= 1600$ requests/s at 100% CPU. Little's Law check: 1600 req/s $\times 55$ ms $= 88$ requests in flight ✓.

Interpretation. The I/O-bound service needs about 11× more threads than cores, because each thread spends 10/11 of its life waiting. **Engineering implication.** The formula assumes the dependency can absorb the load: 88 concurrent requests need about 88 database connections (§14.13). If the database saturates first, more threads only lengthen its queue. Event loops and async runtimes (§6.15) achieve the same concurrency with far fewer threads.

COMMON MISTAKE

“Python threads speed up CPU-bound work.” In CPython the global interpreter lock (GIL) lets only one thread execute Python bytecode at a time, so threads help I/O-bound Python code but not pure-Python CPU-bound code; use processes or native extensions (such as numpy) that release the lock. Free-threaded CPython builds exist experimentally since Python 3.13 (as of 2025), which may change this advice over time.

5.9 Memory latency and bandwidth

5.9.1 Two different numbers

DEFINITION

Definition — Latency and bandwidth (hardware). Latency is the time from issuing one request to receiving its first byte (for example, ~100 ns for a DRAM load). **Bandwidth** is the maximum rate at which data can be moved once transfers are flowing (for example, ~38 GB/s for one DDR5-4800 channel). Achieved bandwidth is **throughput**.

A motorway is the usual picture: latency is how long one car takes to drive from A to B; bandwidth is how many cars per hour the road can carry. Adding lanes raises bandwidth but does not make any single car arrive sooner. For hardware, latency improves slowly (DRAM latency has changed little in two decades), while bandwidth improves steadily with more channels and faster signaling [Hennessy & Patterson 2017, ch. 2]. Designs that need bandwidth can buy it; designs that need low latency must avoid the trip altogether — by caching, prefetching or co-locating data.

FORMULA

Peak DRAM bandwidth

$$BW_{peak} = R_{transfer} \times W_{bus} \times N_{channels}$$

Symbol	Meaning	Units
$R_{transfer}$	Data transfers per second per channel (the "4800" in DDR5-4800)	transfers/second (MT/s)
W_{bus}	Width of a channel's data path: 8 bytes (64 bits)	bytes/transfer
$N_{channels}$	Populated memory channels	channels

WORKED EXAMPLE

Worked example — How much bandwidth does a server socket have?

1. One DDR5-4800 channel: $4800 \times 10^6 \times 8 = 38.4 \times 10^9$ B/s $= 38.4$ GB/s.
2. A socket with 8 channels: $8 \times 38.4 = 307.2$ GB/s. With 12 channels (as on some 2023–2024 server CPUs): $12 \times 38.4 = 460.8$ GB/s.

Interpretation. These are theoretical peaks; sustained throughput on streaming benchmarks is typically 70–90% of peak (approximate). **Engineering implication.** Leaving channels unpopulated to save money (four DIMMs in an eight-channel server) halves memory bandwidth — a common, invisible misconfiguration for analytics and ML workloads.

5.9.2 Why one core cannot use all the bandwidth: Little's Law in memory

Bandwidth is only achieved if enough requests are in flight to cover the latency — the same law as for SSDs (§5.6.5).

WORKED EXAMPLE

Worked example — Memory-level parallelism.

Each core can track a limited number of outstanding L1 misses — on the order of 10–20 on recent x86 cores (as of 2024, approximate; hardware prefetchers into L2 add more). Assume 12 outstanding 64-byte misses and 100 ns latency.

1. Bytes in flight $= 12 \times 64 = 768$ B.
2. Throughput $= 768$ B/100 ns $= 7.68$ B/ns $= 7.68$ GB/s.
3. To saturate an 8-channel socket at 307.2 GB/s with a loaded latency of ~110 ns: bytes in flight $= 307.2 \times 10^9 \times 110 \times 10^{-9} = 33\,792$ B $= 528$ cache lines.

Interpretation. One thread doing demand loads tops out well below the socket's bandwidth; it takes many cores — or very effective prefetching — to keep 500+ lines in flight. **Engineering implication.** Memory-bound jobs (scans, aggregations, embedding lookups) scale with cores *until* the memory channels saturate, then stop, however many more cores you add. Pointer chasing is worst of all: with one outstanding miss, throughput is 64 B/100 ns = 0.64 GB/s.

5.9.3 Worked examples: scanning 1 GB

The single most useful exercise in this chapter is estimating how long it takes to read 1 GB (10^9 bytes) from each place it might live.

WORKED EXAMPLE

Worked example — Time to read 1 GB (as of 2024–2025, approximate rates).

Sequential reads: $T = 10^9$ B/throughput.

1. DRAM, one thread streaming at ~10 GB/s: $10^9/10^{10} = 0.1$ s $= 100$ ms. Whole socket at ~200 GB/s achieved: 5 ms.
2. NVMe SSD (PCIe 4.0) at ~7 GB/s: $10^9/(7\times10^9) \approx 0.143$ s $= 143$ ms.
3. SATA SSD at ~550 MB/s: $10^9/(5.5\times10^8) \approx 1.82$ s.
4. HDD at ~200 MB/s: $10^9/(2\times10^8) = 5$ s.
5. Network at 1 / 10 / 100 Gbit/s ($= 0.125$ / 1.25 / 12.5 GB/s): 8 s / 0.8 s / 0.08 s, before protocol overhead of a few percent.

Random reads: the same gigabyte read as many small, dependent requests.

6. DRAM, random 64-byte lines, one at a time: $10^9/64 = 15\,625\,000$ lines $\times$ 100 ns $= 1.5625$ s — about 16× slower than streaming the same memory.
7. NVMe, random 4 KiB reads at QD 1: $10^9/4096 \approx 244\,141$ reads $\times$ 80 µs ≈ 19.5 s. At QD 80 (~10^6 IOPS): $244\,141/10^6 \approx 0.24$ s.
8. HDD, random 4 KiB reads: $244\,141 \times 12.69$ ms ≈ 3098 s ≈ 51.6 minutes.

Interpretation. Between the best case (5 ms) and the worst (52 minutes) lies a factor of about 600,000 — for the *same* gigabyte. **Engineering implication.** The access pattern matters as much as the device: random HDD access is 620× slower than sequential HDD access, and random DRAM access is slower than sequential NVMe access. This is why databases build indexes that turn scans into a few targeted reads, why analytics systems store columns contiguously, and why caches exist at every level.

Table 5.10 — Time to read 1 GB, summary (as of 2024–2025, approximate).

Source and pattern	Rate or latency assumed	Time
DRAM, all cores streaming	~200 GB/s	~5 ms
DRAM, one thread streaming	~10 GB/s	~0.1 s
NVMe SSD, sequential	~7 GB/s	~0.14 s
10 Gbit/s network	1.25 GB/s	~0.8 s
DRAM, dependent random 64 B reads	100 ns each	~1.6 s
SATA SSD, sequential	0.55 GB/s	~1.8 s
HDD, sequential	0.2 GB/s	~5 s
NVMe SSD, random 4 KiB, QD 1	80 µs each	~20 s
HDD, random 4 KiB	12.7 ms each	~52 min

5.9.4 Latency numbers, then and now

Jeff Dean's list of "numbers everyone should know", presented in 2009 and building on a similar table by Peter Norvig, is the best-known summary of these facts [Dean 2009; Norvig 2001]. §4.17 discusses it in full; Table 5.11 shows how the hardware-specific entries compare with this chapter's figures.

Table 5.11 — Classic latency numbers versus approximate 2024–2025 values.

Operation	Dean 2009	Approx. 2024–2025 (this chapter)	What changed
L1 cache reference	0.5 ns	~1 ns	About the same in cycles; clocks stalled near 3–5 GHz
Branch mispredict	5 ns	~4–5 ns (15–20 cycles)	Little change
L2 cache reference	7 ns	~3–5 ns	Somewhat faster
Main memory reference	100 ns	~80–120 ns	Essentially unchanged
Read 1 MB sequentially from memory	250 µs	~50–100 µs per thread	Bandwidth grew ~3–5×
Disk seek	10 ms	~4–10 ms	Mechanics barely changed
Read 1 MB sequentially from disk	20 ms	~5 ms	Higher areal density
Random read from SSD	Not in 2009 list	~50–100 µs	SSDs became mainstream
Read 1 MB sequentially from NVMe SSD	Not in 2009 list	~0.15–0.3 ms	New tier

Versions of Dean's list differ slightly in individual entries; the durable lesson is the *shape*: memory latency stuck, bandwidth up, disks still mechanical, flash filling the gap between them.

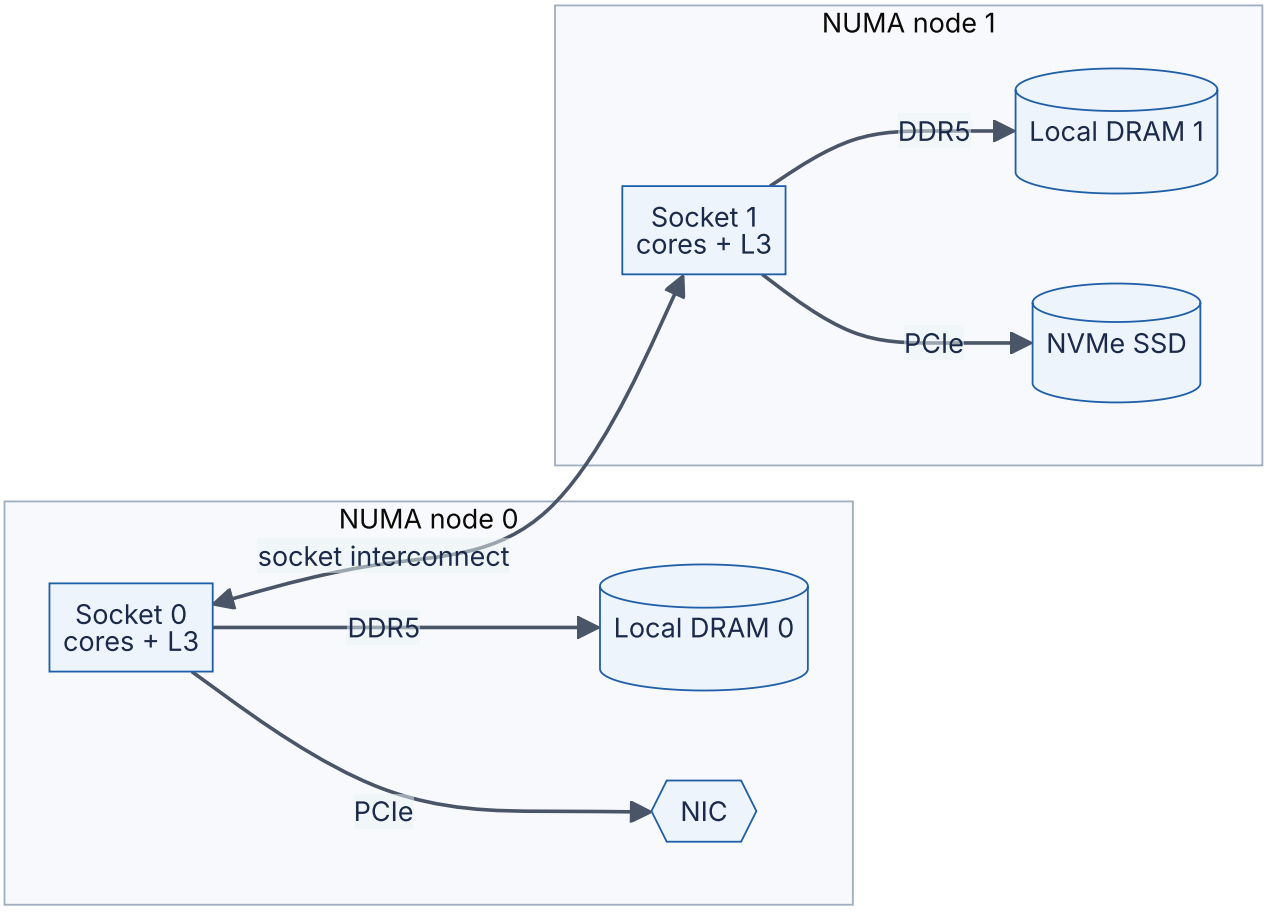
5.10 NUMA

DEFINITION

Definition — NUMA (non-uniform memory access). A memory architecture in which each processor socket (or each chiplet group within a socket) has its own directly attached memory. A core can access all memory in the machine, but accessing its **local** memory is faster than accessing **remote** memory attached to another socket.

Picture two kitchens joined by a corridor, each with its own fridge. A chef can fetch from either fridge, but the far one takes noticeably longer and the corridor can get crowded. NUMA exists because a single shared memory controller cannot scale to dozens of cores and hundreds of GB/s; attaching memory to each socket multiplies total bandwidth, at the price of uneven latency [Lameter 2013].

Figure 5.9 — A two-socket NUMA server.



Reading the diagram. Each socket, with its cores, its L3 cache, its memory channels and its PCIe devices, forms a **NUMA node**. A core on socket 0 reaches DRAM 0 directly; to reach DRAM 1 its request travels over the **socket-to-socket interconnect** (Intel UPI, AMD Infinity Fabric), adding latency — remote accesses commonly cost about 1.5–2× local ones (as of 2024, approximate; Linux reports relative distances via `numactl --hardware`, typically 10 for local and around 20 or more for remote). Devices are local to one node as well: packets arriving on a NIC attached to socket 0 are cheapest to process on socket 0's cores. The interconnect is shared and carries coherence traffic too, so it is both a latency penalty and a bandwidth bottleneck. Failure points are performance failures — a process whose threads run on node 1 while its memory sits on node 0. Scaling within the machine means keeping threads, their memory and their devices on the same node.

WORKED EXAMPLE

Worked example — The cost of ignoring NUMA.

Assume local DRAM latency 80 ns and remote 140 ns.

1. Two sockets, memory placed without regard to which node will use it: about half of accesses are remote. Average $= 0.5 \times 80 + 0.5 \times 140 = 40 + 70 = 110$ ns, 37.5% worse than all-local.
2. Four sockets, memory interleaved evenly: 75% remote. Average $= 0.25 \times 80 + 0.75 \times 140 = 20 + 105 = 125$ ns, 56% worse.

Interpretation. For memory-bound work, NUMA-unaware placement directly inflates latency and congests the interconnect. **Engineering implication.** On large servers, pin latency-sensitive processes to one node (`numactl --cpunodebind=0 --membind=0`), run one process per node rather than one huge process, or prefer smaller single-socket instances. Cloud VMs that span sockets expose NUMA topology too.

INDUSTRY EXAMPLE

MySQL “swap insanity” on two-socket servers. Jeremy Cole described MySQL servers with a buffer pool sized for the whole machine’s RAM that nevertheless swapped to disk. Linux’s default policy allocated memory from the local node of the allocating thread, so one NUMA node filled up while the other had free memory, and the kernel swapped instead of using the remote node. His recommended remedy was to start MySQL with memory interleaved across nodes (`numactl --interleave=all`), accepting uniformly higher latency in exchange for predictable behavior; later MySQL versions added an option to interleave the buffer pool. *Evidence: Primary* — [Cole 2010]

COMMON MISTAKE

“All RAM in a server is equally fast.” On multi-socket and many-chiplet servers it is not. Check `lscpu` and `numactl --hardware` before benchmarking, and keep the benchmark’s threads and memory on the same node or document that you did not.

5.11 SIMD and accelerators

5.11.1 SIMD

DEFINITION

Definition — SIMD (single instruction, multiple data). CPU instructions that apply the same operation to several data elements packed into one wide **vector register** at once — for example, adding eight pairs of 32-bit floats in a single instruction.

Instead of a cook slicing one carrot at a time, SIMD is a mandoline that slices eight at once with a single stroke. It pays off when the same operation runs over large arrays of uniform data: image and audio processing, compression, checksums, string search, database column scans and machine-learning arithmetic.

Table 5.12 — Common SIMD instruction sets (as of 2025).

Instruction set	Platform	Register width	32-bit floats per instruction
SSE family	x86	128 bits	4
AVX / AVX2	x86	256 bits	8
AVX-512	x86 (many server parts)	512 bits	16
NEON (Advanced SIMD)	ARM	128 bits	4
SVE / SVE2	ARM (some server parts)	128–2048 bits, implementation-defined	4–64

You rarely write SIMD instructions by hand. Compilers **auto-vectorize** simple loops, and libraries such as numpy, BLAS and database engines ship vectorized kernels. Your job is to give them data they can vectorize: contiguous arrays of one type, not lists of objects. The database research system MonetDB/X100 showed that processing data in cache-sized *vectors* of column values, rather than one row at a time, cut interpretation overhead and enabled SIMD, which shaped modern vectorized analytics engines [Boncz et al. 2005].

```
"""simd.py — interpreted loop vs vectorized (SIMD-backed) numpy."""
import time
import numpy as np

n = 10_000_000
a = np.random.rand(n).astype(np.float32)
b = np.random.rand(n).astype(np.float32)

t0 = time.perf_counter()
out = [0.0] * n
for i in range(n):                  # one element at a time, interpreted
    out[i] = a[i] * b[i] + 1.0
t1 = time.perf_counter()
c = a * b + 1.0                     # whole-array ops in compiled, SIMD loops
t2 = time.perf_counter()

print(f"python loop: {t1 - t0:.2f} s")
print(f"numpy:       {t2 - t1:.4f} s  ({(t1 - t0) / (t2 - t1):.0f}x faster)")
```

What to notice:

- Both versions compute the same 10 million results.
- The speed-up (about 20× on the test VM in 2026; often larger) comes from two sources: removing the interpreter's per-element overhead, and SIMD in numpy's compiled loops. Do not attribute it all to SIMD.
- `a * b + 1.0` makes two passes over memory and allocates a temporary array; at this size the vectorized version is limited by memory bandwidth (§5.9), not arithmetic.

5.11.2 Accelerators

When one kind of computation dominates a workload, it can pay to build or buy hardware specialized for it:

- **GPUs** — thousands of simple arithmetic lanes and very high-bandwidth memory (HBM), designed for massively parallel, regular computation. They dominate ML training and inference. Chapter 64 covers their architecture, the roofline model [Williams et al. 2009] and the compute-bound versus memory-bound distinction in depth.
- **TPUs and other ML ASICs** — chips built around large matrix-multiply units [Jouppi et al. 2017].
- **Fixed-function units inside the CPU** — AES and SHA instructions make TLS encryption cheap; compression and cryptography offload engines exist on some server CPUs and NICs.
- **SmartNICs / DPUs and FPGAs** — network cards with programmable processors that offload packet processing, storage virtualization or encryption from the host CPU.

The design question is always whether the workload is large and regular enough to justify specialized hardware, and whether data can reach it fast enough — an accelerator starved by PCIe or by CPU-side data preparation delivers a fraction of its rated speed.

5.12 Throughput versus latency at the hardware level

5.12.1 The pattern underneath everything

Every component in this chapter shows the same trade-off. A **pipeline** raises instruction throughput without shortening any instruction. **Out-of-order** cores and **memory-level parallelism** raise memory throughput by keeping many misses in flight. **NVMe queues** raise IOPS by keeping dozens of requests outstanding. **Interrupt coalescing** raises packet throughput by delaying notifications. In each case Little's Law ($L = \lambda W$, §4.11) holds: to get throughput λ from a resource with latency W, you need $L = \lambda W$ operations in flight. Latency is mostly fixed by physics; throughput is bought with parallelism and batching.

FORMULA

Throughput with batching (fixed plus per-item cost)

$$X(B) = \frac{B}{F + B \times v}, \qquad T_{batch}(B) = F + B \times v, \qquad X_{max} = \frac{1}{v}$$

Symbol	Meaning	Units
B	Items per batch	items
F	Fixed cost per operation (system call, device round trip, interrupt, network RTT)	seconds
v	Variable cost per item	seconds/item
$X(B)$	Throughput at batch size B	items/second
$T_{batch}(B)$	Time to complete one batch	seconds

WORKED EXAMPLE

Worked example — Batching small writes.

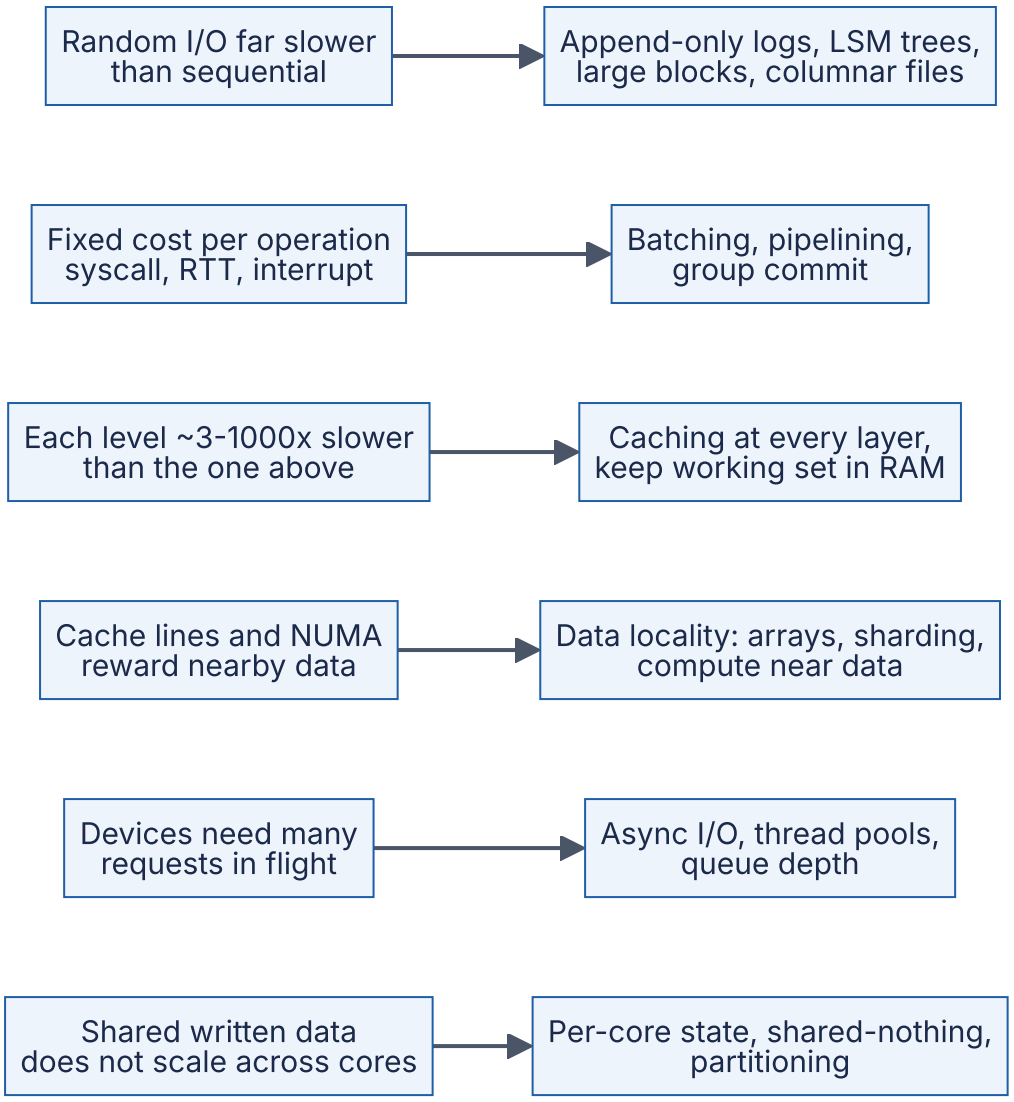
Writing 100-byte records to a log, each write call has fixed cost $F = 50$ µs and per-record cost $v = 0.1$ µs.

1. $B = 1$: $X = 1/50.1\ \mu s \approx 19\,960$ records/s; each write completes in 50.1 µs.
2. $B = 100$: $T = 50 + 100 \times 0.1 = 60$ µs; $X = 100/60\ \mu s \approx 1.67$ million records/s.
3. $B = 1000$: $T = 50 + 100 = 150$ µs; $X = 1000/150\ \mu s \approx 6.67$ million records/s.
4. Ceiling: $X_{max} = 1/0.1\ \mu s = 10$ million records/s.

Interpretation. Batching 100 records multiplies throughput by about 84× while raising per-batch latency only from 50 µs to 60 µs; going to 1000 gains another 4× in throughput but per-batch latency rises 2.5×, plus the time spent *waiting* for the batch to fill. **Engineering implication.** Batch until the fixed cost is amortized (a few times F/v items), then stop; bound the wait with a timer. Kafka producers (`linger.ms` , §31.6), group commit in databases (§13.5) and dynamic batching in model serving (Ch 66) all implement this curve.

5.12.2 How hardware facts drive system design decisions

Figure 5.10 — From hardware facts to design decisions.



Reading the diagram. Each row pairs a physical fact (left) with the design principle it forces (right); arrows mean “therefore”. None of these is a fashion — each follows from numbers you calculated in this chapter. The same principles reappear at datacenter scale: a cross-region network round trip is to a datacenter what a DRAM miss is to a core, so caching, batching, locality and parallelism return in Part IV (scalability, caching, CDNs) and Part V (distributed systems). When a design violates one of these pairs — chatty per-item network calls, random writes to disks, a global counter updated by every core — it usually fails at scale for precisely the reason on the left.

Table 5.13 — Hardware facts and the design decisions they drive.

Hardware fact (§)	Design decision	Examples in this book
Sequential ≫ random on HDD (~620×) and still better on SSD (§5.6)	Write sequentially; read in large chunks	Write-ahead logs (§13.5), LSM trees (§13.8), Kafka (§31.13)
Fixed cost per operation (§5.7, §5.12)	Batch and pipeline	Kafka batching (§31.6), HTTP/2 multiplexing (§7.9), dynamic batching (Ch 66)
Memory hierarchy steps of 3–1000× (§5.5)	Cache hot data close to its consumer	Buffer pools (§13.4), Redis (§20.6), CDNs (Ch 21)
Cache lines, prefetching, NUMA (§5.4, §5.10)	Keep related data together and near the compute	Columnar storage (§13.13), data locality in Spark (§29.7)
Devices need queue depth (§5.6.5)	Concurrency in the I/O path	io_uring and event loops (§6.13), connection pools (§7.16)
Coherence makes shared writes expensive (§5.4.6)	Avoid shared mutable state; partition it	Shard-per-core designs, partitioned databases (Ch 14)
DRAM is volatile (§5.5)	Durability needs persistent media or replicas	fsync (§6.14), replication (Ch 27)

INDUSTRY EXAMPLE

ScyllaDB and the Seastar framework (shard-per-core). ScyllaDB, a Cassandra-compatible database, is built on the Seastar framework, which runs one thread per core, partitions data so that each core owns its shard, and passes messages between cores instead of sharing locked data structures. The design avoids cache-line contention and lock overhead and keeps each core's data in its own caches and local NUMA memory. Trade-off: everything must be asynchronous, and skewed keys can overload a single core. *Evidence: Primary* — [Seastar Documentation]

RECOMMENDATION

Mechanical sympathy in practice. (1) Estimate before you build: write down the latency and bandwidth of each hop. (2) Prefer arrays and contiguous layouts for hot data. (3) Turn many small operations into fewer large ones. (4) Keep enough requests in flight to use your devices. (5) Do not share frequently written data across cores or machines. (6) Measure with hardware counters (`perf`) before and after; intuition about hardware is often wrong.

INTERVIEW TIP

Interview tip. When an interviewer asks "why is X fast?" (Kafka, Redis, a columnar database), the answer almost always maps onto one row of Table 5.13. Say the hardware fact and its number first, then the design choice.

5.13 Summary and revision notes

SUMMARY

- A CPU core repeatedly fetches, decodes and executes instructions. Execution time = instructions × CPI × cycle time; clock rates stalled near 3–5 GHz around 2005, so performance now comes from more work per cycle and more cores.
- Pipelining, superscalar issue and out-of-order execution raise throughput by overlapping instructions; branch prediction keeps the pipeline fed. Mispredictions (~15–20 cycles) and dependent memory accesses are what slow cores down.
- SMT (hyperthreading) shares one core between two hardware threads; a cloud vCPU is usually one such thread, not a full core.
- Caches (L1 ~1 ns, L2 ~4 ns, L3 ~15 ns, DRAM ~100 ns; as of 2024–2025, approximate) exploit temporal and spatial locality. Data moves in 64-byte cache lines; sequential access is the best case and pointer chasing the worst.
- AMAT = hit time + miss rate × miss penalty, applied level by level; layout alone can change it 10×.
- MESI keeps caches coherent per line; writes to one line from several cores cause ping-pong. False sharing is unrelated variables in one line; fix it with padding or per-thread data.
- The memory hierarchy spans ~1 ns to ~10 ms and about 1000× in cost per GB; DRAM and above are volatile.
- HDD access = seek + rotational latency + transfer; ~80–200 random IOPS versus ~200 MB/s sequential (~620× gap for 4 KiB reads).
- Flash writes pages, erases blocks and wears out; the FTL remaps writes, garbage-collects and causes write amplification. NVMe’s many deep queues unlock flash parallelism that SATA’s single 32-entry queue cannot.
- IOPS = queue depth / latency: fast devices need many requests in flight.
- I/O evolves from programmed I/O to interrupts + DMA to polling and kernel bypass as event rates rise.
- CPU-bound work needs ~one thread per core; I/O-bound work needs about N_cores × (1 + W/C) threads, limited by downstream capacity.
- Latency barely improves; bandwidth does. Peak DRAM bandwidth = transfer rate × 8 bytes × channels; achieving it needs hundreds of misses in flight.
- NUMA makes remote memory ~1.5–2× slower; keep threads, memory and devices on one node.
- SIMD and accelerators speed up regular, data-parallel work if data is contiguous and can be fed fast enough.

- Hardware facts drive design: sequential I/O, batching, caching, data locality, concurrency in the I/O path, and no shared mutable hot spots.

5.14 Knowledge checkpoint

1. Using the iron law, what happens to execution time if CPI rises from 0.5 to 0.75 at a fixed clock rate and instruction count?
2. Why does a deeper pipeline make branch mispredictions more expensive?
3. Why can an out-of-order core hide the latency of ten independent loads but not ten loads in a linked list?
4. What is the difference between temporal and spatial locality? Give one example of each.
5. In a 32 KiB, 8-way cache with 64-byte lines, how many sets are there, and why are addresses 4 KiB apart a problem?
6. Compute AMAT for t_{L1} = 1 ns, m_{L1} = 0.1, and a flat 50 ns penalty for every L1 miss.
7. Name the four MESI states and say which one allows a write without any coherence traffic.
8. Two threads increment separate counters and get slower when you add the second thread. What is the likely cause and the fix?
9. A 5400 RPM disk has an average seek of 10 ms. What is its average rotational latency and approximate random-read IOPS for small blocks?
10. Why can a flash SSD not overwrite a page in place, and what does the FTL do instead?
11. A device has 100 µs latency. What queue depth is needed for 400,000 IOPS?
12. What is DMA and what problem does it solve?
13. A service's `time` output shows real = 10 s, user + sys = 0.5 s. Is it CPU-bound or I/O-bound? What would you change first?
14. What is the peak bandwidth of six DDR5-5600 channels?
15. On a two-socket server, local latency is 90 ns and remote 150 ns. What is the average latency if 30% of accesses are remote?

Answers

1. Time rises by 50% (0.75 / 0.5 = 1.5), because time is proportional to CPI.
2. The branch outcome is known later in a deeper pipeline, so more stages hold wrongly fetched instructions that must be flushed; more cycles are lost per misprediction.
3. Independent loads can all be issued and outstanding at once, overlapping their latency. In a linked list each address is known only after the previous load returns, so the latencies add up serially.

4. Temporal: reusing the same data soon (a loop counter, a hot config value). Spatial: using nearby data soon (the next array element, adjacent struct fields).
5. $32\,768/(64 \times 8) = 64$ sets. Addresses $64 \times 64 = 4096$ bytes apart map to the same set, so a 4 KiB stride can use only 8 lines (one set) and suffers conflict misses.
6. $1 + 0.1 \times 50 = 6$ ns.
7. Modified, Exclusive, Shared, Invalid. Writes to an Exclusive line (moving it to Modified) need no bus/interconnect traffic; a line already in Modified also needs none.
8. False sharing: both counters are in one cache line, which ping-pongs between cores. Pad each counter to its own 64-byte line or keep per-thread counters and combine them at the end.
9. One revolution $= 60/5400 = 11.1$ ms; average rotational latency $= 5.56$ ms. Access $\approx 10 + 5.56 \approx 15.6$ ms, so $\approx 1/0.0156 \approx 64$ IOPS.
10. Flash must be erased before being reprogrammed, and erase works only on whole blocks of many pages. The FTL writes the new data to a free page, updates its logical-to-physical map, marks the old page invalid and later garbage-collects blocks.
11. $QD = 400\,000 \times 100 \times 10^{-6} = 40$ outstanding requests.
12. Direct memory access lets a device copy data to or from RAM itself; the CPU only sets up the transfer and handles the completion, so CPU cycles are not spent copying bytes.
13. I/O-bound (or waiting on something): the CPU was busy only 5% of the time. Find what it waits on — disk, network or locks — and add concurrency, caching or batching rather than a faster CPU.
14. $5600 \times 10^6 \times 8 \times 6 = 268.8 \times 10^9$ B/s $= 268.8$ GB/s.
15. $0.7 \times 90 + 0.3 \times 150 = 63 + 45 = 108$ ns.

5.15 Interview questions

Basic

Q1. Why is reading data sequentially faster than reading it randomly, on both disks and memory? Answer. On an HDD a random read pays a seek (~4–10 ms) and half a rotation (~4 ms at 7200 RPM) for every request, while a sequential read pays them once and then streams at ~200 MB/s — about 620× faster for 4 KiB requests. In memory, data arrives in 64-byte cache lines and hardware prefetchers detect sequential patterns and fetch ahead, so sequential reads hit in cache while random reads each pay ~100 ns of DRAM latency. SSDs narrow the gap but still favor large sequential I/O.

Q2. What is a cache line and why should a programmer care? Answer. The 64-byte (on most x86 and ARM servers) unit in which data moves between memory and caches. It means neighbors of anything you read are nearly free (so arrays and compact structs are

fast), and that coherence works per line, so threads writing different variables in the same line interfere (false sharing).

Intermediate

Q3. You moved a service from an 8-vCPU to a 32-vCPU instance and throughput rose only 30%. What could explain this, and how would you investigate? Answer. Candidates: (1) the service is limited by something other than CPU — a database, a lock, a single-threaded component (check per-thread CPU, thread dumps); (2) contention on shared cache lines (atomic counters, false sharing, a JVM internal structure) — check with `perf c2c` or cache-miss counters; (3) NUMA effects if the larger instance spans sockets; (4) vCPUs are hyperthreads, so 32 vCPUs are 16 cores. Measure per-core utilization and IPC before and after, then test hypotheses one by one. Netflix's published 2022 case was exactly this pattern, caused by cache-line sharing in the JVM.

Q4. How would you size a thread pool for a service that calls a database? Answer. Measure CPU time per request C and wait time W. Start with $N = N_{cores} \times U \times (1 + W/C)$; e.g., 8 cores, $C = 5$ ms, $W = 50$ ms gives ~88 threads at full utilization. Then cap it by the downstream: the database connection pool and the database's own capacity. Validate with a load test, watching p99 latency and CPU; if latency rises without throughput gain, the pool is too large. For very high concurrency, consider async I/O instead.

Advanced

Q5. Explain why Kafka can sustain high throughput even on hard disks. Answer. Each partition is an append-only log, so writes are sequential (no seeks). Reads by consumers are also mostly sequential and usually served from the OS page cache because recently written data is still in RAM. Producers and brokers batch many messages per request, amortizing per-request fixed costs (Table 5.13). Data is sent from page cache to socket without copying through user space (zero-copy, §31.13). Each design choice maps to a hardware fact: sequential ≫ random, fixed per-operation costs, and DRAM ≫ disk.

Q6. Design the counters for a metrics library that must record 50 million increments per second across 64 cores. Answer. A single shared atomic counter would ping-pong one cache line between 64 cores and cap at a few tens of millions of updates per second or less, with heavy cross-core traffic. Instead, give each core (or thread) its own counter, each padded to its own cache line (or grouped per core in a per-core region), and increment it with plain or relaxed atomic operations. A reader sums all slots when scraping — reads are rare, so their cost is irrelevant. This trades a little memory (64 lines × 64 B = 4 KiB per counter) and slightly stale reads for near-linear scaling. Keep each core's slots on its own NUMA node. This is the same "partition, don't share" principle used by shard-per-core databases.

5.16 Practical exercises

1. **Cache-line stride benchmark (Python + numpy).** Run the program below. Plot nanoseconds per element against stride. Explain why the cost per element rises until the stride reaches about 64–128 bytes and then levels off, and why total time stays roughly constant for strides up to 64 bytes.

```
"""stride.py — how access stride affects time per element touched.

Run: python3 stride.py   (needs numpy)
"""
import time
import numpy as np

N_BYTES = 256 * 1024 * 1024           # 256 MiB buffer, far larger than any L3
data = np.ones(N_BYTES // 8, dtype=np.int64)   # 8-byte elements

def ns_per_element(stride_elems: int, repeats: int = 3) -> float:
    view = data[::stride_elems]          # strided view, no copy
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        view.sum()                       # touches every element of the view
        best = min(best, time.perf_counter() - t0)
    return best / view.size * 1e9

print(f"{'stride (B)':>10} {'elements':>12} {'ns/element':>11}")
for stride in [1, 2, 4, 8, 16, 32, 64, 128, 512]:
    n = data[::stride].size
    print(f"{stride * 8:>10} {n:>12,} {ns_per_element(stride):>11.2f}")
```

What to notice: up to a 64-byte stride every cache line is still fetched, so total time is dominated by moving 256 MiB from DRAM and the time *per element* grows as fewer elements share each line; beyond 64 bytes each element costs one line fetch (plus adjacent-line prefetch effects up to ~128 bytes). On the test VM the cost rose from ~0.7 ns/element at an 8-byte stride to ~7 ns/element at 256 bytes and above.

2. **False sharing.** Compile and run `false_sharing.c` from §5.4.7. Then change `CACHE_LINE` to 32 and 128 and explain the results. Finally, run the packed version with both threads pinned to the same core (`taskset -c 0 ./fs`) and explain why the penalty disappears.
3. **Disk and SSD.** Using `fio`, measure random 4 KiB read IOPS at queue depths 1, 4, 16 and 64 on a scratch file on your SSD. Check that IOPS ≈ QD / latency until the device saturates.
4. **CPU-bound or I/O-bound?** Run a compression job (`gzip` on a large file) and a script that downloads 100 URLs sequentially, each under `time`. Compare real versus user + sys and watch `vmstat 1` during each.

5. **Topology tour.** On a Linux machine, record the output of `lscpu`, `lscpu -C` (cache sizes and line size) and `numactl --hardware`. Draw your own version of Figure 5.1 with real numbers.

5.17 Design challenges

1. **Log-structured key-value store on HDDs.** You must store 20 TB of 1 KB values on HDDs with 50,000 writes/s and 5,000 random reads/s. Estimate how many disks a random-write design would need versus an append-only log with an in-memory index, and what the memory cost of the index would be.
2. **Analytics scan budget.** Analysts want to scan a 2 TB table in under 10 seconds. Using Table 5.10, decide whether this is feasible from NVMe on one machine, from RAM on one machine, or only across a cluster, and how many nodes and what storage format (row or column) you would choose.
3. **Low-latency order gateway.** A trading gateway must handle 2 million messages/s with p99 under 20 µs on one server. Choose between interrupts, polling and kernel bypass for the NIC; decide thread-to-core and NUMA placement; and explain how you would avoid false sharing between the receive and send threads.

5.18 References

1. Agrawal, N., Prabhakaran, V., Wobber, T., Davis, J. D., Manasse, M., Panigrahy, R. *Design Tradeoffs for SSD Performance*. USENIX Annual Technical Conference, 2008.
2. Apache Software Foundation. *Apache Kafka Documentation — Design*. https://kafka.apache.org/documentation/#design
3. Axboe, J. *Efficient IO with io_uring*. 2019. https://kernel.dk/io_uring.pdf
4. Boncz, P., Zukowski, M., Nes, N. *MonetDB/X100: Hyper-Pipelining Query Execution*. CIDR, 2005.
5. Bryant, R. E., O'Hallaron, D. R. *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.
6. Cole, J. *The MySQL "swap insanity" problem and the effects of the NUMA architecture*. Blog post, 2010.
7. Dean, J. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, LADIS Workshop, 2009.
8. DPDK Project. *Data Plane Development Kit Documentation*. https://doc.dpdk.org/
9. Drepper, U. *What Every Programmer Should Know About Memory*. Red Hat, 2007. https://www.akkadia.org/drepper/cpumemory.pdf
10. Filanovsky, V., Sane, H. *Seeing through hardware counters: a journey to threefold performance increase*. Netflix Technology Blog, 2022.

11. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., Lea, D. *Java Concurrency in Practice*. Addison-Wesley, 2006.
12. Gregg, B. *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
13. Hennessy, J. L., Patterson, D. A. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
14. Jouppi, N. P., et al. *In-Datacenter Performance Analysis of a Tensor Processing Unit*. ISCA, 2017.
15. Lameter, C. *NUMA (Non-Uniform Memory Access): An Overview*. ACM Queue 11(7), 2013.
16. LTO Program. *LTO technology specifications (LTO-9)*. https://www.lto.org/
17. Marr, D. T., Binns, F., Hill, D. L., Hinton, G., Koufaty, D. A., Miller, J. A., Upton, M. *Hyper-Threading Technology Architecture and Microarchitecture*. Intel Technology Journal 6(1), 2002.
18. Norvig, P. *Teach Yourself Programming in Ten Years*. 2001. https://norvig.com/21-days.html
19. NVM Express, Inc. *NVM Express Base Specification*. https://nvmexpress.org/specifications/
20. Papamarcos, M. S., Patel, J. H. *A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories*. ISCA, 1984.
21. Ruemmler, C., Wilkes, J. *An Introduction to Disk Drive Modeling*. IEEE Computer 27(3), 1994.
22. Seastar Project. *Seastar Documentation*. https://seastar.io/
23. Sorin, D. J., Hill, M. D., Wood, D. A. *A Primer on Memory Consistency and Cache Coherence*. Morgan & Claypool, 2011.
24. SPDK Project. *Storage Performance Development Kit Documentation*. https://spdk.io/
25. Thompson, M., Farley, D., Barker, M., Gee, P., Stewart, A. *Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*. LMAX, 2011. https://lmax-exchange.github.io/disruptor/
26. Tomasulo, R. M. *An Efficient Algorithm for Exploiting Multiple Arithmetic Units*. IBM Journal of Research and Development 11(1), 1967.
27. Williams, S., Waterman, A., Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM 52(4), 2009.

5.19 Open verification items

- Cost per GB in Table 5.5 and Table 5.6 reflects approximate 2024 street prices; DRAM and NAND prices moved sharply in 2025–2026. Re-check before publication.
- Exact URLs for [Dean 2009] (LADIS keynote slides), [Cole 2010] and [Filanovsky & Sane 2022] are omitted pending verification; confirm the specific entries of Dean’s

list (versions differ, e.g. 20 ms vs 30 ms for 1 MB from disk).
- Confirm the “up to ~30%” Hyper-Threading gain against [Marr et al. 2002] and the MySQL release that added buffer-pool NUMA interleaving.
- Cite a primary source (Intel investor communication, 2022) for the Optane wind-down.
- Microarchitectural ranges (branch-mispredict penalty 15–20 cycles; 10–20 outstanding L1 misses per core; reorder buffer “several hundred” entries) should be checked against current Intel/AMD/Arm optimization manuals.
- TLC endurance of 1,000–3,000 P/E cycles is a commonly quoted range, not a vendor specification.
- Status of free-threaded CPython (experimental from 3.13) may have changed.

Chapter 7 — Networking Fundamentals

> **Level:** 1–2 · **Track:** Shared · **Estimated reading time:** 5 h

Every system in this book is a set of computers that talk to each other. A database replica talks to its primary, a browser talks to a load balancer, and a model server streams tokens to a phone. Whenever two computers talk, the network sets hard limits on how fast they can do it, how often it fails, and what it costs. Those limits come from physics (the speed of light), from economics (the cost of bandwidth), and from decades-old protocol decisions that you cannot change. You can design around them only if you understand them.

This chapter builds the network up from nothing. We start with the layered model that lets thousands of independently built protocols cooperate, then follow a single request down through addressing, routing, name resolution, transport, security and application protocols. We finish by tracing everything that happens when you type a URL into a browser. Throughout, the goal is quantitative intuition: by the end you should be able to estimate, with arithmetic, how long a request will take and why.

7.1 Learning objectives

After this chapter you should be able to:

- Explain the OSI and TCP/IP layer models and describe how **encapsulation** wraps application data in transport, network and link headers.
- Read and compute IPv4 and IPv6 CIDR blocks: network address, broadcast address, usable host count and whether an address falls in a subnet.
- Describe how packets are routed inside a network and between autonomous systems with BGP, and explain how anycast steers users to a nearby server.
- Draw the DNS resolution sequence (stub, recursive resolver, root, TLD, authoritative), name the common record types, and reason about TTLs and the limits of DNS-based load balancing.
- Explain TCP’s three-way handshake, four-way close, TIME_WAIT, sequence numbers, retransmission, flow control and congestion control (slow start, AIMD, CUBIC, BBR), and identify head-of-line blocking.
- Decide when UDP is the right transport and what you must rebuild on top of it.
- Compare HTTP/1.1, HTTP/2 and HTTP/3 (over QUIC) and justify a choice for a given workload.

- Sequence the TLS 1.3 handshake, explain certificate validation, and state the security caveats of 0-RTT.
- Distinguish forward proxies from reverse proxies, and explain NAT, firewalls and cloud security groups.
- Decompose latency into propagation, transmission, queuing and processing delay; compute the bandwidth-delay product; and apply the Mathis formula to estimate TCP throughput under loss.
- Quantify the cost of connection setup and size a connection pool.
- Describe leaf–spine (Clos) datacenter fabrics and why east–west traffic dominates modern datacenters.

7.2 Prerequisites

- Units: bits vs bytes, SI prefixes, rates — §4.5 (every calculation here converts between bits per second and bytes).
- Percentiles and tail latency — §4.8 (network latency is heavy-tailed; averages mislead).
- Little's Law — §4.11 (used to size connection pools in §7.16).
- Latency numbers every engineer should know — §4.17 (this chapter derives several of them).
- Processes, sockets as file descriptors, and I/O models (blocking, epoll) — §6.7 and §6.13 (a TCP connection is a kernel object your process reads and writes).

No prior networking knowledge is assumed.

7.3 OSI and TCP/IP models

7.3.1 The postal analogy

Imagine you want to send a letter to a friend in another country. You write the letter (the content). You put it in an envelope and write your friend's name on it. The post office puts your envelope into a mail sack addressed to a sorting center in the destination city. The mail sack goes into a truck, then a plane, then another truck. At each sorting center, a worker reads only the sack label, not your letter, and decides where it goes next. At the far end, the sack is opened, the envelope is delivered to the right house, and your friend opens it and reads the letter.

Three ideas in this story are the foundation of all computer networking:

1. **Layering.** Each participant does one job and trusts the others to do theirs. The letter writer does not care whether the mail flies or sails. The truck driver does not read letters.

2. **Encapsulation.** Each layer wraps the thing it was given in its own envelope with its own address, and the matching layer at the far end unwraps it.
3. **Hop-by-hop forwarding.** Intermediate sorting centers look only at the outer address and forward toward the destination. They do not need the whole route in advance.

Computer networks work the same way. Your application writes bytes. The operating system wraps them in a **segment** with port numbers (which program on the machine), then in a **packet** with IP addresses (which machine on the planet), then in a **frame** with hardware addresses (which device on this particular wire). Routers along the way read the packet header and forward it, and the receiving machine unwraps each layer in reverse.

> DEFINITION
>
> **Definition — Protocol.** A set of rules two parties agree on for exchanging messages: message formats, the meaning of each field, and the actions each side takes on sending, receiving or timing out. A **protocol stack** is a set of protocols layered so that each uses the service provided by the layer below and offers a service to the layer above.

> DEFINITION
>
> **Definition — Encapsulation.** The process by which each layer of the stack prepends its own header (and sometimes appends a trailer) to the data unit handed down by the layer above, treating that data unit as an opaque payload. **Decapsulation** is the reverse at the receiver.

Why does this exist?

In the 1970s, every computer vendor had its own networking scheme, and machines from different vendors could not talk. Layering solved two problems. First, it made **interoperability** possible: if two vendors implement the same protocol at a layer, their machines can communicate regardless of what is above or below. Second, it made **independent evolution** possible: Wi-Fi replaced Ethernet cables at the bottom and HTTP/3 replaced HTTP/1.1 at the top, while IP in the middle stayed the same. This “narrow waist” design, with many link technologies below and many applications above one common Internet Protocol, is the main reason the Internet scaled from a few dozen hosts in 1970 to billions of devices.

7.3.2 The two models

There are two reference models you will hear about. The **OSI model** (Open Systems Interconnection), standardized by ISO in the early 1980s, has seven layers. It is a teach-

ing vocabulary; the OSI protocol suite itself lost to TCP/IP in the market. The **TCP/IP model** (also called the Internet model) has four or five layers and describes the protocols the Internet actually runs. Engineers mix the vocabularies freely: “an L4 load balancer” uses the OSI number for the transport layer, and “an L7 proxy” uses the OSI number for the application layer.

Table 7.1 — OSI and TCP/IP layers compared.

OSI layer	TCP/IP layer	Job	Example protocols	Data unit (PDU)	Address used
7 Application	Application	Meaning of messages for a specific application	HTTP, DNS, SMTP, gRPC	Message	URL, hostname
6 Presentation	(folded into application)	Encoding, compression, encryption	TLS*, JSON, Protobuf	—	—
5 Session	(folded into application)	Dialogue management, resumption	TLS sessions*, RPC sessions	—	—
4 Transport	Transport	Process-to-process delivery; reliability; flow and congestion control	TCP, UDP, QUIC**	Segment (TCP), datagram (UDP)	Port number
3 Network	Internet	Host-to-host delivery across many networks	IPv4, IPv6, ICMP	Packet	IP address
2 Data link	Link	Delivery across one physical network segment	Ethernet, Wi-Fi (802.11)	Frame	MAC address
1 Physical	Link (or separate physical layer)	Bits as voltages, light or radio	Copper, fiber, radio	Bit/symbol	—

*TLS does not fit cleanly: it runs on top of TCP but below HTTP, so people place it at 5, 6 or “4.5”. **QUIC runs on top of UDP yet provides transport services (reliability, streams, congestion control), so it is a transport protocol implemented in user space.

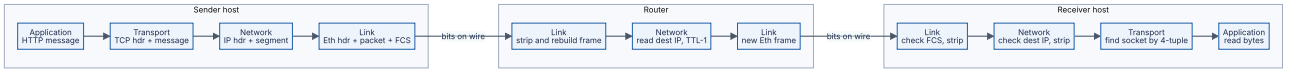
Do not memorize the OSI layers 5 and 6 as separate things that exist in real stacks; they do not. What matters in practice is the five-layer view: physical, link, network, transport, application. This is the organizing structure of the standard textbook treatment [Kurose and Ross 2021], which is the recommended companion for this chapter.

7.3.3 How encapsulation works internally

Follow one HTTP request from a browser to a web server.

1. **Application.** The browser produces the bytes `GET /index.html HTTP/1.1\r\nHost: example.com\r\n\r\n` (about 40 bytes) and calls `write()` on a socket.
2. **Transport.** The kernel's TCP implementation copies the bytes into the socket's send buffer, splits them into segments no larger than the **maximum segment size (MSS)**, and prepends a 20-byte TCP header (up to 60 bytes with options) containing source port (say 52,814), destination port 443, a sequence number, an acknowledgment number, flags and a window size.
3. **Network.** IP prepends a 20-byte IPv4 header (or a 40-byte IPv6 header) containing source and destination IP addresses, a time-to-live (TTL, renamed "hop limit" in IPv6) counter, and the protocol number of the payload (6 for TCP, 17 for UDP).
4. **Link.** The network interface driver looks up the MAC address of the next hop (the local router) using ARP (IPv4) or Neighbor Discovery (IPv6), and prepends a 14-byte Ethernet header (destination MAC, source MAC, EtherType) and appends a 4-byte frame check sequence (a CRC for error detection).
5. **Physical.** The network card serializes the frame onto the wire as electrical or optical symbols.
6. **Each router** receives the frame, checks the CRC, strips the Ethernet header, reads the IP destination, decrements the TTL, looks up the next hop, and wraps the packet in a *new* link-layer frame for the next link. The IP packet travels end to end; link-layer frames live for one hop only.
7. **The server** decapsulates in reverse: the NIC checks the frame, IP checks the destination address, TCP uses the (source IP, source port, destination IP, destination port) **4-tuple** to find the right socket, and the application reads the original bytes.

Figure 7.1 — Encapsulation down the sender's stack and decapsulation up the receiver's.



Reading the diagram. The left group is the sending host; data flows down its stack, gaining one header per layer. The middle group is a single router: it operates only up to the network layer, stripping the incoming Ethernet frame, reading and modifying the IP header (decrementing TTL), and building a fresh frame for the outgoing link. The right group is the receiving host, where data flows up and each layer removes its own header. All arrows are synchronous hand-offs within the same packet's journey; nothing here is queued asynchronously except inside each device's transmit buffer, which is a failure point (buffers overflow and drop packets, see §7.15). Storage boundaries are absent: the network stores nothing durably. To scale, you add links and routers; the layered design means hosts need not change.

Mathematical foundations: header overhead

FORMULA

Protocol efficiency (goodput fraction)

$$\eta = \frac{P}{P + H}$$

Symbol	Meaning	Units
η	Fraction of transmitted bytes that are application payload	dimensionless (0–1)
P	Application payload bytes per frame	bytes
H	Total per-frame overhead from all layers (headers, trailers, and on Ethernet the preamble and inter-frame gap)	bytes

WORKED EXAMPLE

Worked example — How much of a gigabit link carries your data?

A standard Ethernet **maximum transmission unit (MTU)** is 1,500 bytes: the largest IP packet a frame can carry. Inside that, IPv4 uses 20 bytes and TCP 20 bytes, plus 12 bytes for the commonly enabled TCP timestamp option.

1. TCP payload per packet: $1500 - 20 - 20 - 12 = 1448$ bytes.
2. On-the-wire cost per frame: 1,500 (packet) + 14 (Ethernet header) + 4 (FCS) + 8 (preamble) + 12 (inter-frame gap) = 1,538 bytes.
3. Efficiency: $\eta = 1448/1538 \approx 0.941$.
4. Maximum TCP goodput on 1 Gbps Ethernet: $0.941 \times 1000 \approx 941$ Mbps.

Interpretation: about 6% of a full-size frame is overhead. Now consider a tiny 40-byte request carried in one packet: the wire cost is $40 + 20 + 32 + 38 = 130$ bytes, so efficiency is $40/130 \approx 31\%$.

Engineering implication: small messages are dominated by header cost (and, more importantly, by per-packet processing cost in CPUs and switches). This is one reason systems **batch** small writes (§23.8) and why the 941 Mbps "real" ceiling of gigabit Ethernet is a well-known measurement in `iperf` tests.

Common misconceptions

COMMON MISTAKE

"Layer 7 means HTTP and Layer 4 means TCP, so a Layer 4 device cannot see anything about HTTP." Roughly right, but the numbers are labels for *what a device inspects*, not for physical boxes. A "Layer 4 load balancer" makes decisions using IP addresses and ports; a "Layer 7 load balancer" parses HTTP and can route on path or header. Both receive the full packet (§19.3).

7.4 IP addressing, IPv4/IPv6, subnets and CIDR

7.4.1 What an IP address is

DEFINITION

Definition — IP address. A numeric identifier assigned to a network interface that the Internet Protocol uses to deliver packets. An **IPv4** address is 32 bits, written as four decimal octets (`203.0.113.25`) [RFC 791]. An **IPv6** address is 128 bits, written as eight groups of four hexadecimal digits separated by colons (`2001:db8:0:1::25`) [RFC 8200].

First-principles explanation

A postal address is hierarchical: country, city, street, house number. A sorting center in another country only needs to read the country to forward the letter; only the local post office reads the house number. IP addresses are hierarchical in the same way, but the split point is flexible. The leading bits form the **network prefix** (which network), and the trailing bits identify the **host** within that network. Routers far away only need the prefix.

Why does this exist?

Routers must decide where to send every packet in nanoseconds. If they had to store a route for every one of billions of devices, their tables would not fit in fast memory. Grouping addresses into prefixes lets a router store one entry for millions of hosts. The original 1981 scheme fixed the split at 8, 16 or 24 bits ("classes A, B, C"), which wasted addresses: an organization needing 300 addresses had to take a class B with 65,536. **Classless Inter-Domain Routing (CIDR)**, introduced in 1993 and specified today in RFC 4632, allows the split at any bit.

7.4.2 CIDR notation and subnet arithmetic

DEFINITION

Definition — CIDR block. An address range written as `address/prefix-length`, for example `192.168.10.64/26`. The prefix length n says that the first n bits identify the network; the remaining $32 - n$ bits (IPv4) or $128 - n$ bits (IPv6) identify hosts. The **subnet mask** is the same information as a dotted number with n one-bits followed by zeros (`/26` = `255.255.255.192`).

FORMULA

Addresses in a CIDR block

$$N_{\text{addr}} = 2^{W-n} \qquad N_{\text{hosts,IPv4}} = 2^{32-n} - 2$$

Symbol	Meaning	Units
N_{addr}	Total addresses in the block	addresses
W	Address width: 32 for IPv4, 128 for IPv6	bits
n	Prefix length	bits
$N_{\text{hosts,IPv4}}$	Assignable IPv4 hosts on a traditional subnet: subtract the **network address** (all host bits 0) and the **broadcast address** (all host bits 1)	addresses

The "−2" applies to ordinary IPv4 LAN subnets. Point-to-point `/31` links (RFC 3021) and single-host `/32` routes are exceptions, and cloud providers reserve more: AWS reserves five addresses in every VPC subnet (network, VPC router, DNS, one reserved for future use, broadcast), as of 2025 [AWS Docs: VPC subnets].

WORKED EXAMPLE

Worked example 1 — Which subnet is 192.168.10.77/26 in?

1. A `/26` leaves $32 - 26 = 6$ host bits, so each subnet holds $2^6 = 64$ addresses.
2. The mask is 26 one-bits: `11111111.11111111.11111111.11000000` = `255.255.255.192`.
3. Only the last octet is split. Subnets start at multiples of 64 in that octet: .0, .64, .128, .192.
4. 77 lies between 64 and 127, so the network address is **192.168.10.64**.
5. Broadcast address: last address of the block = 64 + 64 − 1 = **192.168.10.127**.
6. Usable hosts: **192.168.10.65 – 192.168.10.126**, which is $64 - 2 = 62$ hosts.

Bitwise check: 77 = `01001101`; AND with mask `11000000` gives `01000000` = 64. Correct.

WORKED EXAMPLE

Worked example 2 — Carving a cloud VPC into subnets.

You are given the VPC range `10.0.0.0/16` and want equal subnets across three availability zones, each with public and private tiers (six subnets), leaving room to grow.

1. The VPC has $2^{32-16} = 65,536$ addresses.
2. Choose `/20` subnets: $2^{20-16} = 16$ subnets of $2^{12} = 4,096$ addresses each. Six are used now; ten remain for growth.
3. With `/20`, the third octet advances in steps of $4096/256 = 16$: `10.0.0.0/20`, `10.0.16.0/20`, `10.0.32.0/20`, … , `10.0.240.0/20`.
4. Usable per subnet on AWS: $4096 - 5 = 4,091$.
5. Membership test: is `10.0.37.200` in `10.0.32.0/20`? The block covers third-octet values 32–47 (32 + 16 − 1). 37 is in that range, so **yes**; its range is `10.0.32.0` – `10.0.47.255`.

Interpretation: a `/20` per subnet sounds generous, but container platforms that give every pod its own VPC address (for example, the AWS VPC CNI for Kubernetes) can consume thousands of addresses per subnet. Running out of IPs in a subnet is a common production incident; size for pods, not for nodes (§42.9).

WORKED EXAMPLE

Worked example 3 — Do two networks overlap?

Team A uses `10.0.0.0/16` (10.0.0.0–10.0.255.255). Team B uses `10.0.128.0/17` (10.0.128.0–10.0.255.255). B's range lies entirely inside A's, so the two VPCs **overlap** and cannot be peered or connected with simple routing: a packet to 10.0.200.1 would be ambiguous. Engineering implication: allocate non-overlapping RFC 1918 ranges centrally (an IP address management plan) *before* teams create networks; renumbering later is painful.

Table 7.2 — CIDR quick reference (IPv4).

Prefix	Mask	Addresses	Typical use
/8	255.0.0.0	16,777,216	Large private network (10.0.0.0/8)
/16	255.255.0.0	65,536	A cloud VPC
/20	255.255.240.0	4,096	A large subnet per availability zone
/24	255.255.255.0	256	Classic LAN; smallest prefix widely accepted in global BGP
/26	255.255.255.192	64	Small subnet
/30	255.255.255.252	4	Legacy point-to-point link (2 usable)
/31	255.255.255.254	2	Point-to-point link (RFC 3021, both usable)
/32	255.255.255.255	1	A single host route

7.4.3 Private and special-purpose ranges

Some ranges are reserved and never routed on the public Internet. The most important are the **private** ranges of RFC 1918, which any organization may reuse internally; they reach the Internet only through NAT (§7.13).

Table 7.3 — Special-purpose address ranges.

Range	Size	Purpose
10.0.0.0/8	16,777,216	Private (RFC 1918)
172.16.0.0/12	1,048,576	Private (RFC 1918): 172.16.0.0–172.31.255.255
192.168.0.0/16	65,536	Private (RFC 1918), common in homes
100.64.0.0/10	4,194,304	Shared address space for carrier-grade NAT (RFC 6598)
127.0.0.0/8	16,777,216	Loopback (`127.0.0.1` = this machine)
169.254.0.0/16	65,536	Link-local; cloud instance metadata services commonly live at `169.254.169.254`
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	256 each	Documentation examples (RFC 5737); used in this book
`::1/128`	1	IPv6 loopback
`fe80::/10`	—	IPv6 link-local
`fc00::/7`	—	IPv6 unique local addresses (RFC 4193), the IPv6 analogue of private space
`2000::/3`	—	IPv6 global unicast
`2001:db8::/32`	—	IPv6 documentation (RFC 3849)

7.4.4 IPv4 exhaustion and IPv6

IPv4 offers $2^{32} \approx 4.29$ billion addresses, fewer than the number of connected devices. The regional Internet registries ran out of freely allocatable IPv4 space during the 2010s, and IPv4 addresses are now bought and sold. Two responses coexist: NAT (§7.13), which lets many devices share one public address, and IPv6, which offers $2^{128} \approx 3.4 \times 10^{38}$ addresses.

IPv6 makes several other changes [RFC 8200]:

- **Fixed 40-byte header** with optional extension headers, simpler for hardware to parse.
- **No fragmentation by routers.** Only the sender fragments; routers send an ICMPv6 “Packet Too Big” message instead, so path MTU discovery is mandatory.
- **No header checksum** (the link and transport layers already check).
- **Standard /64 subnets.** A single LAN gets 2^{64} addresses, enough for hosts to self-assign addresses (SLAAC). A typical site receives a `/48`, which holds $2^{64-48} = 65,536$ `/64` subnets.
- **Notation shortcuts.** Leading zeros in a group may be dropped, and one run of all-zero groups may be replaced by `::`. So `2001:0db8:0000:0000:0000:0000:0000:0025` is

`2001:db8::25`.

Clients with both IPv4 and IPv6 use **Happy Eyeballs** (RFC 8305): they start connecting over IPv6 and, if that has not succeeded within a short delay (around 250 ms is recommended), race an IPv4 attempt, using whichever connects first. This prevents broken IPv6 paths from adding seconds of delay.

COMMON MISTAKE

"Private addresses are a security feature." RFC 1918 ranges are not routed on the Internet, which incidentally makes hosts unreachable from outside without NAT rules. But reachability is not access control: a compromised host inside the network reaches them freely. Use firewalls and authentication (§7.14, §10.10), not address choice, for security.

7.5 Routing basics, BGP and anycast

7.5.1 Forwarding and longest-prefix match

DEFINITION

Definition — Routing and forwarding. Forwarding is the per-packet act of choosing an outgoing interface for a packet based on its destination address and the **forwarding table**. **Routing** is the control-plane process of building those tables by exchanging reachability information with other routers.

Each router holds a table of prefixes and next hops. When a packet arrives, the router selects the **most specific** (longest) prefix that contains the destination. This rule is called **longest-prefix match**.

WORKED EXAMPLE

Worked example — Longest-prefix match.

A router has these entries:

Prefix	Next hop
0.0.0.0/0 (default route)	ISP uplink
10.0.0.0/8	Core router A
10.1.0.0/16	Core router B
10.1.2.0/24	Access switch C

- Destination `10.1.2.3` matches /0, /8, /16 and /24. Longest is /24, so it goes to **C**.
- `10.1.9.9` matches /0, /8, /16: goes to **B**.
- `10.200.0.1` matches /0 and /8: goes to **A**.
- `8.8.8.8` matches only /0: goes to the **ISP uplink**.

Engineering implication: longest-prefix match lets operators override broad routes with specific ones. It also lets an attacker or a misconfiguration hijack traffic by announcing a *more specific* prefix than the legitimate owner, as the industry example below shows.

When several equally good next hops exist, routers use **equal-cost multi-path (ECMP)** routing: they hash each packet's flow identifiers (the 5-tuple of protocol, source and destination address and port) to pick one path. All packets of one flow take the same path, which avoids reordering, while different flows spread across paths. ECMP is the foundation of datacenter fabrics (§7.17) and of horizontally scaled load balancers (§19.8).

7.5.2 Interior and exterior routing

The Internet is a network of networks. Each independently operated network, such as an ISP, a cloud provider or a large enterprise, is an **autonomous system (AS)** identified by an AS number (ASN). Routing happens at two levels:

- **Interior gateway protocols** (OSPF, IS-IS) run *inside* an AS. They compute shortest paths using link costs, converge in seconds or less, and assume everyone is cooperative.
- **The Border Gateway Protocol (BGP)**, version 4 (RFC 4271), runs *between* ASes. It is the protocol that holds the Internet together.

DEFINITION

Definition — BGP. A **path-vector** routing protocol in which neighboring routers (peers) exchange, over a TCP session on port 179, announcements of the form "prefix P is reachable via AS path [X, Y, Z]". Each AS chooses among the paths it hears according to its own **policy**, then re-announces its chosen path (with its own ASN prepended) to the peers its policy allows.

How BGP works internally

1. Two routers in different ASes open a TCP connection on port 179 and exchange OPEN messages.
2. Each sends UPDATE messages announcing prefixes it can reach, with attributes including the **AS path** (the list of ASes traversed), the next hop, and preference hints.
3. Each router stores all received paths, then chooses one best path per prefix. The decision is policy-first: a network prefers routes learned from paying customers over routes from peers, and peers over upstream providers it pays. Only after policy does it prefer the shortest AS path, and then a series of tie-breakers.
4. The router installs the best path in its forwarding table and announces it onward, prepending its own ASN. Loops are prevented because a router rejects any path that already contains its own ASN.
5. When a route disappears, a WITHDRAW propagates. Convergence across the Internet can take seconds to minutes.

Two consequences matter for system designers. First, **BGP picks paths by business policy, not latency**: traffic between two cities may detour through a third. Second, **BGP trusts announcements by default**. The Resource Public Key Infrastructure (RPKI) lets prefix owners publish cryptographically signed statements of which ASN may originate their prefixes, and route-origin validation is increasingly deployed, but it is not universal.

7.5.3 Anycast

DEFINITION

Definition — Anycast. An addressing scheme in which the same IP prefix is announced via BGP from many locations at once. Each client's packets are routed to whichever announcing location is "closest" according to BGP path selection.

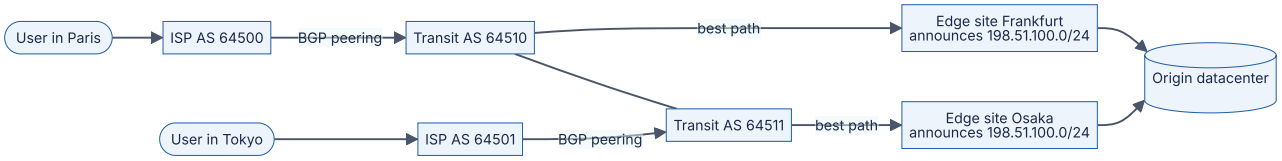
Analogy: a national emergency phone number connects you to your nearest dispatch center, not to a single central office. You dial the same number everywhere.

Anycast is how the DNS root servers, public resolvers such as `8.8.8.8` and `1.1.1.1`, and most CDNs (§21.3) put one address in front of hundreds of sites. Its strengths are auto-

matic proximity, natural absorption of distributed denial-of-service (DDoS) traffic across many sites (§10.11), and failover by **withdrawal**: if a site fails, it stops announcing the prefix, and BGP reroutes clients to the next-nearest site.

Its weaknesses follow from BGP. “Closest” means best by BGP policy, which usually but not always correlates with lowest latency. And because routing can change mid-connection, a long-lived TCP connection can suddenly have its packets delivered to a different site that has no state for it. For this reason anycast was first popular for short UDP exchanges such as DNS; operators that use it for TCP rely on route stability in practice and design for the occasional reset.

Figure 7.2 — BGP routing between autonomous systems, with an anycast prefix announced from two sites.



Reading the diagram. Two users (stadiums) reach their ISPs, which are separate autonomous systems connected to transit providers by BGP sessions. Two edge sites announce the *same* prefix, `198.51.100.0/24`. The Paris user’s ISP hears the shortest policy-preferred path via Frankfurt, and the Tokyo user’s ISP hears a path via Osaka, so the same destination address is served from different continents. Traffic flows synchronously left to right for requests (responses return along a path that may differ, because Internet routing is asymmetric). The origin datacenter is the storage boundary; the edges may cache (§21.4). Failure points: if Frankfurt withdraws its announcement, Paris traffic reconverges toward Osaka (or another site) within seconds to minutes, and in-flight TCP connections break. Scaling: add more announcing sites.

Real-world industry applications

INDUSTRY EXAMPLE

Facebook, 4 October 2021. During routine maintenance, a command intended to assess backbone capacity unintentionally disconnected Facebook’s backbone, and a bug in an audit tool failed to stop it. Facebook’s DNS servers are designed to withdraw their BGP anycast announcements when they cannot reach the datacenters, as a health safeguard. With the backbone gone, every DNS site withdrew at once, so `facebook.com` and related names could not be resolved anywhere. The outage lasted roughly six hours, and recovery was slowed because internal tools depended on the same network. Lesson: a health-driven withdrawal that is correct for one site can become a global outage when all sites share a failure; build in “never withdraw the last route” guards and out-of-band access. *Evidence: Primary* — [Janardhan 2021]

INDUSTRY EXAMPLE

YouTube prefix hijack, February 2008. Pakistan Telecom, intending to block YouTube domestically, announced a more-specific `/24` covering YouTube's address space. Its upstream provider propagated the announcement globally, and by longest-prefix match much of the Internet sent YouTube traffic to Pakistan for about two hours. The incident is a standard case study for BGP's trust model and motivates prefix filtering and RPKI. *Evidence: Secondary* — [RIPE NCC 2008]

Failure scenarios

Table 7.4 — Routing failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Route leak or hijack	A network announces prefixes it does not own, or re-announces routes against policy	External BGP monitors; sudden traffic shifts; latency jumps	Contact upstreams; announce more-specific prefixes	RPKI origin validation, prefix filters, max-prefix limits
Self-inflicted withdrawal	Health checks or automation withdraw all announcements simultaneously	Global reachability probes fail	Out-of-band access to re-announce	Guard rails preventing withdrawal of the last route; staged changes
Slow convergence	BGP path exploration after a failure	Elevated loss for seconds to minutes	Wait; fail over at a higher layer (DNS, client retry)	Multiple upstreams; BFD for fast failure detection
Anycast connection resets	Route change moves an in-flight TCP flow to another site	Spikes in TCP resets during route changes	Client retries	Short-lived connections; stable announcements

7.6 DNS: the Internet's directory

7.6.1 What DNS is and why it exists

DEFINITION

Definition — Domain Name System (DNS). A hierarchical, distributed database that maps human-readable **domain names** (such as `www.example.com`) to data, most often IP addresses, and the protocol used to query it [RFC 1034; RFC 1035].

First-principles explanation

People remember names; routers need numbers. In the early ARPANET, every host downloaded a single file, `HOSTS.TXT`, listing every name and address. As the network grew, that file became too large, changed too often, and depended on one central maintainer. DNS replaced it with a **delegated hierarchy**, much like a library catalog split across buildings: the front desk (the **root**) does not know where every book is, but it knows which building holds each subject (the **top-level domains** such as `.com`), and each building knows which room holds each shelf (the **authoritative servers** for `example.com`). Responsibility for each zone is delegated to whoever owns it, so no one party maintains everything.

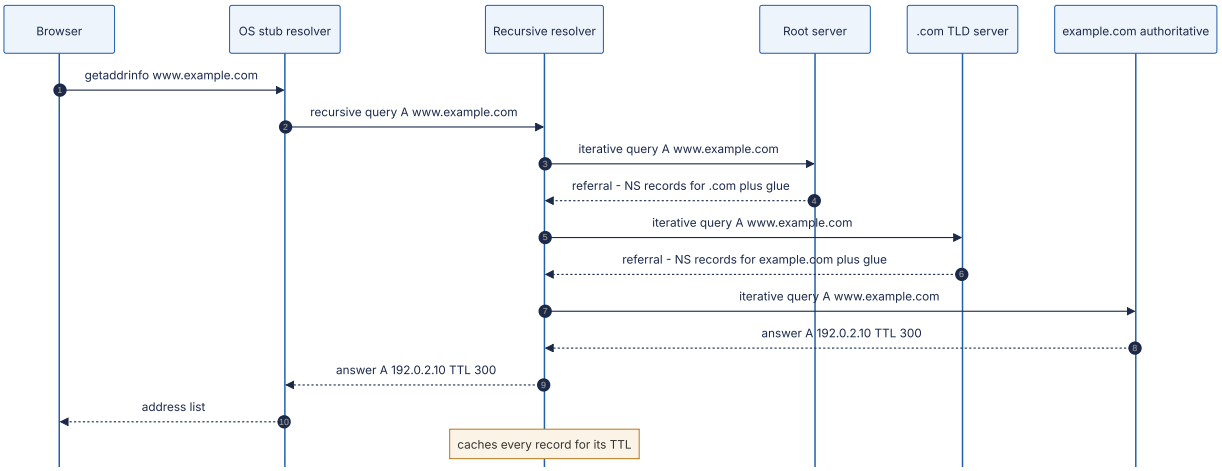
The cast of characters

- **Stub resolver.** A small library in your operating system. It does not do the work itself; it asks a recursive resolver and waits.
- **Recursive resolver** (also called a caching or full-service resolver). Run by your ISP, your company, or a public service such as `8.8.8.8` or `1.1.1.1`. It chases the answer through the hierarchy on your behalf and caches the results.
- **Root name servers.** Thirteen named server identities (`a.root-servers.net` through `m.root-servers.net`), each replicated via anycast (§7.5.3) to many physical instances worldwide. They know where each top-level domain's servers are.
- **TLD name servers.** Authoritative for a top-level domain such as `.com` or `.uk`. They know the name servers for each second-level domain.
- **Authoritative name servers.** Hold the actual records for a zone such as `example.com`. Managed DNS providers and cloud DNS services operate these for most companies.

7.6.2 Recursive versus iterative resolution

There are two query styles. In a **recursive query**, the client asks "give me the final answer", and the server must either return it or an error. Your stub resolver sends recursive queries to the recursive resolver. In an **iterative query**, the server returns the best information it has, often a **referral** ("I don't know, but ask these servers"), and the asker continues. Recursive resolvers send iterative queries to root, TLD and authoritative servers.

Figure 7.3 — DNS resolution of www.example.com with an empty cache.



Reading the diagram. Six participants appear from left to right: the application, the operating system's stub resolver, the recursive resolver, and three levels of authoritative servers. Steps 1–2 are the recursive part: the stub asks once and waits. Steps 3–8 are iterative: the resolver walks down the hierarchy, receiving referrals (dashed return arrows) from the root and the TLD, each containing the name-server (NS) records for the next level and **glue** records giving those servers' IP addresses. Step 8 is the final answer, here an A record with a 300-second TTL. All exchanges are synchronous request/response, normally over UDP port 53. The resolver's cache is the storage boundary: once populated, steps 3–8 are skipped for every client of that resolver until the TTL expires, and the root and TLD referrals are cached for much longer (typically days). Failure points: any server not responding forces a timeout and retry against another listed server (each zone must have at least two). Scaling comes from caching and from anycast replication of the root and TLD servers. (The address 192.0.2.10 is a documentation address used for illustration.)

Mathematical foundations: what a cold lookup costs

WORKED EXAMPLE

Worked example — Cold versus warm DNS lookup.

Assume the client is 5 ms from its recursive resolver, and the resolver is 20 ms from the root instance, 30 ms from the TLD server and 40 ms from the authoritative server (all round-trip times).

1. Cold cache: $5 + 20 + 30 + 40 = 95$ ms before the browser can even open a connection.
2. Root and TLD referrals cached (typical, since they have long TTLs) but the name itself expired: $5 + 40 = 45$ ms.
3. Fully warm at the resolver: 5 ms.
4. Cached in the operating system or browser: effectively 0 ms.

Interpretation: almost all real lookups hit case 3 or 4, which is why DNS feels free. The cold path matters for rarely visited domains, for very low TTLs, and for every new third-party domain a page references. Engineering implication: minimize the number of distinct hostnames on a critical path, and use `<link rel="dns-prefetch">` or `preconnect` hints for domains you know you will need.

7.6.3 Record types

Table 7.5 — Common DNS record types.

Type	Maps a name to	Example	Typical use
A	An IPv4 address	`www.example.com. 300 IN A 192.0.2.10`	Host address
AAAA	An IPv6 address	`www.example.com. 300 IN AAAA 2001:db8::10`	Host address over IPv6
CNAME	Another name (an alias)	`shop.example.com. 3600 IN CNAME shops.saas-provider.example.`	Point your name at a provider's hostname
MX	A mail server name, with priority	`example.com. 3600 IN MX 10 mail1.example.com.`	Email delivery (lower number = preferred)
NS	The authoritative name servers of a zone	`example.com. 86400 IN NS ns1.dnsprovider.example.`	Delegation
TXT	Arbitrary text	`example.com. 300 IN TXT "v=spf1 include:_spf.mailer.example -all"`	Domain ownership proofs, SPF, DKIM, DMARC
SRV	Host and port for a service, with priority and weight	`_sip._tcp.example.com. 300 IN SRV 10 60 5060 sip1.example.com.`	Service discovery (RFC 2782); used by SIP, XMPP, some service meshes
SOA	Zone metadata: primary server, serial, timers, negative-caching TTL	—	Zone administration
PTR	A name, for reverse lookup of an address	`10.2.0.192.in-addr.arpa. PTR www.example.com.`	Logs, mail server checks
CAA	Which certificate authorities may issue for the domain	`example.com. CAA 0 issue "letsencrypt.org"`	Restricting certificate issuance (§7.10)
HTTPS / SVCB	Connection parameters: ALPN protocols, ports, address hints	`example.com. HTTPS 1 . alpn="h3,h2"`	Advertising HTTP/3 before the first connection (RFC 9460)

Three rules trip people up. First, a **CNAME cannot coexist with other records at the same name**, so you cannot put a CNAME at the zone apex (`example.com` itself, which must hold SOA and NS records). DNS providers offer non-standard "ALIAS" or "CNAME flattening" features to work around this. Second, every record in a response carries its own TTL. Third, MX and SRV records point to *names*, never directly to addresses.

7.6.4 TTL and caching

DEFINITION

Definition — DNS time to live (TTL). A per-record value, in seconds, that tells any cache how long it may reuse the record before asking again. **Negative caching** stores "this name does not exist" (NXDOMAIN) answers for a duration derived from the zone's SOA record [RFC 2308].

Caches exist at several levels: the browser, the operating system, the recursive resolver, and sometimes in application runtimes. (Some runtimes have historically cached DNS answers far longer than the TTL, or forever, by default; the Java Virtual Machine's behavior depends on configuration and security manager settings, so check your runtime.)

TTL is a trade-off between **agility** (how quickly a change takes effect) and **load and latency** (how often resolvers must re-query). Short TTLs (30–60 s) let you fail over quickly but raise query volume and the fraction of slow cold lookups. Long TTLs (hours to a day) are efficient but make changes slow.

WORKED EXAMPLE

Worked example — How long does a DNS failover really take?

Your record has TTL 300 s, and your health checker takes 30 s (three failed probes, 10 s apart) to declare the primary site dead and update DNS.

1. Detection: 30 s.
2. Worst-case cache staleness: a resolver that refreshed one second before the update keeps serving the old answer for up to 300 s.
3. Worst-case total for well-behaved resolvers: $30 + 300 = 330$ s, about 5.5 minutes. The average client re-queries halfway through the TTL, so the mean is about $30 + 150 = 180$ s.
4. Clients holding open connections are not affected by DNS at all: they keep sending to the dead address until their connections fail and they reconnect.
5. Misbehaving caches that enforce a minimum TTL or ignore it extend the tail indefinitely.

Engineering implication: DNS failover is a *minutes-scale* mechanism. If you need seconds, put a stable address in front (an anycast virtual IP or a load balancer, §19.8) and fail over behind it.

Before a planned migration, lower the TTL well in advance. If the old TTL is 86,400 s (one day), you must lower it at least one day before the cut-over, because resolvers that cached the record just before you changed it will not re-query for a day.

7.6.5 DNS-based load balancing and its limits

Because a DNS answer can differ per query, DNS is the coarsest and cheapest load balancer:

- **Round-robin DNS.** Return several A records, rotating their order. Clients typically try the first.
- **Weighted answers.** Return each address in proportion to configured weights.
- **Geo-DNS / latency-based DNS.** Answer based on the resolver's (or, with the EDNS Client Subnet extension, the client's) location, pointing users to a nearby region (§44, §19.9).
- **Health-checked DNS.** Remove addresses whose health checks fail.

The limits are fundamental, not implementation details:

1. **Caching defeats fine control.** A large resolver (an ISP serving millions of users) caches one answer and hands it to all its clients for the TTL, so "1/N of queries" is not "1/N of users".
2. **Resolver location is not client location.** Geo-DNS sees the recursive resolver's address. A user in Singapore using a resolver in the US may be sent to the US region, unless EDNS Client Subnet is supported.
3. **Failover speed is bounded by TTL** and by stale clients, as computed above.
4. **No load awareness.** DNS does not see connection counts or server latency; it picks before the connection exists.
5. **Clients behave inconsistently.** Some try only the first address; some try all; some pin forever.

DNS is therefore used to pick a **region or site**, and real load balancers distribute connections *within* the site (Chapter 19).

Transport details

DNS normally uses UDP port 53 because a query and a response each fit in one datagram, avoiding a handshake. Classic DNS limited UDP responses to 512 bytes; the EDNS(0) extension (RFC 6891) lets clients advertise larger buffers, and current operational guidance favors about 1,232 bytes to avoid IP fragmentation. If a response is too large, the server sets the truncation (TC) bit and the client retries over TCP. Zone transfers between servers use TCP. Encrypted variants exist: DNS over TLS (port 853, RFC 7858) and DNS over HTTPS (RFC 8484), which hide queries from on-path observers.

Failure scenarios

Table 7.6 — DNS failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Authoritative provider outage	DDoS or provider failure; all NS records at one provider	Resolution failures from external probes	Wait, or change NS at the registrar (slow: TLD TTLs are long)	Two independent DNS providers serving the same zone
Stale records after migration	TTL was long when you changed the record	Traffic still reaching old addresses	Keep old endpoint alive until TTL × 2 has passed	Lower TTL in advance of planned changes
Resolver overload / latency	Too many lookups per request; no local caching	High p99 on connection setup; resolver timeouts	Add local caching resolver	Cache in-process with TTL respect; reuse connections
Negative-cache trap	Name queried before it was created; NXDOMAIN cached	"Works for me, not for them"	Wait out SOA negative TTL	Create records before announcing names
Split-horizon mismatch	Internal and external views of a zone differ	Errors only inside or only outside the network	Fix the view	Test both views in deployment checks

COMMON MISTAKE

"Changing a DNS record takes 24–48 hours to propagate." DNS does not propagate; caches *expire*. A change is visible to a given resolver as soon as its cached copy's TTL runs out. The folklore "48 hours" comes from long TTLs, especially on NS records at the TLD.

7.7 TCP: reliable byte streams over an unreliable network

7.7.1 What TCP provides

IP is a **best-effort** service: packets may be lost, duplicated, delayed or delivered out of order, and IP will not tell you. Most applications want the opposite: a reliable, ordered pipe of bytes. The **Transmission Control Protocol** builds that pipe on top of IP [RFC 9293].

DEFINITION

Definition — TCP. A connection-oriented transport protocol that provides a reliable, ordered, full-duplex **byte stream** between two endpoints identified by (IP address, port) pairs. It achieves reliability with sequence numbers, acknowledgments and retransmission; it protects a slow receiver with **flow control**; and it protects the network with **congestion control**.

First-principles explanation

Imagine mailing a 300-page manuscript one page per envelope through an unreliable postal service. To make sure your editor gets every page in order, you would number each page, ask the editor to send back postcards saying “I have everything up to page 57”, and resend any page not acknowledged within a reasonable time. You would also avoid sending all 300 pages at once if the editor’s desk is small (flow control) or if the post office is visibly overwhelmed (congestion control). That is TCP. The one twist is that TCP numbers **bytes**, not pages: the stream has no message boundaries, so an application that writes “hello” and then “world” may receive “hellowor” and “ld”. Application protocols must add their own framing (a length prefix or a delimiter).

7.7.2 The segment

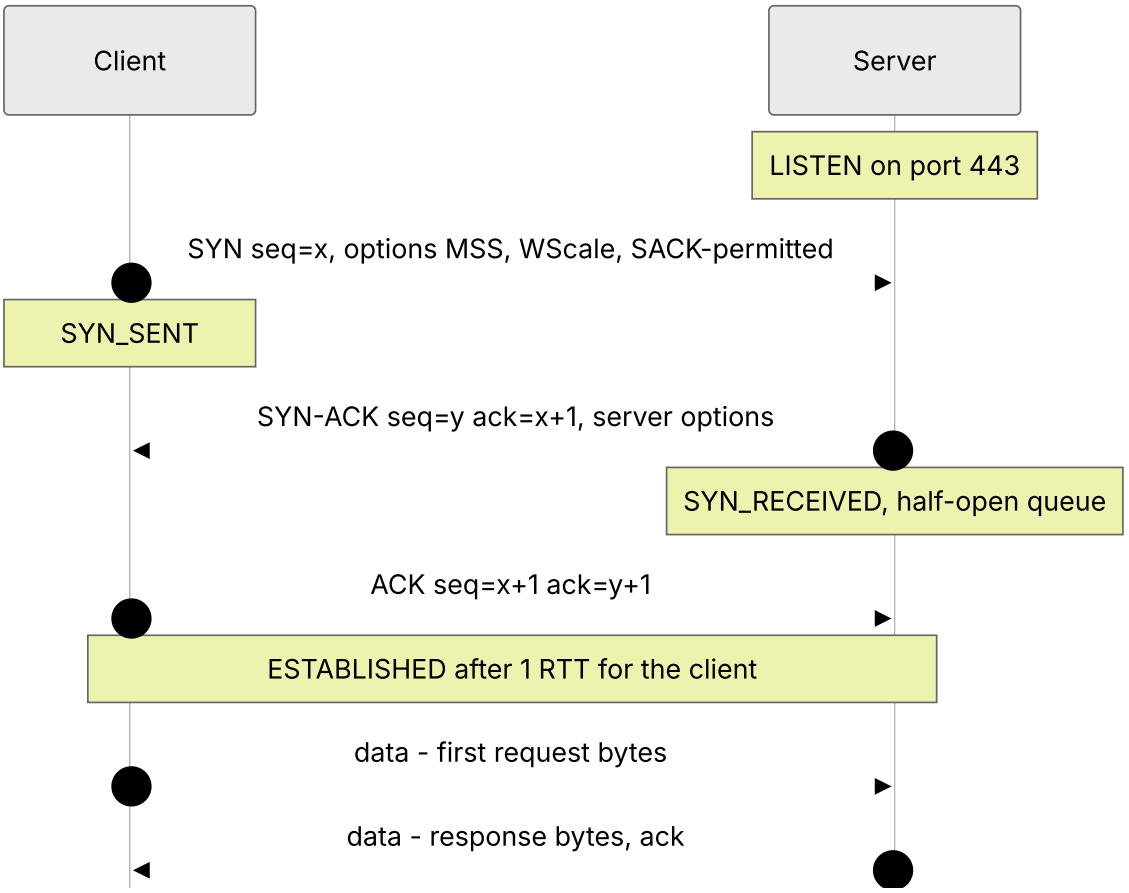
Every TCP segment carries a header of 20 bytes plus up to 40 bytes of options. The fields you will reason about most are:

- **Source and destination ports** (16 bits each). Together with the IP addresses they form the connection’s 4-tuple.
- **Sequence number** (32 bits). The stream offset of the first payload byte in this segment, relative to a random **initial sequence number (ISN)**.
- **Acknowledgment number** (32 bits). The next byte the sender of this segment expects to receive: “I have everything before this”.
- **Flags.** SYN (synchronize, opens a connection), ACK (acknowledgment field valid), FIN (no more data from me), RST (abort), PSH, URG, plus ECE/CWR for explicit congestion notification.
- **Window** (16 bits). How many more bytes the sender of this segment is willing to receive (flow control, §7.7.6).
- **Options.** MSS, window scale, selective acknowledgments (SACK) and timestamps, mostly negotiated in the handshake.

7.7.3 The three-way handshake

Before data flows, both sides must agree that the connection exists and learn each other’s initial sequence numbers. Each ISN is chosen randomly so that stray segments from an old connection, or an off-path attacker, are unlikely to fall into the valid window.

Figure 7.4 — TCP three-way handshake followed by the first request.



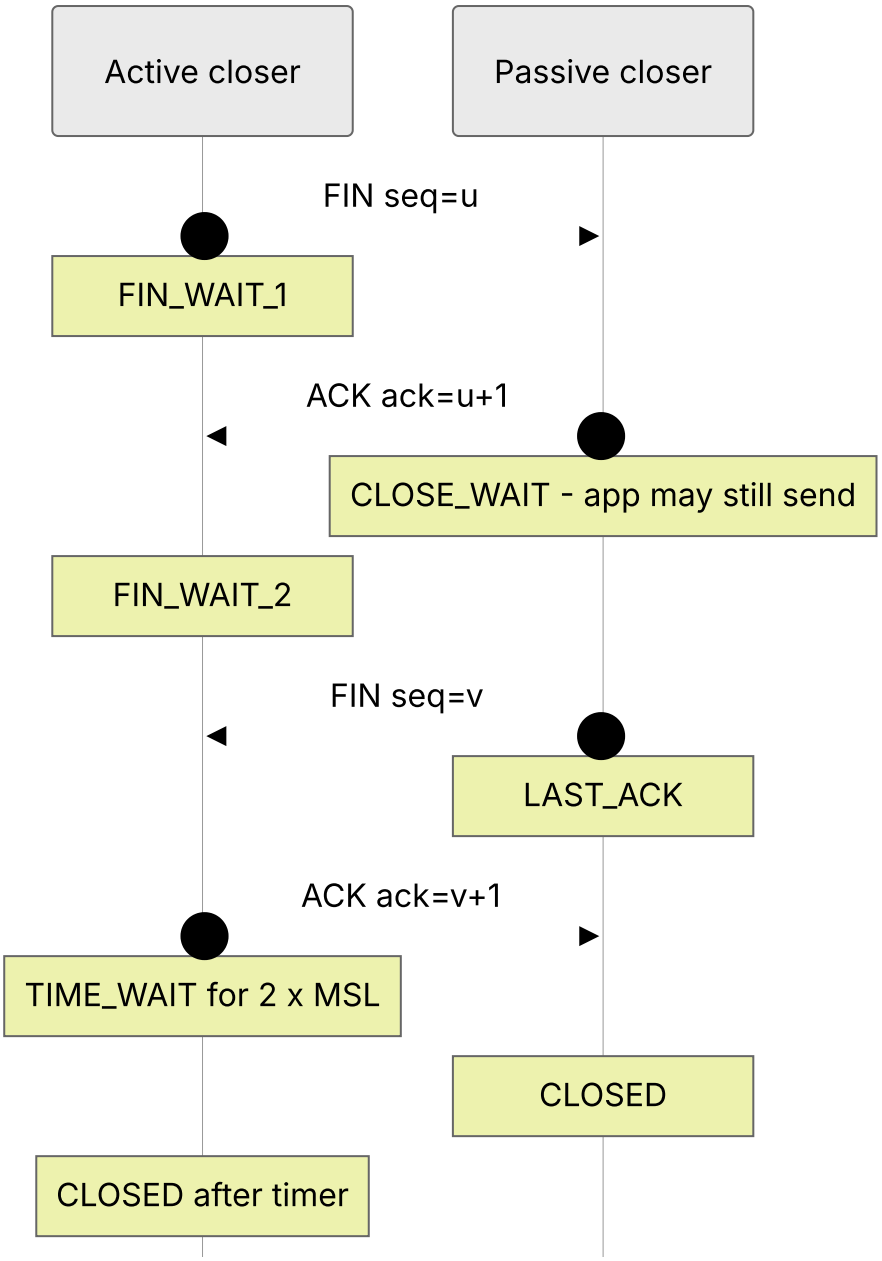
Reading the diagram. Two participants: the client, which actively opens, and the server, which is listening. Step 1 is the SYN, carrying the client's random ISN x and its option offers. Step 2, the SYN-ACK, acknowledges $x + 1$ (a SYN consumes one sequence number) and carries the server's own ISN y. Step 3 completes the handshake. The client may send data immediately after step 3, so the handshake costs the client **one round-trip time (RTT)** before its request can leave; the server sees the request 1.5 RTTs after the client began. All exchanges are synchronous. The server's half-open queue (connections between steps 2 and 3) is a failure point: a **SYN flood** attack fills it with spoofed SYNs, which servers counter with **SYN cookies**, encoding the connection state in the ISN instead of storing it. Scaling: the server's accept queue (`listen` backlog) must be sized for connection bursts.

TCP Fast Open (RFC 7413) lets a returning client put data in the SYN using a cookie from a previous connection, saving one RTT, but middlebox interference and replay concerns have limited its deployment. QUIC (§7.9.4) achieves the same goal more robustly.

7.7.4 The four-way close and TIME_WAIT

TCP is full-duplex, so each direction closes independently. Each side sends a FIN when it has no more data, and the other acknowledges it.

Figure 7.5 — TCP four-way close and the TIME_WAIT state.



Reading the diagram. The **active closer** is whichever side calls `close()` first; the other is the **passive closer**. Steps 1–2 close the active side's direction; the passive side enters CLOSE_WAIT and may keep sending until its application also closes (a pile-up of CLOSE_WAIT sockets is the classic symptom of an application that forgets to close connections). Steps 3–4 close the other direction. Steps 2 and 3 are often combined into one segment, making the close effectively three segments. The active closer then waits in **TIME_WAIT** for twice the **maximum segment lifetime (MSL)**. All steps are synchronous; RST is the abrupt alternative that skips all of this and discards unsent data.

TIME_WAIT exists for two reasons. First, if the final ACK (step 4) is lost, the passive side will retransmit its FIN, and the active side must still be around to re-acknowledge it. Second, it ensures that delayed segments from this connection have died out before a

new connection with the same 4-tuple can be created and misinterpret them. RFC 793 set MSL to two minutes; Linux hard-codes TIME_WAIT at 60 seconds.

WORKED EXAMPLE

Worked example — Ephemeral port exhaustion from TIME_WAIT.

A service opens a new TCP connection for every request to one backend (one destination IP and port) and closes it afterward, so the service is the active closer.

1. Each connection needs a unique source port. Linux's default ephemeral port range is 32,768–60,999: $60,999 - 32,768 + 1 = 28,232$ ports.
2. Each closed connection holds its port in TIME_WAIT for 60 s.
3. Sustainable new-connection rate to that single destination: $28,232/60 \approx 470$ connections per second.
4. At 1,000 requests per second with one connection per request, ports run out after about $28,232/1,000 \approx 28$ s, and `connect()` starts failing with `EADDRNOTAVAIL`.

Interpretation: TIME_WAIT is not a bug, but opening a connection per request is. Fixes, in order of preference: reuse connections with keep-alive and pooling (§7.16); add destination IPs or ports (each distinct 4-tuple has its own port space); widen the ephemeral range; enable `net.ipv4.tcp_tw_reuse` for outgoing connections. Never use the long-removed `tcp_tw_recycle`, which broke clients behind NAT.

7.7.5 Sequence numbers, acknowledgments and retransmission

TCP uses **cumulative acknowledgments**: ACK number *N* means "I have all bytes before *N*". The sender keeps every unacknowledged byte in its send buffer and retransmits on one of two signals:

1. **Retransmission timeout (RTO).** If no ACK arrives within a timeout computed from measured RTTs, the sender retransmits the oldest unacknowledged segment, doubles the timeout (**exponential backoff**), and treats the event as severe congestion.
2. **Fast retransmit.** If the receiver gets a segment out of order, it immediately resends its last ACK. Three **duplicate ACKs** strongly suggest a single lost segment with later ones still arriving, so the sender retransmits at once without waiting for the timer [RFC 5681].

Selective acknowledgments (SACK) let the receiver report exactly which blocks beyond the gap it holds, so the sender retransmits only what is missing.

FORMULA

Retransmission timeout (RFC 6298)

On each new RTT sample R (after the first):

$$\text{RTTVAR} \leftarrow (1-\beta)\,\text{RTTVAR} + \beta\,|\text{SRTT} - R|$$

$$\text{SRTT} \leftarrow (1-\alpha)\,\text{SRTT} + \alpha\,R$$

$$\text{RTO} = \text{SRTT} + \max(G,\ 4\,\text{RTTVAR})$$

Symbol	Meaning	Units
R	Latest measured round-trip time	ms
SRTT	Smoothed (exponentially averaged) RTT	ms
RTTVAR	Smoothed RTT variation	ms
α, β	Smoothing gains, $1/8$ and $1/4$	dimensionless
G	Clock granularity	ms
RTO	Retransmission timeout; RFC 6298 says round up to at least 1 s	ms

WORKED EXAMPLE

Worked example — Computing an RTO.

Current state: SRTT = 100 ms, RTTVAR = 10 ms. A new sample arrives: $R = 140$ ms. Assume G is negligible.

1. RTTVAR $= 0.75 \times 10 + 0.25 \times |100 - 140| = 7.5 + 10 = 17.5$ ms (computed with the *old* SRTT, as the RFC requires).
2. SRTT $= 0.875 \times 100 + 0.125 \times 140 = 87.5 + 17.5 = 105$ ms.
3. RTO $= 105 + 4 \times 17.5 = 175$ ms.
4. RFC 6298 rounds this up to its 1 s minimum; Linux uses a 200 ms minimum, so Linux would wait 200 ms.

Interpretation: inside a datacenter, where RTTs are around 0.1–0.5 ms, a single timeout-driven retransmission costs 200 ms, hundreds of times the normal latency. This is why one lost packet can dominate p99 latency for small RPCs (§23.6), and why datacenter operators tune minimum RTOs down and work hard to avoid drops.

7.7.6 Flow control and the receive window

Flow control prevents a fast sender from overflowing a slow receiver. Each ACK advertises the **receive window** (`rwnd`): the free space in the receiver's socket buffer. The sender never has more than `rwnd` unacknowledged bytes outstanding. If the application stops reading, the buffer fills, the advertised window drops to zero, and the sender pauses, probing periodically until the window reopens. Flow control is how TCP provides **backpressure** end to end (§22.10).

The header's window field is 16 bits, so it can express at most 65,535 bytes. That is far too small for fast long-distance links (see the bandwidth-delay product in §7.15), so the **window scale** option (RFC 7323), negotiated in the SYN, multiplies the window by 2^s for $s \le 14$, allowing windows up to about 1 GiB ($65,535 \times 2^{14} = 1,073,725,440$ bytes).

7.7.7 Congestion control

Flow control protects the receiver. **Congestion control** protects the network. In October 1986, the early Internet suffered **congestion collapse**: throughput between Lawrence Berkeley Laboratory and UC Berkeley fell from 32 kbps to 40 bps [Jacobson 1988] because senders retransmitted aggressively into already-full queues, so most bandwidth carried duplicate packets that were then dropped. Van Jacobson's 1988 algorithms, refined into today's RFC 5681, made each sender infer congestion and back off.

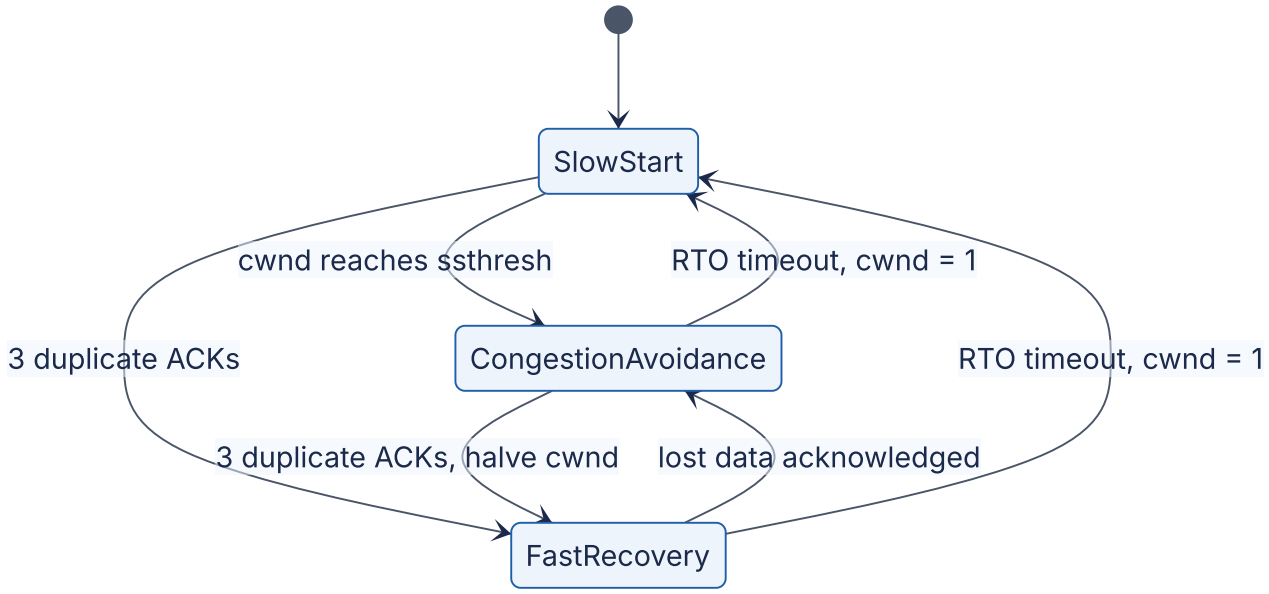
> DEFINITION
>
> **Definition — Congestion window (cwnd).** A sender-side limit, in bytes or segments, on unacknowledged data in flight, based on the sender's estimate of what the network path can absorb. The sender may have at most $\min(\text{cwnd}, \text{rwnd})$ bytes outstanding.

How classic congestion control works

1. **Slow start.** A new connection starts with a small initial window (10 segments, about 14.6 KB, per RFC 6928) and increases cwnd by one segment for every segment acknowledged. That **doubles cwnd every RTT**: "slow" only in comparison with sending everything at once.
2. When cwnd reaches the **slow-start threshold** (`ssthresh`), the sender switches to **congestion avoidance**: it adds about one segment per RTT (**additive increase**).
3. On a loss detected by three duplicate ACKs, the sender halves cwnd (**multiplicative decrease**), sets ssthresh to the new value, retransmits, and continues in congestion avoidance via **fast recovery**.
4. On a retransmission timeout, the sender assumes something severe happened, sets ssthresh to half the flight size, resets cwnd to one segment, and re-enters slow start.

Steps 2 and 3 together are **AIMD (additive increase, multiplicative decrease)**. It produces the familiar sawtooth of cwnd over time, and it has a beautiful property: multiple AIMD flows sharing a bottleneck converge toward an equal share.

Figure 7.6 — Loss-based TCP congestion control states (Reno-style).



Reading the diagram. Each box is a sender state. A connection begins in slow start, where cwnd doubles each RTT. Reaching ssthresh moves it to congestion avoidance (linear growth). Three duplicate ACKs, a mild loss signal, move it to fast recovery with cwnd halved; once the lost data is acknowledged, it returns to congestion avoidance. A timeout, the severe signal, sends it all the way back to slow start with cwnd of one segment. All transitions are driven by asynchronous events (ACK arrivals and timers) on the sender. Failure point: repeated timeouts on a lossy path keep the connection in slow start with exponentially growing RTOs, which users experience as a “hung” transfer.

WORKED EXAMPLE

Worked example — How many RTTs does slow start need for 1 MB?

Assume MSS = 1,460 bytes, initial cwnd = 10 segments, no loss, a large receive window, and ignore delayed ACKs.

1. Segments needed: $1,000,000/1,460 \approx 685$.
2. Segments sent per round (RTT): 10, 20, 40, 80, 160, 320, 640.
3. Cumulative: 10, 30, 70, 150, 310, 630, 1,270.
4. After 6 rounds, 630 < 685; the 7th round finishes the transfer. So **7 RTTs** of data transfer.
5. At an 80 ms RTT that is $7 \times 80 = 560$ ms, plus 1 RTT of TCP handshake and 1 RTT of TLS 1.3 handshake (§7.10): $9 \times 80 = 720$ ms in total, regardless of whether the link is 100 Mbps or 10 Gbps.

Interpretation: for small and medium transfers on long paths, **latency, not bandwidth, determines download time**. A 10× faster link barely helps; reducing RTT (a CDN edge close to the user, §21.3) or reusing a connection whose cwnd is already large (§7.16) helps enormously.

CUBIC

On fast long-distance paths, Reno's "add one segment per RTT" is painfully slow to regain a large window after a loss: after a loss halves a 10,000-segment window, regaining the missing 5,000 segments at one segment per 100 ms RTT takes $5,000 \times 0.1 = 500$ s. **CUBIC** [Ha et al. 2008; RFC 9438] replaces linear growth with a cubic function of *time since the last loss*, independent of RTT. It has been Linux's default since 2006 and is the default in most operating systems today.

FORMULA

CUBIC window growth

$$W(t) = C\,(t - K)^3 + W_{\max}, \qquad K = \sqrt[3]{\frac{W_{\max}\,(1-\beta)}{C}}$$

Symbol	Meaning	Units
$W(t)$	Congestion window t seconds after the last loss	segments
$W_{\max}$	Window just before the last loss	segments
β	Multiplicative decrease factor; window becomes $\beta W_{\max}$ after loss (0.7)	dimensionless
C	Scaling constant (0.4)	segments/s³
K	Time to climb back to $W_{\max}$	s

WORKED EXAMPLE

Worked example — CUBIC recovery time.

Suppose $W_{\max} = 100$ segments when a loss occurs.

1. After the loss: $W(0) = 0.7 \times 100 = 70$ segments. Check with the formula: $K^3 = 100 \times 0.3/0.4 = 75$, so $K = \sqrt[3]{75} \approx 4.22$ s, and $W(0) = 0.4 \times (-4.22)^3 + 100 = 0.4 \times (-75) + 100 = 70$. Consistent.
2. Growth is fast at first, flattens as $t \to K$ (the curve is cautious near the window where loss last occurred), and reaches 100 segments at $t \approx 4.22$ s.
3. Beyond K the window accelerates again, probing for newly available bandwidth.

Interpretation: recovery time depends on $W_{\max}$ and constants, not on RTT, so long-RTT flows are not penalized as much as under Reno. The plateau around $W_{\max}$ keeps CUBIC near the last known-good operating point.

BBR

Loss-based algorithms (Reno, CUBIC) treat packet loss as *the* congestion signal. That has two problems. On paths with large buffers, they keep filling the queue until it overflows, adding hundreds of milliseconds of queuing delay (**bufferbloat**, §7.15). On paths with random loss unrelated to congestion (wireless, long-haul links), they back off when they should not.

BBR (Bottleneck Bandwidth and Round-trip propagation time) [Cardwell et al. 2016] instead builds an explicit model of the path. It continuously estimates the **bottleneck bandwidth** (the maximum recent delivery rate) and the **minimum RTT** (the propagation delay with empty queues). It then paces packets at the estimated bandwidth and keeps roughly one bandwidth-delay product in flight, periodically probing slightly above that rate to discover more bandwidth and periodically draining the queue to re-measure minimum RTT.

Table 7.7 — Congestion control algorithms compared.

Aspect	Reno (AIMD)	CUBIC	BBR (v1)
Congestion signal	Packet loss	Packet loss	Measured bandwidth and RTT model
Growth function	+1 MSS per RTT	Cubic in time since last loss	Pacing at estimated bottleneck rate, with probing
Response to loss	Halve cwnd	Multiply by 0.7	Largely ignores isolated loss
Queue occupancy	Fills buffers until loss	Fills buffers until loss	Aims for little standing queue
Strength	Simple, fair among equals	Good on high-BDP paths; default almost everywhere	Low latency; high throughput with random loss
Weakness	Slow on high-BDP paths	Bufferbloat; collapses with random loss	Fairness issues versus loss-based flows; retransmissions with shallow buffers
Where used	Teaching baseline	Linux, Windows, macOS defaults	Google services, available in Linux since 4.9

INDUSTRY EXAMPLE

BBR at Google. Google reported deploying BBR on its B4 inter-datacenter backbone, where it delivered 2–25× higher throughput than CUBIC, and on YouTube and google.com traffic, where it reduced median RTT (by reducing queuing) while improving throughput. Later work (BBRv2/v3) addresses fairness and loss-sensitivity issues found in BBRv1. *Evidence: Primary* — [Cardwell et al. 2016]

7.7.8 Nagle's algorithm and delayed ACKs

In 1984, John Nagle observed that interactive programs sending one keystroke per write produced 41-byte packets (40 bytes of headers, 1 byte of data), wasting bandwidth. **Nagle's algorithm** (RFC 896) says: if there is unacknowledged data in flight, buffer small writes until either the ACK arrives or a full segment accumulates.

Separately, receivers use **delayed ACKs**: they wait briefly (up to 500 ms by RFC 1122; Linux uses about 40 ms or more) hoping to piggyback the ACK on response data or to acknowledge two segments at once.

Each idea is reasonable; together they cause a notorious stall. A client writes a request header (small write 1), then the body (small write 2), then waits for a response. Write 1 is sent. Nagle holds write 2 until write 1 is acknowledged. The server cannot respond before it has the whole request, so it has nothing to piggyback on and delays its ACK. The pair sit idle for the delayed-ACK timer, adding tens of milliseconds to every request. The fixes are to assemble the full message and write it once, or to set the `TCP_NODELAY` socket option, which disables Nagle. Almost every RPC framework and database driver sets `TCP_NODELAY` by default.

7.7.9 Head-of-line blocking

Head-of-line (HOL) blocking occurs when an item at the front of a queue holds up everything behind it. TCP guarantees in-order delivery of one byte stream, so if segment 5 is lost and segments 6–20 arrive, the receiving kernel holds 6–20 in its buffer and the application can read nothing until segment 5's retransmission arrives, at least one RTT later. If the application multiplexed several independent requests over that connection (HTTP/2, §7.9.3), all of them stall because of one packet that belonged to only one of them. This is the central motivation for QUIC (§7.9.4), which provides independent streams with no cross-stream HOL blocking.

7.7.10 Code example: a TCP echo server and client

```
        # echo_server.py — a minimal, runnable TCP echo server (Python 3.8+).
        import socket

        HOST, PORT = "127.0.0.1", 9000

        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # rebind despite
TIME_WAIT
            srv.bind((HOST, PORT))
            srv.listen(128)                          # backlog of completed handshakes
            print(f"listening on {HOST}:{PORT}")
            while True:
                conn, addr = srv.accept()            # handshake already done by the ker-
nel
                with conn:
                    conn.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
                    while True:
                        data = conn.recv(4096)       # may return fewer bytes than were
sent
                        if not data:                 # empty read: peer sent FIN
                            break
                        conn.sendall(data)           # loops until every byte is queued
```

```
        # echo_client.py — measures connect time and one request/response round trip.
        import socket, time

        t0 = time.perf_counter()
        s = socket.create_connection(("127.0.0.1", 9000))   # SYN, SYN-ACK, ACK
        t1 = time.perf_counter()
        s.sendall(b"hello")
        reply = s.recv(4096)
        t2 = time.perf_counter()
        s.close()                                             # client is active closer:
TIME_WAIT here
        print(reply, f"connect={1e3*(t1-t0):.3f} ms  rtt={1e3*(t2-t1):.3f} ms")
```

What to notice:

- `accept()` returns only after the kernel has completed the three-way handshake; your application never sees SYNs.
- `recv(4096)` returns *up to* 4,096 bytes. TCP is a byte stream, so a real protocol must loop until a complete message has arrived, using a length prefix or delimiter.
- `recv` returning `b""` is how the application learns that the peer sent FIN.
- `SO_REUSEADDR` lets a restarted server bind its port while old connections linger in TIME_WAIT.
- The server handles one client at a time; production servers use threads, an event loop (§6.13) or `asyncio`.

- Over loopback, both timings will be tens of microseconds; the same client against a server 80 ms away would show about 80 ms for each.

Failure scenarios

Table 7.8 — TCP failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Ephemeral port exhaustion	Connection per request; TIME_WAIT holds ports	`EADDRNOTAVAIL`; many TIME_WAIT sockets in `ss -s`	Reuse connections; `tcp_tw_reuse`	Pooling and keep-alive (§7.16)
CLOSE_WAIT leak	Application never closes sockets after peer FIN	Growing CLOSE_WAIT count; file-descriptor exhaustion	Restart process; fix code	Close in `finally`/context managers; timeouts
Half-open connection	Peer crashed or a NAT/firewall dropped state; no FIN sent	Writes hang or eventually reset; reads block forever	Application-level timeouts; reconnect	TCP keep-alive or application heartbeats; read deadlines
Retransmission-driven tail latency	Packet loss; minimum RTO of 200 ms+	p99 spikes; `nstat` retransmission counters	—	Avoid drops; hedged requests (§23.7)
Nagle/delayed-ACK stall	Multiple small writes per request	Latency floor of about 40 ms on small requests	Set `TCP_NODELAY`	Write whole messages; framework defaults
SYN flood	Spoofed SYNs fill the half-open queue	Many SYN_RECV sockets	SYN cookies	SYN cookies on by default; DDoS protection

COMMON MISTAKE

"TCP guarantees my message was processed." An ACK means the peer's *kernel* received the bytes into a buffer, not that the application read them, let alone acted on them or wrote them durably. If the server crashes after ACKing, the data is gone. End-to-end confirmation must come from the application (§24.14, the end-to-end argument).

COMMON MISTAKE

"A TCP connection notices immediately when the other side dies." If the peer vanishes without sending FIN or RST (power loss, cable cut, a firewall silently dropping state), an idle connection looks healthy indefinitely. TCP keep-alive probes default to starting after two hours of idleness on Linux. Always set application-level timeouts.

7.8 UDP and when to use it

DEFINITION

Definition — UDP. The User Datagram Protocol [RFC 768] is a connectionless transport protocol that adds only port numbers, a length and a checksum to IP. Its 8-byte header provides process-to-process delivery of individual **datagrams** with no handshake, no acknowledgments, no retransmission, no ordering, and no flow or congestion control.

If TCP is registered mail with numbered pages and receipts, UDP is a postcard: cheap, immediate, and with no guarantee it arrives or that two postcards arrive in order. UDP does preserve message boundaries: one `sendto()` produces one datagram, and the receiver gets exactly that datagram or nothing.

Why does this exist?

Some applications are hurt more by TCP's guarantees than helped:

- **Request/response with tiny messages.** A DNS query fits in one packet. A TCP handshake would triple the latency; if the reply is lost, the client simply asks again.
- **Real-time media and games.** A voice packet that arrives 300 ms late is useless; it is better to skip it than to stall the stream waiting for a retransmission (TCP's head-of-line blocking).
- **Custom transports.** QUIC (§7.9.4) implements its own reliability, streams and congestion control *on top of* UDP, because UDP passes through existing firewalls and NATs and can be implemented in user space without kernel changes.
- **Multicast and broadcast**, such as DHCP address assignment and market-data feeds, which TCP's one-to-one connections cannot express.
- **High-volume, loss-tolerant telemetry**, such as StatsD metrics, where an occasional lost sample does not matter.

Table 7.9 — TCP versus UDP.

Aspect	TCP	UDP
What it is	Reliable, ordered byte stream	Unreliable datagrams
Connection setup	3-way handshake (1 RTT)	None
Header	20–60 bytes	8 bytes
Reliability and ordering	Yes, via sequence numbers and retransmission	No
Message boundaries	No (byte stream)	Yes (one datagram = one message)
Flow and congestion control	Yes	No; application's responsibility
Head-of-line blocking	Yes, within the stream	No
State on server	Per connection	None required
Typical uses	HTTP/1.1, HTTP/2, databases, RPC	DNS, VoIP, games, QUIC/HTTP/3, DHCP, telemetry
Choose when	You need every byte, in order, and can tolerate setup cost	Timeliness beats completeness, or you are building your own transport

What you must rebuild on UDP

Choosing UDP moves responsibilities into your application: detecting loss (sequence numbers), deciding whether to retransmit, reordering, splitting messages to fit the path MTU (a UDP datagram larger than the MTU gets fragmented by IP, and losing any fragment loses the whole datagram), and above all **congestion control**. A UDP application that sends at a fixed high rate regardless of loss harms every other flow on the path; RFC 8085 gives guidelines for responsible UDP use.

> WARNING / TRADE-OFF
>
> **UDP amplification attacks.** Because UDP has no handshake, an attacker can forge the source address of a small request so that a server sends a much larger response to the victim. Open DNS resolvers, NTP and memcached servers exposed over UDP have been abused for very large DDoS attacks. Never expose UDP services that send responses much larger than requests to the Internet without rate limiting and source validation (§10.11).

7.9 HTTP/1.1, HTTP/2, HTTP/3 and QUIC

7.9.1 HTTP semantics

The **Hypertext Transfer Protocol** is a request/response application protocol. A client sends a request with a **method** (GET, POST, PUT, DELETE, …), a **target** (path), **header fields** (metadata such as `Host`, `Content-Type`, `Cache-Control`) and an optional body. The

server answers with a **status code** (200 OK, 301 redirect, 404 not found, 503 unavailable, …), headers and an optional body. RFC 9110 defines these **semantics** separately from how they are encoded on the wire, and three wire versions coexist: HTTP/1.1 [RFC 9112], HTTP/2 [RFC 9113] and HTTP/3 [RFC 9114]. Your application code sees the same methods, headers and status codes whichever version carries them. API design on top of HTTP is covered in §8.6. For a performance-oriented treatment of everything in this section, see [Grigorik 2013].

7.9.2 HTTP/1.1: text, keep-alive and pipelining

HTTP/1.1 messages are human-readable text:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: curl/8.5.0
Accept: */*
```

The response begins `HTTP/1.1 200 OK`, followed by headers, a blank line and the body. The body's end is marked by `Content-Length` or by **chunked transfer encoding**.

HTTP/1.0 opened a new TCP connection per request. HTTP/1.1 made **persistent connections** (keep-alive) the default: after a response, the connection stays open for the next request, saving a TCP (and TLS) handshake and keeping the warmed-up congestion window.

The remaining problem is that one connection carries **one request at a time**. HTTP/1.1 defined **pipelining**, sending several requests without waiting, but responses must return in request order, so one slow response blocks all those behind it (application-level head-of-line blocking). Buggy intermediaries and that HOL problem meant browsers never enabled pipelining by default. Browsers instead open **multiple parallel connections**, typically six per host, and sites used to "shard" assets across hostnames to get more. Each extra connection costs handshakes, server memory and separate slow starts, and connections compete with each other for bandwidth.

7.9.3 HTTP/2: binary framing and multiplexing

HTTP/2 (originally RFC 7540 in 2015, now RFC 9113) keeps HTTP semantics but changes the wire format.

How it works internally

1. **Binary framing.** Each message is split into **frames** with a 9-byte header giving length, type (HEADERS, DATA, SETTINGS, WINDOW_UPDATE, RST_STREAM, GOAWAY, PING, …), flags and a **stream identifier**.

2. **Streams and multiplexing.** Each request/response pair is a **stream** with its own ID (odd numbers for client-initiated streams). Frames from many streams interleave on one TCP connection, so a slow response no longer blocks others at the HTTP layer. One connection per origin replaces six.
3. **Header compression with HPACK** [RFC 7541]. Requests repeat nearly identical headers (cookies, user agent). HPACK keeps a **static table** of 61 common header fields and a **dynamic table** of recently sent ones on both ends, so repeated headers are sent as small integer indexes; new values may be Huffman-coded. HPACK avoids the compression side-channel attacks (CRIME) that made general-purpose compression of headers unsafe.
4. **Per-stream and connection flow control** with WINDOW_UPDATE frames, so one large download cannot starve other streams of buffer space.
5. **Stream cancellation** with RST_STREAM, without closing the connection.

Two features have faded. The original priority tree was complex and inconsistently implemented, and RFC 9113 deprecates it in favor of the simpler scheme in RFC 9218. **Server push**, which let servers send resources before they were requested, proved hard to use well; major browsers have removed support.

The TCP head-of-line problem remains

HTTP/2 removed head-of-line blocking at the HTTP layer, but all streams still share one TCP byte stream. A single lost packet stalls *every* stream until it is retransmitted (§7.7.9). On a clean network HTTP/2's single connection beats six HTTP/1.1 connections; at loss rates of a few percent, typical of poor mobile networks, it can do worse, because six independent connections each suffer only their own losses.

7.9.4 QUIC and HTTP/3

QUIC [RFC 9000] is a general-purpose transport protocol, developed first at Google and standardized by the IETF in 2021, that runs over UDP. **HTTP/3** [RFC 9114] is HTTP mapped onto QUIC.

Why does this exist?

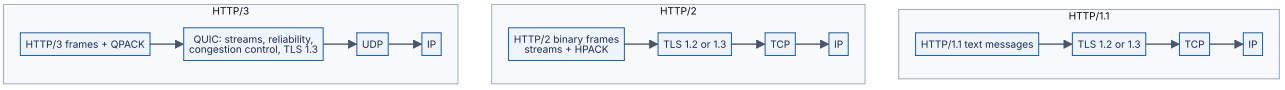
Three problems could not be fixed inside TCP. First, TCP's head-of-line blocking is inherent in its single ordered stream. Second, a secure connection costs two sequential handshakes (TCP, then TLS). Third, TCP lives in operating-system kernels and is inspected and sometimes rewritten by **middleboxes** (firewalls, NATs, load balancers), so new TCP features take a decade to deploy, a problem called **ossification**. Building on UDP, implementing the transport in user space, and encrypting nearly all transport headers solves all three.

How it works internally

1. **Integrated handshake.** QUIC carries the TLS 1.3 handshake inside its own packets [RFC 9001]. Transport and cryptographic setup happen together in **1 RTT**; a resumed connection can send application data in its first flight (**0-RTT**, with the caveats in §7.10.4).
2. **Independent streams.** Reliability and ordering are per stream. A lost packet delays only the streams whose data it carried.
3. **Connection IDs.** A QUIC connection is identified by connection IDs chosen by the endpoints, not by the IP/port 4-tuple. When a phone moves from Wi-Fi to cellular and its IP address changes, the connection **migrates** without a new handshake.
4. **Encrypted headers.** Packet numbers and most transport metadata are encrypted, so middleboxes cannot depend on (and ossify) them.
5. **Better loss recovery.** Packet numbers are never reused (a retransmission gets a new number), which removes TCP's ambiguity about whether an ACK refers to the original or the retransmission [RFC 9002].
6. **QPACK** [RFC 9204] replaces HPACK, because HPACK assumes in-order delivery of headers, which independent streams no longer guarantee.

Clients learn that a server supports HTTP/3 from an `Alt-Svc` response header on an earlier HTTP/2 connection, or from a DNS HTTPS record (§7.6.3), and fall back to HTTP/2 over TCP when UDP is blocked, as it is on some enterprise networks.

Figure 7.7 — Protocol stacks for the three HTTP versions.



Reading the diagram. Each column is one protocol stack, top (application) to bottom (network). In HTTP/1.1 and HTTP/2, the security layer (TLS) sits between HTTP and TCP, and reliability lives in the kernel's TCP. HTTP/2 changes only the top box: binary framing and multiplexed streams over a single TCP connection. In HTTP/3, QUIC absorbs the jobs of both TCP and TLS and runs over UDP, so the stream abstraction moves into the transport and loss on one stream no longer blocks others. Arrows represent encapsulation (synchronous hand-off of each message down the stack). Failure point specific to HTTP/3: networks that block or throttle UDP, handled by falling back to HTTP/2. Scaling consideration: QUIC runs in user space and encrypts every packet individually, so it has historically cost more server CPU per byte than kernel TCP with TLS; implementations close the gap with UDP segmentation offload and batching.

Table 7.10 — HTTP/1.1, HTTP/2 and HTTP/3 compared.

Aspect	HTTP/1.1	HTTP/2	HTTP/3
Specification	RFC 9112	RFC 9113	RFC 9114 (over QUIC, RFC 9000)
Encoding	Text	Binary frames	Binary frames
Transport	TCP	TCP	QUIC over UDP
Concurrency per connection	One request at a time	Many multiplexed streams	Many independent streams
Head-of-line blocking	HTTP layer and TCP	TCP layer only	None across streams
Header compression	None	HPACK	QPACK
Encryption	Optional (TLS)	Effectively required by browsers	Always (TLS 1.3 built in)
New-connection setup with TLS 1.3	2 RTT (TCP + TLS)	2 RTT	1 RTT; 0 RTT on resumption
Connection migration	No	No	Yes, via connection IDs
Debuggability	Plain text with telnet or curl	Needs tools	Needs tools and decrypting keys
Choose when	Simple clients, internal tooling, legacy	Default for APIs and gRPC; most traffic	Mobile and lossy networks, latency-sensitive web traffic

RECOMMENDATION

Choosing an HTTP version. Terminate HTTP/2 (and HTTP/3 where your load balancer or CDN supports it) at the edge for browsers and mobile clients. Inside a datacenter, HTTP/2 over long-lived connections (for example, gRPC, §8.8) is the common default: loss is rare, so TCP HOL blocking matters little, while kernel TCP is cheap and mature. Do not hand-roll HTTP/3 between internal services without a measured reason.

COMMON MISTAKE

“HTTP/2 is always faster than HTTP/1.1.” HTTP/2 wins in most browser workloads because it removes connection limits and compresses headers. But one connection means one congestion window and one point of HOL blocking: under heavy loss, or when a single connection’s cwnd is the bottleneck for bulk transfer, several HTTP/1.1 connections can outperform it. Measure on your traffic.

7.10 TLS 1.3 and HTTPS

7.10.1 What TLS provides

DEFINITION

Definition — Transport Layer Security (TLS). A protocol that gives two communicating applications **confidentiality** (eavesdroppers cannot read the data), **integrity** (tampering is detected) and **authentication** (at least the server proves its identity, usually with an X.509 certificate). **HTTPS** is HTTP carried over TLS, on TCP port 443 by default. TLS 1.3 is specified in RFC 8446 (2018).

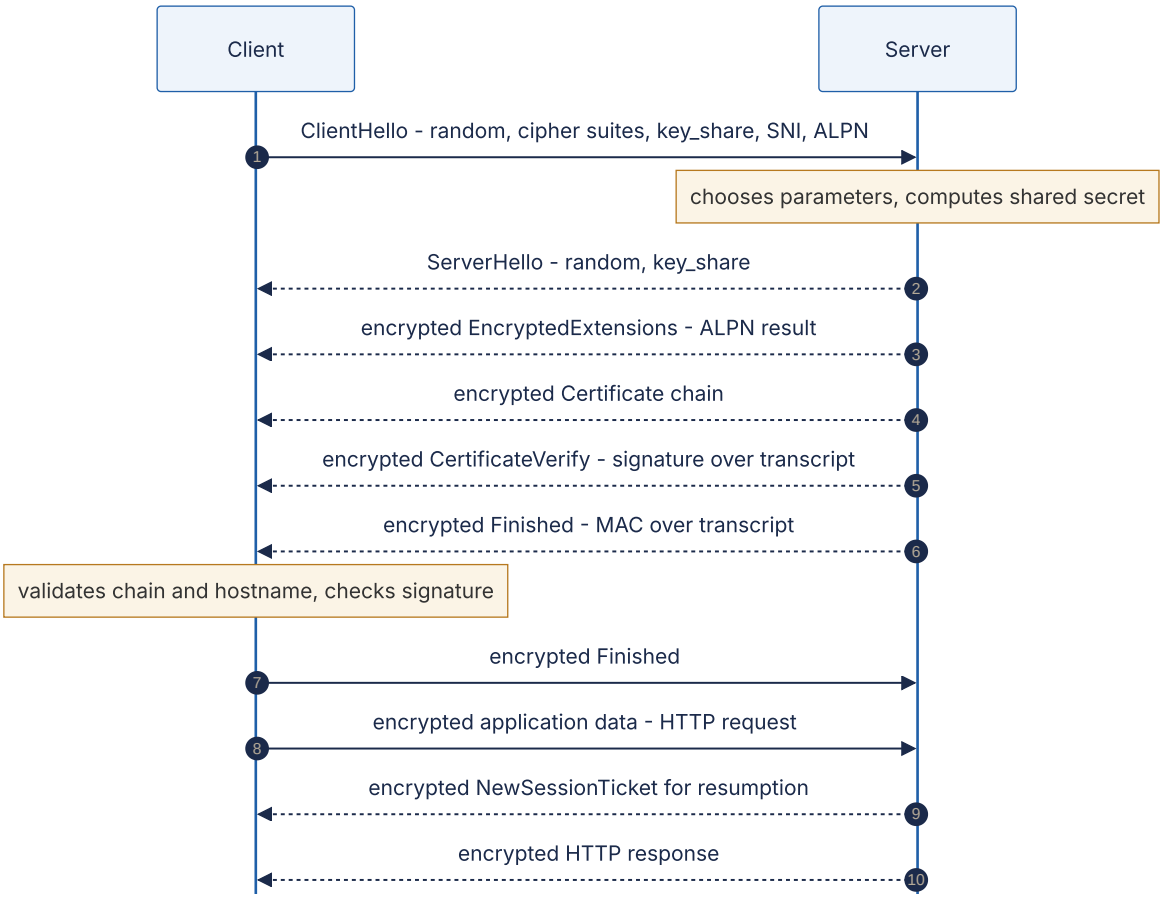
Analogy: before discussing secrets with a stranger on the phone, you check their government-issued ID (the certificate, vouched for by a trusted issuer), then the two of you agree on a code that only you two know, without ever saying it aloud on the line (a **key exchange**), and from then on you speak only in that code and seal every sentence so changes would be noticed (authenticated encryption).

The cryptography itself (Diffie–Hellman, signatures, AEAD ciphers) is covered in §10.4. Here we focus on the round trips and their operational consequences.

7.10.2 The TLS 1.3 handshake

TLS 1.2 needed **two** round trips before application data could flow. TLS 1.3 needs **one**, because the client guesses the key-exchange group the server will accept and sends its key share in the very first message.

Figure 7.8 — TLS 1.3 full handshake (after the TCP handshake).



Reading the diagram. Step 1, the **ClientHello**, carries a random nonce, the cipher suites the client supports, its ephemeral Diffie–Hellman **key share**, the **Server Name Indication (SNI)** naming the host it wants (so one IP address can serve many sites), and the **ALPN** list of application protocols it speaks (`h2`, `http/1.1`). Steps 2–6 are the server's single flight. From the ServerHello onward, both sides can derive handshake keys, so everything after it, including the certificate, is encrypted (in TLS 1.2 the certificate travelled in clear text). CertificateVerify proves the server holds the private key for the certificate; Finished proves nobody tampered with the handshake. In step 7 the client sends its Finished and, in step 8, its first request, all after **one RTT**. Steps 9–10 arrive later: a session ticket for faster resumption, and the response. Everything is synchronous request/response over the already-established TCP connection. Failure points: certificate validation (expired, wrong name, untrusted issuer) aborts the handshake; mismatched protocol versions or cipher suites fail at step 2. Scaling consideration: the server's signature in step 5 is the main CPU cost of a handshake, which is a reason to reuse connections (§7.16) and resume sessions.

7.10.3 Certificates and the chain of trust

A **certificate** binds a public key to one or more names (in the Subject Alternative Name field), with a validity period, signed by a **certificate authority (CA)**. Browsers and operating systems ship a **root store** of trusted root CAs. Servers send their own (**leaf**) certifi-

cate plus **intermediate** certificates; the client builds a chain from leaf to a trusted root and checks:

1. each signature in the chain verifies with the issuer's public key;
2. the current time is within every certificate's validity period;
3. the requested hostname matches a name in the leaf;
4. the certificate has not been revoked (via OCSP or CRLs, imperfectly in practice); and
5. publicly trusted certificates appear in Certificate Transparency logs, as major browsers require.

Operationally, the most common TLS outage is mundane: **an expired certificate**. Publicly trusted certificates have a maximum lifetime of 398 days as of 2025, and the CA/Browser Forum has adopted a schedule to shorten it further over the coming years; ACME-based automation (as popularized by Let's Encrypt) is the only sane response. Monitor expiry dates of every certificate, including internal ones.

Mutual TLS (mTLS), in which the client also presents a certificate, is the standard way to authenticate services to each other inside a zero-trust network (§10.10).

Table 7.11 — TLS 1.2 versus TLS 1.3.

Aspect	TLS 1.2 (RFC 5246)	TLS 1.3 (RFC 8446)
Full handshake	2 RTT	1 RTT
Resumption	1 RTT	1 RTT, or 0-RTT with early data
Key exchange	RSA key transport or (EC)DHE	(EC)DHE only, so forward secrecy is mandatory
Cipher suites	Many, including weak legacy options	Five AEAD suites
Certificate visibility	Sent in clear text	Encrypted
Removed features	—	RSA key transport, static DH, CBC-mode ciphers, compression, renegotiation

Forward secrecy means that stealing a server's long-term private key later does not let an attacker decrypt previously recorded sessions, because each session's keys came from ephemeral Diffie–Hellman values that were discarded.

7.10.4 0-RTT and its caveats

With a session ticket from a previous connection, a TLS 1.3 client can send **early data** in its first flight, encrypted under a key derived from the resumption pre-shared key.

Combined with QUIC, a returning client can deliver a request with zero round trips of setup. Two caveats make 0-RTT dangerous if used carelessly [RFC 8446 §8]:

1. **Replay.** An attacker who records the first flight can resend it, and the server may process the early data twice. TLS cannot fully prevent this across a distributed server fleet.
2. **Weaker forward secrecy for early data**, which is protected only by the resumption key, not by fresh ephemeral key exchange.

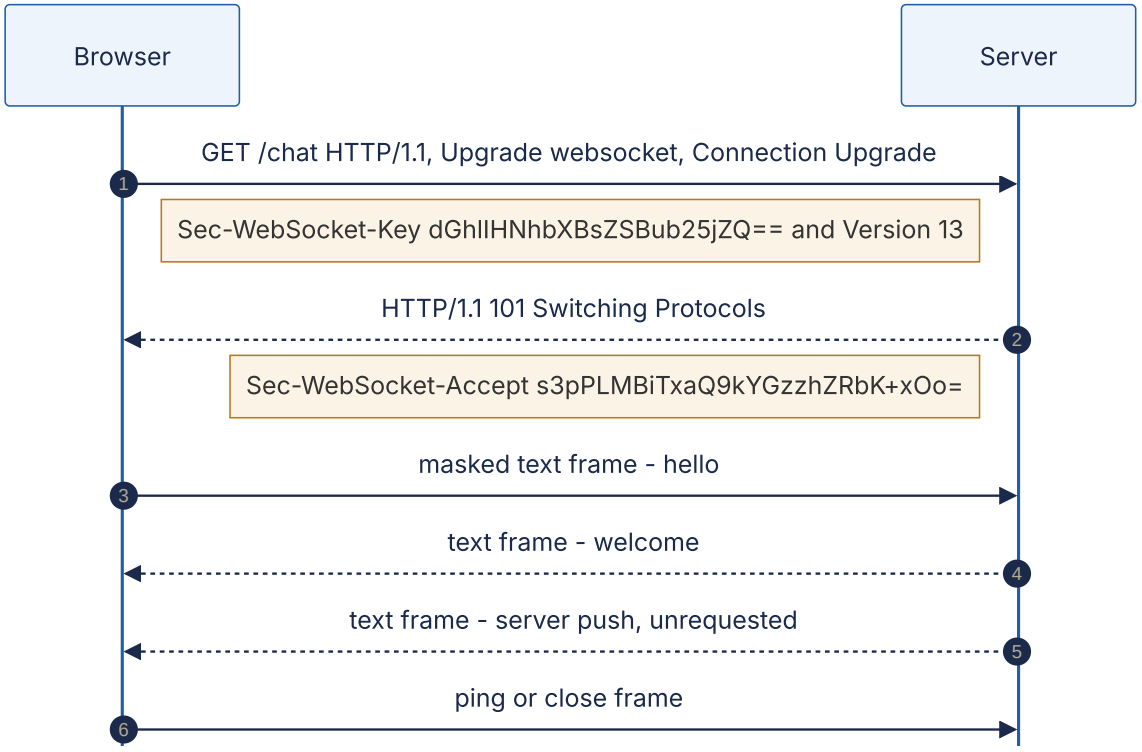
The rule: accept 0-RTT only for **idempotent, safe requests** (GET without side effects). Servers and CDNs that support it typically restrict it to such methods, and HTTP defines the `Early-Data` header and the 425 (Too Early) status code (RFC 8470) so origins can reject replay-unsafe early requests. Never let a payment or any non-idempotent POST ride in 0-RTT (see idempotency keys, §22.15).

> COMMON MISTAKE
>
> **"TLS is too slow to use internally."** With TLS 1.3, a handshake costs one RTT (sub-millisecond inside a datacenter) and some CPU, and modern CPUs encrypt with AES-GCM at several GB/s per core using hardware instructions. With connection reuse, the steady-state cost is small. The exceptions are extreme-throughput paths, where kernel TLS and NIC offload exist for exactly this reason.

7.11 WebSockets (introduction)

HTTP is client-initiated: the server cannot speak until asked. Chat, live dashboards, collaborative editors and multiplayer games need the server to push data whenever it has some. **WebSocket** [RFC 6455] provides a persistent, full-duplex, message-oriented channel between browser and server, created by **upgrading** an HTTP/1.1 connection.

Figure 7.9 — WebSocket opening handshake and framing.



Reading the diagram. Steps 1–2 are an ordinary HTTP/1.1 exchange over an existing TCP (usually TLS) connection. The client sends a random base64 key; the server proves it understood the WebSocket protocol by returning `Sec-WebSocket-Accept`, the base64-encoded SHA-1 hash of the key concatenated with the fixed GUID `258EAFA5-E914-47DA-95CA-C5AB0DC85B11` (the values shown are the example from RFC 6455). After the `101` response the same TCP connection carries WebSocket **frames** in both directions, at any time (steps 3–6): either side may send without being asked, so the arrows are asynchronous in the application sense. Frames from client to server are **masked** with a random 4-byte key to stop malicious pages from crafting bytes that confuse caching proxies. Control frames (ping, pong, close) keep the connection alive and close it cleanly. Failure points: intermediaries with idle timeouts (load balancers commonly default to around 60 s) silently kill quiet connections, so applications send periodic pings; and every connection pins state to one server, which complicates deploys and load balancing.

The full design space (long polling, Server-Sent Events, WebSockets at scale, sticky routing, presence) is Chapter 9. WebSockets can also run over HTTP/2 (RFC 8441) and HTTP/3 (RFC 9220) using an extended CONNECT method.

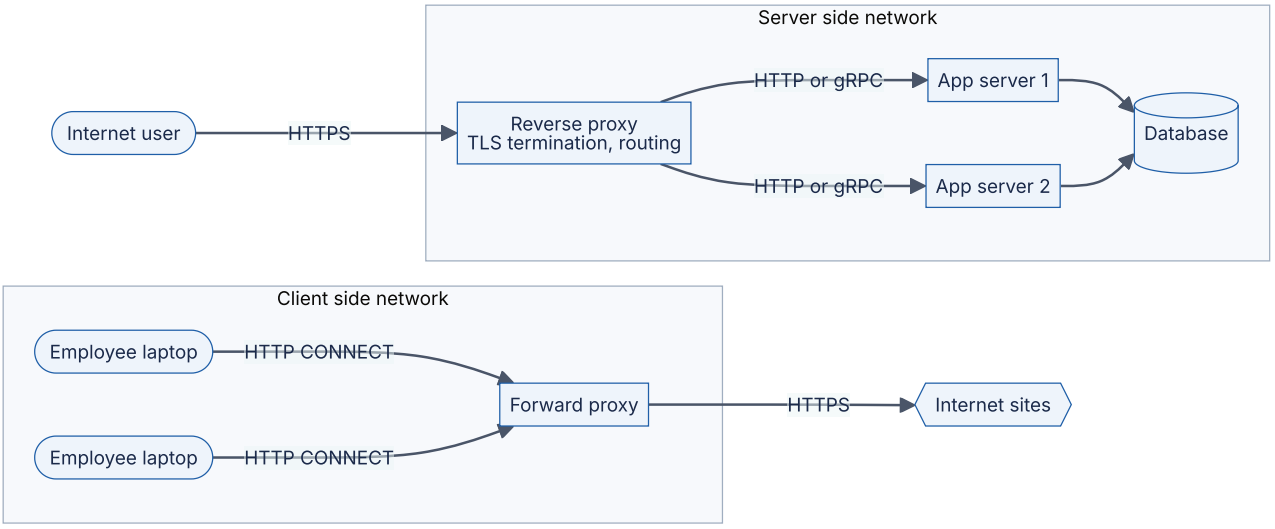
7.12 Forward proxies and reverse proxies

DEFINITION

Definition — Proxy. An intermediary that terminates a client's connection and opens its own connection onward, relaying requests and responses. A **forward proxy** acts on behalf of **clients** and is chosen by them (or their network administrator). A **reverse proxy** acts on behalf of **servers** and is chosen by the site operator; clients usually do not know it exists.

Analogy: a forward proxy is your personal assistant who places all your calls, so callees see the assistant's number. A reverse proxy is a company switchboard: callers dial one number and the receptionist routes them to the right employee, who never has to expose a direct line.

Figure 7.10 — Forward proxy versus reverse proxy.



Reading the diagram. On the left, client machines inside an organization send all outbound traffic through a forward proxy, using the HTTP `CONNECT` method to tunnel TLS connections; destination sites see the proxy's address. On the right, Internet users connect to a reverse proxy, which terminates TLS, inspects HTTP and forwards each request to one of several application servers over internal HTTP or gRPC; users see only the proxy. Both paths are synchronous request/response. The database behind the app servers is the storage boundary. Failure points: each proxy is a single point of failure unless replicated behind anycast, ECMP or DNS (§19.8). Scaling: reverse proxies scale horizontally and absorb connection-handling work (TLS, slow clients) so app servers do not have to.

Table 7.12 — Forward and reverse proxies compared.

Aspect	Forward proxy	Reverse proxy
Acts for	Clients	Servers
Configured by	Client or client's network	Service operator
Hides	Client identity from servers	Server topology from clients
Typical functions	Egress filtering, caching, auditing, access control, anonymity	TLS termination, load balancing, routing by path or host, caching, compression, rate limiting, WAF, request buffering
Examples	Squid, corporate secure web gateways, egress proxies in VPCs	NGINX, HAProxy, Envoy, cloud load balancers, CDNs, API gateways
System-design relevance	Controlled egress from private networks	Nearly every web architecture's front door

A reverse proxy that distributes load is a **load balancer** (Chapter 19); one that adds authentication, quotas and API management is an **API gateway** (§22); one deployed beside every service instance is a **sidecar** in a **service mesh**. Proxies that terminate TCP also shield back-ends from slow clients: the proxy buffers a slowly uploaded request and hands the application a complete one, so the application's limited workers are not tied up for seconds.

WARNING / TRADE-OFF

The client IP problem. Behind a reverse proxy, the application sees the proxy's IP as the source. Proxies pass the original address in the `X-Forwarded-For` or standard `Forwarded` header (RFC 7239), or at L4 via the PROXY protocol. Trust these headers only when they were set by your own proxy; otherwise clients can forge them to evade rate limits or IP allow-lists.

7.13 NAT

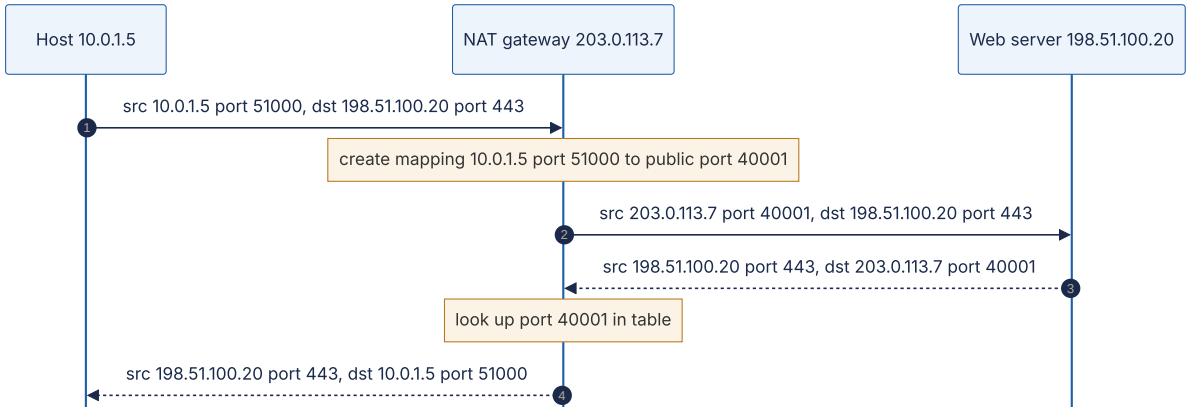
DEFINITION

Definition — Network Address Translation (NAT). A function, usually in a router or gateway, that rewrites the source and/or destination IP addresses (and usually ports) of packets as they pass, keeping a **translation table** so that replies can be rewritten back. The most common form, **source NAT with port translation** (NAPT, often just "NAT"), lets many hosts with private addresses share one public address [RFC 3022].

Analogy: an office building with one street address and a mailroom. Outgoing letters carry the building's return address with an internal "room code" written on them; when

replies come back with that code, the mailroom forwards them to the right room. A letter that arrives with no room code the mailroom has issued has nowhere to go.

Figure 7.11 — Source NAT for an outbound connection.



Reading the diagram. A private host (left) opens a connection to a public server (right) through a NAT gateway (middle). In step 1 the packet carries the host's private address. The NAT rewrites the source to its own public address and a free public port, recording the mapping (steps 2–3). The server replies to the public address (step 4), and the NAT reverses the translation using the table (steps 5–6). All traffic is synchronous and flows through the same NAT in both directions: the NAT is **stateful**, which makes it a failure point (if it restarts or the mapping times out, existing connections die) and a scaling limit (ports per destination).

Consequences for system design

- **Inbound connections are impossible by default**, because no mapping exists until the inside host sends first. Peer-to-peer systems (WebRTC, §9.7) use STUN to discover their public mapping and fall back to TURN relays when NAT behavior prevents direct connections.
- **Mappings time out.** Idle UDP mappings may expire in tens of seconds and TCP mappings in minutes to hours, depending on the device. Long-lived idle connections need keep-alives shorter than the NAT timeout, or they become half-open (§7.7).
- **Port exhaustion.** A single public IP has about 64,000 usable ports per destination (IP, port, protocol). AWS documents a NAT gateway limit of 55,000 simultaneous connections per assigned IP address to each unique destination, raisable by assigning more IPs (as of 2025) [AWS Docs: NAT gateways]. A fleet of services calling one external API through one NAT IP can exhaust this; connection pooling and more NAT IPs fix it.
- **Carrier-grade NAT (CGNAT)**: mobile and some fixed ISPs place many subscribers behind shared addresses from `100.64.0.0/10`. Thousands of users can share one public IP, so **IP-based rate limiting and blocking punish innocent users**.

- **Cost.** Managed cloud NAT gateways usually charge per gigabyte processed; routing bulk traffic to cloud storage through them instead of private endpoints is a common surprise on the bill.

7.14 Firewalls and security groups

DEFINITION

Definition — Firewall. A device or software function that permits or denies traffic according to rules. A **stateless** (packet-filter) firewall evaluates each packet independently against rules on addresses, ports and protocol. A **stateful** firewall tracks connections, so a reply to an allowed outbound connection is automatically permitted. **Application-layer firewalls**, including web application firewalls (WAFs), parse protocols such as HTTP to block malicious requests.

Cloud providers expose two common forms. A **security group** (AWS, similar constructs elsewhere) is a stateful, allow-only rule set attached to a network interface or instance. A **network ACL** is a stateless, ordered allow-and-deny rule list attached to a subnet. Because NACLs are stateless, you must explicitly allow return traffic, typically to the client's ephemeral port range, which is a frequent source of "connection hangs" misconfigurations.

Table 7.13 — Firewall mechanisms compared.

Aspect	Stateless ACL	Stateful firewall / security group	Web application firewall
Inspects	Packet headers	Headers plus connection state	HTTP requests (L7)
Return traffic	Must be explicitly allowed	Allowed automatically	N/A (proxy)
Rule style	Ordered allow/deny	Usually allow-only	Signatures, rate rules, custom logic
Cost of state	None	Connection table (can be exhausted)	Full proxying
Typical placement	Subnet boundary	Each instance or interface	In front of web apps, often at the CDN
Choose when	Coarse subnet guardrails	Default micro-segmentation between tiers	Blocking injection, bots, abusive patterns

Practical rules

1. **Default deny.** Allow only the ports and sources each tier needs: the load balancer may reach the app tier on port 8080; only the app tier may reach the database on

5432.

2. **Reference groups, not addresses.** Security groups can name other groups as sources (“allow from sg-app”), which keeps rules correct as instances scale in and out.
3. **Firewalls are one layer of defense in depth.** Combine them with authentication and encryption (mTLS, §10.10); a network position is not an identity.
4. **Watch the state table.** Stateful devices track every connection; a flood of connections (legitimate or attack) can exhaust their tables and drop everything. Linux’s `conntrack` table is a common culprit on busy proxies.
5. **Silent drops cause timeouts, rejections cause fast failure.** A firewall that drops packets makes clients wait for a timeout; one that sends RST or ICMP unreachable fails fast. Choose deliberately.

7.15 Latency, bandwidth, the bandwidth-delay product and packet loss

7.15.1 The four components of latency

Think of a highway. **Bandwidth** is the number of lanes: how many cars per second can pass a point. **Latency** is how long one car takes to travel from entrance to exit. Adding lanes does not make the drive shorter. A packet’s one-way delay through each hop is the sum of four parts.

> DEFINITION
>
> **Definition — Components of network delay. Propagation delay**: time for a signal to travel the physical distance. **Transmission (serialization) delay**: time to push all of a packet’s bits onto the link. **Queuing delay**: time spent waiting in buffers behind other packets. **Processing delay**: time for a device to examine headers, look up routes and copy the packet.

FORMULA

One-hop delay

$$d_{\text{hop}} = \underbrace{\frac{D}{v}}_{\text{propagation}} + \underbrace{\frac{L}{R}}_{\text{transmission}} + d_{\text{queue}} + d_{\text{proc}}$$

Symbol	Meaning	Units
D	Physical length of the link	m (or km)
v	Signal speed in the medium: about 2×10^8 m/s in optical fiber (roughly two-thirds of c)	m/s
L	Packet size	bits
R	Link rate (bandwidth)	bits/s
d_{queue}	Queuing delay: bytes waiting ahead ÷ link rate	s
d_{proc}	Processing delay: microseconds in software, sub-microsecond in switch hardware	s

Table 7.14 — Delay components at a glance.

Component	Depends on	Typical magnitude	How to reduce it
Propagation	Distance, medium	~5 µs per km of fiber; tens of ms across oceans	Move closer: CDNs, edge, regional replicas
Transmission	Packet size, link rate	12 µs for 1,500 B at 1 Gbps; 1.2 µs at 10 Gbps	Faster links, smaller packets
Queuing	Load, buffer size	0 to hundreds of ms	Lower utilization, AQM, BBR
Processing	Device and software path	~µs (hardware) to tens of µs (software)	Kernel bypass, offloads, fewer hops

WORKED EXAMPLE

Worked example 1 — The speed-of-light floor from New York to London.

The great-circle distance between New York and London is about 5,570 km.

1. One-way propagation in fiber: $5,570 \text{ km} / 200,000 \text{ km/s} = 0.02785 \text{ s} = 27.85 \text{ ms}$.
2. Round-trip time: $2 \times 27.85 = 55.7 \text{ ms}$.
3. Even in a vacuum at $c \approx 299,792$ km/s, the RTT would be $2 \times 5,570 / 299,792 \approx 37.2 \text{ ms}$.
4. Real cables do not follow the great circle, and routers, transmission and queuing add more, so measured RTTs are noticeably higher than 55.7 ms (often in the 70 ms range).

Interpretation: no engineering can make a New York–London round trip faster than about 56 ms over fiber. A user in London calling a single-region service in New York pays at least that on every round trip, and a page needing five sequential round trips pays at least 280 ms. Engineering implication: for global users, reduce round trips (connection reuse, TLS 1.3, QUIC) and move content and computation closer (CDNs, §21; multi-region, §44).

WORKED EXAMPLE

Worked example 2 — Transmission versus propagation.

Send one 1,500-byte packet ($L = 12,000$ bits).

- On a 10 Mbps link: $12,000 / 10^7 = 1.2$ ms.
- On a 1 Gbps link: $12,000 / 10^9 = 12$ µs.
- On a 100 Gbps link: $12,000 / 10^{11} = 120$ ns.

Compare with propagation across a 100 m datacenter cable: $100 / (2 \times 10^8) = 0.5$ µs. Interpretation: on slow access links, serialization dominates; on fast datacenter links, serialization becomes negligible and per-hop processing and software stacks dominate.

WORKED EXAMPLE

Worked example 3 — Bufferbloat.

A home router has a 10 MB buffer in front of a 1 Gbps uplink. When a large upload fills that buffer, a newly arriving packet waits behind $10 \times 10^6 \times 8 = 8 \times 10^7$ bits, so its queuing delay is $8 \times 10^7 / 10^9 = 0.08$ s = 80 ms. On a 20 Mbps uplink the same buffer would add $8 \times 10^7 / (2 \times 10^7) = 4$ s. Loss-based congestion control (CUBIC) keeps such buffers full, so a video call alongside a big upload suffers seconds of lag. Fixes: active queue management (CoDel, FQ-CoDel) on the bottleneck, and delay-aware congestion control such as BBR.

7.15.2 Throughput and the bandwidth-delay product

A reliable sender can only have a window's worth of unacknowledged data in flight; it then waits for ACKs, which take one RTT. So:

FORMULA

Window-limited throughput and the bandwidth-delay product (BDP)

$$\text{Throughput} \le \frac{W}{\text{RTT}} \qquad \text{BDP} = R \times \text{RTT}$$

Symbol	Meaning	Units
W	Effective window, $\min(\text{cwnd}, \text{rwnd})$	bytes
RTT	Round-trip time	s
R	Bottleneck link rate	bits/s (divide by 8 for bytes/s)
BDP	Data that must be in flight to keep the link full	bits or bytes

WORKED EXAMPLE

Worked example — BDP for a 10 Gbps, 80 ms path.

1. BDP $= 10 \times 10^9$ bits/s $\times\ 0.080$ s $= 8 \times 10^8$ bits.
2. In bytes: $8 \times 10^8 / 8 = 10^8$ bytes $=$ **100 MB**.
3. Without window scaling, W is at most 65,535 bytes, giving throughput $\le 65,535 \times 8 / 0.08 \approx 6.55$ Mbps, about 0.07% of the link.
4. To fill the link, the receiver must advertise (and the sender buffer) about 100 MB per connection: window scaling must be on and kernel buffer limits (`net.ipv4.tcp_rmem` and `tcp_wmem` maximums on Linux) must be at least that large.

Interpretation: on “long fat networks”, a single TCP connection is limited by buffers and window, not by the link. Engineering implication: bulk cross-region transfers (backups, replication, dataset copies) need tuned buffers, parallel streams, or both.

7.15.3 Packet loss and the Mathis formula

Loss hurts TCP twice: each lost packet must be retransmitted, and each loss signal halves (Reno) or cuts (CUBIC) the congestion window. For loss-based congestion avoidance with random loss probability p, Mathis, Semke, Mahdavi and Ott derived a simple upper bound on steady-state throughput [Mathis et al. 1997].

FORMULA

Mathis throughput formula

$$\text{Throughput} \le \frac{\text{MSS}}{\text{RTT}} \times \frac{C}{\sqrt{p}}$$

Symbol	Meaning	Units
MSS	Maximum segment size	bits (or bytes, for bytes/s)
RTT	Round-trip time	s
p	Packet loss probability	dimensionless
C	Constant, $\sqrt{3/2} \approx 1.22$ for periodic loss with every packet ACKed	dimensionless

WORKED EXAMPLE

Worked example — Loss caps a long-distance connection.

MSS = 1,460 bytes = 11,680 bits; RTT = 80 ms.

1. MSS/RTT $= 11,680/0.08 = 146,000$ bits/s.
2. With $p = 10^{-4}$ (0.01% loss): $C/\sqrt{p} = 1.22/0.01 = 122$, so throughput $\leq 146,000 \times 122 \approx 17.8$ Mbps.
3. With $p = 10^{-6}$: $C/\sqrt{p} = 1,220$, so throughput ≤ 178 Mbps.
4. To reach 10 Gbps: $\sqrt{p} = 1.22 \times 146,000/10^{10} \approx 1.79 \times 10^{-5}$, so $p \approx 3.2 \times 10^{-10}$, about one loss in three billion packets.

Interpretation: throughput falls with $1/\text{RTT}$ and $1/\sqrt{p}$, independent of link speed. A tiny 0.01% loss caps a single transcontinental CUBIC- or Reno-style flow at under 20 Mbps on a 10 Gbps link. Engineering implications: (a) find and fix lossy links, which are often a dirty optic or a bad cable; (b) reduce RTT; (c) for bulk transfers, use parallel connections or a model-based algorithm such as BBR that does not treat random loss as congestion. The formula is an approximation for loss-based algorithms in congestion avoidance; it does not describe BBR, short flows still in slow start, or timeout-dominated regimes.

7.16 Connection pooling and keep-alive

7.16.1 The cost of a new connection

Before the first byte of a response can arrive over a new HTTPS connection, the client pays for DNS, the TCP handshake, the TLS handshake, and the request itself.

Table 7.15 — Round trips before the first response byte (DNS cached).

Stack	Setup RTTs	Request RTT	Total	At RTT 80 ms	At RTT 0.5 ms (same datacenter)
HTTP over TCP, no TLS	1	1	2	160 ms	1.0 ms
HTTPS, TCP + TLS 1.2	1 + 2	1	4	320 ms	2.0 ms
HTTPS, TCP + TLS 1.3	1 + 1	1	3	240 ms	1.5 ms
HTTP/3, QUIC first connection	1	1	2	160 ms	1.0 ms
HTTP/3, QUIC 0-RTT resumption	0	1	1	80 ms	0.5 ms
Any stack, reused connection	0	1	1	80 ms	0.5 ms

Round trips are only part of the cost. Each new connection also burns CPU (the TLS asymmetric cryptography is the expensive part), allocates kernel and proxy memory, starts in slow start with a small congestion window (§7.7.7), and leaves a TIME_WAIT entry when closed (§7.7.4).

WORKED EXAMPLE

Worked example — Reusing connections to a cross-region dependency.

Service A in Virginia calls service B in Oregon (RTT about 70 ms) 200 times per second. Each call takes 10 ms of server time.

1. New TCP + TLS 1.3 connection per call: latency $\approx 3 \times 70 + 10 = 220$ ms per call.
2. Reused connection: $1 \times 70 + 10 = 80$ ms per call, a 2.75× reduction.
3. Handshake work avoided: 200 TLS handshakes per second, each involving a signature on B and verification on A.
4. Ports: with per-call connections and A as active closer, $200 \times 60 = 12,000$ sockets sit in TIME_WAIT at any time, about 43% of the 28,232 default ephemeral ports.

Interpretation: connection reuse is not a micro-optimization; it changes latency by a factor of nearly three and removes a scaling cliff.

7.16.2 Sizing a pool with Little's Law

A **connection pool** keeps a set of open connections that callers borrow and return. For protocols that carry one request at a time per connection (HTTP/1.1, most database protocols), the number of connections in use equals the number of requests in flight, which Little's Law gives directly (§4.11).

FORMULA

Connections needed for a one-request-per-connection protocol

$$N = \lambda \times W$$

Symbol	Meaning	Units
N	Average connections busy	connections
λ	Request rate	requests/s
W	Time a request holds a connection (network RTT plus server time)	s

WORKED EXAMPLE

Worked example — Database pool sizing.

An application server issues 2,000 queries per second; each holds a connection for 5 ms on average (0.5 ms RTT plus 4.5 ms of query execution).

1. $N = 2,000 \times 0.005 = 10$ busy connections on average.
2. Add headroom for bursts and tail latency: if p99 hold time is 25 ms, bursts can need $2,000 \times 0.025 = 50$. A pool of 20–50 is reasonable.
3. With 40 application servers each holding a pool of 50, the database must accept 2,000 connections. Many databases degrade with thousands of connections (PostgreSQL uses a process per connection), which is why connection proxies such as PgBouncer sit in between (Chapter 14).

Interpretation: size pools from rate × hold time, not by guessing; and remember pools multiply across instances.

7.16.3 Keep-alive pitfalls

- **Idle-timeout races.** If the server (or a load balancer) closes idle connections after 60 s and the client's pool keeps them for 90 s, the client will occasionally send a request on a connection the server has just closed, getting a reset. Set client idle

timeouts **shorter** than every server-side timeout along the path, and retry idempotent requests on this specific failure.

- **Stale connections after failover.** Long-lived connections keep talking to an old backend after DNS changes, so set a maximum connection lifetime.
- **Load imbalance with multiplexed protocols.** An HTTP/2 or gRPC client may send all its traffic over one long-lived connection to one backend; an L4 load balancer balances connections, not requests, so new backends receive nothing. Use L7 (request-level) load balancing or client-side balancing, and cap connection age so connections periodically redistribute (§19.10).
- **TCP keep-alive is not HTTP keep-alive.** HTTP keep-alive means "reuse this connection for more requests". TCP keep-alive sends empty probes on an idle connection to detect dead peers and refresh NAT and firewall state.

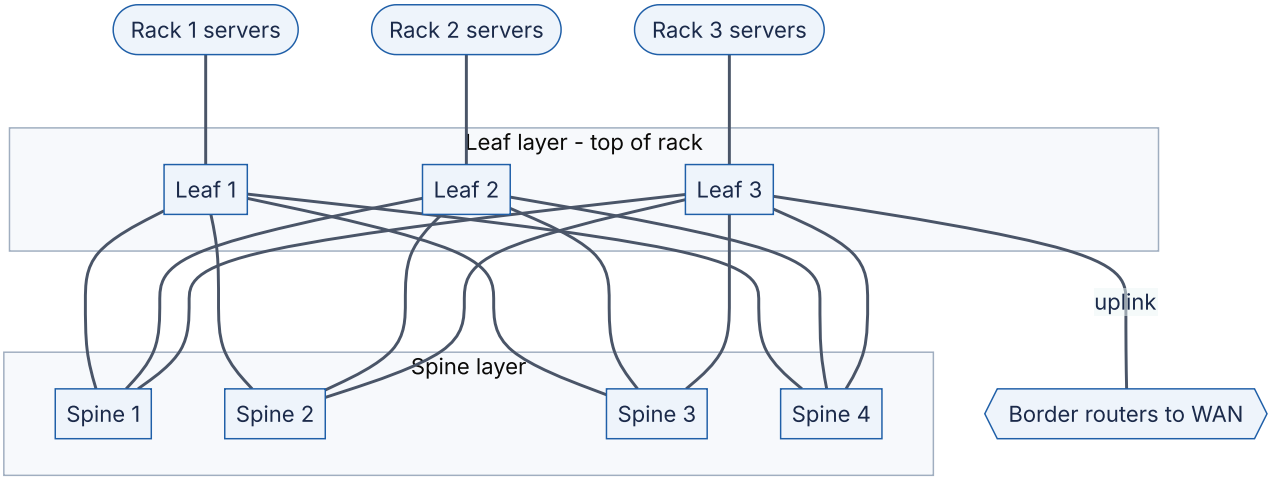
7.17 Datacenter networking

7.17.1 From trees to leaf–spine

Traditional enterprise networks were **trees**: servers connect to access switches, which connect to aggregation switches, which connect to a pair of large core routers. Trees suit **north–south** traffic (between users outside and servers inside). Modern applications are different: a single user request fans out to dozens of internal services, caches and databases, and batch jobs shuffle terabytes between servers. Most bytes now flow **east–west**, server to server inside the datacenter. In a tree, east–west traffic converges on the few links near the top, which become the bottleneck, and the expensive core routers become both a cost and a failure concentration.

> DEFINITION
>
> **Definition — Leaf-spine (folded Clos) network.** A two-tier topology in which every **leaf** (top-of-rack) switch connects to every **spine** switch, and servers attach only to leaves. Any two servers on different leaves are exactly two switch hops apart (leaf → spine → leaf), and traffic is spread across all spines with ECMP. It is a form of the multistage **Clos network**, first described by Charles Clos in 1953 for telephone switching. Larger fabrics add a third tier.

Figure 7.12 — A leaf-spine fabric with ECMP.



Reading the diagram. Servers in each rack connect to their leaf switch. Every leaf has one uplink to every spine (a full bipartite mesh), so a flow from rack 1 to rack 3 has four equal-cost two-hop paths, and leaves pick among them by hashing each flow's 5-tuple (ECMP). Spines never connect to each other or to servers. One leaf (or a dedicated pair) connects to border routers for north–south traffic. Links are undirected because traffic flows both ways; all forwarding is synchronous hardware switching. Failure points: losing a spine removes one quarter of cross-rack capacity rather than isolating anything; losing a leaf isolates its rack, which is why critical services spread replicas across racks. Scaling: add spines for more bandwidth, add leaves (up to the spine port count) for more racks.

FORMULA

Oversubscription ratio at a leaf

$$\text{Oversubscription} = \frac{\text{total server-facing bandwidth}}{\text{total uplink bandwidth}}$$

Symbol	Meaning	Units
Server-facing bandwidth	Sum of the leaf's downlink port speeds	Gbps
Uplink bandwidth	Sum of the leaf's spine-facing port speeds	Gbps

WORKED EXAMPLE

Worked example — Sizing a leaf-spine fabric.

Each leaf has 48 × 25 Gbps server ports and 6 × 100 Gbps uplinks; each spine has 64 × 100 Gbps ports.

1. Downlink: $48 \times 25 = 1,200$ Gbps. Uplink: $6 \times 100 = 600$ Gbps.
2. Oversubscription: $1,200/600 = 2{:}1$. If every server sends cross-rack at full rate, each gets at most 12.5 Gbps.
3. Each leaf uses one port on each of 6 spines, so there are 6 spines. Each spine has 64 ports, so at most 64 leaves.
4. Maximum servers: $64 \times 48 = 3,072$.
5. Total leaf-to-spine capacity: $64 \text{ leaves} \times 600 \text{ Gbps} = 38.4$ Tbps. The **bisection bandwidth** (capacity between any half of the leaves and the other half) is $32 \times 600 = 19.2$ Tbps.

Interpretation: oversubscription is a cost lever. 1:1 (“non-blocking”) fabrics cost more uplinks and spines; 3:1 is acceptable for workloads that are mostly rack-local. Larger fabrics add a third tier of switches to go beyond the port count of one spine.

INDUSTRY EXAMPLE

Google Jupiter. Google’s SIGCOMM 2015 paper describes a decade of five datacenter network generations, culminating in Jupiter. Instead of buying ever-larger proprietary routers, Google built multistage Clos topologies from many small switches using commodity merchant silicon, managed them with centralized, software-defined control rather than traditional distributed routing protocols, and scaled capacity about 100× over ten years to more than 1 Pbps of bisection bandwidth. The paper’s argument (that bandwidth demand from east–west traffic grows faster than any single box can, so the network must scale out like the servers) is now standard practice across large datacenter operators. *Evidence: Primary* — [Singh et al. 2015]

7.17.2 Datacenter-specific problems

- **Incast.** When a client fans a request out to 100 servers that all reply at the same instant, the replies converge on one leaf port, overflow its shallow buffer, and drop packets. The dropped flows wait for a 200 ms minimum RTO in a network whose RTT is about 100 µs, producing extreme tail latency (§23.6). Mitigations: limit fan-out concurrency, jittered responses, smaller minimum RTOs, and ECN-based congestion control such as DCTCP.
- **ECMP hash collisions.** Two elephant flows hashed onto the same spine link share it while other links sit idle. Flowlet switching, packet spraying and adaptive routing

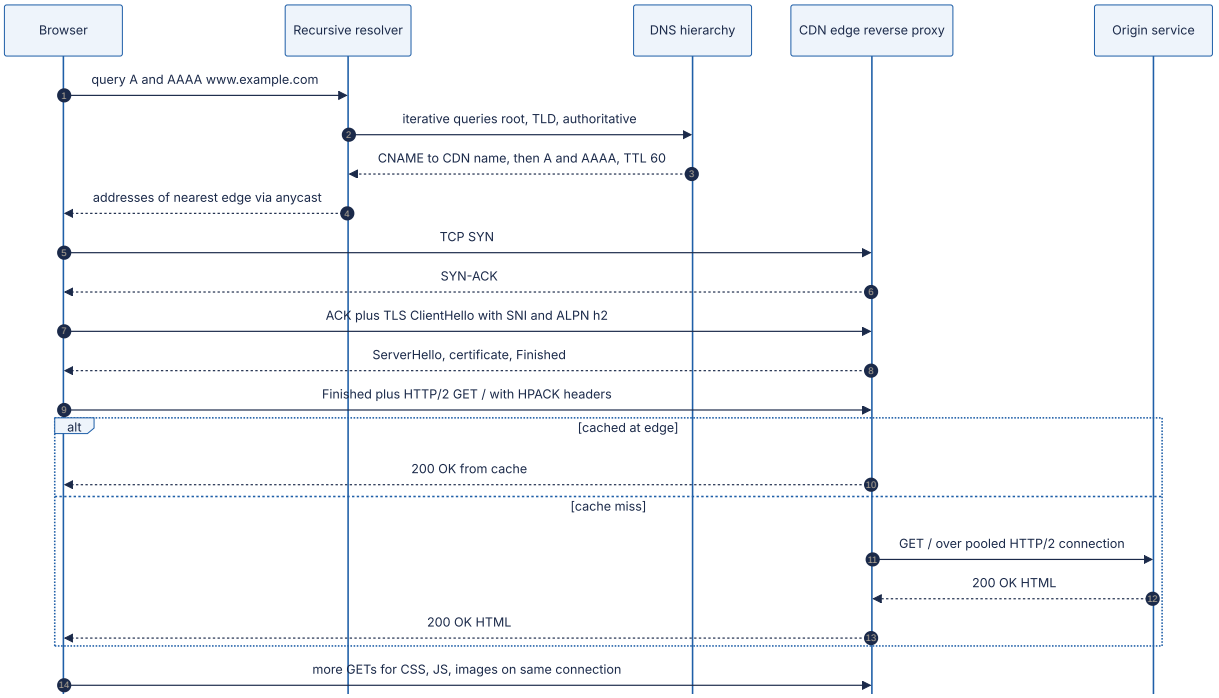
reduce this.

- **Gray failures.** A link that drops 1% of packets is worse than a dead one, because routing still uses it while TCP throughput collapses (recall the Mathis formula). Operators monitor per-link error counters and drain suspect links automatically.

7.18 What happens when you type a URL

Everything in this chapter appears in the life of one request. Suppose you type `https://www.example.com/` into a browser on a laptop connected by Wi-Fi at home, with a cold DNS cache, and the site is served by a CDN edge 20 ms away (RTT) with HTTP/2.

Figure 7.13 — Loading https://www.example.com/ end to end.



Reading the diagram. Steps 1–4: the browser (via the operating system stub) asks a recursive resolver, which walks the DNS hierarchy (§7.6); the authoritative server returns a CNAME pointing into the CDN, whose own authoritative servers answer with an anycast address (§7.5.3). Steps 5–7: the TCP handshake (§7.7.3), with the TLS ClientHello piggybacked on the final ACK. Step 8: the server's TLS 1.3 flight (§7.10.2), including the certificate the browser validates. Step 9: the client's Finished and the first HTTP/2 request, with ALPN having selected `h2` (§7.9.3). Steps 10–13: the edge serves from cache or fetches from the origin over a pre-warmed, pooled connection (§7.16), so the user does not pay the origin's handshakes. Step 14: subresources reuse the same multiplexed connection. All steps are synchronous; the edge cache and origin are the storage boundaries. Failure points include DNS (every step depends on it), certificate validity, and the origin path on cache misses.

7.18.1 Steps the diagram leaves out

Figure 7.13 shows the protocol messages. Several steps happen around them:

1. **Before any network traffic**, the browser parses the URL into scheme (`https`, so port 443), host and path, applies its HSTS list (which may force HTTPS), and checks its HTTP cache and any existing connection to the origin. A reused connection skips steps 1–9 of the figure entirely.
2. **Choosing an address (§7.4).** With both AAAA and A answers, Happy Eyeballs tries IPv6 first and races IPv4 if needed.
3. **The first hop (§7.3).** The destination is off-subnet, so the laptop frames the packet for its default gateway, learning the gateway's MAC address with ARP or Neighbor Discovery.
4. **NAT (§7.13).** For IPv4, the home router rewrites the private source address (for example `192.168.1.23`) to its public address and records the mapping; the ISP may add carrier-grade NAT.
5. **Routing (§7.5).** ISP routers forward by longest-prefix match, and BGP delivers the anycast destination to a nearby edge site.
6. **Edge admission (§7.14, Chapter 19).** DDoS filtering and an L4 load balancer (ECMP plus consistent hashing) pick the server running the reverse proxy.
7. **Behind the edge (Chapters 8 and 21).** On a cache miss, the origin's reverse proxy routes the request to an application server, which may call caches and databases across a leaf–spine fabric (§7.17).
8. **After the response.** The browser discovers CSS, JavaScript and images and fetches them as concurrent streams on the same connection, ramping up under slow start (§7.7.7). An `Alt-Svc` header may advertise HTTP/3 for later connections.

WORKED EXAMPLE

Worked example — Time to first byte budget.

Assume: 5 ms RTT to the resolver; resolver needs root, TLD and authoritative lookups costing 15, 20 and 30 ms, plus 25 ms for the CDN’s authoritative servers; 20 ms RTT to the edge; 2 ms edge processing on a cache hit.

1. DNS: $5 + 15 + 20 + 30 + 25 = 95$ ms (cold); 5 ms warm.
2. TCP handshake: 20 ms.
3. TLS 1.3 handshake: 20 ms.
4. Request and first response byte: $20 + 2 = 22$ ms.
5. Total cold: $95 + 20 + 20 + 22 = 157$ ms. Warm DNS: $5 + 62 = 67$ ms. Reused connection: 22 ms.
6. On a cache miss with the origin 70 ms from the edge over a pooled connection, add $70 + $ origin processing (say 30 ms) = 100 ms.

Interpretation: on a cold visit, connection setup (DNS, TCP, TLS) is most of the time to first byte, and the server’s own work is a small slice. This is why the biggest wins for global latency are fewer round trips and shorter round trips, not faster servers.

7.19 Summary and revision notes

SUMMARY

- **Layering and encapsulation.** Each layer adds its own header: application message → TCP segment or UDP datagram (ports) → IP packet (addresses) → link frame (MAC). Routers rewrite link frames at every hop; IP packets travel end to end.
- **Addressing.** IPv4 is 32 bits, IPv6 is 128. CIDR `/n` leaves 2^{32-n} IPv4 addresses (minus 2 on ordinary subnets; minus 5 on AWS). Private ranges: 10/8, 172.16/12, 192.168/16. Plan non-overlapping ranges before networks need to connect.
- **Routing.** Forwarding uses longest-prefix match. BGP connects autonomous systems by policy, not latency, and trusts announcements by default. Anycast announces one prefix from many sites to steer users to a nearby one.
- **DNS.** Stub → recursive resolver (recursive query) → root → TLD → authoritative (iterative queries). Caches expire by TTL; nothing "propagates". DNS load balancing picks sites, not servers, and fails over in minutes.
- **TCP.** 1-RTT handshake; four-way close; the active closer holds TIME_WAIT (60 s on Linux), which can exhaust ephemeral ports. Reliability via sequence numbers, cumulative ACKs, fast retransmit and RTO (200 ms minimum on Linux). Flow control uses the receive window; congestion control uses cwnd: slow start doubles per RTT, AIMD halves on loss, CUBIC grows by a cubic of time, BBR paces at measured bandwidth. One lost segment blocks the whole stream (head-of-line blocking). Set `TCP_NODELAY` for RPC.
- **UDP.** 8-byte header, no setup, no reliability. Use it when timeliness beats completeness or to build your own transport; you then own congestion control.
- **HTTP.** 1.1: one request at a time per connection, about six connections per host. 2: binary frames, multiplexed streams, HPACK, but TCP HOL blocking. 3: QUIC over UDP with independent streams, built-in TLS 1.3, 1-RTT (0-RTT on resumption) setup and connection migration.
- **TLS 1.3.** 1-RTT handshake, encrypted certificates, mandatory forward secrecy. 0-RTT early data is replayable: use only for idempotent requests. Expired certificates are the most common TLS outage; automate renewal.
- **Middle of the network.** Forward proxies act for clients; reverse proxies act for servers (TLS termination, routing, load balancing). NAT shares public IPs, blocks unsolicited inbound traffic, and has finite ports per destination. Security groups are stateful; network ACLs are stateless.
- **Numbers.** Fiber propagation is about 5 µs/km; NY–London RTT is at least 56 ms. BDP $= R \times \text{RTT}$: 10 Gbps × 80 ms = 100 MB in flight. Mathis: throughput $\leq (\text{MSS}/\text{RTT}) \times 1.22/\sqrt{p}$, so 0.01% loss caps an 80 ms flow near 18 Mbps.

- **Connections.** A new HTTPS connection with TLS 1.3 costs 3 RTTs to first byte, versus 1 on a reused connection. Pool connections; size pools with $N = \lambda W$; set client idle timeouts shorter than server timeouts.
- **Datacenters.** East–west traffic dominates; leaf–spine Clos fabrics with ECMP give every rack equal-cost two-hop paths and scale out with commodity switches (Google Jupiter: over 1 Pbps of bisection bandwidth).

7.20 Knowledge checkpoint

1. Which headers does a router remove and rebuild at each hop, and which header fields does it modify?
2. What are the network address, broadcast address and usable host range of `172.20.14.200/22`?
3. A router has routes for `10.0.0.0/8`, `10.4.0.0/14` and `10.4.4.0/24`. Which is used for `10.5.1.1`?
4. Why can anycast break long-lived TCP connections, and why is it nevertheless ideal for DNS?
5. Distinguish recursive and iterative DNS queries, and say which participants send each kind.
6. Why can’t you place a CNAME at the zone apex?
7. You need to move a service to new IP addresses; the A record’s TTL is 86,400 s. What must you do and when?
8. What does an acknowledgment number of 5,001 mean?
9. Why does the active closer of a TCP connection enter TIME_WAIT, and for how long on Linux?
10. Explain the difference between flow control and congestion control, naming the window each uses.
11. Starting from an initial window of 10 segments, how many RTTs of slow start are needed to send 300 segments with no loss?
12. Explain how Nagle’s algorithm and delayed ACKs together add latency, and give the standard fix.
13. HTTP/2 multiplexes streams over one connection. Why does it still suffer head-of-line blocking, and how does HTTP/3 avoid it?
14. Why must 0-RTT early data be limited to idempotent requests?
15. Compute the BDP of a 1 Gbps link with a 40 ms RTT, in megabytes.
16. Using the Mathis formula, what is the maximum throughput with MSS 1,460 bytes, RTT 20 ms and 1% loss?
17. Why are network ACLs a frequent cause of “connection hangs” while security groups are not?

18. What is the oversubscription ratio of a leaf with 32 × 100 Gbps server ports and 8 × 400 Gbps uplinks?

Answers

1. The router strips the incoming link-layer (Ethernet) header and trailer and builds a new frame with new source and destination MACs for the next link. In the IP header it decrements TTL (hop limit) and, for IPv4, recomputes the header checksum. NAT devices also rewrite addresses and ports.
2. A `/22` has $2^{10} = 1,024$ addresses; the third octet advances in steps of 4. 14 lies in 12–15, so network `172.20.12.0`, broadcast `172.20.15.255`, usable `172.20.12.1` – `172.20.15.254` (1,022 hosts).
3. `10.4.0.0/14` covers 10.4.0.0–10.7.255.255, which includes 10.5.1.1; the /24 does not. Longest match is the /14.
4. A BGP route change can deliver a flow's later packets to a different site that has no state for the connection, which resets it. DNS queries are typically single UDP request/response exchanges with no connection state, so a route change between queries is harmless.
5. A recursive query asks the server for a final answer; the stub resolver sends it to the recursive resolver. An iterative query accepts referrals; the recursive resolver sends these to root, TLD and authoritative servers.
6. A CNAME may not coexist with any other record at the same name, and the apex must hold SOA and NS records.
7. Lower the TTL (for example to 60 s) at least 86,400 s before the change, so all caches have refreshed with the short TTL; make the change; keep the old addresses serving until at least the short TTL (plus margin for misbehaving caches) has passed; then raise the TTL again.
8. The receiver has received every byte up to and including byte 5,000 (relative to the ISN) and expects byte 5,001 next.
9. To retransmit the final ACK if the peer resends its FIN, and to let delayed segments of the old connection expire before the same 4-tuple can be reused. Linux uses 60 s (2 × MSL).
10. Flow control protects the receiver's buffer using the advertised receive window (rwnd). Congestion control protects the network using the sender's congestion window (cwnd). The sender is limited by $\min(\text{cwnd}, \text{rwnd})$.
11. Cumulative segments: 10, 30, 70, 150, 310. 300 segments complete in the 5th RTT.
12. With unacknowledged data in flight, Nagle holds a second small write; the receiver delays its ACK because it has no response ready until the full request arrives; both wait for the delayed-ACK timer. Fix: write whole messages at once and set `TCP_NODELAY`.
13. All streams share one ordered TCP byte stream, so a lost segment blocks delivery of later bytes for every stream. QUIC orders data per stream, so loss stalls only the affected stream.

14. An attacker can capture and replay the first flight, causing the server to process the early request again; idempotent requests are safe to repeat.
15. $10^9 \times 0.04 = 4 \times 10^7$ bits $= 5 \times 10^6$ bytes $= 5$ MB.
16. MSS/RTT $= 11,680/0.02 = 584,000$ bits/s; $1.22/\sqrt{0.01} = 12.2$; throughput ≤ about 7.1 Mbps.
17. NACLs are stateless, so return traffic to the client's ephemeral ports must be explicitly allowed; if it is not, SYN-ACKs or responses are silently dropped and clients wait for timeouts. Security groups are stateful and allow return traffic automatically.
18. Downlink $32 \times 100 = 3,200$ Gbps; uplink $8 \times 400 = 3,200$ Gbps; ratio 1:1 (non-blocking).

7.21 Interview questions

Basic

Q1. What is the difference between TCP and UDP, and when would you choose UDP?

Answer. TCP provides a connection-oriented, reliable, ordered byte stream with flow and congestion control, at the cost of a 1-RTT handshake, head-of-line blocking and per-connection state. UDP provides unreliable, unordered datagrams with an 8-byte header and no setup. Choose UDP when timeliness matters more than completeness (voice, video, games), when exchanges are a single small request and response (DNS), or when you need a custom transport (QUIC). Mention the cost: the application must handle loss, ordering, message sizing to the MTU and, critically, congestion control.

Q2. Walk me through what happens during a DNS lookup.

Answer. The application calls the OS stub resolver, which checks local caches and then sends a recursive query to a recursive resolver. If the resolver's cache is cold, it queries a root server, which refers it to the TLD servers; the TLD server refers it to the domain's authoritative servers; the authoritative server returns the record with a TTL. The resolver caches every record for its TTL and returns the answer. Mention UDP port 53 with TCP fallback, TTL trade-offs, and that DNS changes do not "propagate"; caches expire.

Q3. What happens during a TCP three-way handshake, and why three messages?

Answer. The client sends SYN with its random ISN; the server replies SYN-ACK with its own ISN and acknowledges the client's; the client ACKs the server's ISN. Three messages are the minimum for each side to announce its ISN and confirm the other's has been received, in both directions. It costs the client one RTT before sending data. Strong answers mention option negotiation (MSS, window scaling, SACK), SYN floods and SYN cookies.

Intermediate

Q4. Your service's p99 latency to a dependency in another region is far higher than its p50. What network causes would you investigate?

Answer. First, connection setup: if connections are not reused, some requests pay DNS plus TCP plus TLS handshakes (3+ RTTs instead of 1); check pool hit rates and idle-timeout races that produce resets and retries. Second, packet loss: a single retransmission timeout costs at least 200 ms on Linux, and loss also shrinks cwnd; check retransmission counters (`nstat`, `ss -ti`). Third, slow start after idle: large responses on cold or idle connections need several RTTs to ramp. Fourth, queuing (bufferbloat) on congested links, Nagle/delayed-ACK stalls on small writes, DNS resolution timeouts, and NAT port exhaustion. Measure with connection-phase timings (e.g., `curl -w`) and distributed traces before changing anything.

Q5. Compare HTTP/1.1, HTTP/2 and HTTP/3 for a mobile application.

Answer. HTTP/1.1 serializes requests per connection, so clients open several connections, each paying handshakes and slow start. HTTP/2 multiplexes streams on one connection with HPACK header compression, a big improvement, but a lost TCP segment stalls all streams, which hurts on lossy mobile links. HTTP/3 over QUIC gives per-stream reliability (no cross-stream HOL blocking), a 1-RTT combined transport and TLS handshake with 0-RTT resumption, and connection migration when the device switches networks. Costs: UDP may be blocked (keep HTTP/2 fallback), and QUIC uses more server CPU. For a mobile app, enable HTTP/3 at the CDN or edge with HTTP/2 fallback, and use 0-RTT only for idempotent requests.

Q6. How would you use DNS for global traffic distribution, and what are its limitations?

Answer. Use latency-based or geo-DNS with health checks to return the nearest healthy region's address, typically pointing at a regional load balancer or anycast address. Limitations: resolvers cache answers for the TTL, so control is coarse and failover takes the detection time plus up to one TTL (and longer for misbehaving caches); geo decisions see the resolver, not the client, unless EDNS Client Subnet is used; DNS has no knowledge of load; clients with open connections ignore DNS changes. So use DNS for region selection with moderate TTLs (30–300 s), anycast or load balancers for fast failover within and across sites, and client retries across regions for resilience.

Advanced

Q7. A nightly replication job copies data between regions 80 ms apart over a 10 Gbps link but achieves only about 200 Mbps. Diagnose and fix.

Answer. Compute the BDP: $10\ \text{Gbps} \times 80\ \text{ms} = 100$ MB must be in flight. Check whether the window is the limit: 200 Mbps × 0.08 s = 2 MB, so the effective window is about 2 MB, possibly because of `tcp_rmem` / `tcp_wmem` maximums or application socket buffer settings. Check loss: by Mathis, 200 Mbps at 80 ms RTT corresponds to $p \approx (1.22 \times 146{,}000/2 \times 10^{8})^{2} \approx 8 \times 10^{-7}$, so even tiny loss could explain it with CUBIC. Fixes: raise buffer limits and ensure window scaling; use several parallel streams (each has its own cwnd); consider BBR for the bulk sender, since it does not treat random loss as congestion; fix lossy links; compress data. Verify with `ss -ti` (cwnd, rtt, retransmits) and `iperf3`.

Q8. Design the network path for a global API with low latency, high availability and DDoS resilience.

Answer. Publish the API hostname with DNS from two independent providers, pointing at an anycast address announced from many edge sites, so users reach a nearby edge and attacks spread across sites. At each edge, L4 load balancers using ECMP and consistent hashing feed L7 reverse proxies that terminate TLS 1.3 and HTTP/2/HTTP/3, apply WAF rules and rate limits, and cache what they can. Edges forward to the nearest healthy regional origin over pre-warmed, pooled, long-lived connections (mTLS), avoiding per-request handshakes on long paths. Regions are active–active behind health-checked routing, with DNS TTLs of about 60 s as a slower backstop. Inside regions, leaf–spine fabrics and security groups segment tiers. Discuss trade-offs: anycast and TCP stability, 0-RTT replay restrictions, certificate automation, and cost of egress.

Q9. Explain BBR to someone who knows CUBIC, including when you would and would not deploy it.

Answer. CUBIC infers congestion from loss: it grows cwnd as a cubic function of time since the last loss and multiplies cwnd by 0.7 on loss, so it fills buffers until they overflow and reacts badly to random loss. BBR builds a model: it measures the bottleneck bandwidth (maximum delivery rate) and the minimum RTT, paces at the bandwidth estimate, and keeps about one BDP in flight, cycling its pacing gain to probe for more bandwidth and periodically draining queues to re-measure min RTT. Benefits: much higher throughput on lossy long paths (Google reported 2–25× on its B4 backbone) and lower queuing delay. Concerns: BBRv1 can be unfair to loss-based flows sharing a bottleneck and can cause high retransmission rates with shallow buffers; later versions address this. Deploy it on senders serving users over the WAN or doing bulk transfers on lossy paths; be cautious in shared, shallow-buffer datacenter fabrics, and A/B test.

7.22 Practical exercises

1. **Trace DNS by hand.** Run `dig +trace www.wikipedia.org` and identify the root, TLD and authoritative steps. Then run `dig www.wikipedia.org` twice against your resolver

and compare the TTL values in the two answers; explain the difference. Query `dig example.com MX`, `dig example.com TXT`, and `dig _xmpp-server._tcp.jabber.org SRV`, and read each record using Table 7.5.

2. **Time a request phase by phase.** Run:

```
        curl -o /dev/null -s -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%
{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n'
https://www.example.com/
```

   All values are cumulative seconds from the start. Compute the TCP handshake (`tcp − dns`), TLS handshake (`tls − tcp`) and server time (`ttfb − tls`). Repeat with `--http1.1`, `--http2` and (if your curl supports it) `--http3`, and against a nearby and a distant site. Estimate RTT from the TCP phase and check it against `ping`.

3. **Watch a handshake.** Start the echo server from §7.7.10, then in another terminal run `sudo tcpdump -i lo -nn 'tcp port 9000'` and run the client. Identify the SYN, SYN-ACK, ACK, data segments, FIN/ACK exchanges and their sequence and acknowledgment numbers. Run `ss -tan state time-wait` afterward and find the TIME_WAIT socket on the client side. Repeat the capture in Wireshark and use "Follow TCP Stream".

4. **Exhaust and fix.** Write a client that opens and closes a new connection to the echo server in a tight loop, and count TIME_WAIT sockets with `ss -s`. Then rewrite it to reuse one connection, and compare throughput.

5. **Feel the Mathis formula.** On a Linux test machine, add delay and loss to loopback with `sudo tc qdisc add dev lo root netem delay 40ms loss 0.1%` (giving roughly an 80 ms RTT), measure throughput with `iperf3`, then remove it with `sudo tc qdisc del dev lo root`. Compare your result with the formula, and repeat with `-C bbr` if the BBR module is available.

6. **Subnet calculator.** Verify the worked examples in §7.4 with Python's standard library:

```
        import ipaddress
        n = ipaddress.ip_network("192.168.10.77/26", strict=False)
        print(n, n.netmask, n.broadcast_address, n.num_addresses - 2)   #
192.168.10.64/26 255.255.255.192 192.168.10.127 62
        print(ipaddress.ip_address("10.0.37.200") in
ipaddress.ip_network("10.0.32.0/20"))  # True
        print(list(ipaddress.ip_network("10.0.0.0/16").subnets(new_prefix=20))
[:3])
        print(ipaddress.ip_network("10.0.0.0/16").overlaps(ipaddress.ip_network("1
0.0.128.0/17")))  # True
```

7. **Inspect a certificate.** Run `openssl s_client -connect www.example.com:443 -servername www.example.com -tls1_3 </dev/null` and identify the chain, the negotiated protocol and cipher, and the leaf's validity period.

7.23 Design challenges

1. **Global login service.** Users on five continents must log in with p95 under 300 ms. The credential database must live in one region. Design the network path (DNS, anycast or regional endpoints, TLS termination location, connection reuse between edge and origin) and compute the round-trip budget for a user in Sydney with the database in Virginia (great-circle distance about 15,700 km). Which steps can 0-RTT safely accelerate, and which cannot?
2. **Egress at scale.** 3,000 containers in private subnets call a single third-party payments API through cloud NAT. Calls take 300 ms, peak load is 20,000 requests per second, and each call currently opens a new TLS connection. Estimate concurrent connections with Little's Law, identify the NAT and TIME_WAIT limits you will hit, and redesign (pooling, egress proxies, multiple NAT IPs, keep-alive settings).
3. **Datacenter fabric for an analytics cluster.** 2,000 servers with 25 Gbps NICs run shuffle-heavy batch jobs. Choose leaf and spine port counts and an oversubscription ratio, compute the bisection bandwidth, and explain how you would detect and mitigate incast and ECMP hot spots.

7.24 References

1. Postel, J. *RFC 791: Internet Protocol*. IETF, 1981. https://www.rfc-editor.org/rfc/rfc791
2. Deering, S., Hinden, R. *RFC 8200: Internet Protocol, Version 6 (IPv6) Specification*. IETF, 2017. https://www.rfc-editor.org/rfc/rfc8200
3. Mockapetris, P. *RFC 1034: Domain Names — Concepts and Facilities*. IETF, 1987. https://www.rfc-editor.org/rfc/rfc1034
4. Mockapetris, P. *RFC 1035: Domain Names — Implementation and Specification*. IETF, 1987. https://www.rfc-editor.org/rfc/rfc1035
5. Eddy, W. (Ed.). *RFC 9293: Transmission Control Protocol (TCP)*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9293
6. Postel, J. *RFC 768: User Datagram Protocol*. IETF, 1980. https://www.rfc-editor.org/rfc/rfc768
7. Fielding, R., Nottingham, M., Reschke, J. (Eds.). *RFC 9110: HTTP Semantics*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110
8. Fielding, R., Nottingham, M., Reschke, J. (Eds.). *RFC 9112: HTTP/1.1*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9112

9. Thomson, M., Benfield, C. (Eds.). *RFC 9113: HTTP/2*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9113
10. Bishop, M. (Ed.). *RFC 9114: HTTP/3*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9114
11. Iyengar, J., Thomson, M. (Eds.). *RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport*. IETF, 2021. https://www.rfc-editor.org/rfc/rfc9000
12. Rescorla, E. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018. https://www.rfc-editor.org/rfc/rfc8446
13. Fette, I., Melnikov, A. *RFC 6455: The WebSocket Protocol*. IETF, 2011. https://www.rfc-editor.org/rfc/rfc6455
14. Allman, M., Paxson, V., Blanton, E. *RFC 5681: TCP Congestion Control*. IETF, 2009. https://www.rfc-editor.org/rfc/rfc5681
15. Other IETF RFCs cited by number in this chapter, each available at `https://www.rfc-editor.org/rfc/rfcNNNN`: 896 (Nagle's algorithm), 1122 (host requirements), 1918 (private addresses), 2308 (negative caching), 2782 (SRV), 3021 (/31 links), 3022 (NAT), 3849 and 5737 (documentation prefixes), 4193 (IPv6 ULA), 4271 (BGP-4), 4632 (CIDR), 5246 (TLS 1.2), 6298 (RTO), 6598 (CGNAT space), 6891 (EDNS(0)), 6928 (initial window), 7239 (Forwarded), 7323 (window scaling), 7413 (TCP Fast Open), 7541 (HPACK), 7858 (DNS over TLS), 8085 (UDP guidelines), 8305 (Happy Eyeballs v2), 8441 and 9220 (WebSockets over HTTP/2 and HTTP/3), 8470 (early data in HTTP), 8484 (DNS over HTTPS), 9001 (QUIC-TLS), 9002 (QUIC loss recovery), 9204 (QPACK), 9218 (HTTP priorities), 9438 (CUBIC), 9460 (SVCB/HTTPS records).
16. Kurose, J. F., Ross, K. W. *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
17. Grigorik, I. *High Performance Browser Networking*. O'Reilly Media, 2013. https://hpbn.co
18. Mathis, M., Semke, J., Mahdavi, J., Ott, T. *The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm*. ACM SIGCOMM Computer Communication Review 27(3), 1997.
19. Cardwell, N., Cheng, Y., Gunn, C. S., Hassas Yeganeh, S., Jacobson, V. *BBR: Congestion-Based Congestion Control*. ACM Queue 14(5), 2016. https://queue.acm.org/detail.cfm?id=3022184
20. Singh, A., Ong, J., Agarwal, A., et al. *Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network*. ACM SIGCOMM, 2015. https://research.google/pubs/pub43837/
21. Jacobson, V. *Congestion Avoidance and Control*. ACM SIGCOMM, 1988.
22. Ha, S., Rhee, I., Xu, L. *CUBIC: A New TCP-Friendly High-Speed TCP Variant*. ACM SIGOPS Operating Systems Review 42(5), 2008.
23. Clos, C. *A Study of Non-Blocking Switching Networks*. Bell System Technical Journal 32(2), 1953.

24. Janardhan, S. *More details about the October 4 outage*. Engineering at Meta, 2021. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
25. RIPE NCC. *YouTube Hijacking: A RIPE NCC RIS case study*. RIPE NCC, 2008.
26. Amazon Web Services. *NAT gateways* (Amazon VPC User Guide). AWS Documentation, accessed 2025. https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-working-with.html
27. Amazon Web Services. *Subnet CIDR blocks* (Amazon VPC User Guide). AWS Documentation, accessed 2025.

7.25 Open verification items

- [RIPE NCC 2008]: confirm the exact title and URL of the YouTube hijacking case study and the “about two hours” duration.
- [AWS Docs: VPC subnets]: confirm the URL and that five addresses per subnet are still reserved (stated “as of 2025”).
- The current CA/Browser Forum schedule for reducing maximum public certificate lifetime below 398 days (deliberately not quantified in §7.10.3).
- “Measured NY–London RTTs are often in the 70 ms range” (§7.15.1) is an order-of-magnitude statement; replace with a dated measurement if one is wanted.
- Jacobson’s 1986 collapse figure (32 kbps to 40 bps) and Google’s BBR results (2–25× on B4) should be checked against the primary texts.
- Linux defaults quoted (TIME_WAIT 60 s, minimum RTO 200 ms, delayed-ACK minimum about 40 ms, ephemeral range 32,768–60,999, keep-alive after two hours) are version- and distribution-dependent.
- Runnable code is inline only; per style guide §8 it should also be copied to `code/ch07/` (not done, since this task was limited to the chapter file).

Chapter 10 — Security Fundamentals for System Designers

> **Level:** 2 · **Track:** Shared · **Estimated reading time:** 5 h

Every system you design will be attacked. Not “might be”: a service with a public IP address receives automated scanning traffic within minutes of going online, and any system that stores money, personal data or compute capacity is worth an attacker’s time. Security is therefore not a feature you add at the end. It is a property of the architecture, in the same way that availability and latency are, and the cheapest time to get it right is when the boxes and arrows are still on the whiteboard.

This chapter gives you the vocabulary and mental models a system designer needs: finding threats systematically, choosing cryptographic building blocks, storing passwords and secrets, encrypting data with managed keys, limiting the blast radius of a breach, surviving denial-of-service attacks and keeping records an intruder cannot quietly rewrite. The focus is defensive and architectural. Chapter 50 builds on it with identity architecture, supply-chain security and privacy engineering.

10.1 Learning objectives

After this chapter you should be able to:

- Draw a data-flow diagram for a system, mark its trust boundaries and enumerate threats with STRIDE.
- Choose the correct cryptographic primitive (hash, MAC, symmetric encryption, asymmetric encryption, signature, key exchange) for a stated security goal and explain what it does *not* guarantee.
- Explain how a browser decides to trust a TLS certificate, and design certificate issuance and rotation for internal services.
- Store passwords with Argon2id, scrypt or bcrypt, choose cost parameters, and compute an attacker’s expected brute-force cost.
- Map each OWASP Top 10 (2021) category to an architectural control.
- Design secrets management with a vault, short-lived credentials and automated rotation.
- Explain envelope encryption with a key management service (KMS) and draw its sequence diagram.

- Contrast perimeter security with zero trust, and explain mTLS, least privilege and network segmentation.
- Classify DDoS attacks as volumetric, protocol or application-layer and place mitigations at the right layer.
- Design a tamper-evident audit log.
- Apply a security checklist to any system design.

10.2 Prerequisites

- TCP, UDP and the OSI layers — §7.3, §7.7, §7.8 (DDoS attacks target specific layers).
- TLS 1.3 and HTTPS — §7.10 (this chapter explains the cryptography and certificates underneath it).
- Firewalls and security groups — §7.14 (the starting point for network segmentation).
- Authentication, authorization, sessions, JWT and OAuth 2.0 — §8.10–§8.14 (we assume you know what a session token is).
- Powers of two and logarithms — §4.3, §4.4 (key sizes and brute-force estimates are exponential).
- Probability essentials — §4.6 (collision and risk estimates).

10.3 Threat modeling

10.3.1 What “secure” means: security goals

DEFINITION

Definition — Security property. A statement about what must remain true about a system even when an adversary is actively trying to violate it. The classic triad is **confidentiality** (only authorized parties can read data), **integrity** (data and behavior cannot be changed without detection) and **availability** (authorized parties can use the system when they need it), often abbreviated **CIA**. System designers add **authentication** (knowing who is acting), **authorization** (deciding what that actor may do) and **non-repudiation** (an actor cannot credibly deny an action they took).

First-principles explanation

Think about a house. Confidentiality is curtains: people outside cannot see your belongings. Integrity is a tamper seal on the medicine cabinet: you may not stop someone opening it, but you will know if they did. Availability is being able to get into your own home: a burglar who glues your lock shut has harmed you without stealing anything.
Authentication is the doorman recognizing your face; authorization is the rule that your

face opens the front door but not the neighbor's flat. Non-repudiation is the signed visitor's book: a visitor cannot later claim they were never there.

"Is this system secure?" has no answer on its own. The only answerable question is "secure against *whom*, protecting *what*, with respect to *which* property?" A public weather website needs integrity and availability but almost no confidentiality. A password database needs all three, and confidentiality failures there are catastrophic. Threat modeling is the discipline of asking these questions systematically instead of by intuition.

10.3.2 Assets, adversaries and trust boundaries

DEFINITION

Definition — Threat model. A structured description of a system, the **assets** in it that are worth protecting, the **adversaries** who might attack it and their capabilities, the **threats** (ways a security property could be violated), and the **mitigations** chosen for each threat [Shostack 2014].

DEFINITION

Definition — Trust boundary. A line in a system where the level of trust changes: data or requests crossing it come from a party that the receiving side does not fully control or trust. Examples: the internet-to-load-balancer edge, the browser-to-server connection, a service calling a third-party API, a tenant's code running on shared infrastructure.

Assets are the things an attacker wants or the things you cannot afford to lose: user credentials, payment data, personal data, signing keys, the ability to send email from your domain, compute capacity (attackers mine cryptocurrency on stolen cloud accounts), and your reputation. Rank them: a leaked public product catalog is embarrassing; leaked password hashes or private keys are a crisis.

Adversaries range from automated opportunists (bots scanning the whole internet for default passwords and known vulnerabilities), through targeted external attackers who study *your* system and phish *your* employees, malicious or compromised **insiders** who already hold legitimate credentials, and supply-chain attackers who compromise a library or vendor you depend on, up to well-resourced state actors. Every internet-facing design must handle the first three. The insider is the one beginners forget, and the reason zero trust (§10.10) and audit logging (§10.12) exist.

Trust boundaries are where attacks happen. Inside a single process, data you created yourself can be trusted; the moment data crosses a boundary, it must be **authenticated**

(who sent it?), **authorized** (are they allowed?) and **validated** (is it well-formed?). Almost every vulnerability class in §10.7 is a failure to do one of these three things at a boundary.

10.3.3 Data-flow diagrams

How it works internally

A **data-flow diagram (DFD)** is the standard picture for threat modeling. It uses only four element types, plus boundaries:

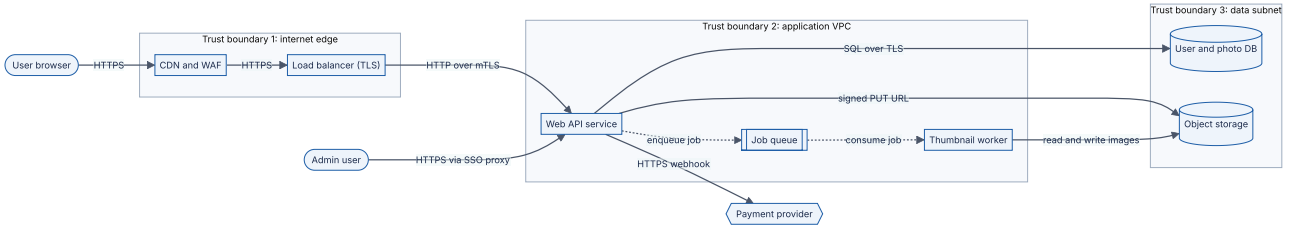
1. **External entity** — something outside your control that sends or receives data (a user, a partner API).
2. **Process** — code that transforms data (a web server, a worker).
3. **Data store** — where data rests (a database, a bucket, a cache, a log).
4. **Data flow** — an arrow showing data moving between the above, labeled with what moves and over which protocol.
5. **Trust boundary** — a dashed box or line where trust changes.

The rules are deliberately simple so that every engineer on a team can draw one in ten minutes. The value is not the drawing; it is that each arrow crossing a boundary becomes a question: *how is this authenticated, authorized, validated and protected in transit?*

Shostack frames the whole exercise as four questions [Shostack 2014]: **What are we working on?** (draw the DFD) · **What can go wrong?** (apply STRIDE) · **What are we going to do about it?** (choose mitigations) · **Did we do a good job?** (review and test).

Architecture diagram

Figure 10.1 — Data-flow diagram of a photo-sharing web application with trust boundaries.



Reading the diagram. Two external entities (the user's browser and an administrator) and one external system (a payment provider, hexagon) sit outside every boundary. Trust boundary 1 is the internet edge: the CDN with a web application firewall (WAF) and the TLS-terminating load balancer are the first components that see untrusted traffic, so they carry rate limiting and coarse filtering. Trust boundary 2 is the application network: the web API is the only process that authenticates users and authorizes actions; it talks to the load balancer over mutual TLS. Solid arrows are synchronous request/response flows; the

dotted arrows to and from the job queue are asynchronous — the thumbnail worker processes uploads later, so a failure there delays thumbnails but does not break uploads. Trust boundary 3 contains the storage: the database and object store accept connections only from the application subnet. Notice where the risky crossings are: every arrow from `U`, `ADM` and `PAY` crosses at least one boundary, and the worker parses user-uploaded image files, which is a classic place for parser vulnerabilities. Scaling options (more web replicas, more workers) do not change the boundaries, but each new data store or third-party integration adds a new arrow that must be modeled.

10.3.4 STRIDE

DEFINITION

Definition — STRIDE. A mnemonic for six threat categories, developed at Microsoft in 1999 by Loren Kohnfelder and Praerit Garg and popularized by Shostack [Shostack 2014]: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service and **E**levation of privilege. Each category is the violation of one security property.

Table 10.1 — STRIDE categories, the property each violates, and typical mitigations.

Threat	Violates	Plain-language question	Typical mitigations
Spoofing	Authentication	Can someone pretend to be another user or service?	Strong authentication, MFA, mTLS, signed tokens
Tampering	Integrity	Can someone change data in transit or at rest?	TLS, MACs, signatures, AEAD encryption, DB permissions
Repudiation	Non-repudiation	Can someone deny doing something?	Audit logs, signatures, tamper-evident storage
Information disclosure	Confidentiality	Can someone read what they should not?	Encryption, access control, data minimization
Denial of service	Availability	Can someone stop legitimate use?	Rate limits, quotas, autoscaling, DDoS protection
Elevation of privilege	Authorization	Can someone gain rights they were not granted?	Least privilege, authorization checks on every request, sandboxing

You apply STRIDE **per element** of the DFD: for each process, data store and flow, walk through the six letters and ask whether the threat applies. Not every letter applies to every element type: an external entity can be spoofed but cannot be “tampered with” by you; a data flow can be tampered with, disclosed or flooded. This mechanical walk-

through is what makes STRIDE useful — it stops you from fixating on the one threat you happened to think of first.

10.3.5 Worked example: STRIDE on the photo-sharing application

WORKED EXAMPLE

Worked example — STRIDE pass over Figure 10.1.

Step 1 — Assets. User credentials and session tokens; private photos; payment tokens returned by the provider; admin capabilities (delete any account); the database master credentials; the TLS private keys.

Step 2 — Walk each boundary-crossing element. The table below shows a representative (not exhaustive) result.

Step 3 — Rank. Score each threat for likelihood and impact on a 1–3 scale (see §10.3.6) and fix high scores first.

Step 4 — Record. The result is a short table in the design document, owned by the team, revisited whenever the DFD changes.

Table 10.2 — STRIDE threats for the photo-sharing application (excerpt).

#	Element	STRIDE	Threat	Mitigation	L × I
T1	User → Web API	S	Credential stuffing with passwords leaked from other sites	Rate limit per account and IP; breached-password check; MFA; Argon2id storage	3 × 3 = 9
T2	Web API	E	User changes `photo_id` in URL and reads another user's private photo (IDOR)	Object-level authorization check on every read, enforced in a shared data-access layer	3 × 3 = 9
T3	Web API → Object store	I	Bucket accidentally made public	Block public access at account level; serve via short-lived signed URLs	2 × 3 = 6
T4	Thumbnail worker	E	Malicious image exploits image-parsing library, attacker runs code in worker	Sandbox worker (no network, minimal IAM role), patch libraries, separate account	2 × 3 = 6
T5	Payment webhook	S	Attacker forges "payment succeeded" webhook	Verify provider's HMAC signature and timestamp (§10.4.7)	2 × 3 = 6
T6	Admin → Web API	R	Admin deletes an account and later denies it	Append-only, tamper-evident audit log of admin actions (§10.12)	1 × 2 = 2
T7	Upload endpoint	D	Attacker uploads thousands of 1 GB files	Size limits, per-user quotas, rate limits	2 × 2 = 4
T8	Web API → DB	T	SQL injection modifies rows	Parameterized queries; DB user without DDL rights	2 × 3 = 6

Look at what the exercise produced: eight concrete, testable requirements, most of which are architectural (where the authorization check lives, which account the worker runs in, how the bucket is exposed) rather than "write secure code". That is the point of threat modeling for a system designer.

Mathematical foundations

FORMULA

Qualitative risk score

$$R = L \times I$$

Symbol	Meaning	Units
R	Risk score used to rank threats	dimensionless (1–9 on a 3-point scale)
L	Likelihood that the threat is exploited within the planning horizon (1 = unlikely, 2 = plausible, 3 = expected)	ordinal score
I	Impact if exploited (1 = minor, 2 = serious, 3 = severe: data breach, financial loss, regulatory)	ordinal score

WORKED EXAMPLE

Worked example — Ranking T1 against T6. Credential stuffing (T1) is *expected* for any consumer login page, because attackers replay billions of leaked username–password pairs automatically: $L = 3$. Its impact is account takeover and private-photo exposure: $I = 3$. So $R = 3 \times 3 = 9$. Admin repudiation (T6) requires a malicious administrator ($L = 1$) and its impact is a dispute rather than a breach ($I = 2$): $R = 2$. *Interpretation:* fix T1 in the first release; T6 can follow. *Engineering implication:* the score is ordinal, not a probability — do not multiply it by dollar figures or average it across threats. Its only job is to order the work and make the reasoning visible to reviewers.

Advantages, disadvantages and alternatives

STRIDE is easy to teach, systematic and tied to properties, which makes it ideal for design reviews. Its weaknesses: it can produce long lists of low-value threats, it does not rank risk by itself, and it focuses on technical threats more than abuse of legitimate features (spam, fraud, harassment). Alternatives include **attack trees** (a root goal such as “read another user’s photo” decomposed into AND/OR subgoals — good for deep analysis of one critical asset), **PASTA** (a seven-stage risk-centric process), **LINDDUN** (privacy threats) and **kill-chain** analysis (focused on detection and response). Many teams use STRIDE for breadth and attack trees for the top two or three risks.

Failure scenarios

Table 10.3 — Threat-modeling failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Threat model is stale	Architecture changed, DFD did not	New data store or vendor absent from DFD in review	Re-run STRIDE on changed elements	Make DFD update a checklist item for design changes
Threat model is a document nobody reads	Produced for compliance, not engineering	Mitigations not traced to tickets/tests	Convert each mitigation to a tracked task	Keep it short; review it in design review
Insider threats ignored	"Everything inside the VPC is trusted" assumption	Red-team exercise with internal credentials	Add authorization and audit inside the network	Include insider row in adversary list

Common misconceptions

COMMON MISTAKE

"We use HTTPS, so we are secure." TLS protects data *in transit* between two endpoints. It does nothing about a logged-in user requesting someone else's data (T2), a forged webhook (T5) or SQL injection (T8). Most real breaches happen through authenticated, encrypted connections.

10.4 Cryptography essentials

Cryptography is the toolbox that turns security goals into mathematics. As a system designer you will almost never implement a cryptographic algorithm, but you will constantly *choose* one, and choosing the wrong primitive for a goal (encrypting when you needed to authenticate, hashing when you needed a secret key) is one of the most common design errors. This section explains each primitive by analogy, then by mechanism, then with the numbers that tell you how strong it is.

Table 10.4 — Which primitive achieves which goal.

Goal	Primitive	Needs a key?	Typical algorithm (as of 2026)	Analogy
Detect accidental or malicious change, given a trusted reference value	Cryptographic hash	No	SHA-256, SHA-3, BLAKE2/3	Fingerprint
Prove a message came from someone holding a shared secret, unchanged	MAC	Shared secret	HMAC-SHA-256	Wax seal pressed with a ring both parties own
Keep data secret *and* detect tampering, between parties sharing a key	Authenticated (AEAD) symmetric encryption	Shared secret	AES-256-GCM, ChaCha20-Poly1305	Locked box with a tamper seal
Let anyone send you a secret without a pre-shared key	Asymmetric encryption / KEM	Public + private key	RSA-OAEP, ECIES, ML-KEM	Padlock you hand out open; only you hold its key
Agree on a shared secret over a public channel	Key exchange	Ephemeral key pairs	ECDHE (X25519, P-256)	Mixing paint colors (see §10.4.5)
Prove to *anyone* that a specific key holder approved a message	Digital signature	Private key signs, public key verifies	Ed25519, ECDSA P-256, RSA-PSS	Notarized handwritten signature
Produce unpredictable keys, nonces, tokens	CSPRNG	Seeded by OS	`/dev/urandom`, `getrandom()`, `secrets`	Fair dice nobody can watch

10.4.1 Hash functions

DEFINITION

Definition — Cryptographic hash function. A deterministic function H that maps an input of any length to a fixed-length output (the **digest**), such that it is computationally infeasible to (1) find an input that hashes to a given digest (**preimage resistance**), (2) given one input, find a different input with the same digest (**second-preimage resistance**), and (3) find any two distinct inputs with the same digest (**collision resistance**).

First-principles explanation

A hash is a fingerprint. You cannot reconstruct a person from a fingerprint, two people practically never share one, and the tiniest change in the person (a cut on the finger) changes the print. SHA-256 takes a 4 GB disk image or the single letter “a” and produces 256 bits; flip one input bit and roughly half of the output bits flip (the **avalanche effect**).

Hashes give you integrity *only if the reference digest itself is trustworthy*. If an attacker can change a download *and* the digest published next to it, the hash proves nothing. That is why hashes are combined with keys (MACs) or signatures, and why a plain hash is never a password-storage scheme on its own (§10.6).

How it works internally

SHA-2 hashes pad the input, split it into 512-bit (SHA-256) blocks and mix each block into a running internal state with a **compression function** (the **Merkle–Damgård construction**); the final state is the digest. A side effect is the **length-extension property**: given $H(m)$ and the length of m, anyone can compute $H(m \parallel \text{padding} \parallel x)$ without knowing m. That is harmless for fingerprints but fatal if you build a MAC as $H(\text{key} \parallel \text{message})$ — the reason HMAC exists (§10.4.6). SHA-3 uses a different "sponge" construction without this property.

MD5 and SHA-1 are broken for collision resistance: practical collisions were published for MD5 in 2004 and for SHA-1 in 2017 (the "SHAttered" attack). Do not use them for any security purpose.

Mathematical foundations

FORMULA

Birthday bound for hash collisions

$$P_{\text{collision}} \approx \frac{k^2}{2^{n+1}}$$

Symbol	Meaning	Units
k	Number of distinct inputs hashed	count
n	Digest length	bits
$P_{\text{collision}}$	Probability that at least two of the k digests are equal (valid while $P \ll 1$)	probability

Consequence: an n-bit hash offers about $n/2$ bits of collision resistance, because P approaches $1/2$ when $k \approx 2^{n/2}$.

WORKED EXAMPLE

Worked example — Is a 64-bit truncated hash safe as a content ID? Suppose a storage service names objects by the first 64 bits of their SHA-256 digest and stores $k = 10^{10}$ objects (ten billion).

1. $k^2 = 10^{20}$.
2. $2^{65} \approx 3.69 \times 10^{19}$.
3. $P \approx 10^{20} / 3.69 \times 10^{19} \approx 2.7$ — the approximation has broken down because P is not small; in reality a collision is all but certain.

With the full 256-bit digest: $P \approx 10^{20} / 2^{257} \approx 10^{20} / 2.3 \times 10^{77} \approx 4 \times 10^{-58}$.

Interpretation: 64 bits feels large, but birthday collisions arrive at around $2^{32} \approx 4.3$ billion items. *Engineering implication:* if an adversary benefits from a collision (deduplication, content addressing, signatures), use at least 256-bit digests; truncate only for non-adversarial uses such as cache sharding.

10.4.2 Symmetric encryption and AES-GCM

DEFINITION

Definition — Symmetric encryption. Encryption in which the same secret key is used to encrypt and decrypt. A **block cipher** such as AES (Advanced Encryption Standard) encrypts fixed 128-bit blocks under a 128-, 192- or 256-bit key; a **mode of operation** turns the block cipher into a scheme for messages of any length.

DEFINITION

Definition — Authenticated encryption with associated data (AEAD). A symmetric scheme that provides confidentiality for the plaintext *and* integrity for both the ciphertext and optional unencrypted **associated data** (AAD), producing an authentication **tag** that decryption checks before releasing any plaintext. AES-GCM and ChaCha20-Poly1305 are AEAD schemes.

First-principles explanation

Symmetric encryption is a lockbox where sender and receiver hold copies of the same key. It is extremely fast: modern CPUs have AES instructions (AES-NI on x86, equivalent extensions on ARM) that encrypt at multiple gigabytes per second per core, which is why every bulk-data protocol — TLS records, disk encryption, database encryption — uses symmetric ciphers for the data itself. The hard part is not encrypting; it is getting the

same key to both parties securely and keeping it secret, which the rest of this chapter addresses.

An *unauthenticated* mode such as AES-CBC or AES-CTR hides the content but does not detect changes. An attacker who flips bit 7 of a CTR ciphertext flips bit 7 of the plaintext — turning “pay $100” into “pay $900” without knowing the key. AEAD adds the tamper seal: any change to ciphertext, tag, nonce or associated data makes decryption fail.

How it works internally

AES-GCM (Galois/Counter Mode, NIST SP 800-38D):

1. Take a key K and a 96-bit **nonce** (number used once) N.
2. Generate a keystream by encrypting the counter values $N \parallel 2, N \parallel 3, \ldots$ with AES; XOR it with the plaintext to get ciphertext (counter mode).
3. Compute a 128-bit tag over the AAD and ciphertext using GHASH, a polynomial hash keyed by a value derived from K, and mask it with the encryption of $N \parallel 1$.
4. Output (nonce, ciphertext, tag). Decryption recomputes the tag and rejects mismatches before returning plaintext.

The single most important rule: **never reuse a nonce with the same key.** A repeated nonce produces the same keystream, so XOR of two ciphertexts reveals the XOR of the two plaintexts, and it also leaks enough to forge tags. Use a 96-bit random nonce per message, or a counter that is guaranteed never to repeat, and rotate keys before the nonce collision probability becomes meaningful.

FORMULA

Random-nonce collision probability for AES-GCM

$$P_{\text{reuse}} \approx \frac{q^2}{2^{97}}$$

Symbol	Meaning	Units
q	Number of messages encrypted under one key with random 96-bit nonces	messages
P_{reuse}	Probability that any two of them share a nonce (birthday bound with $n = 96$)	probability

WORKED EXAMPLE

Worked example — How many messages per key? NIST SP 800-38D caps random-nonce GCM at 2^{32} invocations per key. Check the risk: $q = 2^{32}$ gives $q^2 = 2^{64}$, so $P \approx 2^{64}/2^{97} = 2^{-33} \approx 1.2 \times 10^{-10}$. A service encrypting 50,000 messages per second reaches $2^{32} \approx 4.3 \times 10^9$ messages in $4.3 \times 10^9 / 5 \times 10^4 \approx 86,000$ s, about one day. *Engineering implication:* a high-throughput service must rotate data keys at least daily, or use per-object keys (envelope encryption, §10.9), or a nonce-misuse-resistant mode such as AES-GCM-SIV, or a scheme with larger nonces (XChaCha20-Poly1305 uses 192-bit nonces).

The associated data is a powerful design tool. If you encrypt a user's tax ID in row 42 of the `users` table and put `table=users;id=42` in the AAD, an attacker with database write access cannot copy that ciphertext into row 43 — decryption would fail because the context differs. The runnable example in §10.4.8 demonstrates exactly this.

10.4.3 Asymmetric cryptography: RSA and elliptic curves

DEFINITION

Definition — Asymmetric (public-key) cryptography. Cryptography using a mathematically linked **key pair**: a **public key** that can be shared with anyone and a **private key** kept secret by its owner. What one key does, only the other can undo or verify. Security rests on problems believed hard to invert: factoring large integers (RSA) or computing discrete logarithms on elliptic curves (ECC).

First-principles explanation

Picture a padlock factory. You manufacture thousands of open padlocks, all of which only your key opens, and leave them in a bowl on the street. Anyone can put a message in a box, snap one of your padlocks shut, and mail it to you; nobody, including the sender, can reopen it — only you. The open padlocks are your public key; your key is the private key. Asymmetric cryptography solves the problem symmetric encryption cannot: two parties who have never met can communicate securely, provided the sender knows the padlock really is yours (that is what certificates in §10.5 establish).

The price is speed and size. An RSA-2048 private-key operation costs on the order of a millisecond of CPU, thousands of times more than encrypting a short message with AES. So real systems use asymmetric cryptography only to establish or protect a symmetric key and to sign things, never for bulk data. This combination is called **hybrid encryption**.

How it works internally

- **RSA**: pick two large random primes p and q; the public key is $(n = pq, e)$, the private key is d with $ed \equiv 1 \pmod{(p-1)(q-1)}$. Encryption is $c = m^e \bmod n$, decryption $m = c^d \bmod n$. Breaking it requires factoring n. RSA must always be used with a padding scheme (OAEP for encryption, PSS for signatures); "textbook RSA" is insecure.
- **Elliptic-curve cryptography (ECC)**: the private key is a random integer a; the public key is the point $A = a \cdot G$ obtained by "adding" a fixed base point G on a curve to itself a times. Recovering a from A (the elliptic-curve discrete logarithm problem) is believed to be infeasible. ECC achieves the same security as RSA with much smaller keys and faster operations, which is why TLS 1.3 deployments overwhelmingly use X25519 and P-256.

Table 10.5 — Approximate equivalent security strengths (NIST SP 800-57 Part 1).

Security strength (bits)	Symmetric	RSA modulus	Elliptic curve key	Hash for collision resistance	Status
112	3-key 3DES (legacy)	2048	224	SHA-224	Minimum acceptable today
128	AES-128	3072	256 (P-256, X25519, Ed25519)	SHA-256	Recommended default
192	AES-192	7680	384 (P-384)	SHA-384	High assurance
256	AES-256	15360	512+ (P-521)	SHA-512	Long-term / post-quantum margin for symmetric

"128-bit security" means the best known attack takes about 2^{128} operations. At 10^{18} operations per second (far beyond all computers on Earth combined) that is $3.4 \times 10^{38} / 10^{18} = 3.4 \times 10^{20}$ s, about 10^{13} years.

WARNING / TRADE-OFF

Quantum computers and "harvest now, decrypt later." A large fault-tolerant quantum computer running Shor's algorithm would break RSA and ECC; Grover's algorithm only roughly halves symmetric strength, leaving AES-256 a large margin. No such machine exists as of 2026, but recorded traffic could be decrypted later. NIST published post-quantum standards in August 2024 — FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA) — and TLS libraries have begun deploying hybrid X25519 + ML-KEM key exchange. Design for **crypto agility** (configurable algorithms) and prefer AES-256 for long-lived data.

10.4.4 Key exchange and forward secrecy

First-principles explanation

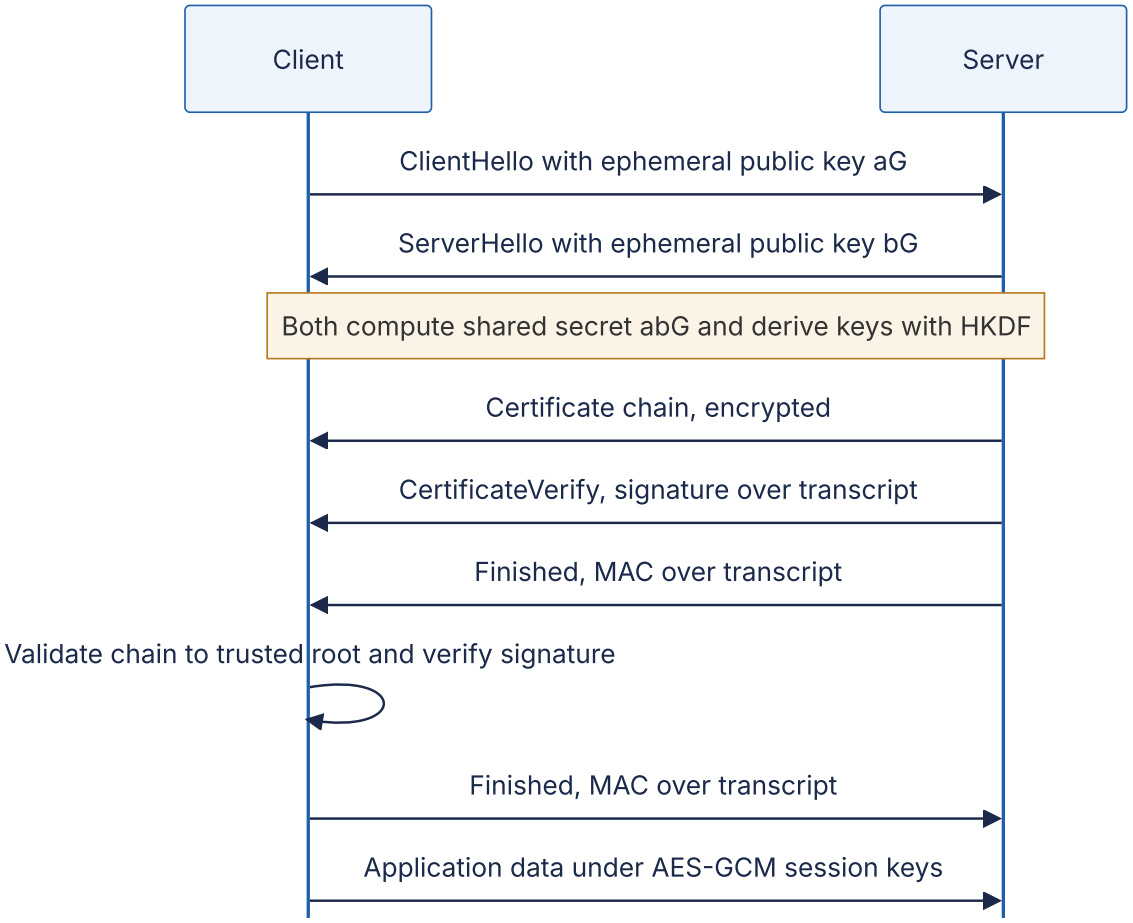
The classic analogy for **Diffie–Hellman key exchange** uses paint. Alice and Bob publicly agree on a common color, yellow. Alice secretly picks red, Bob secretly picks blue. Alice mixes yellow + red and sends the result (orange) across the open channel; Bob mixes yellow + blue and sends green. Alice adds her secret red to Bob's green; Bob adds his secret blue to Alice's orange. Both end up with the same yellow + red + blue brown. An eavesdropper saw yellow, orange and green but cannot "unmix" paint to recover red or blue.

In the elliptic-curve version (ECDH), Alice's secret is a, her public value aG; Bob's are b and bG; both compute the shared point abG. The shared secret is then fed through a **key-derivation function** such as HKDF to produce symmetric session keys.

> DEFINITION
>
> **Definition — Forward secrecy.** A property of a key-exchange protocol whereby compromise of a party's long-term private key does not reveal session keys from past sessions. It is achieved by using fresh **ephemeral** key pairs for every session (the "E" in ECDHE) and deleting them afterward.

Why it matters: before TLS 1.3, many servers used RSA key transport, where the client encrypted the session secret with the server's long-term RSA public key. Anyone who recorded traffic and later stole that one private key could decrypt *years* of recorded sessions. TLS 1.3 removed RSA key transport entirely; every full handshake uses ephemeral (EC)DHE, and the server's long-term key is used only to *sign* the handshake, proving its identity.

Figure 10.2 — Simplified TLS 1.3 handshake: ephemeral key exchange plus signature.



Reading the diagram. All messages flow synchronously over one TCP (or QUIC) connection, and the full handshake completes in one round trip before application data flows. The ephemeral public values travel in the clear; the shared secret never crosses the network. From the ServerHello onward, everything is encrypted with keys derived from that secret, including the certificate (hiding which certificate the server uses from passive observers). The server proves it owns the certificate's private key by **signing** the handshake transcript (CertificateVerify); the client validates the certificate chain (§10.5). The Finished messages are MACs over everything exchanged so far, so a man-in-the-middle who altered any earlier message is detected. Failure points: an expired or untrusted certificate (handshake aborts), a client and server with no common cipher suite, or clock skew on the client breaking validity checks.

10.4.5 Digital signatures

DEFINITION

Definition — Digital signature. A scheme with three algorithms: key generation, **sign** (using the private key, produces a signature over a message) and **verify** (using the public key, checks a signature against a message). A valid signature shows that the message was approved by the holder of the private key and has not changed since. Because verification needs only the public key, *anyone* can verify, which provides non-repudiation.

Signatures are the notary of cryptography. They secure software updates (the update client holds the vendor's public key and refuses unsigned binaries), container images, JWTs signed with RS256 or ES256 (§8.13), certificates themselves, Git commits and the TLS handshake. In practice you sign a *hash* of the message: the signing algorithm hashes internally, so signing a 2 GB file costs one SHA-256 pass plus one fast private-key operation.

Prefer **Ed25519** (fast, deterministic, small 64-byte signatures, few ways to misuse) or **ECDSA P-256** where compliance requires NIST curves. ECDSA is fragile when the per-signature random value repeats or is biased: in 2010 researchers extracted Sony's PlayStation 3 code-signing key because the same random value was reused for every signature. Ed25519 derives this value deterministically, eliminating the failure mode.

10.4.6 Message authentication codes and HMAC

DEFINITION

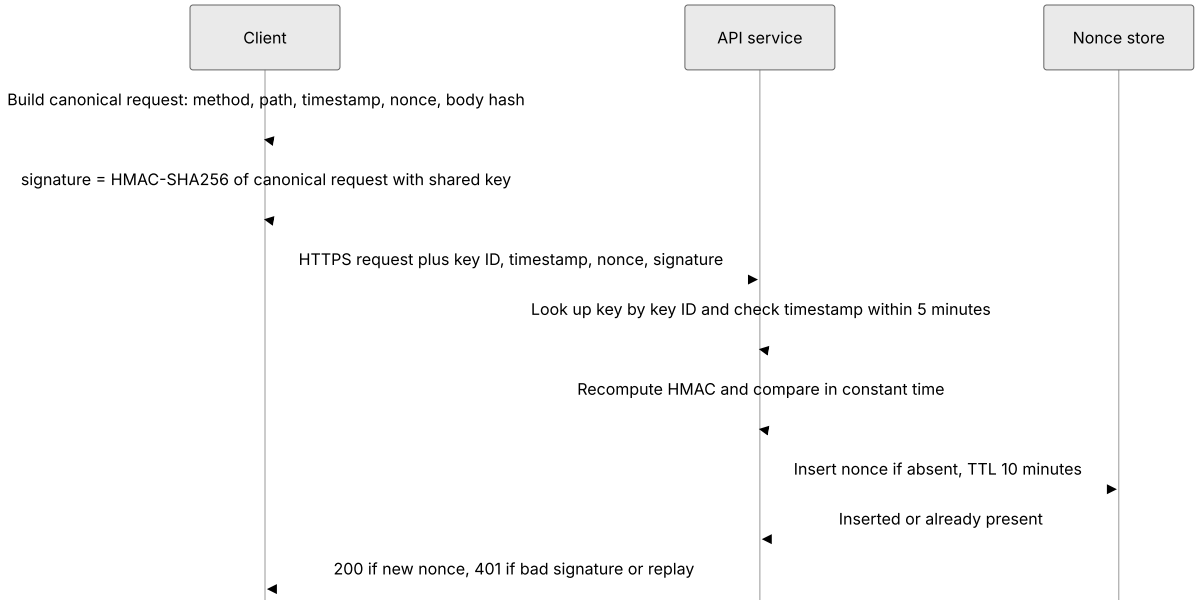
Definition — Message authentication code (MAC). A short tag computed from a message and a secret key, $t = \mathrm{MAC}_K(m)$, such that without K it is infeasible to produce a valid tag for any new message. **HMAC** is the standard construction from a hash function: $\mathrm{HMAC}_K(m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$ [Krawczyk et al. 1997].

A MAC is a wax seal pressed with a ring that both parties own. The receiver, holding the same ring, can tell the letter is genuine and unopened. But because both own the ring, a third party cannot tell *which* of them sealed it — so MACs provide integrity and authenticity between the two key holders, but **not** non-repudiation. That is the essential difference from a signature, and the reason payment providers sign webhooks with an HMAC (they only need you to know it came from them) while software vendors sign releases with asymmetric signatures (millions of verifiers who must not be able to forge).

HMAC's nested structure defeats the length-extension attack described in §10.4.1. It is also fast (two hash computations) and needs only a shared key, which makes it the stan-

dard tool for **request signing**: AWS Signature Version 4, Stripe and GitHub webhook signatures and many internal service-to-service schemes use HMAC-SHA-256.

Figure 10.3 — HMAC request signing with replay protection.



Reading the diagram. The client and API service share a secret key identified by a key ID (so keys can be rotated: during rotation the server accepts both the old and new key IDs). The client signs a **canonical** representation — a byte-exact serialization of every field the server will act on — because if a field is not covered (say, the HTTP method), an attacker can change it without invalidating the signature. The timestamp bounds how long a captured request remains useful; the nonce store (in production a shared cache such as Redis with a TTL slightly longer than twice the allowed skew) rejects exact replays inside that window. The signature comparison must be constant-time to avoid leaking, byte by byte, how much of a forged signature was correct. All calls are synchronous; the nonce store is the only shared state and a single point of failure — if it is unavailable, the service must choose between failing closed (reject requests) and failing open (accept possible replays), a decision that belongs in the design document.

Code example: HMAC request signing

```
        """HMAC-SHA256 request signing and verification with replay protection."""
        import hashlib
        import hmac
        import secrets
        import time

        MAX_SKEW_SECONDS = 300          # accept requests at most 5 minutes old/new

        def canonical_request(method: str, path: str, timestamp: str,
                              nonce: str, body: bytes) -> bytes:
            body_hash = hashlib.sha256(body).hexdigest()
            # Every field the server relies on must be covered by the signature.
            return "\n".join([method.upper(), path, timestamp, nonce,
body_hash]).encode()

        def sign(key: bytes, method: str, path: str, body: bytes) -> dict:
            ts = str(int(time.time()))
            nonce = secrets.token_hex(16)
            mac = hmac.new(key, canonical_request(method, path, ts, nonce, body),
                           hashlib.sha256).hexdigest()
            return {"X-Key-Id": "client-42", "X-Timestamp": ts,
                    "X-Nonce": nonce, "X-Signature": mac}

        class Verifier:
            def __init__(self, keys: dict[str, bytes]):
                self.keys = keys
                self.seen_nonces: dict[str, int] = {}    # production: shared store with
TTL

            def verify(self, method: str, path: str, body: bytes, headers: dict) ->
bool:
                key = self.keys.get(headers.get("X-Key-Id", ""))
                if key is None:
                    return False
                ts = int(headers["X-Timestamp"])
                now = int(time.time())
                if abs(now - ts) > MAX_SKEW_SECONDS:
                    return False                                    # stale or future
                expected = hmac.new(key, canonical_request(method, path, headers["X-
Timestamp"],
                                                           headers["X-Nonce"], body),
                                    hashlib.sha256).hexdigest()
                if not hmac.compare_digest(expected, headers["X-Signature"]):
                    return False                                    # forged or tampered
                # Expire old nonces, then reject a nonce we have already seen (replay).
                self.seen_nonces = {n: t for n, t in self.seen_nonces.items()
                                    if now - t <= 2 * MAX_SKEW_SECONDS}
                if headers["X-Nonce"] in self.seen_nonces:
                    return False
                self.seen_nonces[headers["X-Nonce"]] = ts
                return True
```

Tested with Python 3.13: a correctly signed request verifies; replaying it, altering one byte of the body, or signing with a different key all return `False`.

What to notice:

- `hmac.compare_digest` compares in constant time; `==` returns early at the first differing byte and can leak timing information.
- The body is covered via its SHA-256 hash, and the method and path are included, so neither can be swapped.
- The order of checks matters: the nonce is recorded only *after* the signature verifies, so an attacker cannot fill the nonce store with garbage that blocks legitimate nonces.
- The in-memory `seen_nonces` dictionary works for one process only; with several API replicas, use a shared store with an atomic "set if absent" operation (for example Redis `SET key value NX EX 600`).

10.4.7 Randomness

Every key, nonce, salt, session token and password-reset link depends on unpredictable random numbers. General-purpose generators (`random` in Python, `Math.random()` in JavaScript, `rand()` in C) are designed for simulations: they are fast and statistically uniform but **predictable** — observing a few hundred outputs of Python's Mersenne Twister lets an attacker compute all future outputs. Security code must use a **cryptographically secure pseudo-random number generator (CSPRNG)** seeded from the operating system's entropy pool: `secrets` or `os.urandom` in Python, `crypto.getRandomValues` in browsers, `SecureRandom` in Java, `crypto/rand` in Go.

FORMULA

Entropy of a uniformly random secret

$$H = L \cdot \log_2 N$$

Symbol	Meaning	Units
H	Entropy: the number of bits an attacker must effectively guess	bits
L	Length of the secret	symbols
N	Number of equally likely choices per symbol	symbols

Valid only when every symbol is chosen independently and uniformly at random — true for machine-generated tokens, false for human-chosen passwords.

WORKED EXAMPLE

Worked example — How long should a session token be?
`secrets.token_urlsafe(32)` draws 32 random bytes ($N = 256$, $L = 32$): $H = 32 \times 8 = 256$ bits. An attacker making 10^6 guesses per second for a year (3.15×10^{13} guesses) against 10^8 live sessions succeeds with probability about $3.15 \times 10^{13} \times 10^8 / 2^{256} \approx 3 \times 10^{-56}$.
Engineering implication: 128 bits of CSPRNG output is already ample; the realistic risk is tokens leaking through logs, URLs and referrer headers, not guessing.

10.4.8 Don’t roll your own crypto

“Don’t roll your own crypto” does not only mean “don’t invent a cipher”. It means: do not assemble primitives into your own protocol or storage format when a vetted, high-level construction exists. The canonical failures are all compositions of *correct* primitives: encrypting without authenticating (bit-flipping attacks), MAC-then-encrypt with distinguishable padding errors (padding-oracle attacks against CBC), `SHA256(key || msg)` as a MAC (length extension), reused GCM nonces, comparing MACs with `==` (timing attacks), and ECB mode, which encrypts identical blocks to identical ciphertext so that patterns in an image remain visible. Ferguson, Schneier and Kohno make the case that most cryptographic failures in practice are engineering failures in how primitives are combined and keys are handled, not breaks of the primitives themselves [Ferguson et al. 2010].

RECOMMENDATION

Practical rule set. (1) Use TLS 1.3 for data in transit — do not design a transport protocol. (2) Use a high-level library API (libsodium/NaCl `secretbox`, Google Tink, or the `cryptography` package’s AEAD classes) that chooses nonces, modes and tag checks for you. (3) Use a managed KMS for key storage and envelope encryption (§10.9). (4) Use a password-hashing function, not a hash (§10.6). (5) Use your platform’s JWT/OAuth libraries, pinned to an explicit algorithm allow-list. (6) Have a cryptographer review any design where you still feel you need to combine primitives.

Code example: AEAD, signatures and key agreement

```
        """AES-256-GCM with associated data, Ed25519 signatures and X25519 key agree-
ment."""
        import os
        from cryptography.exceptions import InvalidSignature, InvalidTag
        from cryptography.hazmat.primitives import hashes
        from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
        from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM
        from cryptography.hazmat.primitives.kdf.hkdf import HKDF

        # --- 1. Authenticated symmetric encryption: AES-256-GCM ---
        key = AESGCM.generate_key(bit_length=256)
        aead = AESGCM(key)
        nonce = os.urandom(12)                      # 96-bit nonce; NEVER reuse with
same key
        aad = b"user_id=42;table=payments"          # authenticated but not encrypted
        ct = aead.encrypt(nonce, b"card ending 4242", aad)
        print("ciphertext+tag bytes:", len(ct))     # plaintext length + 16-byte tag
        print("decrypted:", aead.decrypt(nonce, ct, aad))
        try:
            aead.decrypt(nonce, ct, b"user_id=43;table=payments")   # wrong context
        except InvalidTag:
            print("AAD mismatch -> InvalidTag (ciphertext cannot be moved to another
row)")
        tampered = bytearray(ct); tampered[0] ^= 1
        try:
            aead.decrypt(nonce, bytes(tampered), aad)
        except InvalidTag:
            print("bit flip -> InvalidTag")

        # --- 2. Digital signatures: Ed25519 ---
        signing_key = Ed25519PrivateKey.generate()
        verify_key = signing_key.public_key()
        release = b"release v2.4.1 sha256=9f86d08..."
        sig = signing_key.sign(release)
        print("signature bytes:", len(sig))
        verify_key.verify(sig, release)             # raises if invalid
        try:
            verify_key.verify(sig, release + b"!")
        except InvalidSignature:
            print("modified message -> InvalidSignature")

        # --- 3. Key agreement: X25519 + HKDF ---
        alice, bob = X25519PrivateKey.generate(), X25519PrivateKey.generate()
        shared_a = alice.exchange(bob.public_key())  # only public keys cross the net-
work
        shared_b = bob.exchange(alice.public_key())
        assert shared_a == shared_b
        session_key = HKDF(algorithm=hashes.SHA256(), length=32, salt=None,
                           info=b"example-protocol v1 session key").derive(shared_a)
        print("both sides derived the same 256-bit session key:", session_key.hex()
[:16], "...")
```

Tested with `cryptography` 50.0.1: decryption succeeds with the right AAD; a changed AAD or a single flipped ciphertext bit raises `InvalidTag`; a modified release string raises `InvalidSignature`; both X25519 parties derive the same key.

What to notice:

- The 16-byte plaintext became 32 bytes: GCM ciphertext is the same length as the plaintext plus a 16-byte tag. The 12-byte nonce must be stored alongside it (it is not secret).
- Decryption fails *loudly* on any tampering or context mismatch; there is no way to obtain partially decrypted garbage.
- The raw X25519 output is never used directly as a key; HKDF with a protocol-specific `info` string turns it into a uniformly random key bound to its purpose.

Table 10.6 — Hash vs MAC vs signature vs encryption.

	Hash	MAC (HMAC)	Digital signature	AEAD encryption
Confidentiality	No	No	No	Yes
Integrity	Only against a trusted digest	Yes	Yes	Yes
Authenticity	No	Yes, among key holders	Yes, publicly verifiable	Yes, among key holders
Non-repudiation	No	No	Yes	No
Key	None	Shared secret	Private / public pair	Shared secret
Speed	GB/s	GB/s	~10^4–10^5 signatures/s per core (Ed25519)	GB/s with AES-NI
Typical use	Checksums, content addressing, Merkle trees	Webhooks, API request signing, cookies	Software updates, certificates, JWT (RS/ES), audit anchors	Data at rest and in transit

Common misconceptions

COMMON MISTAKE

"Encoding, hashing and encryption are interchangeable." Base64 is an *encoding*: reversible by anyone, provides no security. A hash is one-way and keyless: anyone can compute it, so it cannot keep a low-entropy value (a phone number, a password) secret — an attacker hashes every candidate. Encryption is reversible *with the key*. Kubernetes Secrets stored as base64 in etcd without encryption at rest are, from a security standpoint, plaintext.

10.5 PKI and certificates

DEFINITION

Definition — Public key infrastructure (PKI). The set of roles, policies and systems that bind public keys to identities. A **certificate** (X.509) is a document stating "this public key belongs to this name (for example `api.example.com`), valid from date A to date B", signed by a **certificate authority (CA)**. A client trusts a certificate if it can build a **chain** of valid signatures from it to a **root CA** certificate in its local **trust store**.

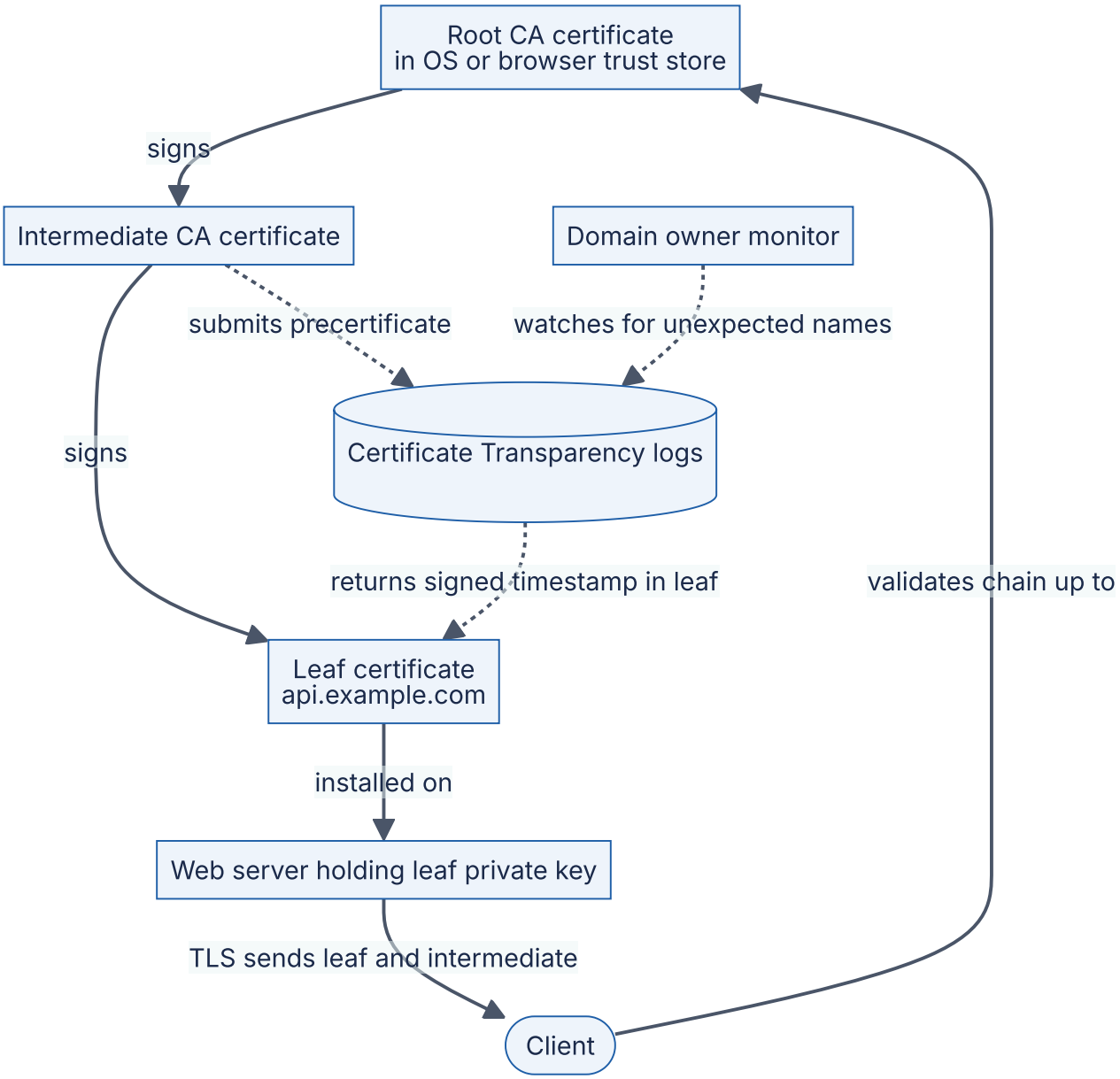
First-principles explanation

Asymmetric cryptography lets anyone lock a box with your padlock — but how does a stranger know the padlock in the bowl is *yours* and not one an impostor swapped in? A passport solves the same problem for people: a government you already trust vouches for the link between a photo and a name. A CA is that government for public keys. Your browser or operating system ships with roughly a hundred-odd root CA certificates (the exact number varies by vendor), and it trusts any certificate that chains back to one of them.

How it works internally

1. The server operator generates a key pair and sends the CA a **certificate signing request** containing the public key and the domain name.
2. The CA verifies control of the domain — for automated CAs, via the **ACME** protocol (RFC 8555): placing a token at a well-known HTTP path or in a DNS TXT record.
3. The CA signs a certificate using an **intermediate CA** key. Root keys are kept offline in hardware security modules and only sign intermediates, so a root key is used a few times per decade.
4. In the TLS handshake the server sends its **leaf** certificate plus intermediates. The client checks each signature up to a trusted root, the validity dates, the domain name, key-usage constraints and revocation status.

Figure 10.4 — Certificate chain of trust and Certificate Transparency.



Reading the diagram. Solid arrows show the signing hierarchy and the synchronous TLS path: the root signs the intermediate, the intermediate signs the leaf, and the server presents leaf plus intermediate to the client, which already holds the root. Dotted arrows show the asynchronous transparency path: before issuance the CA submits a precertificate to public append-only logs and embeds the logs' signed timestamps in the final certificate; browsers that enforce CT reject publicly trusted certificates without them. A domain owner's monitor watches the logs and alerts on any certificate for its names it did not request. Failure points: an expired leaf or intermediate (the most common certificate outage), a server that forgets to send the intermediate (works in some clients, fails in others), and a compromised or misbehaving CA — the reason CT exists.

Revocation — withdrawing a certificate before expiry because its key leaked — works poorly in practice: revocation lists grow large, online status checks (OCSP) add latency and leak browsing history, and clients usually "soft-fail" when the responder is unreach-

able. The industry's answer is shorter lifetimes. Let's Encrypt has issued 90-day certificates since 2015 to force automation, and in 2025 the CA/Browser Forum voted to shrink the maximum lifetime of publicly trusted TLS certificates in stages, reaching 47 days by 2029 (as of 2026; check the current Baseline Requirements).

DEFINITION

Definition — Certificate Transparency (CT). A system of public, append-only, Merkle-tree-based logs into which CAs record every certificate they issue, so that mis-issued certificates become publicly detectable [RFC 6962].

INDUSTRY EXAMPLE

DigiNotar (2011). Attackers compromised the Dutch CA DigiNotar and issued hundreds of fraudulent certificates, including one for `*.google.com` that was used to intercept traffic of users in Iran. Browsers removed DigiNotar from their trust stores and the company went bankrupt within weeks. The incident directly motivated Certificate Transparency: had CT existed, Google's monitors would have seen the rogue certificate within hours of issuance. *Evidence: Secondary* — [Fox-IT 2012]

Certificate rotation for internal services

For service-to-service mTLS (§10.10) you run a private CA (cloud private CA services, HashiCorp Vault's PKI engine, or a service mesh's built-in CA). The design rules: issue **short-lived** certificates (hours to days); rotate automatically well before expiry (renew at about two-thirds of lifetime); have services reload certificates without restarting; alert on any certificate with less than, say, 25% of its lifetime remaining; and treat the CA's private key as the most sensitive key in the company (store it in an HSM or KMS).

Table 10.7 — Certificate failure scenarios.

What goes wrong	Why	Detection	Recovery	Prevention
Expired certificate outage	Manual renewal forgotten; long lifetime meant the process was never exercised	Synthetic TLS probes; expiry metrics	Emergency renewal and deploy	Automated issuance (ACME), short lifetimes, expiry alerts
Private key leak	Key in a repo, image or backup	Secret scanning; CT monitor shows unexpected certs	Revoke, reissue, rotate	Keys generated on host or in KMS/HSM, never copied
Rogue certificate for your domain	Compromised or careless CA	CT log monitoring	Request revocation; report to browser vendors	CAA DNS records restricting which CAs may issue

10.6 Password storage

10.6.1 Why passwords need special treatment

First-principles explanation

Suppose you run a coat check. You could keep each customer's ticket number in a ledger, but if the ledger is stolen, the thief can claim every coat. Better: keep only a "fingerprint" of each ticket, so a stolen ledger is useless — *unless* the tickets are so predictable that the thief can fingerprint every possible ticket and match them. Human passwords are predictable tickets: "Summer2024!" and its millions of cousins appear in every attacker's wordlist.

The design goal therefore is: **assume the password database will one day be stolen** (via SQL injection, a leaked backup or an insider), and make each password as expensive as possible to recover from what was stolen. Three techniques combine to achieve this.

1. **Hash, don't encrypt.** The server never needs the password back; it only needs to check a candidate. A one-way function means there is no decryption key to steal.
2. **Salt.** A **salt** is a unique random value (at least 16 bytes) stored next to each hash and mixed into it. Without salts, identical passwords produce identical hashes, so one computation cracks every user with "123456", and attackers can use precomputed **rainbow tables**. With salts, every guess must be computed separately per user.
3. **Slow, memory-hard hashing.** SHA-256 is designed to be fast; a single high-end GPU computes on the order of 10^{10} SHA-256 hashes per second (order-of-magnitude figure as of 2024). A **password-hashing function** (bcrypt, scrypt, Argon2id) is deliberately slow and, for scrypt and Argon2, **memory-hard**: each guess needs tens of megabytes of RAM, which defeats the massive parallelism of GPUs and custom chips.

Optionally add a **pepper**: a secret key, stored outside the database (in a KMS or HSM), applied as an HMAC before or after hashing. A database-only leak then yields hashes that cannot be attacked at all without also stealing the pepper. NIST SP 800-63B recommends this additional keyed step.

INDUSTRY EXAMPLE

LinkedIn (2012). About 6.5 million LinkedIn password hashes were posted publicly in 2012; they were unsalted SHA-1 hashes, and a large fraction were cracked within days. In 2016 a much larger dataset from the same breach (reported at over 100 million accounts) surfaced. The lesson is both technical (fast, unsalted hashes) and architectural (credential reuse turns one site's breach into credential stuffing against every other site — threat T1 in Table 10.2). *Evidence: Secondary* — widely reported in the technology press, 2012 and 2016.

10.6.2 Algorithms and cost parameters

Table 10.8 — Password-hashing functions compared (parameters per [OWASP Password Storage Cheat Sheet], as of 2026).

	Argon2id	scrypt	bcrypt	PBKDF2-HMAC-SHA-256
What it is	Winner of the 2015 Password Hashing Competition, RFC 9106	Memory-hard KDF, RFC 7914	Blowfish-based adaptive hash (1999)	Iterated HMAC, NIST-approved
Cost knobs	memory m, iterations t, parallelism p	N (CPU/memory), r (block size), p	work factor (cost c ⇒ 2^c rounds)	iteration count
Memory-hard	Yes	Yes	No (4 KiB)	No
OWASP minimum	m = 19 MiB, t = 2, p = 1	$N = 2^{17}$, r = 8, p = 1	cost ≥ 10	≥ 600,000 iterations
Gotchas	Library support varies	Memory = $128 \cdot N \cdot r$ bytes (128 MiB at OWASP minimum)	Truncates input at 72 bytes	GPU-friendly; use only when FIPS-140 is required
Choose when	Default for new systems	Argon2id unavailable	Legacy systems, wide library support	Compliance mandates FIPS

RFC 9106 also gives two general-purpose Argon2id recommendations: $t = 1$, $p = 4$, $m = 2$ GiB where memory allows, or $t = 3$, $p = 4$, $m = 64$ MiB otherwise [RFC 9106]. The OWASP minimums are lower because login servers must handle many concurrent hashes.

Choosing cost parameters is a capacity-planning decision. Pick the most expensive setting that keeps a single hash at roughly 50–500 ms on your production hardware and that your login fleet can afford at peak. If peak login rate is 200 per second and one hash costs 100 ms of one core and 64 MiB, you need about $200 \times 0.1 = 20$ cores and 20×64 MiB ≈ 1.3 GiB of RAM dedicated to hashing — plus headroom, because an attacker can make login attempts to burn your CPU (rate-limit the endpoint). Store the algorithm and parameters with each hash so you can raise them later and **rehash on next successful login**.

Mathematical foundations

FORMULA

Expected offline brute-force time

$$T = \frac{S}{2 \cdot R \cdot G}$$

Symbol	Meaning	Units
T	Expected time to find one password (average: half the space)	seconds
S	Size of the search space: N^L for random passwords, or the size of the attacker's ranked wordlist up to the target	candidates
R	Guesses per second per attacking device for this hash function and cost	guesses/s/device
G	Number of attacking devices (GPUs)	devices

Salting means T is paid **per user**; without salts, one pass covers all users.

WORKED EXAMPLE

Worked example — Fast hash vs slow hash. A user picks a random 8-character password from the 95 printable ASCII characters.

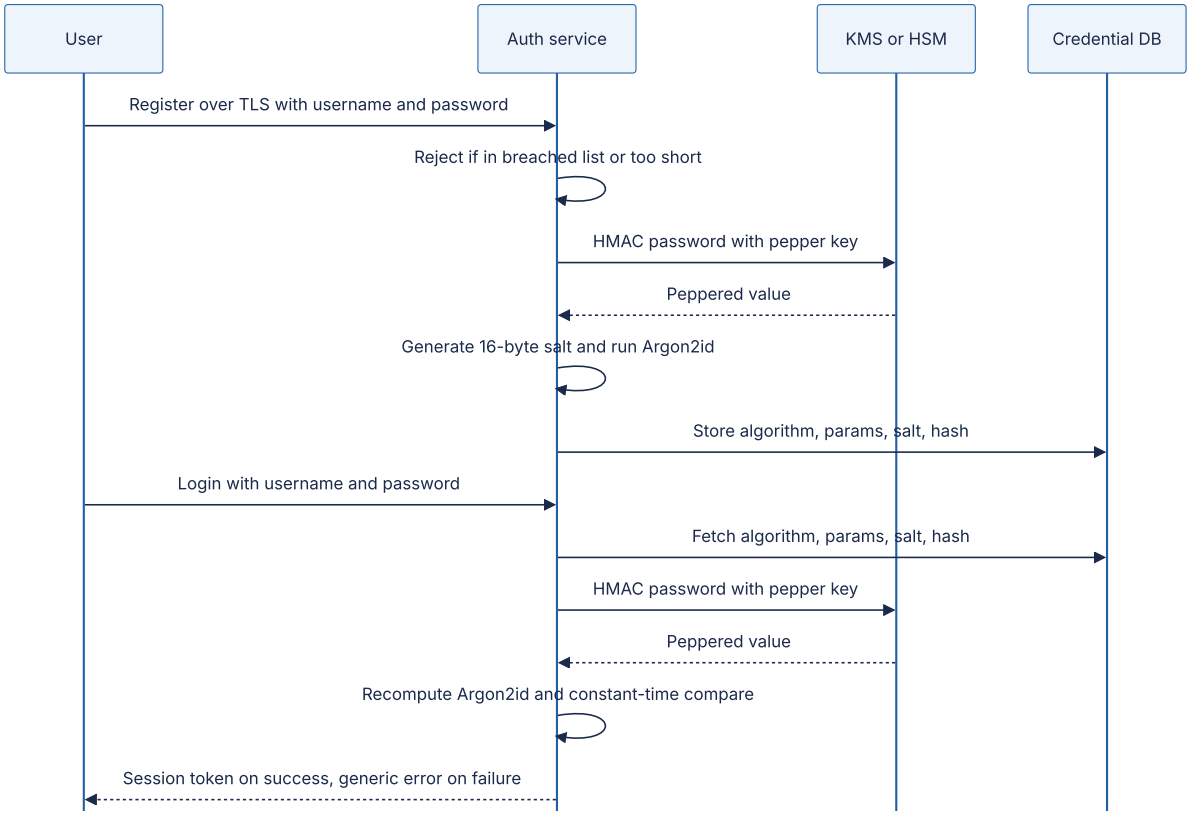
1. Search space: $S = 95^8 \approx 6.63 \times 10^{15}$ (entropy $8 \log_2 95 \approx 52.6$ bits).
2. *Salted SHA-256,* $R \approx 10^{10}$ /s, $G = 10$ GPUs: $T = 6.63 \times 10^{15} / (2 \times 10^{11}) \approx 3.3 \times 10^4$ s $\approx$ **9 hours**.
3. *Argon2id or bcrypt tuned to ~100 ms on a server core*. An attacker's GPU is far less advantaged on memory-hard functions; assume an optimistic $R \approx 10^4$ /s per device: $T = 6.63 \times 10^{15} / (2 \times 10^5) \approx 3.3 \times 10^{10}$ s $\approx$ **1,050 years** per password.
4. A 12-character random password ($S = 95^{12} \approx 5.4 \times 10^{23}$) against the same slow hash: $\approx 2.7 \times 10^{18}$ s, about 8.6×10^{10} years.

Interpretation: the slow hash multiplies the attacker's cost by 10^6; length multiplies it exponentially. *But* human passwords are not random: "Summer2024!" appears near the top of real wordlists, so S for it is perhaps 10^6, and even at 10^4 guesses/s it falls in under a minute. *Engineering implication:* slow hashing protects the middle of the distribution; protecting the weakest passwords requires refusing them at registration (breached-password lists) and adding MFA.

NIST guidance. NIST SP 800-63B, revised as SP 800-63B-4 (final in 2025), reverses several folk rules: do **not** impose composition rules (“one uppercase, one symbol”), do **not** force periodic password changes without evidence of compromise, **do** check new passwords against lists of breached and commonly used passwords, allow at least 64 characters and all printable characters including spaces, and require a minimum length — 15 characters when the password is the only authentication factor, 8 when it is part of multi-factor authentication [NIST SP 800-63B-4].

Architecture diagram

Figure 10.5 — Registration and login with salted Argon2id and a pepper.



Reading the diagram. All calls are synchronous. The password exists in plaintext only in memory of the auth service, briefly. The pepper key never leaves the KMS or HSM; the service sends it data to MAC (some designs instead load the pepper into service memory at startup, trading isolation for one fewer network call per login). The credential database stores only the self-describing record — for example `argon2id$m=19456,t=2,p=1$<salt>$<hash>` — so a stolen database alone is not enough to start guessing. The failure response is deliberately generic (“invalid username or password”) and takes the same time for unknown users to avoid revealing which accounts exist. Scaling: the Argon2id step dominates CPU and memory, so login capacity scales with auth-service replicas; rate limiting per account and per IP sits in front of it.

Code example

```
        """Password hashing with scrypt (standard library) and Argon2id (cryptography >=
44)."""
        import base64
        import hashlib
        import hmac
        import os
        import time

        # ---------- scrypt (stdlib) ----------
        SCRYPT_N, SCRYPT_R, SCRYPT_P = 2**17, 8, 1   # OWASP minimum: N=2^17, r=8, p=1
        SCRYPT_MAXMEM = 256 * 1024 * 1024            # memory needed is ~128*N*r = 128
MiB

        def b64(b: bytes) -> str:
            return base64.b64encode(b).decode().rstrip("=")

        def unb64(s: str) -> bytes:
            return base64.b64decode(s + "=" * (-len(s) % 4))

        def scrypt_hash(password: str) -> str:
            salt = os.urandom(16)                                  # unique per pass-
word
            dk = hashlib.scrypt(password.encode("utf-8"), salt=salt,
                                n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P,
                                maxmem=SCRYPT_MAXMEM, dklen=32)
            # Store algorithm, parameters, salt and hash together so they can evolve.
            return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${b64(salt)}${b64(dk)}"

        def scrypt_verify(password: str, stored: str) -> bool:
            alg, n, r, p, salt_s, dk_s = stored.split("$")
            if alg != "scrypt":
                raise ValueError("unknown algorithm")
            n, r, p = int(n), int(r), int(p)
            candidate = hashlib.scrypt(password.encode("utf-8"), salt=unb64(salt_s),
                                       n=n, r=r, p=p,
                                       maxmem=max(SCRYPT_MAXMEM, 256 * n * r), dklen=32)
            return hmac.compare_digest(candidate, unb64(dk_s))      # constant-time com-
pare

        def needs_rehash(stored: str) -> bool:
            _, n, r, p, *_ = stored.split("$")
            return (int(n), int(r), int(p)) != (SCRYPT_N, SCRYPT_R, SCRYPT_P)

        # ---------- Argon2id (cryptography library) ----------
        from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
        from cryptography.exceptions import InvalidKey

        ARGON_T, ARGON_M_KIB, ARGON_P = 2, 19 * 1024, 1   # OWASP minimum: m=19 MiB,
t=2, p=1

        def argon2id_hash(password: str) -> str:
            salt = os.urandom(16)
            kdf = Argon2id(salt=salt, length=32, iterations=ARGON_T,
                           lanes=ARGON_P, memory_cost=ARGON_M_KIB)
```

```
            tag = kdf.derive(password.encode("utf-8"))
            return f"argon2id${ARGON_M_KIB}${ARGON_T}${ARGON_P}${b64(salt)}${b64(tag)}"

        def argon2id_verify(password: str, stored: str) -> bool:
            alg, m, t, p, salt_s, tag_s = stored.split("$")
            if alg != "argon2id":
                raise ValueError("unknown algorithm")
            kdf = Argon2id(salt=unb64(salt_s), length=32, iterations=int(t),
                           lanes=int(p), memory_cost=int(m))
            try:
                kdf.verify(password.encode("utf-8"), unb64(tag_s))  # constant-time in-
side
                return True
            except InvalidKey:
                return False
```

Tested with Python 3.13 and `cryptography` 50.0.1 on a cloud VM: one scrypt hash at OWASP-minimum settings took about 400 ms, one Argon2id hash about 33 ms; correct passwords verify, a one-letter change fails, and hashing the same password twice gives different strings.

What to notice:

- `hashlib.scrypt` refuses to use more than 32 MiB by default; `maxmem` must be raised for $N = 2^{17}$, $r = 8$ (128 MiB).
- The stored string is self-describing (algorithm, parameters, salt, hash), which is what makes `needs_rehash` and gradual parameter upgrades possible. Production code should prefer the standard PHC string format produced by maintained libraries such as `argon2-cffi`.
- ~33 ms for Argon2id at OWASP-minimum settings is a floor; raise m or t until the cost reaches your latency and capacity budget.

> COMMON MISTAKE
>
> **“Encrypting passwords is safer than hashing them.”** Encryption is reversible, so whoever obtains the key — an attacker with application-server access, a rogue administrator — recovers every password at once. Systems should never need to recover a user’s password; if a product requirement says “email users their password”, the requirement is wrong.

10.7 The OWASP Top 10 in architectural terms

DEFINITION

Definition — OWASP Top 10. A consensus ranking of the most critical web application security risk categories, published by the Open Worldwide Application Security Project from vulnerability data contributed by testing organizations plus a community survey. This section uses the **2021 edition** [OWASP 2021]. A 2025 edition has since been published; it keeps broken access control at #1, folds SSRF into it, and adds categories for software supply-chain failures and mishandling of exceptional conditions, so the architectural controls below remain applicable.

The Top 10 is often taught as a list of coding bugs. For a system designer, each category is better read as a *missing architectural control*: something that should exist once, centrally, rather than being re-implemented correctly by every developer on every endpoint.

Table 10.9 — OWASP Top 10 (2021) mapped to architectural controls.

ID	Category	What goes wrong	Architectural control
A01	Broken Access Control	Users act outside their permissions: IDOR, missing function-level checks, CORS misconfiguration	Deny by default; central policy engine or authorization middleware; object-level checks in the data-access layer; tenant ID derived from the token, never from the request body
A02	Cryptographic Failures	Sensitive data unencrypted or protected with weak/misused crypto	TLS everywhere; KMS-managed keys; AEAD; data classification deciding what is encrypted (§10.9)
A03	Injection	Untrusted data interpreted as code: SQL, NoSQL, OS command, LDAP; cross-site scripting is included here in 2021	Parameterized queries/ORMs; no shell invocation with user input; context-aware output encoding; Content Security Policy
A04	Insecure Design	Missing or ineffective security controls by design, not by implementation	Threat modeling (§10.3); abuse-case requirements; rate limits and business-logic limits designed in
A05	Security Misconfiguration	Default credentials, verbose errors, open buckets, unnecessary features enabled	Infrastructure as code with policy checks; hardened base images; configuration drift detection
A06	Vulnerable and Outdated Components	Known-vulnerable libraries, frameworks, OS packages	Software bill of materials (SBOM); dependency scanning in CI; patch SLAs; automated update PRs
A07	Identification and Authentication Failures	Credential stuffing, weak session handling, missing MFA	Central identity provider; MFA; rate limiting; secure session tokens (§10.4.7)
A08	Software and Data Integrity Failures	Unsigned updates, compromised CI/CD, insecure deserialization	Signed artifacts and provenance; protected build pipeline; no deserialization of untrusted native objects
A09	Security Logging and Monitoring Failures	Breaches go undetected for months	Central, tamper-evident audit logs (§10.12); alerting on security events
A10	Server-Side Request Forgery (SSRF)	Server fetches an attacker-chosen URL, reaching internal services	Egress allow-lists via a dedicated fetch proxy; block metadata and private ranges; IMDSv2-style token-protected metadata

10.7.1 Broken access control (A01)

Broken access control is #1 because authorization is the one control that cannot be bolted on by infrastructure: only the application knows that photo 9001 belongs to user 42. The classic form is the **insecure direct object reference (IDOR)**: `GET /api/photos/9001` returns the photo to anyone authenticated, because the handler checks *that* the user is logged in but not *whether* they own photo 9001.

Architectural patterns that make the correct behavior the default:

- **Deny by default.** A new endpoint without an explicit policy rejects all requests.
- **Centralize decisions.** Route every authorization question through one library or service (a policy engine such as Open Policy Agent, or a relationship-based system modeled on Google's Zanzibar — §8.12), so the rules are written and reviewed once.
- **Push checks to the data layer.** Repository methods take the caller's identity and add `WHERE owner_id = :caller` (or tenant filters) automatically; database row-level security can enforce the same rule underneath.
- **Use unguessable identifiers as defense in depth, not as the control.** Random UUIDs make enumeration harder but are not authorization; IDs leak through URLs, logs and shares.

10.7.2 Injection (A03)

Injection happens when data and code share a channel. A query built as `"SELECT * FROM users WHERE name = '" + name + "'"` lets an attacker supply `' OR '1'='1` and change the query's *structure*. **Parameterized queries** separate the channels: the query text is sent to the database with placeholders, and values are sent separately, so a value can never become syntax.

```
        import sqlite3
        db = sqlite3.connect(":memory:")
        db.execute("CREATE TABLE users (name TEXT, email TEXT)")
        db.execute("INSERT INTO users VALUES ('alice', 'a@example.com')")
        attacker_input = "x' OR '1'='1"
        # Vulnerable (illustrative, never do this): f"... WHERE name =
'{attacker_input}'"
        rows = db.execute("SELECT email FROM users WHERE name = ?",
(attacker_input,)).fetchall()
        print(rows)   # [] -- the input is treated purely as a value
```

What to notice: the `?` placeholder is bound by the driver; no escaping function is needed or involved. The architectural layer of defense is to give the application's database user only the rights it needs (no `DROP`, no access to other schemas), so a missed injection cannot become a full compromise.

10.7.3 Server-side request forgery (A10)

Many features make the server fetch a URL the user supplied: link previews, webhooks, “import from URL”, PDF rendering. If the server can reach internal addresses, the attacker can make it fetch `http://169.254.169.254/` (the cloud instance-metadata endpoint that hands out the instance’s IAM credentials), `http://localhost:8080/admin` or internal databases’ HTTP interfaces.

> INDUSTRY EXAMPLE
>
> **Capital One (2019).** An attacker exploited a misconfigured web application firewall to make requests to the AWS instance-metadata service, obtained temporary credentials for the instance’s IAM role, and used that role’s overly broad permissions to copy data on roughly 100 million credit-card applicants from S3. Two architectural failures compounded: an SSRF path to metadata, and a role with far more S3 access than the WAF needed. Later that year AWS introduced IMDSv2, which requires a session token obtained via an HTTP `PUT`, defeating most simple SSRF requests. *Evidence: Secondary* — public court filings and widespread reporting; [AWS Docs: IMDSv2].

Defense in depth for SSRF: (1) route all user-driven outbound fetches through a dedicated **egress proxy** in its own network segment with no route to internal ranges; (2) validate URLs (scheme, port, resolved IP) and **pin** the validated IP for the connection, otherwise **DNS rebinding** — a hostname that resolves to a public IP at check time and a private IP at fetch time — bypasses the check; (3) disable redirects or re-validate each hop; (4) require IMDSv2 and give instance roles least privilege.

```
        """Defense-in-depth check for user-supplied URLs fetched by a server (SSRF)."""
        import ipaddress
        import socket
        from urllib.parse import urlsplit

        ALLOWED_SCHEMES = {"https"}
        ALLOWED_PORTS = {443}

        def resolve_and_check(url: str) -> list[str]:
            parts = urlsplit(url)
            if parts.scheme not in ALLOWED_SCHEMES:
                raise ValueError(f"scheme {parts.scheme!r} not allowed")
            port = parts.port or 443
            if port not in ALLOWED_PORTS or not parts.hostname:
                raise ValueError("port or host not allowed")
            infos = socket.getaddrinfo(parts.hostname, port, proto=socket.IPPROTO_TCP)
            addrs = sorted({info[4][0] for info in infos})
            for a in addrs:
                ip = ipaddress.ip_address(a)
                if not ip.is_global:   # rejects private, loopback, link-local
(169.254.169.254), etc.
                    raise ValueError(f"{parts.hostname} resolves to non-public {ip}")
            # Caller must connect to one of THESE addresses (pin them) to avoid DNS re-
binding.
            return addrs
```

Tested results: `http://example.com/` is denied (scheme), `https://127.0.0.1/admin`, `https://169.254.169.254/latest/meta-data/`, `https://10.0.0.5:443/` and `https://[::1]/` are denied (non-public address), and `https://localhost:8443/` is denied (port). What to notice: the check uses *every* resolved address, not just the first, and `ip.is_global` covers private, loopback, link-local and other reserved ranges for both IPv4 and IPv6. Application-level validation is the inner layer; the network-level egress proxy is the layer that still holds when a developer forgets to call this function.

> COMMON MISTAKE
>
> **“A WAF fixes the OWASP Top 10.”** A web application firewall blocks known attack *patterns* at the edge and buys time for patching. It cannot know who owns photo 9001 (A01), cannot fix insecure design (A04), and can itself be a component with an SSRF flaw, as the Capital One case shows. Treat the WAF as one layer, not the control.

10.8 Secrets management

DEFINITION

Definition — Secret. Any value that grants access and must be kept confidential: database passwords, API keys, OAuth client secrets, TLS and signing private keys, encryption keys, webhook HMAC keys. **Secrets management** is the practice of storing, distributing, rotating, auditing and revoking secrets so that they are available to the workloads that need them and to nobody else.

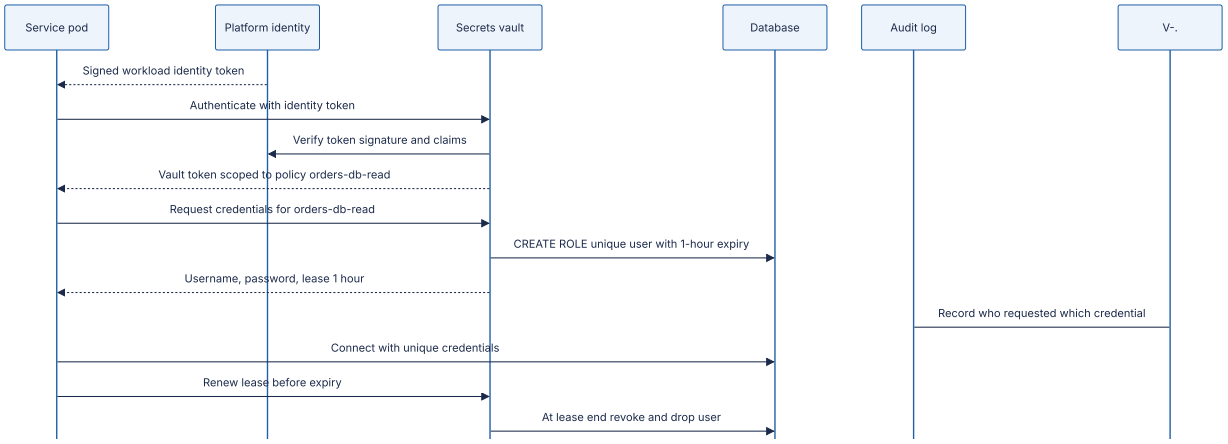
First-principles explanation

In an office, physical keys drift: copies are cut for contractors, left in drawers, never returned. Eventually nobody knows who can open the server room. Software secrets drift the same way and faster: a database password gets pasted into a configuration file, committed to Git, baked into a container image, printed in a debug log, copied into a CI variable and a teammate's laptop. Each copy is a place it can leak from, and a secret that has existed unchanged for three years has had three years to leak.

The progression of maturity is:

1. **Hard-coded** in source — anyone with repository access has production access, forever (Git history keeps it).
2. **Environment variables / config files** injected at deploy time — out of source, but static, widely copied, visible in crash dumps.
3. **Central vault** — a secrets manager (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault) stores secrets encrypted, releases them only to authenticated workloads, logs every read and supports rotation.
4. **Dynamic, short-lived credentials** — the vault *creates* a unique credential per workload on demand with a lease (say one hour) and revokes it at expiry.
5. **Identity-based ("secretless")** — workloads authenticate with a platform-attested identity (cloud IAM role, Kubernetes service-account token, SPIFFE ID) and receive short-lived tokens; there is no long-lived secret to leak.

Figure 10.6 — A workload obtaining dynamic database credentials from a vault.



Reading the diagram. The pod starts with no secret at all: the platform (for example Kubernetes or a cloud provider) gives it a signed identity token describing which service it is. The vault verifies that token and maps the identity to a policy. Instead of returning a shared password, the vault connects to the database with its own administrative credential and creates a unique, time-limited database user. The workload receives credentials it alone holds; the vault records the issuance asynchronously in an audit log (dotted arrow). Leases are renewed while the pod lives; when the pod dies the lease expires and the vault drops the user. Failure points: the vault is now a critical dependency — if it is unavailable, new pods cannot get credentials, so vaults are run as replicated clusters (Vault uses Raft consensus, §26.7) and clients cache credentials for their lease. Scaling: one credential per pod means the database must tolerate many distinct users; for very large fleets, issue per-service rather than per-pod credentials.

FORMULA

Credential misuse window

$$W = \begin{cases} T_{\text{detect}} + T_{\text{rotate}} & \text{long-lived credential} \\ \min\left(T_{\text{TTL}},\ T_{\text{detect}} + T_{\text{revoke}}\right) & \text{short-lived credential} \end{cases}$$

Symbol	Meaning	Units
W	Time during which a leaked credential remains usable	hours
T_{detect}	Time from leak to detection	hours
$T_{\text{rotate}}, T_{\text{revoke}}$	Time to rotate or revoke after detection	hours
T_{TTL}	Lifetime of a short-lived credential	hours

WORKED EXAMPLE

Worked example — Static key vs one-hour credential. Industry breach studies commonly report detection times measured in months; assume $T_{\text{detect}} = 200$ days $= 4,800$ h and $T_{\text{rotate}} = 24$ h. A static API key leaked to a public repository is usable for $W = 4,824$ h. A one-hour dynamic credential leaked in the same way is usable for at most $W = \min(1, 4,824) = 1$ h. *Interpretation:* short lifetimes shrink the window by more than three orders of magnitude *without* requiring detection to improve. *Engineering implication:* invest first in making credentials short-lived and automatically renewed; detection and response remain necessary for the vault and identity provider themselves.

Rotation must be designed, not improvised. A safe rotation for a static secret uses **two valid versions**: (1) create the new secret and register it with the receiving side, which now accepts both; (2) roll out the new secret to all callers; (3) verify no traffic uses the old one (metrics per key ID); (4) revoke the old one. This is why key IDs appear in the HMAC design of Figure 10.3. Complement rotation with **secret scanning** in repositories and CI, pre-commit hooks, and an incident runbook that answers "this key leaked — what do we do in the next 15 minutes?"

WARNING / TRADE-OFF

Kubernetes Secrets are not encrypted by default. They are base64-encoded objects in etcd; anyone with etcd access or broad RBAC `get secrets` permission can read them. Enable encryption at rest for etcd with a KMS provider, restrict RBAC, and prefer mounting secrets from an external vault (§42.11).

10.9 Encryption at rest and in transit, KMS and envelope encryption

10.9.1 In transit and at rest

Encryption in transit protects data moving over a network from eavesdropping and tampering. The rule is simple: TLS 1.2 or later (prefer 1.3) on *every* hop, including internal ones — between load balancer and service, service and database, service and cache. "The internal network is trusted" is exactly the perimeter assumption that zero trust (§10.10) rejects.

Encryption at rest protects stored data from someone who obtains the storage medium or a copy of it. It is applied at different layers, and the layer determines *which* attacker it stops.

Table 10.10 — Layers of encryption at rest and what each protects against.

Layer	Example	Protects against	Does **not** protect against	Cost
Full-disk / volume	Cloud volume encryption, LUKS	Stolen disks, discarded hardware	Anyone who can query the running system	Near-zero; usually default
Storage service	S3 SSE-KMS, managed DB storage encryption	Above, plus snapshots/backups copied without key permission	Application or DB credentials compromise	Low; KMS call costs
Database (TDE)	Transparent data encryption	Stolen DB files and backups	SQL injection, compromised DB user	Low
Application / field-level	App encrypts SSN column with a KMS-managed key before writing	DB admins, SQL injection reading raw columns, backup leaks	Compromise of the application with decrypt permission	Higher: no querying on ciphertext, key plumbing
Client-side / end-to-end	Messaging apps, client-encrypted backups	The service operator itself	Compromised client device	Highest: server cannot read or index data

The key insight: lower layers are cheap and defeat *physical* and *backup* threats; higher layers are expensive and defeat *logical* threats from people and processes with access to the running system. Most designs use volume or service encryption everywhere, and field-level encryption for a small set of highly sensitive fields (government IDs, health data, payment details).

10.9.2 Key management services and envelope encryption

DEFINITION

Definition — Key management service (KMS). A managed service, backed by hardware security modules (HSMs), that generates and stores **key-encryption keys (KEKs)** that never leave it in plaintext, performs cryptographic operations with them on request, enforces access policies on every use and logs every call. Examples: AWS KMS, Google Cloud KMS, Azure Key Vault, HashiCorp Vault's Transit engine.

DEFINITION

Definition — Envelope encryption. Encrypting data with a **data-encryption key (DEK)** generated for that data, then encrypting ("wrapping") the DEK with a KEK held in a KMS, and storing the wrapped DEK alongside the ciphertext [AWS Docs: KMS concepts].

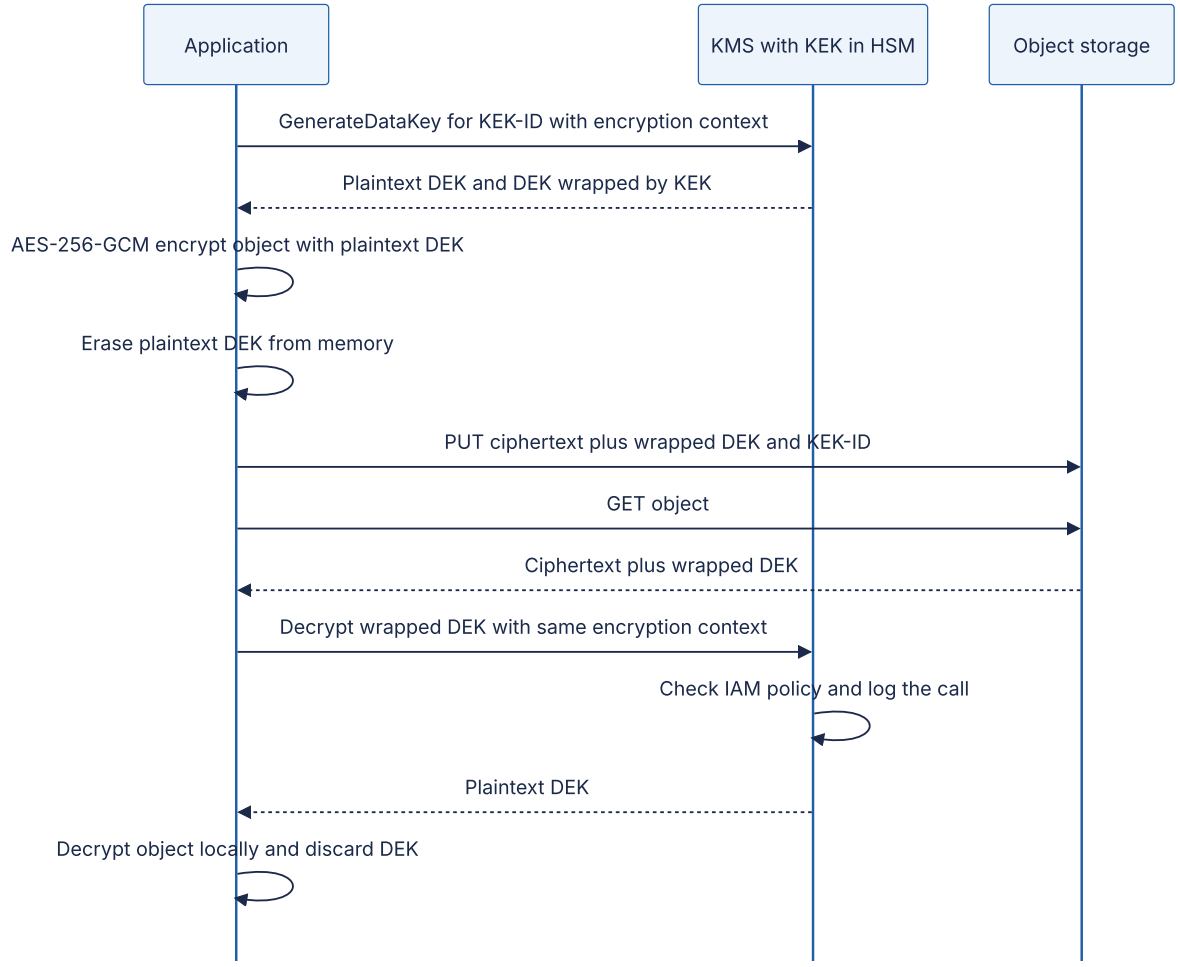
First-principles explanation

A hotel does not give each guest the master key. Each room has its own key (the DEK); the room keys are kept in a safe at the front desk, and only the manager has the key to the safe (the KEK in the KMS). Losing one room key affects one room; changing the safe's lock does not require changing every room's lock.

Why does this exist?

Three problems with sending all data to the KMS directly: (1) **size** — KMS encrypt APIs accept only small payloads (AWS KMS `Encrypt` takes at most 4,096 bytes of plaintext); (2) **latency and throughput** — every encryption would cost a network round trip and count against request quotas; (3) **blast radius** — one key encrypting everything means one leak exposes everything. Envelope encryption sends only 32-byte keys to the KMS and encrypts the bulk data locally at memory speed.

Figure 10.7 — Envelope encryption: write and read paths.



Reading the diagram. All calls are synchronous. On the write path the application asks the KMS for a fresh DEK; the KMS returns it twice — in plaintext for immediate use, and wrapped under the KEK for storage. The object is encrypted locally and stored with its

wrapped DEK, so the storage system holds only ciphertext. On the read path the application sends the small wrapped DEK to the KMS; the KMS enforces the access policy *at decrypt time* and writes an audit record, which means “who read this data” becomes visible in KMS logs even if storage access logging is off. The **encryption context** (non-secret key–value pairs such as tenant and object name) is bound to the wrapped DEK as associated data; a wrapped DEK copied onto another tenant’s object fails to unwrap. Failure points: KMS unavailability blocks all reads and writes of new objects (mitigate with regional KMS keys, retries and, carefully, short-lived DEK caching); a deleted KEK makes all data under it unrecoverable (KMS services enforce deletion waiting periods for this reason). Scaling: KMS calls grow with the number of objects, not their size.

Code example: envelope encryption with a simulated KMS

```
        """Envelope encryption with a simulated KMS (the KEK never leaves the 'KMS')."""
        import json
        import os
        from dataclasses import dataclass
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM

        class FakeKMS:
            """Stands in for AWS KMS / Cloud KMS / Vault Transit. Holds KEKs inter-
nally."""
            def __init__(self):
                self._keks = {"kek-2026-10": AESGCM.generate_key(bit_length=256)}
                self.current = "kek-2026-10"

            def generate_data_key(self, context: dict) -> tuple[bytes, dict]:
                dek = AESGCM.generate_key(bit_length=256)            # fresh DEK per ob-
ject
                nonce = os.urandom(12)
                aad = json.dumps(context, sort_keys=True).encode()   # "encryption con-
text"
                wrapped = AESGCM(self._keks[self.current]).encrypt(nonce, dek, aad)
                return dek, {"kek_id": self.current, "nonce": nonce.hex(), "wrapped":
wrapped.hex()}

            def decrypt_data_key(self, blob: dict, context: dict) -> bytes:
                aad = json.dumps(context, sort_keys=True).encode()
                return AESGCM(self._keks[blob["kek_id"]]).decrypt(
                    bytes.fromhex(blob["nonce"]), bytes.fromhex(blob["wrapped"]), aad)

            def rotate(self) -> None:
                new_id = f"kek-{len(self._keks) + 1}"
                self._keks[new_id] = AESGCM.generate_key(bit_length=256)
                self.current = new_id                                 # old KEKs kept
for decrypt

            def rewrap(self, blob: dict, context: dict) -> dict:
                """Re-encrypt a DEK under the current KEK without touching the data."""
                dek = self.decrypt_data_key(blob, context)
                nonce = os.urandom(12)
                aad = json.dumps(context, sort_keys=True).encode()
                return {"kek_id": self.current, "nonce": nonce.hex(),
                        "wrapped": AESGCM(self._keks[self.current]).encrypt(nonce, dek,
aad).hex()}

        @dataclass
        class EncryptedObject:
            wrapped_dek: dict
            nonce: bytes
            ciphertext: bytes

        def encrypt_object(kms: FakeKMS, plaintext: bytes, context: dict) ->
EncryptedObject:
            dek, wrapped = kms.generate_data_key(context)             # 1 KMS call per
object
            nonce = os.urandom(12)
```

```
            ct = AESGCM(dek).encrypt(nonce, plaintext, None)          # bulk data en-
crypted locally
            del dek                                                   # drop plaintext
DEK ASAP
            return EncryptedObject(wrapped, nonce, ct)

        def decrypt_object(kms: FakeKMS, obj: EncryptedObject, context: dict) -> bytes:
            dek = kms.decrypt_data_key(obj.wrapped_dek, context)
            return AESGCM(dek).decrypt(obj.nonce, obj.ciphertext, None)

        if __name__ == "__main__":
            kms = FakeKMS()
            ctx = {"tenant": "acme", "object": "invoices/2026-10.pdf"}
            data = os.urandom(5_000_000)                              # 5 MB document
            obj = encrypt_object(kms, data, ctx)
            assert decrypt_object(kms, obj, ctx) == data
            print("round trip OK; wrapped DEK is",
len(bytes.fromhex(obj.wrapped_dek["wrapped"])), "bytes")

            kms.rotate()
            obj.wrapped_dek = kms.rewrap(obj.wrapped_dek, ctx)        # rotate KEK: re-
wrap ~60 bytes,
            assert decrypt_object(kms, obj, ctx) == data              # not re-encrypt 5
MB
            print("after KEK rotation, DEK now wrapped by", obj.wrapped_dek["kek_id"])

            try:
                decrypt_object(kms, obj, {"tenant": "evil", "object": "invoices/2026-
10.pdf"})
            except Exception as e:
                print("wrong encryption context ->", type(e).__name__)
```

Output (tested):

```
round trip OK; wrapped DEK is 48 bytes
after KEK rotation, DEK now wrapped by kek-2
wrong encryption context -> InvalidTag
```

What to notice:

- **Rotation is cheap.** Rotating the KEK means rewrapping a 48-byte wrapped DEK per object, not re-encrypting 5 MB of data. Managed KMS services go further: automatic rotation keeps old key versions for decryption, so existing wrapped DEKs remain valid without any rewrap.
- **Crypto-shredding.** Deleting a tenant's KEK (or an object's DEK) renders the data permanently unreadable, including every backup copy — a practical way to honor deletion requests in systems with immutable backups.
- **Context binding.** The encryption context is authenticated, so a confused-deputy bug that passes the wrong tenant fails closed.

- `del dek` only removes the Python reference; Python cannot guarantee the bytes are wiped from memory. Production envelope encryption should use a maintained SDK (for example the AWS Encryption SDK or Google Tink), which also defines a standard on-disk message format.

FORMULA

KMS request rate under envelope encryption

$$Q_{\text{KMS}} = \frac{\lambda_{\text{obj}}}{B}$$

Symbol	Meaning	Units
Q_{KMS}	KMS requests generated	requests/s
λ_{obj}	Rate of objects (or records) encrypted or decrypted	objects/s
B	Objects protected per KMS call (1 for a DEK per object; larger with DEK caching or per-batch keys)	objects/request

WORKED EXAMPLE

Worked example — Sizing KMS usage. An ingestion service writes $\lambda = 20,000$ records per second. With one DEK per record ($B = 1$), it would send 20,000 KMS requests per second — likely above a default per-account quota (AWS KMS cryptographic-operation quotas are in the thousands to tens of thousands of requests per second depending on region and key type, as of 2026) and costly at roughly $0.03 per 10,000 requests: $20,000 \times 86,400/10,000 \times \$0.03 \approx \$5,184$ per day. If each of 40 ingestion nodes instead generates a DEK once per minute and uses it for a batch of records (storing the wrapped DEK once per batch), then $Q = 40/60 \approx 0.67$ requests per second, about $0.17 per day. *Interpretation:* DEK granularity is a trade-off between cost/throughput and blast radius: a per-minute DEK exposes at most one minute of one node's records if leaked from memory. *Engineering implication:* choose per-object DEKs for large or individually deletable objects, and time- or count-bounded cached DEKs for high-rate small records — and keep under the GCM per-key message limit from §10.4.2.

10.10 Network security, zero trust and mTLS

10.10.1 From the castle to zero trust

The traditional **perimeter model** is a castle with a moat: a firewall at the edge, a VPN for remote workers, and implicit trust for anything inside. Its failure mode is total. Once an attacker is inside — through a phished laptop, a vulnerable VPN appliance or a compromised internal service — they can reach everything, a technique called **lateral movement**. Most large breaches are a small initial foothold followed by weeks of lateral movement.

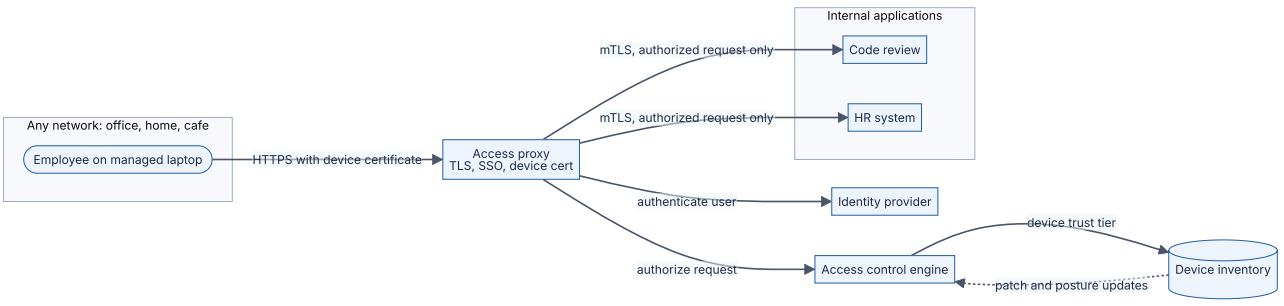
DEFINITION

Definition — Zero trust. A security model in which network location confers no trust: every request is authenticated, authorized and encrypted based on the identity of the user, the device and the workload, plus contextual signals, regardless of where it originates. NIST SP 800-207 describes it as moving defenses "from static, network-based perimeters to focus on users, assets, and resources" [NIST SP 800-207].

INDUSTRY EXAMPLE

Google BeyondCorp. After the 2009 "Operation Aurora" intrusions, Google began moving its corporate applications off the privileged internal network. In BeyondCorp, as described by Ward and Beyer, employee devices sit on an *unprivileged* network even inside Google offices; every request to an internal application goes through an internet-facing **access proxy**; access decisions are made per request by an **access control engine** using the user's identity and group membership, a **device inventory** with per-device certificates, and a computed **trust tier** for the device (patch level, management state). Remote and in-office access become identical, and the VPN is retired. Follow-up papers describe the multi-year migration and the access-proxy design. *Evidence: Primary* — [Ward & Beyer 2014]; [Osborn et al. 2016]

Figure 10.8 — Zero-trust access in the style of BeyondCorp.



Reading the diagram. The employee's location is irrelevant: the office network has no special privileges. Every request reaches the access proxy over HTTPS, presenting a device certificate (identifying the machine) and a single-sign-on session (identifying the person). The proxy asks the identity provider to authenticate the user and the access control engine to authorize *this request to this application*, using group membership and the device's current trust tier from the inventory (fed asynchronously by device-management data). Only authorized requests are forwarded, over mTLS, to applications that accept connections from the proxy alone. Failure points: the proxy and access engine become critical, highly available infrastructure; an over-permissive policy reproduces the old flat network. Scaling: the proxy is a stateless, horizontally scaled L7 tier (§19.3).

10.10.2 Mutual TLS and workload identity

Ordinary TLS authenticates only the server. **Mutual TLS (mTLS)** has *both* sides present certificates and prove possession of the private key, so a service knows cryptographically which workload is calling it — not merely which IP address, which can be spoofed or reused. In practice mTLS between services is provided by a **service mesh** sidecar or proxy (§34.10) that obtains short-lived certificates automatically. The **SPIFFE** standard defines a portable workload identity such as `spiffe://example.org/ns/payments/sa/api` carried in an X.509 certificate (an SVID), so authorization policies can be written as "only `orders` may call `payments` `POST /charge`", independent of IP addresses.

10.10.3 Least privilege and segmentation

Least privilege grants every user, service and credential the minimum permissions needed for its task, for the minimum time [Adkins et al. 2020]. In practice: one IAM role per service, not a shared "app" role; read-only database users for read-only services; no standing production admin access (engineers request time-boxed, approved, logged elevation — "just-in-time access"); and periodic review that removes unused permissions (cloud providers report "last used" data for this).

Segmentation limits blast radius at the network layer: separate VPCs or accounts for production and non-production; subnets for edge, application and data tiers with security groups allowing only the required flows (the boundaries in Figure 10.1); Kubernetes NetworkPolicies that default-deny pod-to-pod traffic; and dedicated egress paths. **Microsegmentation** pushes this down to per-workload policies. Zero trust does not abolish network controls — it stops *relying* on them. Defense in depth means an attacker must defeat identity, authorization, network policy and monitoring, not just one.

Table 10.11 — Perimeter model vs zero trust.

	Perimeter ("castle and moat")	Zero trust
What it is	Trust based on network location	Trust based on verified identity and context per request
How it works	Edge firewall, VPN, flat internal network	Identity-aware proxies, mTLS, per-request policy
Advantages	Simple; works with legacy apps	Limits lateral movement; uniform remote access
Disadvantages	One breach exposes everything	Requires strong identity, device inventory, mature tooling
Operational complexity	Low initially, high after incidents	High upfront; policy management is ongoing
Choose when	Never as the only control	Default target architecture for new systems

10.11 DDoS basics

DEFINITION

Definition — Distributed denial of service (DDoS). An attack that makes a service unavailable by overwhelming some finite resource — network bandwidth, packet-processing capacity, connection-state tables, or application CPU, memory and database capacity — with traffic from many sources at once, typically a **botnet** of compromised devices or abused open servers.

First-principles explanation

Imagine a restaurant. A volumetric attack is a crowd so large it blocks the street, so no customer reaches the door. A protocol attack is a crowd that walks in, occupies every table, and orders nothing. An application-layer attack is a crowd of well-dressed customers each ordering the single dish that takes the kitchen an hour to cook. Each needs a different defense, applied at a different place.

Table 10.12 — DDoS attack classes.

	Volumetric	Protocol (state exhaustion)	Application layer (L7)
Target resource	Link bandwidth	Connection tables in fire-walls, load balancers, servers	Application CPU, DB, cache, downstream APIs
Examples	UDP floods; reflection/amplification via DNS, NTP, memcached	SYN floods, fragmented-packet floods	HTTP floods on expensive endpoints; HTTP/2 Rapid Reset
Measured in	Bits per second (Gbps–Tbps)	Packets per second	Requests per second
Looks like	Obviously abnormal traffic	Half-open connections	Legitimate-looking requests
Primary defense	Upstream capacity: any-cast scrubbing networks, CDN	SYN cookies, stateless edge proxies, scrubbing	WAF rules, rate limits, bot detection, caching, au-toscaling, load shedding

Reflection and amplification are what make volumetric attacks cheap. The attacker sends small UDP requests to thousands of open servers with the *victim's* IP address forged as the source; each server sends its (much larger) reply to the victim.

FORMULA

Amplified attack bandwidth

$$B_{\text{victim}} = B_{\text{attacker}} \times A$$

Symbol	Meaning	Units
B_{victim}	Traffic arriving at the victim	bits/s
B_{attacker}	Spoofed request traffic the at-tacker sends	bits/s
A	Bandwidth amplification factor of the abused protocol (response bytes / request bytes)	dimensionless

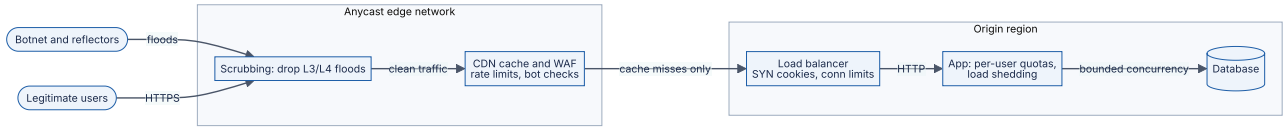
WORKED EXAMPLE

Worked example — DNS amplification. US-CERT lists DNS amplification factors of roughly 28–54 [US-CERT TA14-017A]. An attacker with 2 Gbps of spoofing capacity and $A = 50$ produces $B = 2 \times 50 = 100$ Gbps at the victim. A single origin data center with a 10 Gbps uplink is saturated tenfold; nothing the origin does locally helps, because the link is full before packets reach any server. *Engineering implication:* volumetric defense must happen **upstream**, in networks whose capacity exceeds the attack — anycast CDNs and scrubbing services that advertise your IP prefixes from many locations and absorb the flood across their global capacity.

INDUSTRY EXAMPLE

GitHub memcached attack (2018) and HTTP/2 Rapid Reset (2023). On 28 February 2018 GitHub received a 1.35 Tbps memcached-reflection attack; memcached servers exposed on UDP can amplify requests by factors in the tens of thousands. GitHub routed traffic to its DDoS-mitigation provider and the attack was mitigated within about ten minutes. In 2023 the “HTTP/2 Rapid Reset” technique (CVE-2023-44487) — opening and immediately cancelling streams on a single connection — produced application-layer floods that Google reported peaking at about 398 million requests per second. *Evidence: Primary* — [GitHub 2018]; [Google Cloud 2023]

Figure 10.9 — DDoS mitigation in layers.



Reading the diagram. Traffic flows left to right through successively narrower and more expensive layers. The anycast edge, spread across many locations, absorbs volumetric and protocol floods (the scrubbing stage) because its aggregate capacity is far larger than any single origin’s. The CDN serves cacheable content without touching the origin and applies L7 controls: WAF rules, per-IP and per-session rate limits, and bot challenges. The origin should accept traffic only from the edge (allow-list the CDN’s ranges or use authenticated origin pulls), otherwise attackers bypass the edge by hitting the origin IP directly. Inside, the load balancer defends connection state, and the application enforces per-user quotas and sheds load (§22.9) so that the database — the scarcest resource — sees bounded concurrency. Scaling: autoscaling helps against L7 floods only up to your budget; an attacker’s marginal cost per request is usually lower than yours, so the main lever is making requests cheap (caching) or refusing them early.

RECOMMENDATION

Design rules. Put every public endpoint behind an anycast CDN or DDoS-protection service; hide origin IPs; cache aggressively; make expensive endpoints (search, report generation, login) authenticated and rate-limited; set timeouts and bounded queues everywhere (§22.11); know in advance who you call and what you toggle during an attack (a runbook, rehearsed).

10.12 Audit logging and tamper evidence

DEFINITION

Definition — Audit log. An append-only record of security-relevant events — who (actor), did what (action), to which resource, when, from where, and with what result — retained so that actions can be investigated and attributed. A log is **tamper-evident** if any modification, deletion or reordering of past entries can be detected.

First-principles explanation

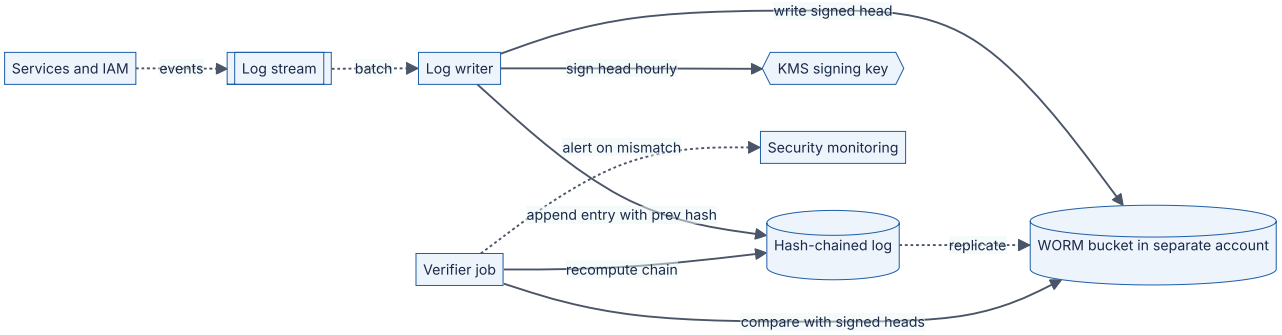
An audit log is a bank's bound, page-numbered ledger written in ink: you can add lines but cannot tear out a page or erase a line without it being obvious. Application debug logs are the opposite — a whiteboard. Attackers who gain privileged access routinely delete or edit logs to hide their tracks, and malicious insiders are precisely the people with access to logs. So audit logs must be designed so that the people being audited cannot rewrite them.

What to log. Authentication events (success, failure, MFA changes), authorization failures, privilege and role changes, access to sensitive data (reads, exports), administrative and configuration changes, key and secret usage (KMS logs this for you), and security-control changes (disabling logging is itself an event). Each entry: timestamp (UTC, synchronized clock), actor identity (user and workload), source (IP, device), action, resource, result, and a request or trace ID. **What not to log:** passwords, session tokens, full card numbers, secret values, or more personal data than necessary — the log becomes a sensitive data store itself.

How tamper evidence works. Each entry includes the hash of the previous entry, forming a **hash chain**: $h_i = H(h_{i-1} \parallel e_i)$. Changing any past entry changes its hash and breaks every link after it. A chain alone is not enough — whoever can write the log can recompute the whole chain — so the latest hash (the **head**) is periodically **anchored** somewhere the writer cannot alter: signed by a key in a KMS, written to a separate account's write-once (WORM) storage, or published to an external transparency log. Schneier and Kelsey described keyed hash-chain audit logs in 1999; Crosby and Wallach later showed

how Merkle-tree structures make such logs efficiently verifiable — the same idea behind Certificate Transparency (§10.5) [Schneier & Kelsey 1999; Crosby & Wallach 2009].

Figure 10.10 — Tamper-evident audit logging pipeline.



Reading the diagram. Services emit events asynchronously to a stream (dotted arrows), so logging never blocks user requests — at the cost that events can be delayed; the stream must be durable so events are not lost. A single writer appends each event with the previous entry's hash. Every hour it asks the KMS to sign the current head (the signing key is usable only by the writer's role and never leaves the KMS) and stores signed heads and log replicas in a WORM bucket in a separate account with object lock, so even production administrators cannot delete them before retention expires. An independent verifier recomputes the chain and compares it with the signed heads, alerting the security team on any mismatch. Failure points: the writer is a bottleneck and single point of ordering (partition the chain per service or per tenant to scale); clock skew complicates ordering (use the chain order, not timestamps, as the authority).

```
        """Tamper-evident audit log: each entry commits to the previous one (hash
chain)."""
        import hashlib
        import hmac
        import json

        GENESIS = "0" * 64

        def entry_hash(prev_hash: str, event: dict) -> str:
            payload = json.dumps(event, sort_keys=True, separators=(",", ":")).encode()
            return hashlib.sha256(prev_hash.encode() + payload).hexdigest()

        class AuditLog:
            def __init__(self):
                self.entries: list[dict] = []

            def append(self, event: dict) -> str:
                prev = self.entries[-1]["hash"] if self.entries else GENESIS
                h = entry_hash(prev, event)
                self.entries.append({"event": event, "prev": prev, "hash": h})
                return h                        # publish/anchor this "head" externally

            def verify(self, anchored_head: str | None = None) -> int | None:
                """Return index of first bad entry, or None if the chain is intact."""
                prev = GENESIS
                for i, e in enumerate(self.entries):
                    if e["prev"] != prev or not hmac.compare_digest(e["hash"],
entry_hash(prev, e["event"])):
                        return i
                    prev = e["hash"]
                if anchored_head is not None and prev != anchored_head:
                    return len(self.entries)   # tail truncated or rewritten
                return None
```

Tested: an intact chain verifies; editing the `resource` field of entry 1 makes `verify` return index 1; deleting the last entry is caught only when the verifier is given the previously anchored head.

What to notice: canonical JSON (`sort_keys`, fixed separators) makes the hash reproducible; truncation is detectable *only* because the verifier holds an anchored head from outside the log; and in production the anchor is signed and stored where the writer cannot overwrite it.

INDUSTRY EXAMPLE

AWS CloudTrail log file integrity validation. CloudTrail delivers hourly digest files that contain the SHA-256 hashes of the log files delivered in that hour, are signed with an RSA key held by AWS, and include the signature of the previous digest — a signed hash chain. Customers verify with a CLI command and are advised to store logs in a separate account with S3 Object Lock. *Evidence: Primary* — [AWS Docs: CloudTrail log file integrity validation]

10.13 A security checklist for every design

Use this table in every design document and design review — and in interviews, where spending two minutes on it distinguishes senior candidates.

Table 10.13 — Security checklist for every design.

Area	Question to answer in the design	Default good answer	Section
Threat model	What are the assets, adversaries and trust boundaries? Top 3 threats?	DFD with boundaries; STRIDE table; owners for mitigations	§10.3
Authentication	How is every user and every service identified?	Central IdP, MFA for humans; mTLS or signed tokens for services	§10.10, §8.11
Authorization	Where is each access decision made? Default?	Deny by default; central policy; object-level checks in data layer	§10.7.1
Input handling	What untrusted input crosses boundaries, and how is it validated?	Schema validation; parameterized queries; output encoding; SSRF egress proxy	§10.7
Data in transit	Is every hop encrypted, including internal ones?	TLS 1.3 everywhere; mTLS between services	§10.4.4, §10.10
Data at rest	What is sensitive? At which layer is it encrypted? Who can decrypt?	Volume encryption everywhere; KMS envelope/field encryption for sensitive fields	§10.9
Keys and secrets	Where do secrets live? Lifetime? Rotation?	Vault or KMS; short-lived dynamic credentials; no secrets in code or images	§10.8
Passwords	If you store them, how?	Argon2id (or scrypt/bcrypt), unique salt, optional pepper, breached-list check	§10.6
Least privilege	What is the minimum each component can do? Blast radius if compromised?	Per-service roles; segmented networks; no standing admin access	§10.10.3
Availability under attack	What happens under a 100× traffic flood?	CDN/anycast, rate limits, quotas, load shedding, hidden origin	§10.11
Logging and detection	Can you reconstruct who did what? Can logs be altered?	Structured audit log; tamper-evident; separate account; alerts	§10.12
Supply chain	How do dependencies and builds stay trustworthy?	Pinned dependencies, scanning, signed artifacts, protected CI	§10.7, Ch 50
Incident response	Who is paged, what is revoked, how do you rotate everything?	Runbooks; tested key rotation; backups that attackers cannot delete	§39.12
Privacy and retention	Do you need this data? For how long?	Collect minimum; retention limits; crypto-shredding for deletion	§10.9, Ch 50

10.14 Summary and revision notes

SUMMARY

- Security is a property of the architecture. Ask “against whom, protecting what, for which property (CIA, authentication, authorization, non-repudiation)?”
- Threat modeling: draw a DFD, mark trust boundaries, walk STRIDE per element, rank with likelihood × impact, and turn mitigations into tracked requirements. Most mitigations are architectural.
- Pick primitives by goal: hash (fingerprint, no key), MAC (shared-key authenticity), AEAD such as AES-GCM (secrecy + integrity), asymmetric encryption/KEM (no pre-shared key), signatures (publicly verifiable, non-repudiation), ECDHE (key agreement with forward secrecy), CSPRNG (all randomness).
- Never reuse a GCM nonce with the same key; never use `H(key‖msg)` as a MAC; never compare MACs with `==`; never use MD5/SHA-1 for security; use high-level libraries.
- PKI binds keys to names through CA signatures; Certificate Transparency makes mis-issuance detectable; short-lived, automatically rotated certificates beat revocation.
- Passwords: Argon2id (OWASP minimum m = 19 MiB, t = 2, p = 1), scrypt or bcrypt; unique salt; optional pepper in a KMS; store parameters; rehash on login; check breached lists; MFA. Expected cracking time $T = S/(2RG)$.
- OWASP Top 10 categories are missing central controls: deny-by-default authorization, parameterized queries, egress proxies against SSRF, signed artifacts, audit logging.
- Secrets: vault, short-lived dynamic credentials, workload identity, two-version rotation, secret scanning. Misuse window shrinks from months to the TTL.
- Encrypt in transit everywhere (TLS 1.3, mTLS). At rest, the layer decides which attacker you stop. Envelope encryption: DEK encrypts data locally, KMS-held KEK wraps the DEK; cheap rotation, auditable decrypts, crypto-shredding.
- Zero trust: no trust from network location; per-request authentication and authorization of user, device and workload (BeyondCorp, NIST SP 800-207), plus least privilege and segmentation for defense in depth.
- DDoS: volumetric (bandwidth, absorb upstream), protocol (state, SYN cookies/stateless edge), application (requests, rate limits, caching, shedding). Amplification $B_{\text{victim}} = B_{\text{attacker}} \times A$.
- Audit logs: who/what/which/when/where/result, no secrets; hash-chained, anchored heads signed and stored in WORM storage outside the writer’s control.

10.15 Knowledge checkpoint

1. Which STRIDE category does each of the following violate: (a) a forged webhook, (b) an admin denying a deletion, (c) reading another user's invoice by changing an ID?
2. Why is SHA-256(key ‖ message) not a safe MAC, and what does HMAC change?
3. What happens if AES-GCM encrypts two messages with the same key and nonce?
4. What is the purpose of associated data (AAD) in an AEAD scheme? Give a database example.
5. Why does TLS 1.3 use ephemeral Diffie–Hellman rather than RSA key transport?
6. A MAC and a signature both detect tampering. Which one provides non-repudiation, and why?
7. What problem does Certificate Transparency solve that the CA chain alone does not?
8. Name three reasons salts are required in password storage.
9. Compute the entropy of a 10-character password chosen uniformly from 62 alphanumeric characters.
10. Give two architectural controls for SSRF beyond URL validation.
11. Classify: SYN flood, DNS amplification, HTTP flood on `/search`.
12. Why must the head of a hash-chained audit log be anchored outside the log?

Answers

1. a. Spoofing; (b) Repudiation; (c) Elevation of privilege (an authorization failure — an IDOR; it also causes information disclosure).
2. SHA-256 is vulnerable to length extension: from the tag of `key‖m`, an attacker can compute a valid tag for `key‖m‖padding‖x` without the key. HMAC's nested two-pass construction with inner and outer padded keys prevents this.
3. The same keystream is used twice, so XORing the ciphertexts reveals the XOR of the plaintexts, and the authentication key can be recovered, enabling forgeries. Confidentiality and integrity both fail.
4. AAD is authenticated but not encrypted; it binds ciphertext to its context. Putting `table=users;id=42` in the AAD prevents an attacker with DB write access from moving that ciphertext to row 43.
5. Forward secrecy: ephemeral keys are discarded after each session, so stealing the server's long-term key later does not decrypt recorded traffic. The long-term key only signs.
6. The signature. A MAC key is shared by both parties, so either could have produced the tag; only the private-key holder can produce a signature, and anyone can verify it.
7. It detects a trusted CA issuing a certificate for your domain to someone else (compromise or error). The chain only shows *some* trusted CA signed it; CT logs make every issuance public so domain owners can notice.

8. Identical passwords no longer have identical hashes; precomputed (rainbow) tables become useless; attackers must attack each user separately, multiplying cost by the number of users.
9. $H = 10 \times \log_2 62 \approx 10 \times 5.954 = 59.5$ bits.
10. A dedicated egress proxy in a network segment without routes to internal ranges; blocking or token-protecting the metadata service (IMDSv2); least-privilege instance roles; pinning resolved IPs and disabling redirects.
11. SYN flood: protocol (state exhaustion). DNS amplification: volumetric. HTTP flood on `/search`: application layer.
12. Anyone who can write the log can recompute the entire chain after editing it. A head signed by a separate key and stored in WORM storage the writer cannot modify fixes a point the rewritten chain will not match.

10.16 Interview questions

Basic

Q1. How should a web application store user passwords? Answer. Never in plaintext or reversibly encrypted. Hash each password with a slow, memory-hard password-hashing function — Argon2id by default (at least m = 19 MiB, t = 2, p = 1 per OWASP), otherwise scrypt or bcrypt — with a unique random salt of at least 16 bytes. Store algorithm and parameters with the hash so you can raise cost later and rehash on next login. Optionally apply a pepper (a secret key in a KMS/HSM) so a database-only leak is not attackable. Compare in constant time, rate-limit login, check new passwords against breached lists, and offer MFA. Size the login fleet for the hashing cost.

Q2. What is the difference between encryption, hashing and signing? Answer. Encryption provides confidentiality and is reversible with a key. Hashing is a keyless one-way fingerprint for integrity against a trusted reference. Signing uses a private key to produce a tag anyone can verify with the public key, providing integrity, authenticity and non-repudiation. A MAC is the shared-key analogue of a signature without non-repudiation. AEAD encryption combines confidentiality with integrity.

Intermediate

Q3. Design encryption for a multi-tenant document store where each tenant's data must be deletable on request, including from backups. Answer. Use envelope encryption with a per-tenant KEK in a KMS and a per-document DEK. Each document is encrypted locally with AES-256-GCM under its DEK; the wrapped DEK and KEK ID are stored with it, and the encryption context includes tenant ID and document ID. IAM policy on each KEK allows decrypt only by the service's role, and KMS logs every decrypt for audit. To delete a tenant, schedule deletion of its KEK: all its DEKs, and therefore all copies

of its documents including immutable backups, become unreadable (crypto-shredding). Discuss KMS quotas (DEK per document is fine for large documents; batch keys for tiny records), KMS availability as a dependency, regional keys for residency, and the waiting period that protects against accidental KEK deletion.

Q4. A partner sends you webhooks. How do you make sure they are genuine and not replayed? Answer. Share a high-entropy secret per partner (with a key ID for rotation). The partner computes HMAC-SHA-256 over a canonical string of timestamp, a unique event ID or nonce, and the raw body. You verify by recomputing over the raw bytes, comparing in constant time, rejecting timestamps outside a ±5-minute window and rejecting event IDs already seen (a shared store with TTL just longer than the window). Process idempotently by event ID (§22.15), since retries are legitimate. Support two active keys during rotation. Use TLS for confidentiality; HMAC provides authenticity.

Advanced

Q5. Your company runs 400 microservices on a flat internal network with shared database passwords in environment variables. Propose a two-year security roadmap. Answer. Start with visibility: inventory services and data flows, threat-model the most sensitive data stores, centralize tamper-evident audit logging. Phase 1 (0–6 months): move secrets into a vault and rotate all of them (assume compromise); add secret scanning; one IAM role and one database user per service. Phase 2 (6–12 months): issue short-lived SPIFFE-style workload certificates via a mesh or library; enable mTLS in permissive then strict mode; move databases to dynamic credentials. Phase 3 (12–24 months): default-deny network policies, per-service authorization policies, an identity-aware proxy for internal tools, just-in-time admin access. Measure progress (share of traffic on mTLS, secrets older than 30 days, wildcard permissions), roll out with dashboards and per-phase rollback switches. Trade-offs: mesh latency and operational load, database connection limits with per-workload users, and the need for owned, reviewed policies.

Q6. How would you design an audit logging system for a bank that regulators and internal investigators must be able to trust, even against a malicious administrator? Answer. Define the event schema (actor, workload, action, resource, result, source, trace ID) and emit from every service through a durable stream so logging does not block requests. A writer service builds per-partition hash chains; hourly it signs each chain head using a KMS key restricted to the writer role and writes signed heads plus log segments to WORM storage (object lock in compliance mode) in a separate account administered by a different team. An independent verifier recomputes chains and compares with signed heads; mismatches alert security operations. Separate duties: no single administrator controls the production account, the log account and the KMS key policy. Index a copy in a SIEM for search, treating the SIEM as derived and untrusted. Address clock sync, retention (years, per regulation), personal data minimization, and Merkle-tree structures if third parties need efficient inclusion proofs.

10.17 Practical exercises

1. **Password hashing benchmark.** Run the code in §10.6.2. Increase Argon2id memory to 64 MiB and t to 3 and measure latency. Compute how many cores you would need for 500 logins per second at peak with 50% headroom.
2. **HMAC request signing.** Extend the code in §10.4.6 to support two active keys per client, and write a test proving that removing the HTTP method from the canonical string lets an attacker turn a signed `GET` into a `DELETE`.
3. **Envelope encryption.** Extend the `FakeKMS` example in §10.9.2 with a per-tenant KEK and a `delete_tenant` operation. Show that encrypted objects for a deleted tenant can no longer be decrypted while other tenants are unaffected. Then replace `FakeKMS` with a real KMS client in a sandbox account if you have one.
4. **Brute-force calculator.** Write a function that takes alphabet size, length, guesses per second and number of GPUs and prints expected time. Use it to compare a 4-word and a 6-word Diceware passphrase (7,776 words) against bcrypt and against unsalted SHA-256.
5. **Tamper-evident log.** Modify the audit log in §10.12 so that every 100 entries the head is signed with an Ed25519 key (§10.4.8). Write a verifier that accepts only chains whose signed heads verify.
6. **Threat model.** Draw a DFD for a URL shortener (Ch 52.1) with trust boundaries and produce a STRIDE table with at least eight threats ranked by $L \times I$.

10.18 Design challenges

1. **Secure file-sharing link service.** Users share files by link; links can expire and be revoked; recipients may be anonymous. Design storage encryption, link-token generation (entropy and length), revocation, download rate limiting and audit logging. Identify where signed URLs, HMAC and envelope encryption each fit.
2. **Zero-trust migration for an admin console.** A company's internal admin console is reachable over VPN by 300 support agents and offers "view any customer" functionality. Redesign access with an identity-aware proxy, device posture, just-in-time elevation for sensitive actions and an audit trail that customers can be shown. State the failure mode if the identity provider is down.
3. **DDoS-resilient public API.** A public API serves 20,000 RPS normally, with an expensive search endpoint costing 50 ms of database time per request. Design layered defenses for a 1 Tbps volumetric attack and for a 2 million RPS application-layer flood from residential IPs. Quantify how much the cache hit ratio and per-key rate limits reduce database load.

10.19 References

1. Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.
2. Kohnfelder, L., Garg, P. *The Threats to Our Products*. Microsoft internal paper, 1999.
3. OWASP Foundation. *OWASP Top 10:2021*. 2021. https://owasp.org/Top10/
4. OWASP Foundation. *OWASP Top 10:2025*. 2025–2026. https://owasp.org/Top10/2025/
5. OWASP Cheat Sheet Series. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
6. NIST. *SP 800-63B-4: Digital Identity Guidelines — Authentication and Authenticator Management*. National Institute of Standards and Technology, 2025. https://pages.nist.gov/800-63-4/sp800-63b.html
7. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, IETF, 2021. https://www.rfc-editor.org/rfc/rfc9106
8. Percival, C., Josefsson, S. *The scrypt Password-Based Key Derivation Function*. RFC 7914, IETF, 2016. https://www.rfc-editor.org/rfc/rfc7914
9. Ward, R., Beyer, B. “BeyondCorp: A New Approach to Enterprise Security.” *;login:* 39(6), USENIX, 2014.
10. Osborn, B., McWilliams, J., Beyer, B., Saltonstall, M. “BeyondCorp: Design to Deployment at Google.” *;login:* 41(1), USENIX, 2016.
11. Rose, S., Borchert, O., Mitchell, S., Connelly, S. *Zero Trust Architecture*. NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207
12. Ferguson, N., Schneier, B., Kohno, T. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010.
13. Adkins, H., Beyer, B., Blankinship, P., Lewandowski, P., Oprea, A., Stubblefield, A. *Building Secure and Reliable Systems*. O’Reilly, 2020.
14. Amazon Web Services. *AWS Key Management Service Developer Guide — AWS KMS concepts (envelope encryption, data keys, encryption context)*. https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html
15. Amazon Web Services. *Amazon EC2 User Guide — Use the Instance Metadata Service Version 2 (IMDSv2)*. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html
16. Amazon Web Services. *AWS CloudTrail User Guide — Validating CloudTrail log file integrity*. https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html
17. Krawczyk, H., Bellare, M., Canetti, R. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, IETF, 1997. https://www.rfc-editor.org/rfc/rfc2104
18. Rescorla, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018. https://www.rfc-editor.org/rfc/rfc8446

19. Laurie, B., Langley, A., Kasper, E. *Certificate Transparency*. RFC 6962, IETF, 2013. https://www.rfc-editor.org/rfc/rfc6962
20. Barnes, R., Hoffman-Andrews, J., McCarney, D., Kasten, J. *Automatic Certificate Management Environment (ACME)*. RFC 8555, IETF, 2019. https://www.rfc-editor.org/rfc/rfc8555
21. Dworkin, M. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.
22. Barker, E. *Recommendation for Key Management: Part 1 — General*. NIST SP 800-57 Part 1 Rev. 5, 2020.
23. NIST. *FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)*. August 2024.
24. US-CERT (CISA). *Alert TA14-017A: UDP-Based Amplification Attacks*. 2014 (updated).
25. GitHub Engineering. “February 28th DDoS Incident Report.” GitHub Blog, 2018.
26. Google Cloud. “How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack.” Google Cloud Blog, 2023.
27. Fox-IT. *Black Tulip: Report of the Investigation into the DigiNotar Certificate Authority Breach*. 2012.
28. Schneier, B., Kelsey, J. “Secure Audit Logs to Support Computer Forensics.” *ACM Transactions on Information and System Security* 2(2), 1999.
29. Crosby, S. A., Wallach, D. S. “Efficient Data Structures for Tamper-Evident Logging.” *USENIX Security Symposium*, 2009.

10.20 Open verification items

- OWASP Top 10:2025 final category names, numbering and release status (reported as unveiled November 2025, finalized early 2026); owasp.org was not reachable from the authoring environment.
- NIST SP 800-63B-4 exact final wording on 15-character single-factor minimum and on peppers/keyed hashing (confirmed only through secondary sources).
- CA/Browser Forum ballot SC-081 lifetime schedule (200 → 100 → 47 days, 2026–2029).
- GPU guessing rates (~10^{10} SHA-256/s per GPU; ~10^{4}/s for tuned memory-hard hashes) are order-of-magnitude estimates.
- AWS KMS request quotas and pricing ($0.03 per 10,000 requests) as of 2026.
- LinkedIn 2012/2016 and Capital One 2019 figures (secondary reporting).
- Exact URLs for BeyondCorp ;login: papers and the AWS KMS envelope-encryption page anchor.